

6. 时序逻辑电路

6.1 时序逻辑电路的基本概念

6.2 同步时序逻辑电路的分析

6.3 同步时序逻辑电路的设计

6.4 异步时序逻辑电路的分析

6.5 若干典型的时序逻辑电路

6.6 简单的时序可编程逻辑器件GAL

6.7 用Verilog描述时序逻辑电路

6.8 VerilogHDL在时序电路的设计



教学基本要求

- 1、熟练掌握时序逻辑电路的描述方式及其相互转换。
- 2、熟练掌握时序逻辑电路的分析方法
- 3、熟练掌握时序逻辑电路的设计方法
- 4、熟练掌握典型时序逻辑电路计数器、寄存器、移位寄存器的逻辑功能及其应用。
- 5、正确理解时序可编程器件的原理及其应用。
- 6、学会用Virelog HDL设计时序电路及时序可编程逻辑器件的方法。

6.1 时序逻辑电路的基本概念

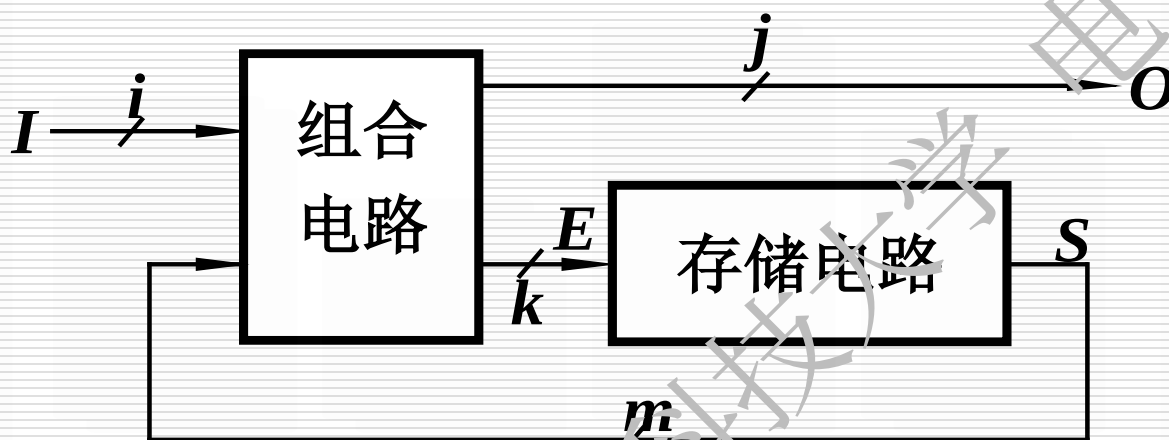
6.1.1 时序逻辑电路的模型与分类

6.1.2 时序电路逻辑功能的表达

6.1 时序逻辑电路的基本概念

6.1.1 时序逻辑电路的模型与分类

1. 时序电路的一般化模型



结构特征:

*电路由组合电路和存储电路组成。

*电路存在反馈。

工作特征: 时序逻辑电路的工作特点是任意时刻的输出状态不仅与该当前的输入信号有关, 而且与此前电路的状态有关。

输出方程: $O = f_1(I, S)$

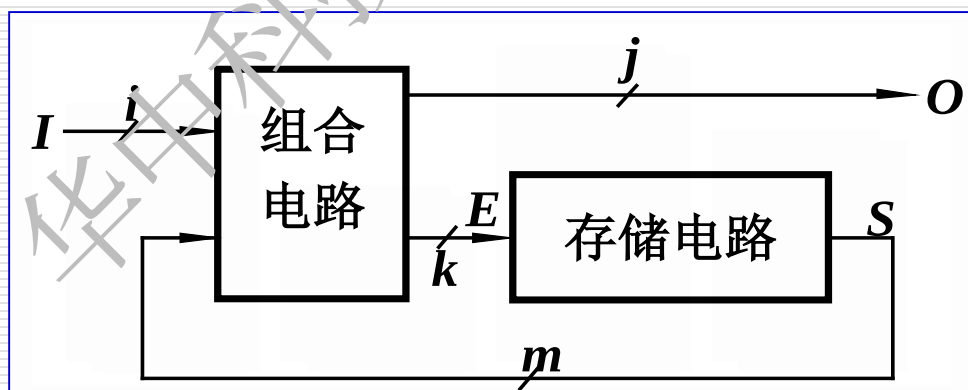
表达输出信号与输入信号、状态变量的关系式

激励方程: $E = f_2(I, S)$

表达了激励信号与输入信号、状态变量的关系式

状态方程: $S^{n+1} = f_3(E, S^n)$

表达存储电路从现态到次态的转换关系式

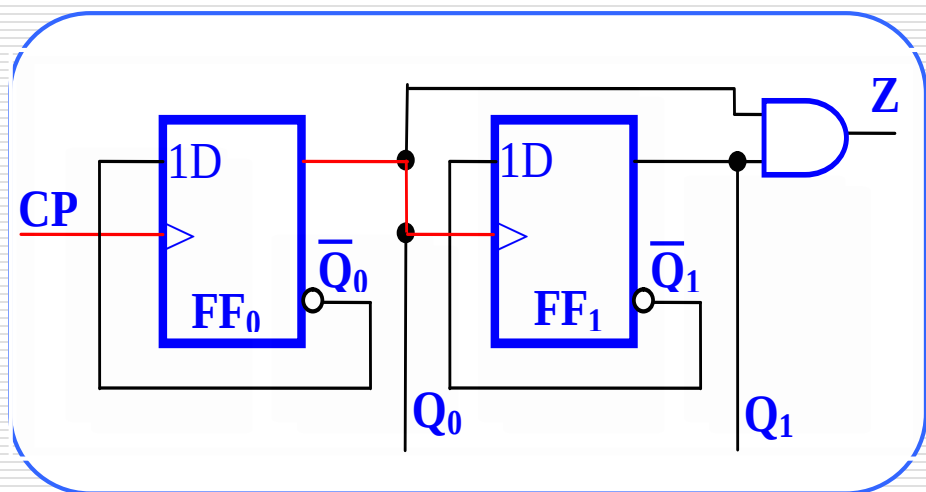
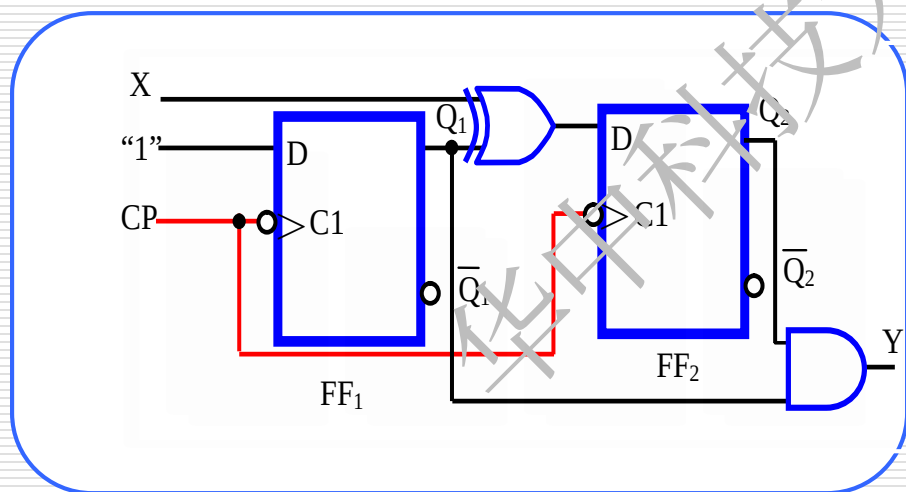


2、异步时序电路与同步时序电路

时序电路

同步：存储电路里所有触发器有一个**统一的时钟源**，
它们的状态在**同一时刻更新**。

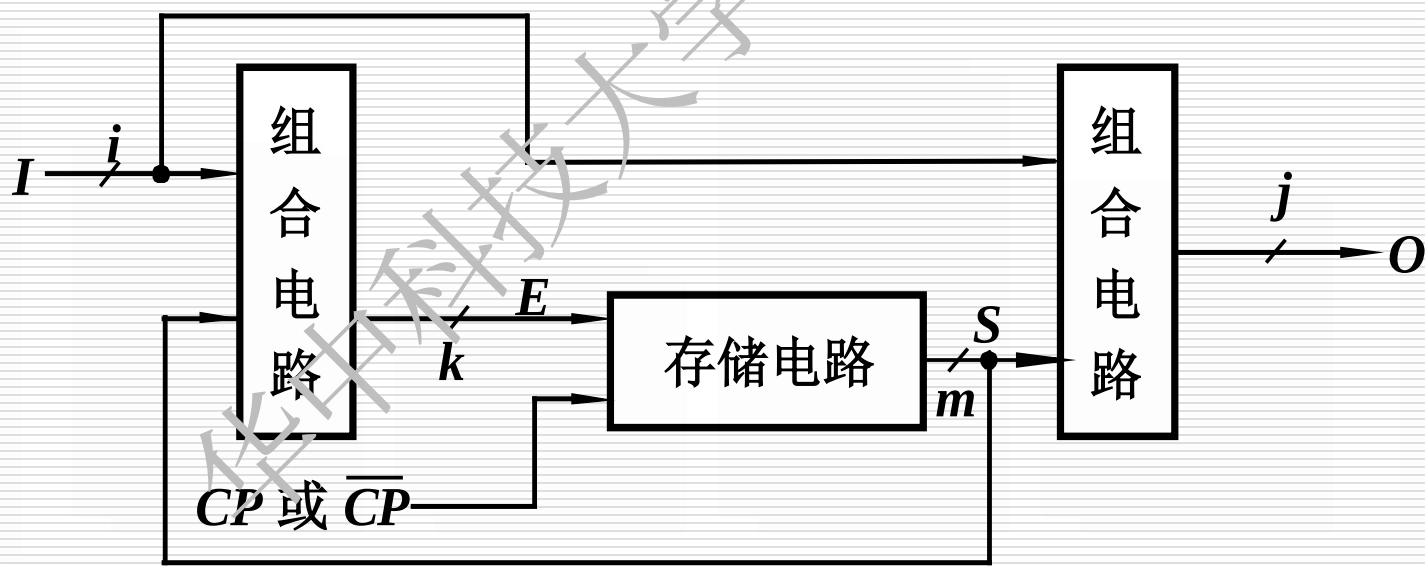
异步：**没有统一的时钟脉冲**或没有时钟脉冲，电路
的**状态更新不是同时发生的**。



3. 米利型和穆尔型时序电路

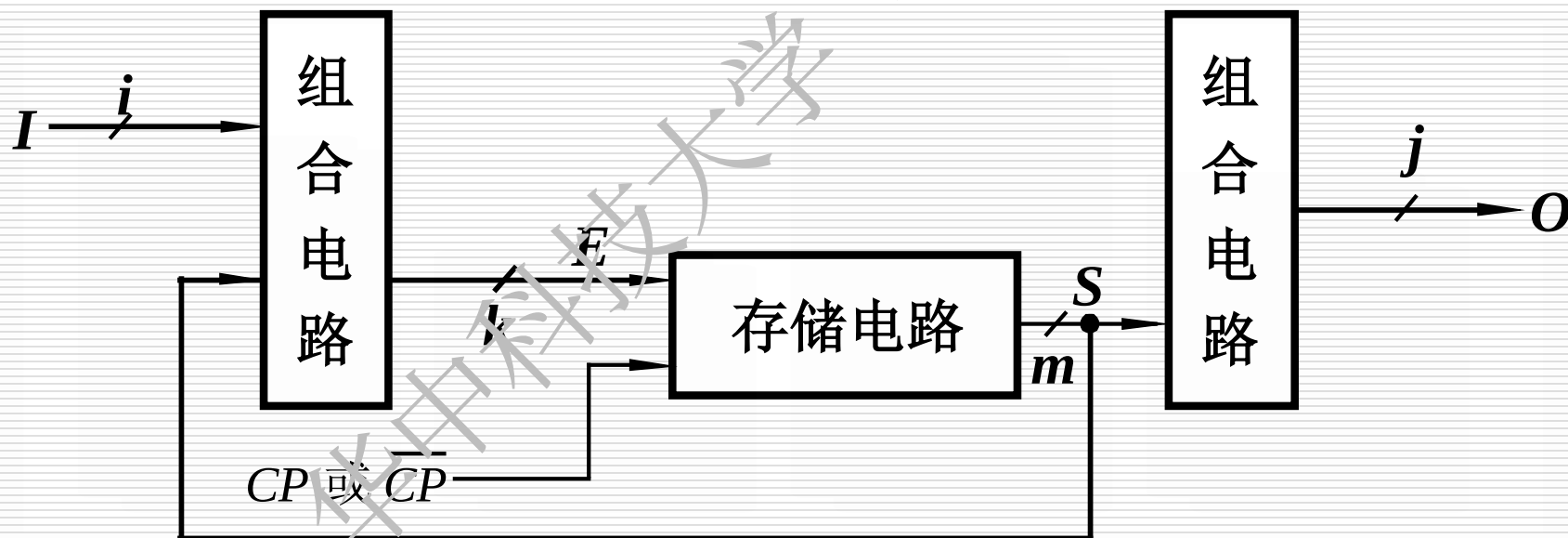
米利型电路

电路的输出是输入变量 A 及触发器输出 Q_1 、 Q_0 的函数，这类时序电路亦称为米利型电路



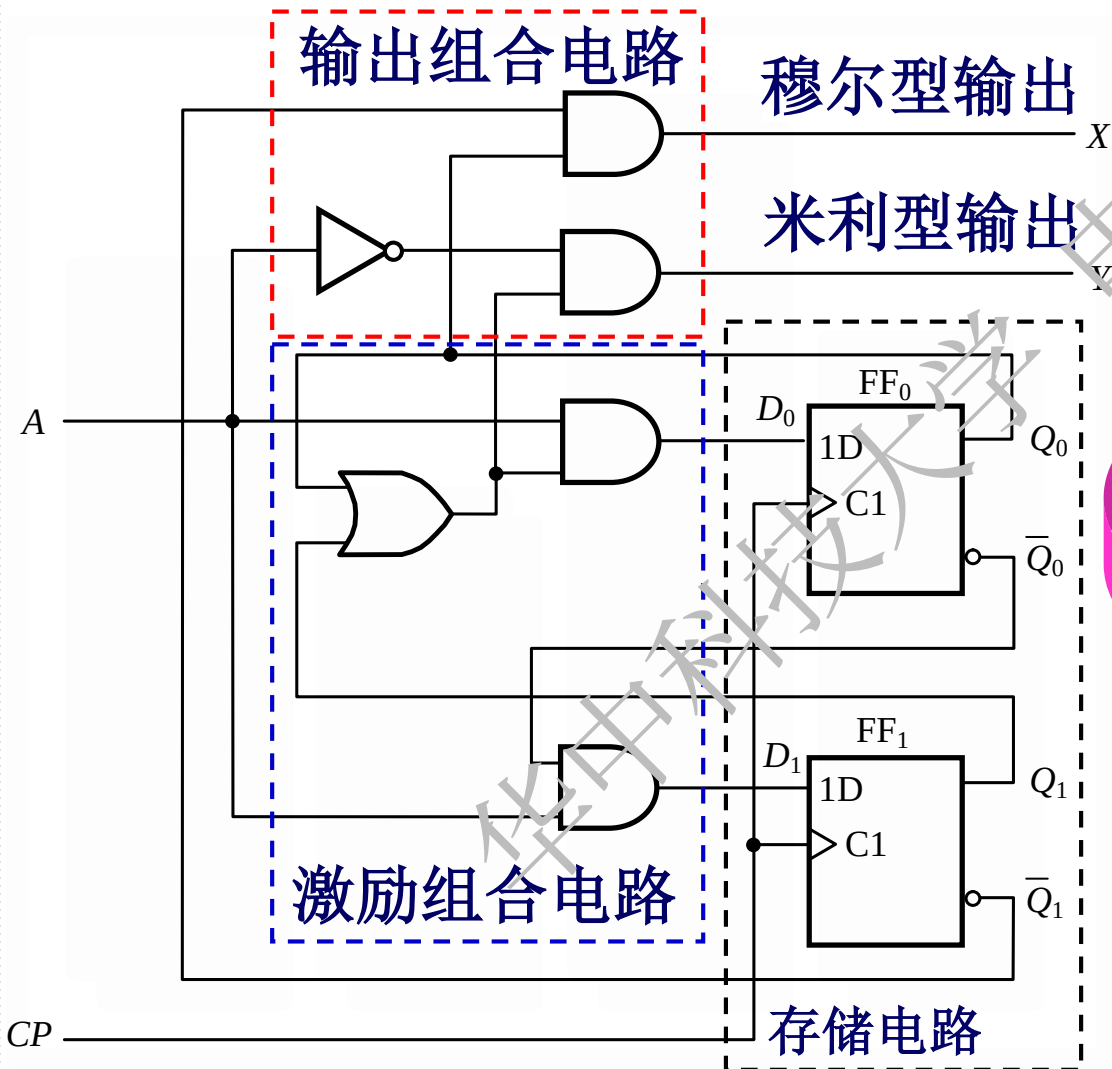
穆尔型电路

电路输出仅仅取决于各触发器的状态，而不受电路当时的输入信号影响或没有输入变量，这类电路称为穆尔型电路



6.1.2 时序电路功能的表达方法

1. 逻辑方程组



输出方程

$$X = \overline{Q_1} Q_0$$
$$Y = (Q_0 + Q_1) \overline{A}$$

激励方程组

$$D_0 = (Q_0 + Q_1) A$$

$$D_1 = \overline{Q_0} A$$

状态方程组

$$Q_1^{n+1} = D$$

$$Q_0^{n+1} = (Q_0^n + Q_1^n) A$$

$$Q_1^{n+1} = \overline{Q_0^n} A$$

2. 根据方程组列出状态转换真值表

输出方程

$$X = \overline{Q_1} Q_0$$

$$Y = (Q_0 + Q_1) \overline{A}$$

状态方程组

$$Q_1^{n+1} = \overline{Q_0^n} A$$

$$Q_0^{n+1} = (Q_0^n + Q_1^n) A$$

状态转换真值表

Q_1^n	Q_0^n	A	Q_1^{n+1}	Q_0^{n+1}	X	Y
0	0	0	0	0	0	0
0	0	1	1	0	0	0
0	1	0	0	0	1	1
0	1	1	0	1	1	0
1	0	0	0	0	0	1
1	0	1	1	1	0	0
1	1	0	0	0	0	1
1	1	1	0	1	0	0

3. 将状态转换真值表转换为状态表

状态转换真值表

Q_1^n	Q_0^n	A	Q_1^{n+1}	Q_0^{n+1}	X	Y
0	0	0	0	0	0	0
0	0	1	1	0	0	0
0	1	0	0	0	1	1
0	1	1	0	1	1	0
1	0	0	0	0	0	1
1	0	1	1	1	0	0
1	1	0	0	0	0	1
1	1	1	0	1	0	0

转换表

$Q_1^n Q_0^n$	$Q_1^{n+1} Q_0^{n+1} / Y$		X
	$A=0$	$A=1$	
00	00 / 0	10 / 0	0
01	00 / 1	01 / 0	1
10	00 / 1	11 / 0	0
11	00 / 1	01 / 0	0

4.根据转换表得状态表

令4个状态为00=a, 01=b, 10=c, 11=d, 得:

转换表

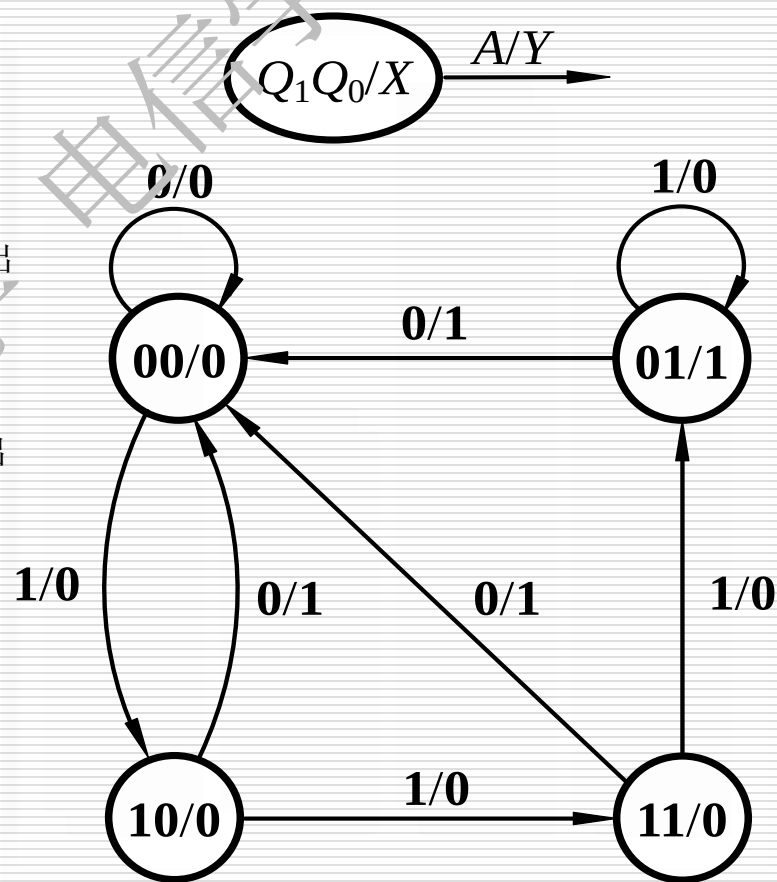
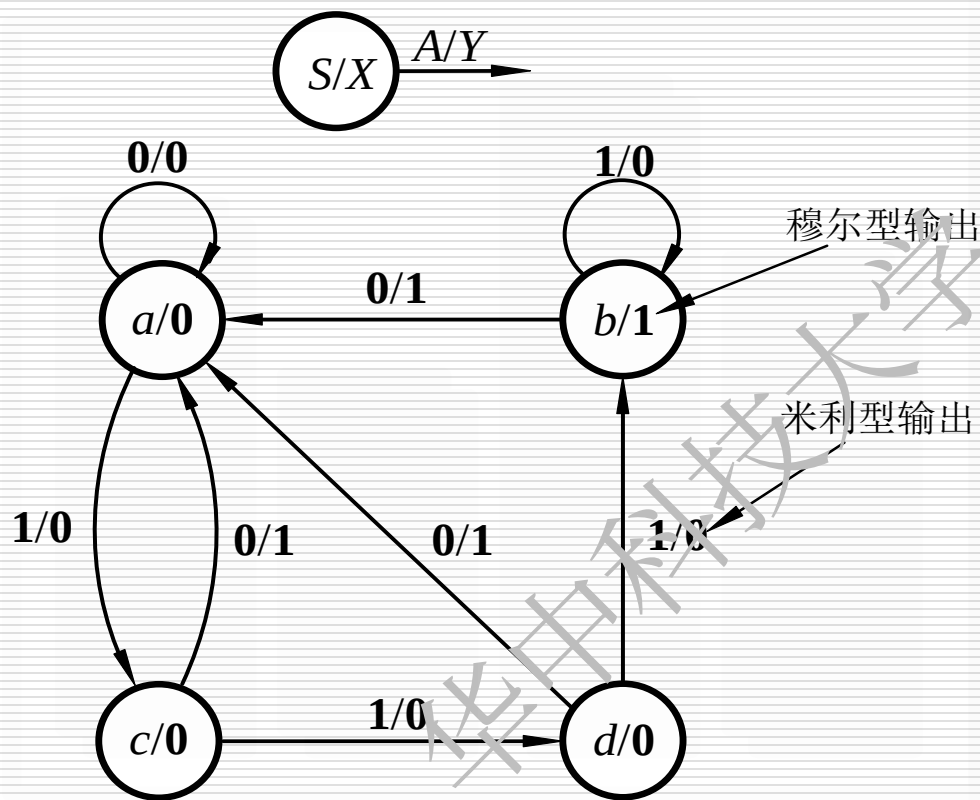
$Q_1^n Q_0^n$	$Q_1^{n+1} Q_0^{n+1} / Y$		X
	$A=0$	$A=1$	
00	00 / 0	10 / 0	0
01	00 / 1	01 / 0	1
10	00 / 1	11 / 0	0
11	00 / 1	01 / 0	0

状态表

S^n	S^{n+1} / Y		X
	$A=0$	$A=1$	
a	a / 0	c / 0	0
b	a / 1	b / 0	1
c	a / 1	d / 0	0
d	a / 1	b / 0	0

5.画状态图---有两种

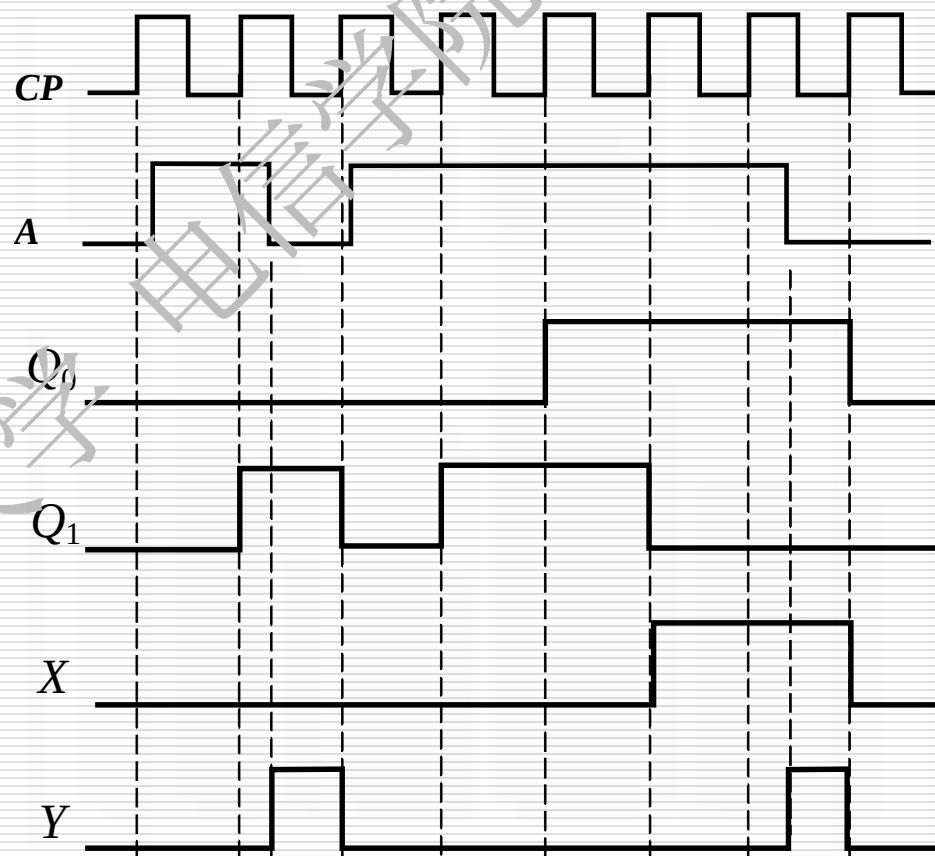
米利型输出标在方向线旁。穆尔型标在圆圈状态名旁。



6. 时序图 根据转换表画出波形图

转换表

$Q_1^n Q_0^n$	$Q_1^{n+1} Q_0^{n+1} / Y$		X
	$A=0$	$A=1$	
00	00 / 0	10 / 0	0
01	00 / 1	01 / 0	1
10	00 / 1	11 / 0	0
11	00 / 1	01 / 0	0



时序逻辑电路的五种描述方式是可以相互转换的

6.2 时序逻辑电路的分析

6.2.1 分析同步时序逻辑电路的一般步骤

6.2.2 同步时序逻辑电路分析举例

6.2 时序逻辑电路的分析

时序逻辑电路分析的任务：

分析时序逻辑电路在输入信号的作用下，其状态和输出信号变化的规律，进而确定电路的逻辑功能。

分析过程的主要表现形式：

时序电路的逻辑功能是由其状态和输出信号的变化规律呈现出来的。所以,分析过程主要是列出**电路状态表**或画出**状态图**、**工作波形图**。

6.2.1 分析同步时序逻辑电路的一般步骤:

1.了解电路的组成:

电路的输入、输出信号、触发器的类型等

2. 根据给定的时序电路图,写出下列各逻辑方程式:

(1) **输出方程**;

(2) 各触发器的**激励方程**;

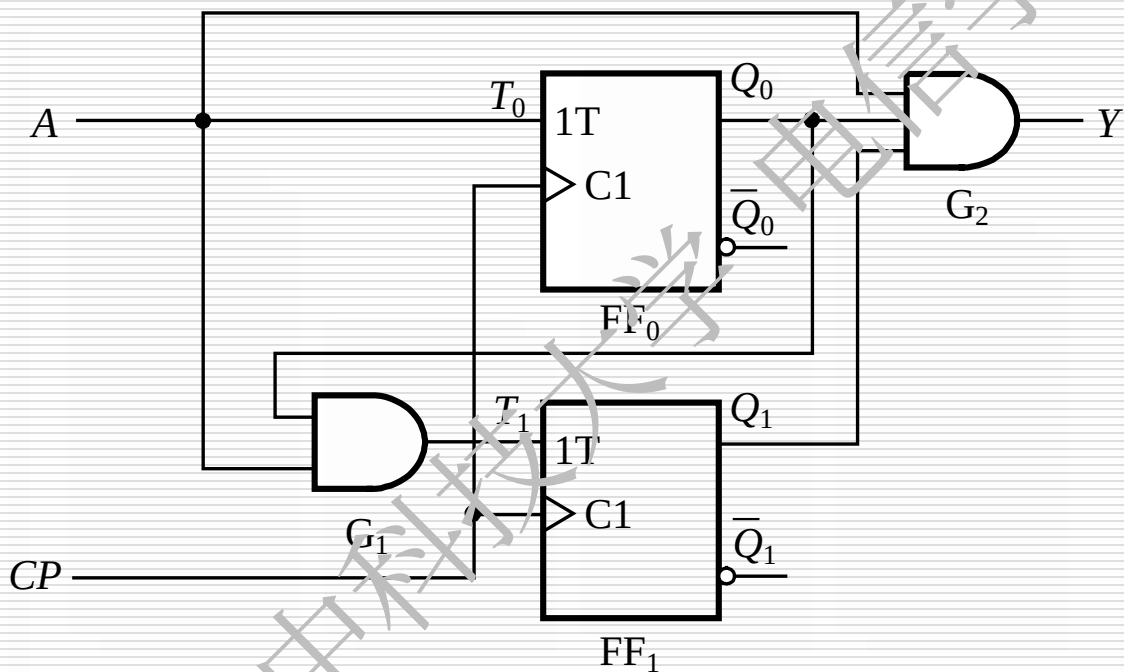
(3) **状态方程**: 将每个触发器的驱动方程代入其特性方程得状态方程.

3.列出状态转换表或画出状态图和波形图;

4 .确定电路的逻辑功能.

6.2.2 同步时序逻辑电路分析举例

例1 试分析如图所示时序电路的逻辑功能。



解： (1) 了解电路组成。

电路是由两个T 触发器组成的同步时序电路。

(2) 根据电路列出三个方程组

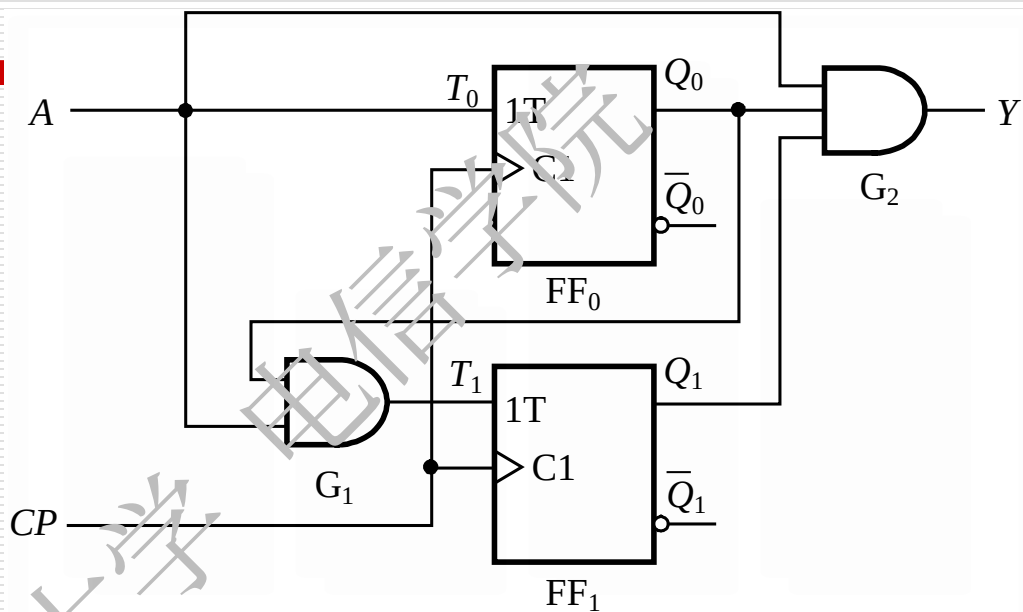
输出方程组:

$$Y=AQ_1Q_0$$

激励方程组:

$$T_0=A$$

$$T_1=AQ_0$$



将激励方程组代入T触发器的特性方程得状态方程组

$$Q^{n+1} = T \oplus Q^n = TQ^n + \overline{T}Q^n$$

$$Q_0^{n+1} = A \oplus Q_0^n$$

$$Q_1^{n+1} = (AQ_0^n) \oplus Q_1^n$$

(3) 根据状态方程组和输出方程列出状态表

$$Q_0^{n+1} = A \oplus Q_0^n$$

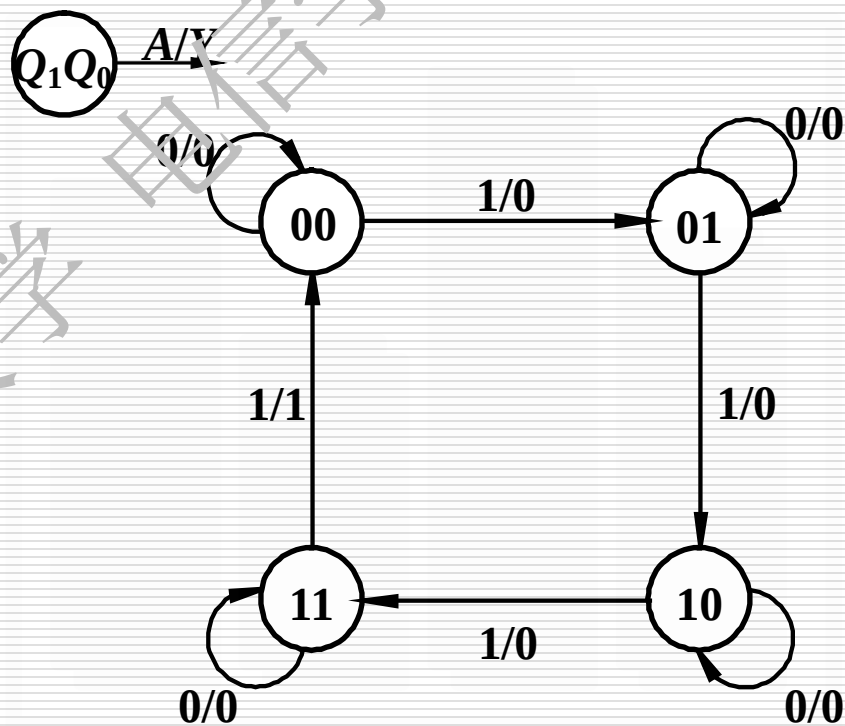
$$Q_1^{n+1} = (A Q_0^n) \oplus Q_1^n$$

$$Y = A Q_1 Q_0$$

$Q_1^n Q_0^n$	$Q_1^{n+1} Q_0^{n+1} / Y$	
	$A=0$	$A=1$
0 0	0 0 / 0	0 1 / 0
0 1	0 1 / 0	1 0 / 0
1 0	1 0 / 0	1 1 / 0
1 1	1 1 / 0	0 0 / 1

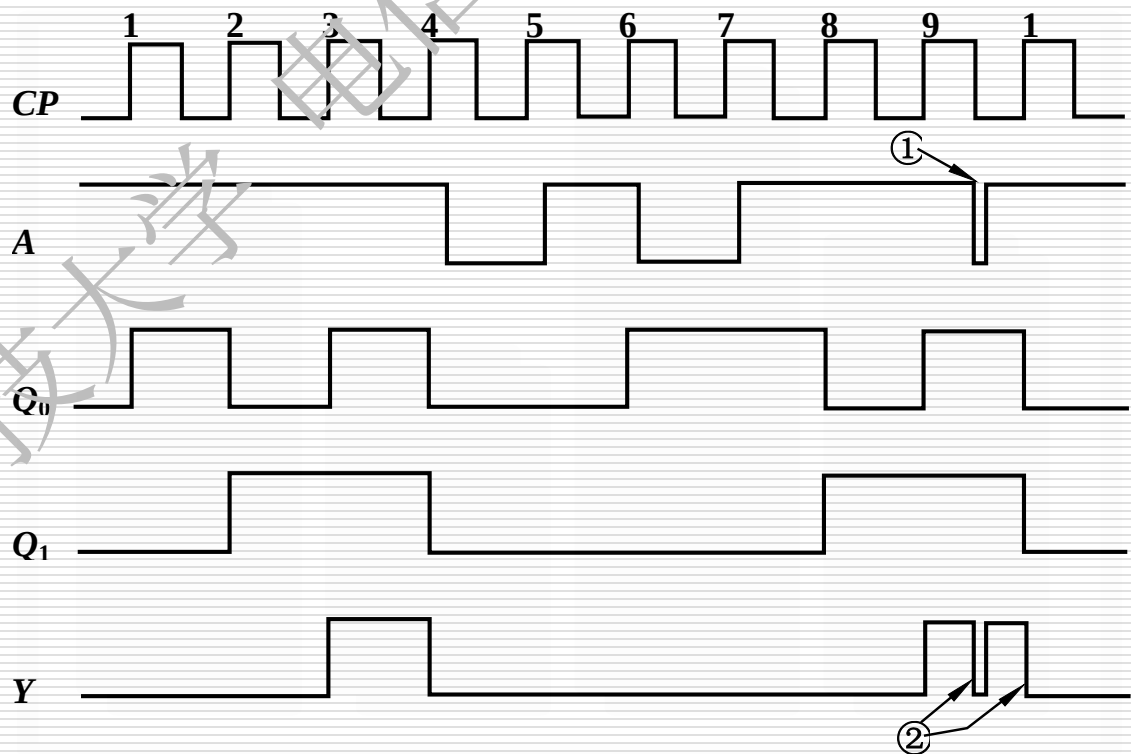
(4) 画出状态图

$Q_1^n Q_0^n$	$Q_1^{n+1} Q_0^{n+1} / Y$	
	$A=0$	$A=1$
0 0	0 0 / 0	0 1 / 0
0 1	0 1 / 0	1 0 / 0
1 0	1 0 / 0	1 1 / 0
1 1	1 1 / 0	0 0 / 1



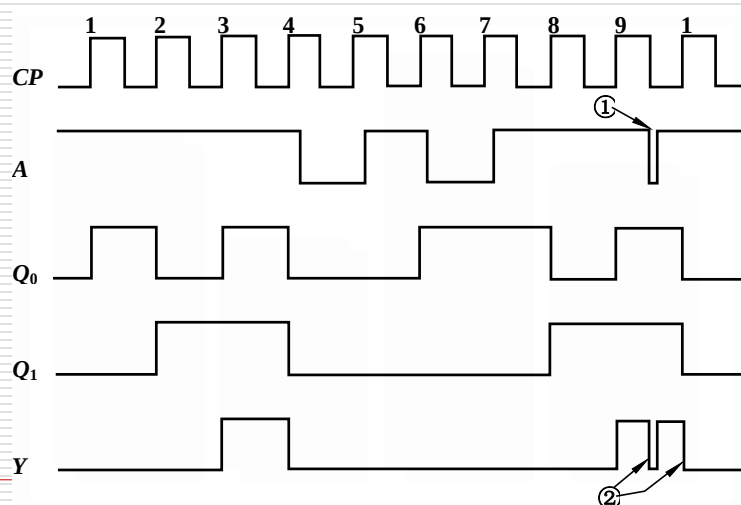
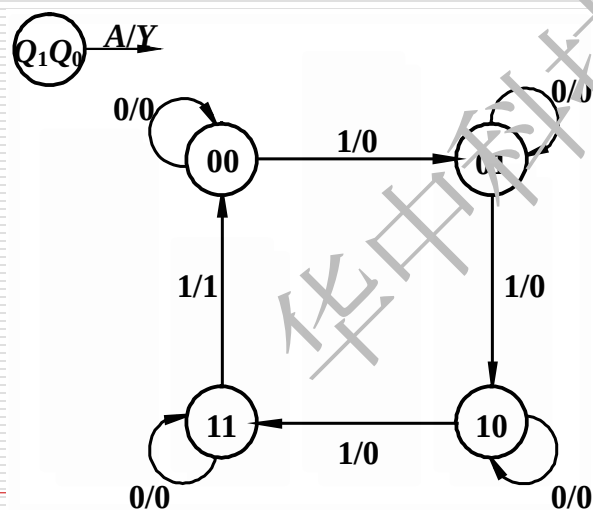
(5) 画出时序图

$Q_1^n Q_0^n$	$Q_1^{n+1} Q_0^{n+1} / Y$	
	$A=0$	$A=1$
0 0	0 0 / 0	0 1 / 0
0 1	0 1 / 0	1 0 / 0
1 0	1 0 / 0	1 1 / 0
1 1	1 1 / 0	0 0 / 1



(6) 逻辑功能分析

观察状态图和时序图可知，电路是一个由信号A控制的可控二进制计数器。当A=0时停止计数，电路状态保持不变；当A=1时，在CP上升沿到来后电路状态值加1，一旦计数到11状态，Y输出1，且电路状态将在下一个CP上升沿回到00。输出信号Y的下降沿可用于触发进位操作。



例2 试分析如图所示时序电路的逻辑功能。

解： 1. 了解电路组成。

电路是由两个T触发器组成的莫尔型同步时序电路。

2. 写出下列各逻辑方程式：

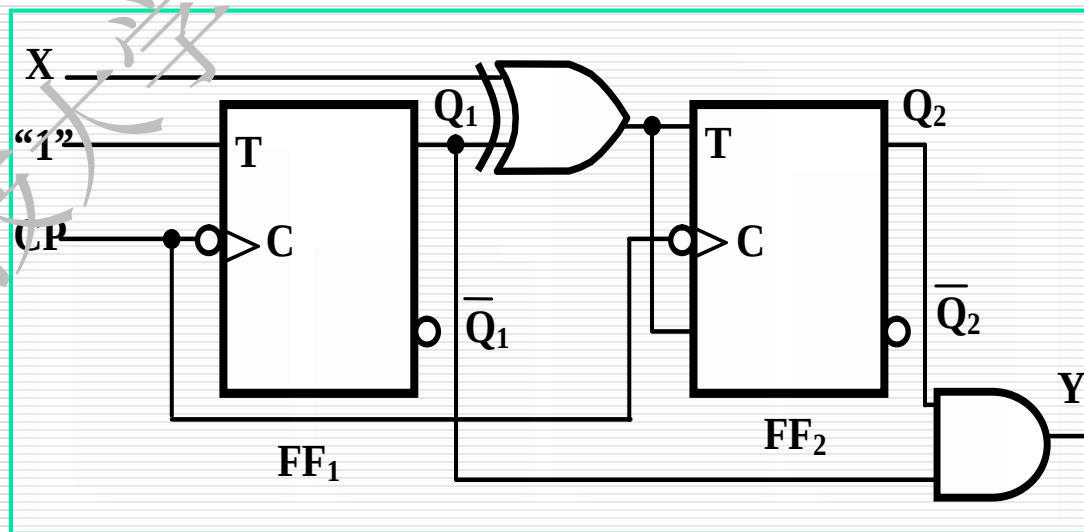
激励方程

$$T_1 = 1$$

$$T_2 = X \oplus Q_1$$

输出方程

$$Y = Q_2 Q_1$$



将激励方程代入T触发器的特性方程得状态方程

FF₁ **$T_1=1$**



$$Q^{n+1} = T\bar{Q}^n + \bar{T}Q^n$$



$$Q_1^{n+1} = 1 \cdot \bar{Q}_1^n + \bar{1} \cdot Q_1^n = \bar{Q}_1^n$$

FF₂ **$T_2=X \oplus Q_1$**



$$Q^{n+1} = T\bar{Q}^n + \bar{T}Q^n$$



$$Q_2^{n+1} = X \oplus Q_1^n \cdot \bar{Q}_2^n + \overline{X \oplus Q_1^n} \cdot Q_2^n$$

整理得:

$$Q_2^{n+1} = X \oplus Q_1^n \oplus Q_2^n$$

3.列出其状态转换表，画出状态转换图和波形图

$$Q_1^{n+1} = \overline{Q_1^n} \quad Q_2^{n+1} = X \oplus Q_1^n \oplus Q_2^n \quad Y = Q_2 Q_1$$

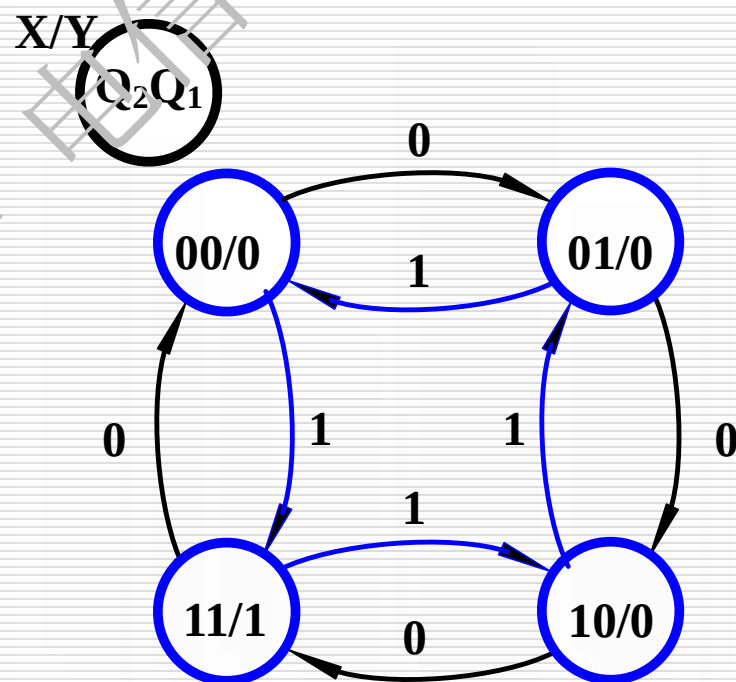
状态转换表

$Q_1^n Q_0^n$	$Q_1^{n+1} Q_0^{n+1}$		Y
	$X=0$	$X=1$	
0 0	0 1	1 1	0
0 1	1 0	0 0	0
1 0	1 1	0 1	0
1 1	0 0	1 0	1

画出状态图

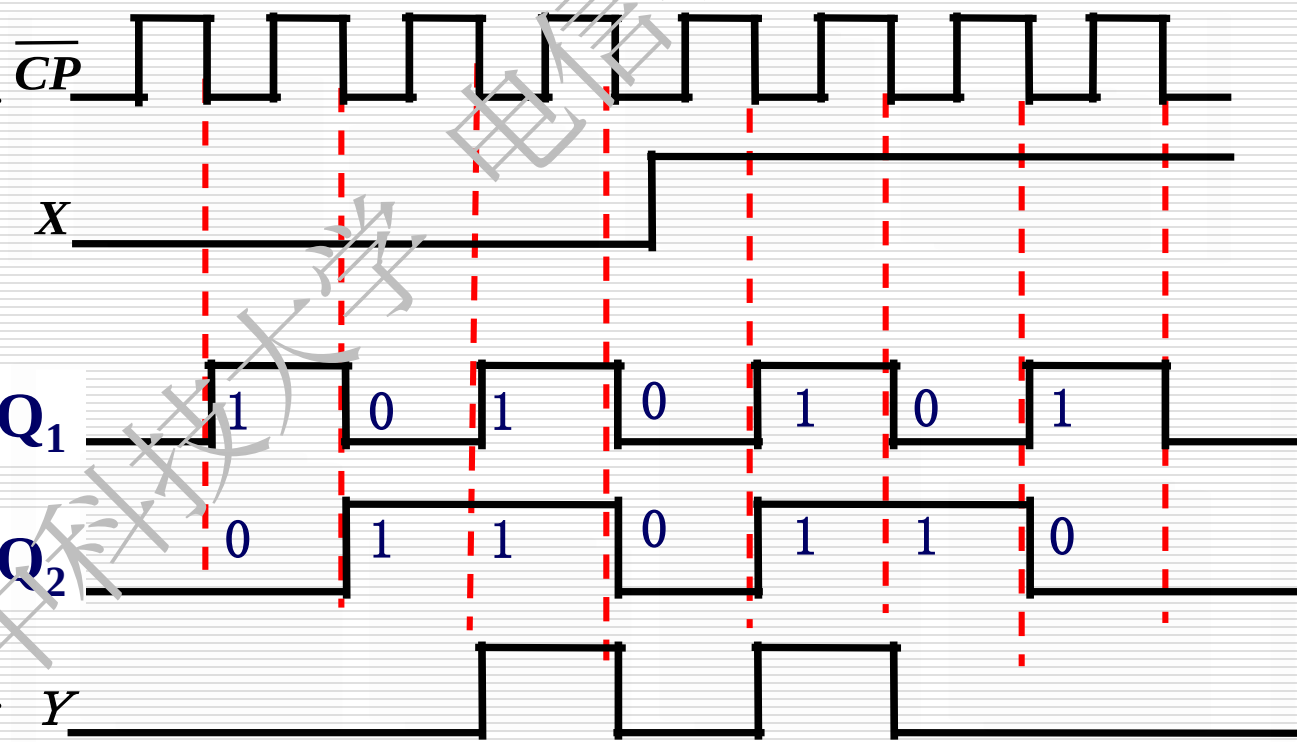
$Q_1^n Q_0^n$	$Q_1^{n+1} Q_0^{n+1}$		Y
	$X=0$	$X=1$	
0 0	0 1	1 1	0
0 1	1 0	0 0	0
1 0	1 1	0 1	0
1 1	0 0	1 0	1

状态图



根据状态转换表，画出波形图。

$Q_1^n Q_0^n$	$Q_1^{n+1} Q_0^{n+1}$		Y
	$X=0$	$X=1$	
00	01	11	0
01	10	00	0
10	11	01	0
11	00	10	1



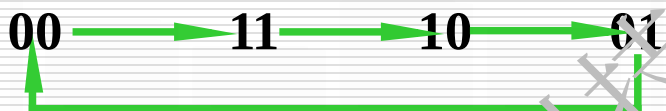
4. 确定电路的逻辑功能.

•X=0时



电路进行加1计数

•X=1时

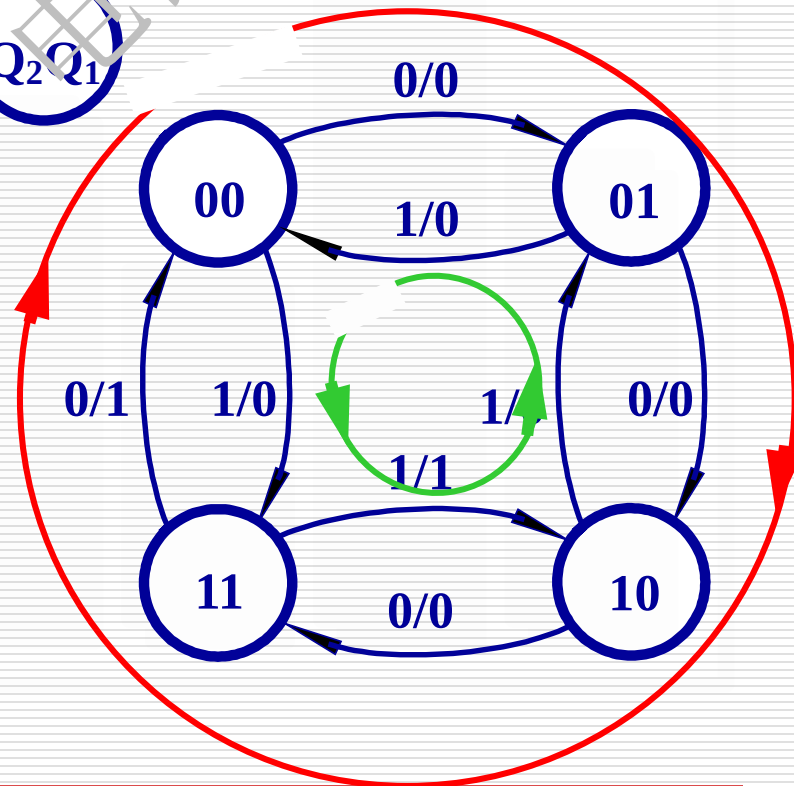


电路进行减1计数。

电路功能：可逆计数器

Y可理解为进位或借位端。

X/Y



© 2006 The Authors
Journal compilation © 2006 Blackwell Publishing Ltd



$$0 \leq \epsilon_0 < \epsilon_1 < \epsilon_2 < \dots < \epsilon_{n-1} < \epsilon_n = 1$$

Figure 1. The effect of the number of trials on the number of correct responses.

$$D_0 = Q_1 Q_0$$

100

将激励方程代入D 触发器的特性方程得状态方程

$$Q^{n+1} = D$$

状态表

$Q_2^n Q_1^n Q_0^n$	$Q_2^{n+1} Q_1^{n+1} Q_0^{n+1}$
0 0 0	0 0 1
0 0 1	0 1 0
0 1 0	1 0 0
0 1 1	1 1 0
1 0 0	0 0 1
1 0 1	0 1 0
1 1 0	1 0 0
1 1 1	1 1 0

得状态方程

$$Q_0^{n+1} = D_0 = \overline{Q_1^n} \overline{Q_0^n}$$

$$Q_1^{n+1} = D_1 = Q_0^n$$

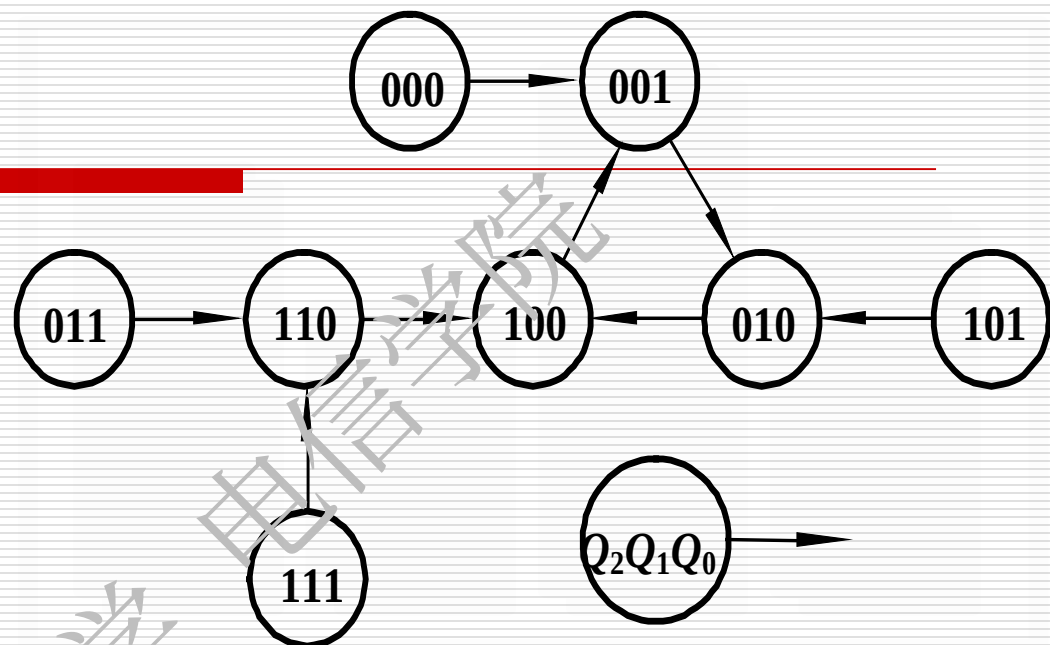
$$Q_2^{n+1} = D_2 = Q_1^n$$

2.列出其状态表

3. 画出状态图

状态表

$Q_2^n Q_1^{n+1} Q_0^n$	$Q_2^{n+1} Q_1^{n+1} Q_0^{n+1}$
0 0 0	0 0 1
0 0 1	0 1 0
0 1 0	1 0 0
0 1 1	1 1 0
1 0 0	0 0 1
1 0 1	0 1 0
1 1 0	1 0 0
1 1 1	1 1 0



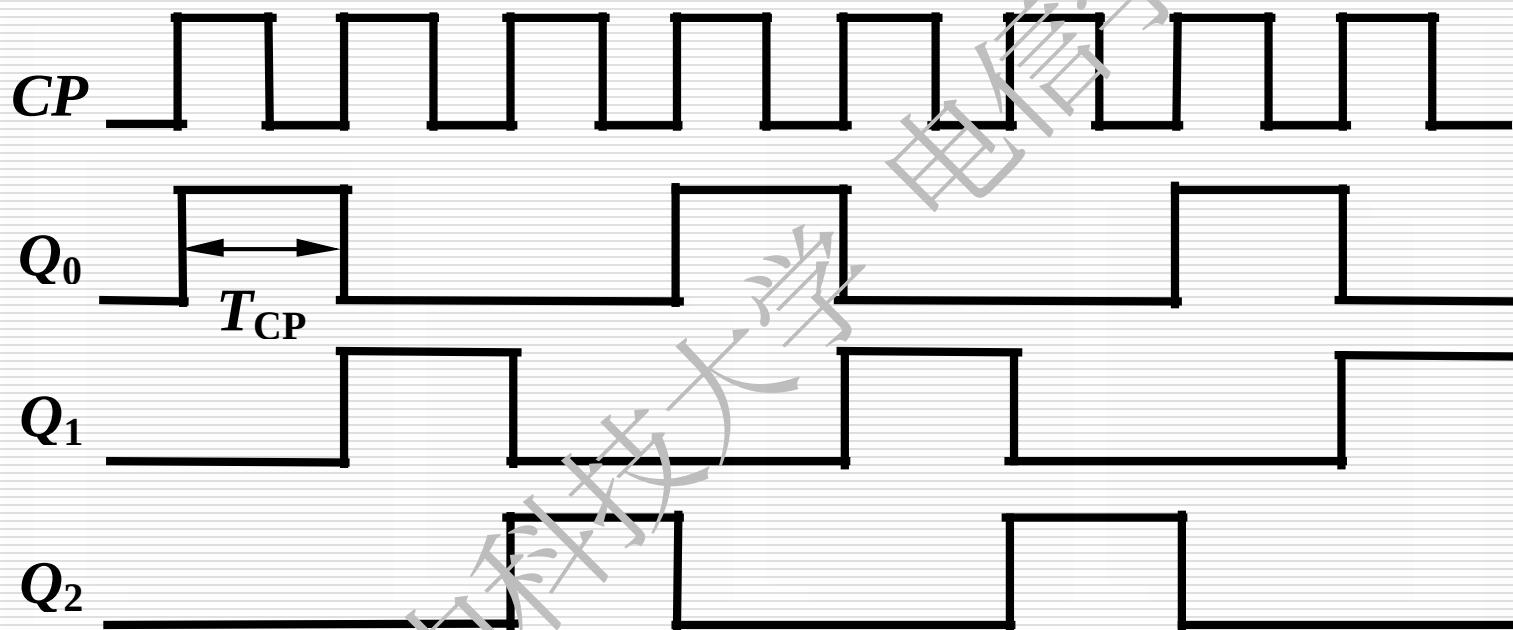
电路具有自启动能力

如何判断电路是否具有自启动能力？

画出完整的状态图（包括所有状态）

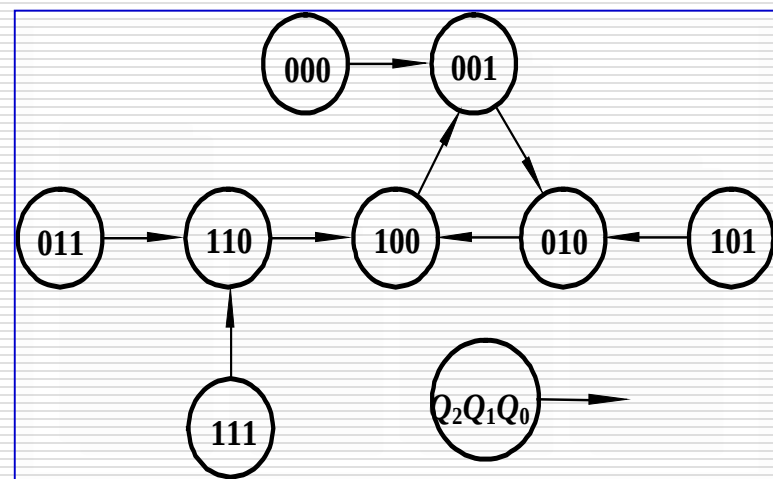
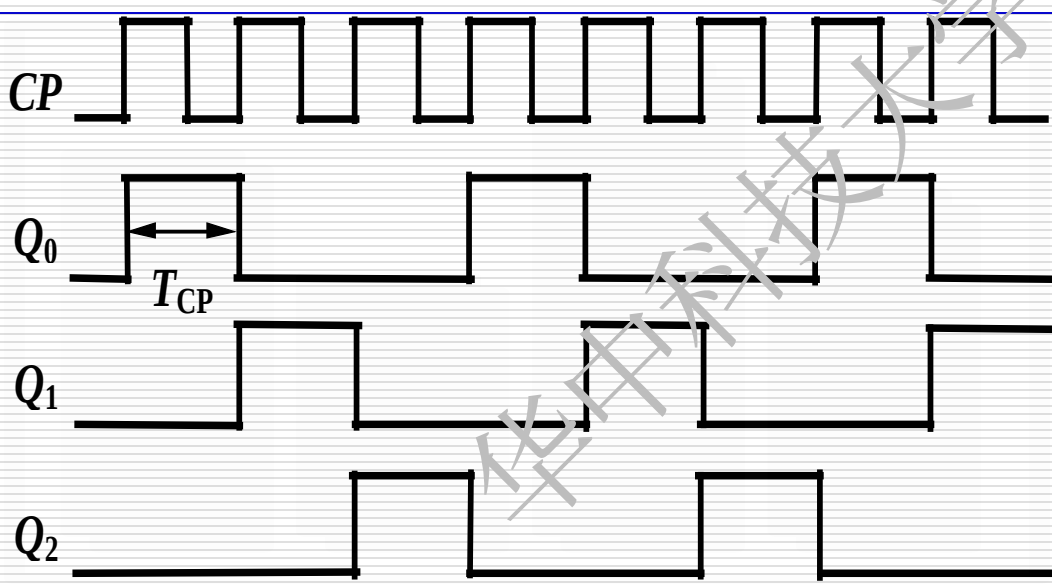
在任何无效状态下都能进入有效循环

3. 画出时序图



4、逻辑功能分析

由状态图可见，电路的有效状态是三位循环码。
从时序图可看出，电路正常工作时，各触发器的Q端轮流出现一个宽度为一个 CP 周期脉冲信号，循环周期为 $3T_{CP}$ 。电路的功能为脉冲分配器或节拍脉冲产生器。



6.3 同步时序逻辑电路的设计

6.3.1 设计同步时序逻辑电路的一般步骤

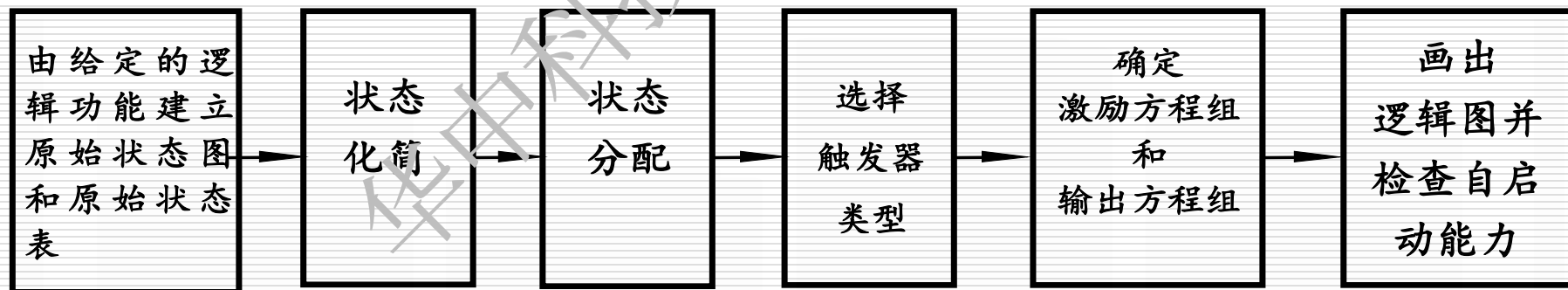
6.3.2 同步时序逻辑电路设计举例

6.3 同步时序逻辑电路的设计

同步时序逻辑电路的设计是分析的逆过程,其任务是根据实际逻辑问题的要求,设计出能实现给定逻辑功能的电路。

6.3.1 设计同步时序逻辑电路的一般步骤

同步时序电路的设计过程



(1) 根据给定的逻辑功能建立原始状态图和原始状态表

- ①明确电路的输入条件和相应的输出要求，分别确定输入变量和输出变量的数目和符号。
- ②找出所有可能的状态和状态转换之间的关系。
- ③根据原始状态图建立原始状态表。

(2) 状态化简-----求出最简状态图；

合并等价状态，消去多余状态的过程称为状态化简

等价状态：在相同的输入下有相同的输出，并转换到同一个次态去的两个状态称为等价状态。

(3)状态编码（状态分配）；

给每个状态赋以二进制代码的过程。

根据状态数确定触发器的个数，

$$2^{n-1} < M \leq 2^n \quad (M: \text{状态数}; n: \text{触发器的个数})$$

(4)选择触发器的类型

(5)求出电路的激励方程和输出方程；

(6)画出逻辑图并检查自启动能力。

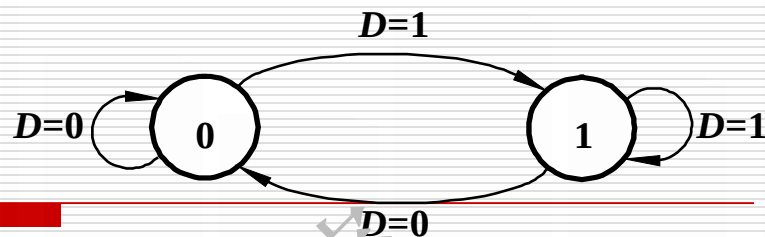
6.3.2 同步时序逻辑电路设计举例

例1 用D触发器设计一个8421 BCD码同步十进制加计数器。

8421码同步十进制加计数器的状态表

计数脉冲CP的顺序	现 态				次 态			
	Q_3^n	Q_2^n	Q_1^n	Q_0^n	Q_3^{n+1}	Q_2^{n+1}	Q_1^{n+1}	Q_0^{n+1}
0	0	0	0	0	0	0	0	1
1	0	0	0	1	0	0	1	0
2	0	0	1	0	0	0	1	1
3	0	0	1	1	0	1	0	0
4	0	1	0	0	0	1	0	1
5	0	1	0	1	0	1	1	0
6	0	1	1	0	0	1	1	1
7	0	1	1	1	1	0	0	0
8	1	0	0	0	1	0	0	1
9	1	0	0	1	0	0	0	0

(2) 确定激励方程组

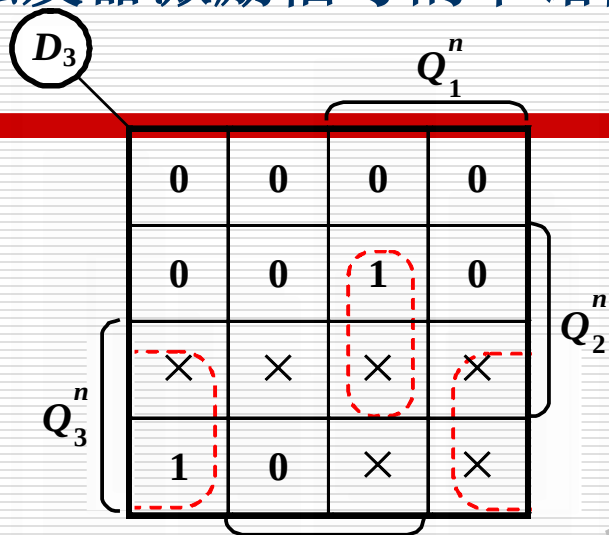


计数脉冲CP的顺序	现 态				次 态				激励信号			
	Q_3^n	Q_2^n	Q_1^n	Q_0^n	Q_3^{n+1}	Q_2^{n+1}	Q_1^{n+1}	Q_0^{n+1}	D_3	D_2	D_1	D_0
0	0	0	0	0	0	0	0	1	0	0	0	1
1	0	0	0	1	0	0	1	0	0	0	1	0
2	0	0	1	0	0	0	1	1	0	0	1	1
3	0	0	1	1	0	1	0	0	0	1	0	0
4	0	1	0	0	0	1	0	1	0	1	0	1
5	0	1	0	1	0	1	1	0	0	1	1	0
6	0	1	1	0	0	1	1	1	0	1	1	1
7	0	1	1	1	1	0	0	0	1	0	0	0
8	1	0	0	0	1	0	0	1	1	0	0	1
9	1	0	0	1	0	0	0	0	0	0	0	0

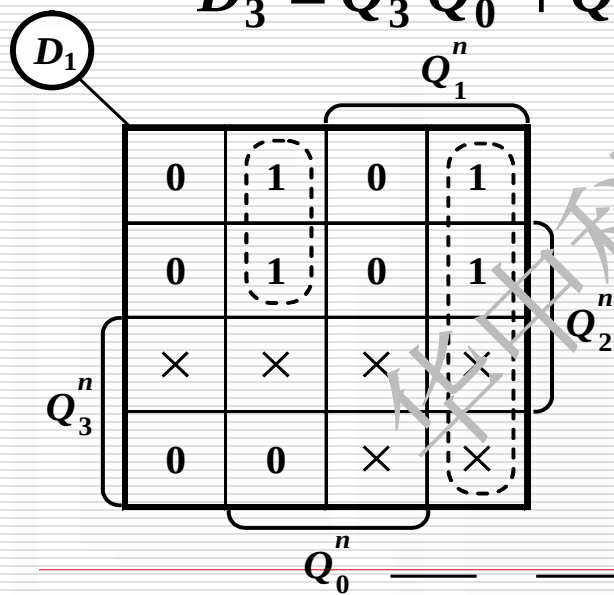
D_3 、 D_2 、 D_1 、 D_0 、是触发器现态还是次态的函数？

D_3 、 D_2 、 D_1 、 D_0 是触发器现态的函数

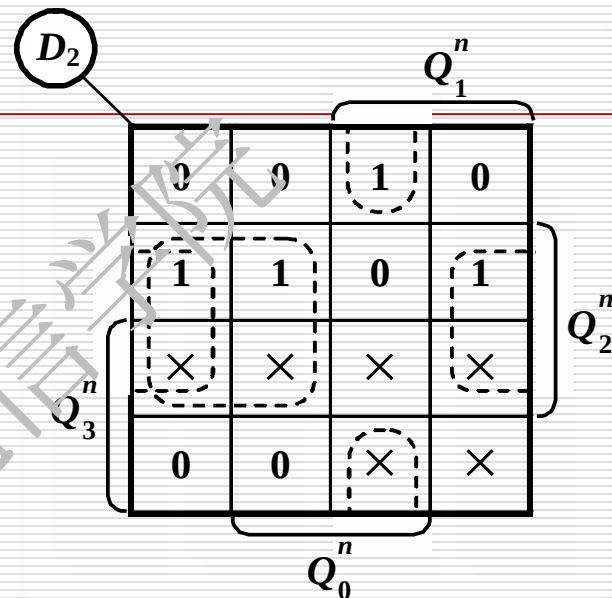
画出各触发器激励信号的卡诺图



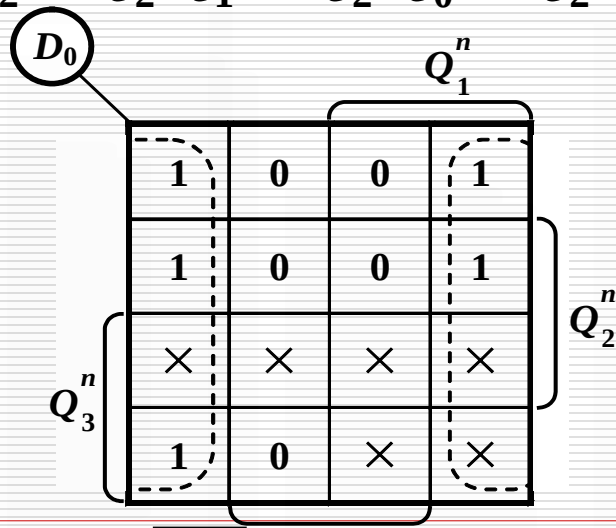
$$D_3 = Q_3^n \overline{Q_0^n} + Q_2^n Q_1^n \overline{Q_0^n}$$



$$D_1 = Q_1^n \overline{Q_0^n} + Q_3^n Q_1^n \overline{Q_0^n}$$



$$D_2 = Q_2^n \overline{Q_1^n} + Q_2^n \overline{Q_0^n} + Q_2^n Q_1^n \overline{Q_0^n}$$



$$D_0 = Q_0^n$$

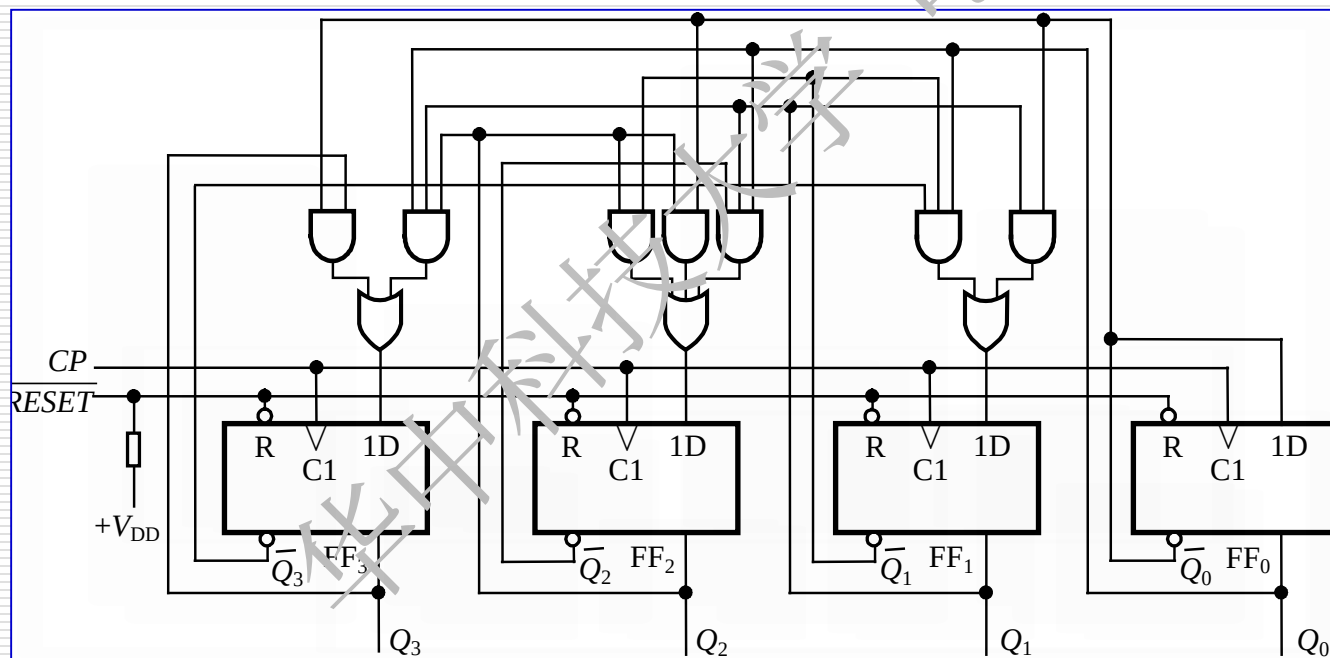
画出逻辑图

$$D_3 = Q_3^n \overline{Q_0^n} + \overline{Q_2^n} Q_1^n Q_0^n$$

$$D_2 = Q_2^n \overline{Q_1^n} + \overline{Q_2^n} \overline{Q_0^n} + \overline{Q_2^n} Q_1^n Q_0^n$$

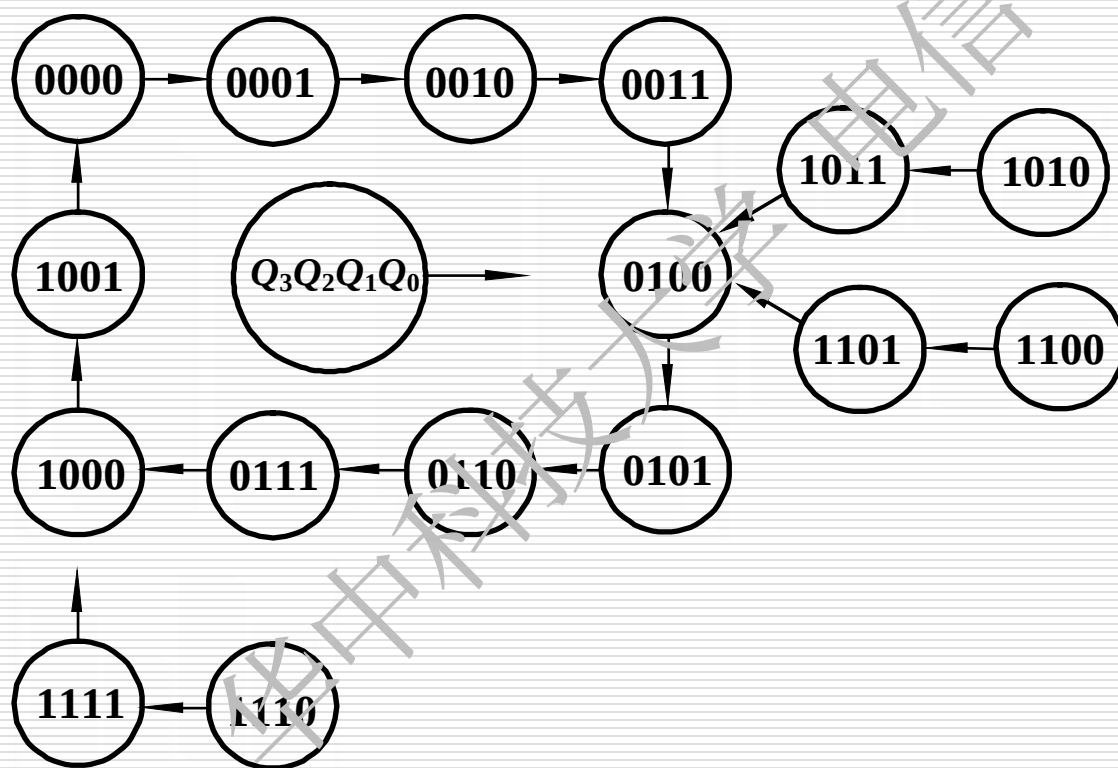
$$D_1 = Q_1^n \overline{Q_0^n} + \overline{Q_3^n} Q_1^n Q_0^n$$

$$D_0 = Q_0^n$$



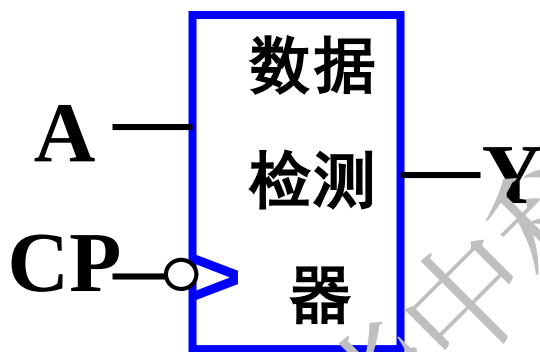
(3) 画出状态图，并检查自启动能力

画出完全状态图

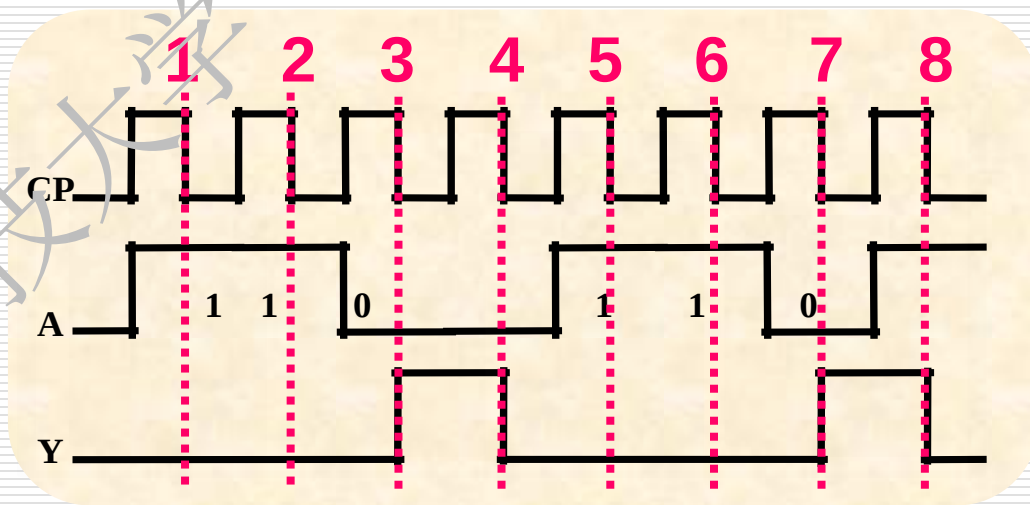


电路具有自启动能力

例2: 设计一个串行数据检测器。电路的输入信号A是与时钟脉冲同步的串行数据，其时序关系如下图所示。输出信号为Y；要求电路在A信号输入出现110序列时，输出信号Y为1，否则为0。



电路框图



例2: 设计一个串行数据检测器。电路的输入信号A是与时钟脉冲同步的串行数据，其时序关系如下图所示。输出信号为Y；要求电路在A信号输入出现110序列时，输出信号Y为1，否则为0。

解: (1) 根据给定的逻辑功能建立原始状态图和原始状态表

1.) 确定输入、输出变量及电路的状态数:

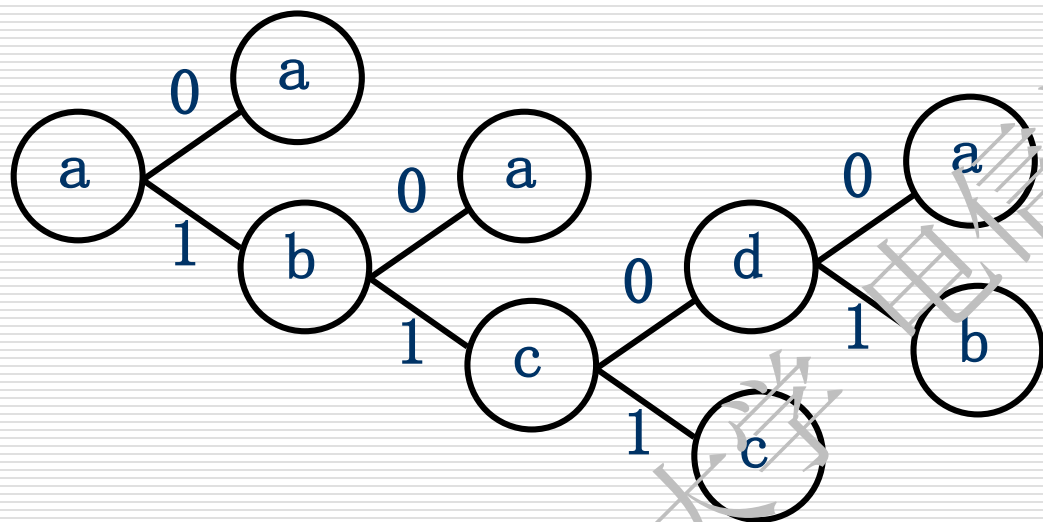
输入变量: A 输出变量: Y 状态数: 4个

2.) 定义输入 输出逻辑状态和每个电路状态的含义;

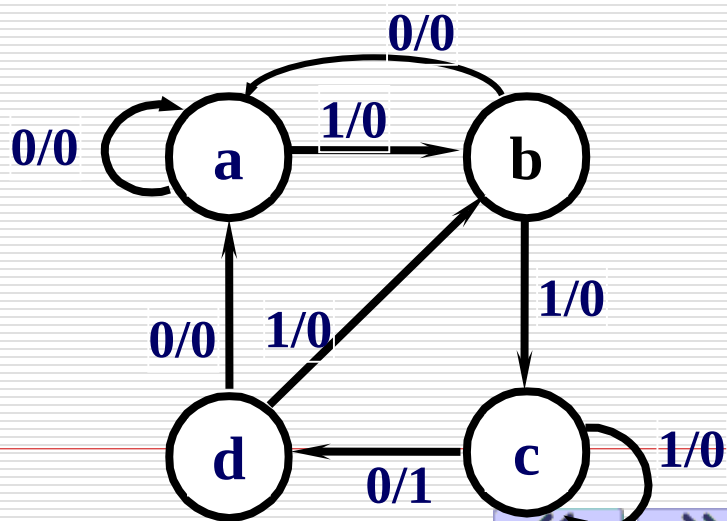
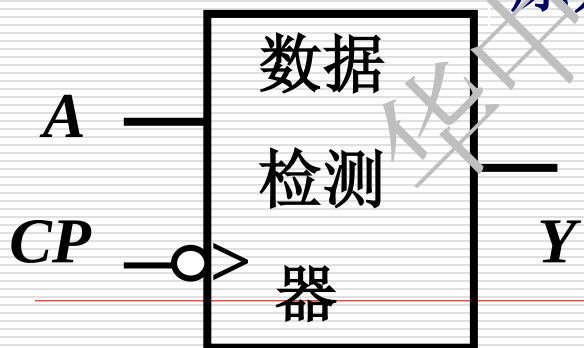
a —— 初始状态; b —— A输入1后;

c —— A输入11后; d —— A输入110后。

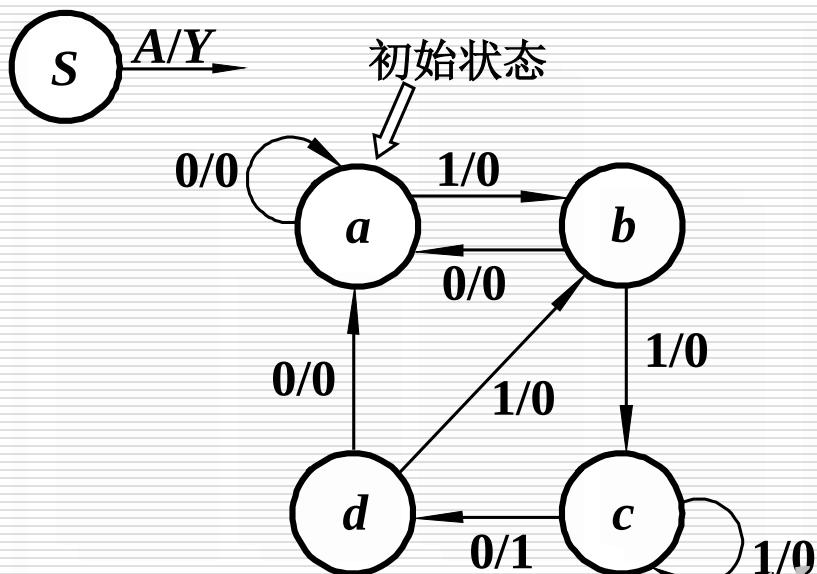
1、逻辑抽象建立原始状态图或状态表。



原始状态图

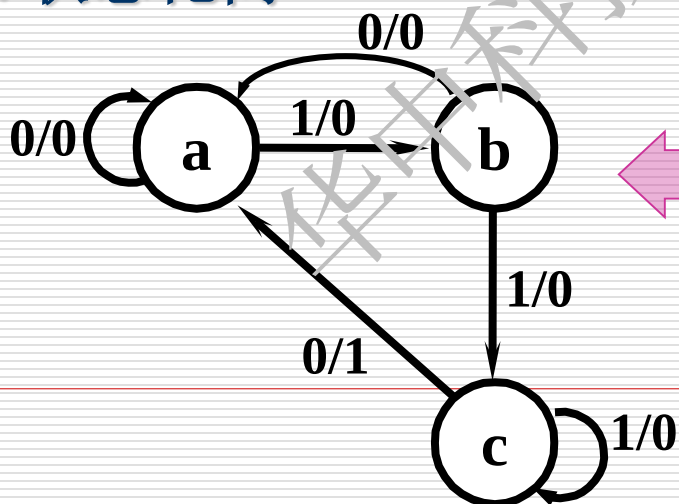


列出原始状态转换表



现态	次态/输出	
	A=0	A=1
a	a / 0	b / 0
b	a / 0	c / 0
c	d / 1	c / 0
d	a / 0	b / 0

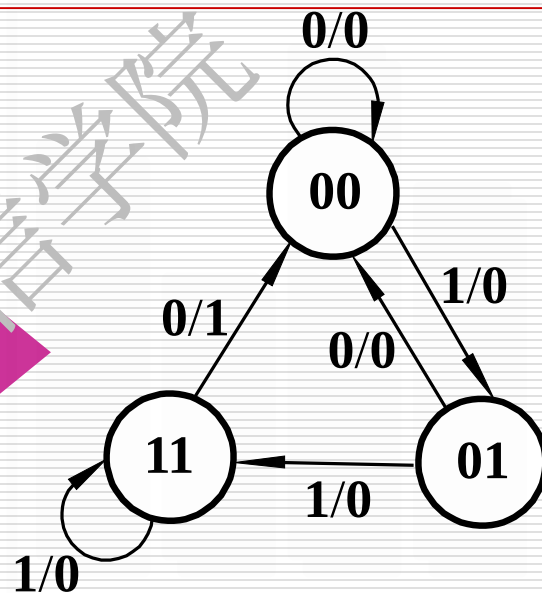
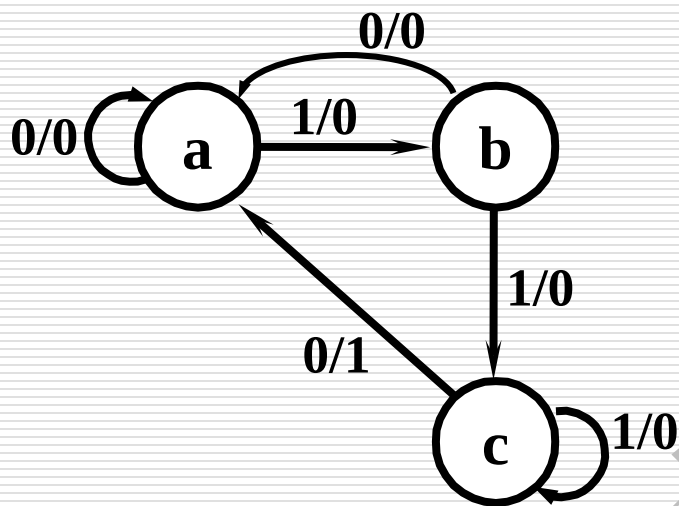
2. 状态化简



现态	次态 / 输出	
	A=0	A=1
a	a / 0	b / 0
b	a / 0	c / 0
c	a / 1	c / 0

3、状态分配

令 $a = 00$, $b = 01$, $c = 11$,



4、选择触发器的类型

触发器个数：两个。

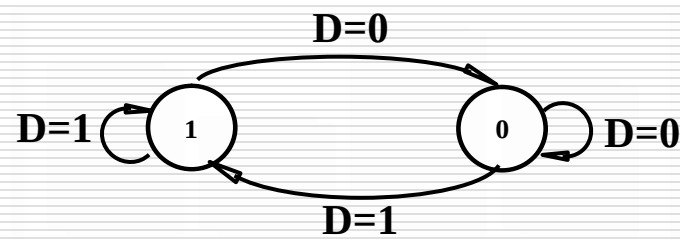
类型：采用对 CP 下降沿敏感的
 D 触发器。

现态 $Q_1 Q_0$	$Q_1^{n+1} Q_0^{n+1} / Y$	
	$A=0$	$A=1$
00	00 / 0	01 / 0
01	00 / 0	11 / 0
11	00 / 1	11 / 0

方法1：采用D 触发器。

5. 求激励方程和输出方程

现态 Q_1Q_0	$Q_1^{n+1}Q_0^{n+1} / Y$	
	A=0	A=1
00	00 / 0	01 / 0
01	00 / 0	11 / 0
11	00 / 1	11 / 0



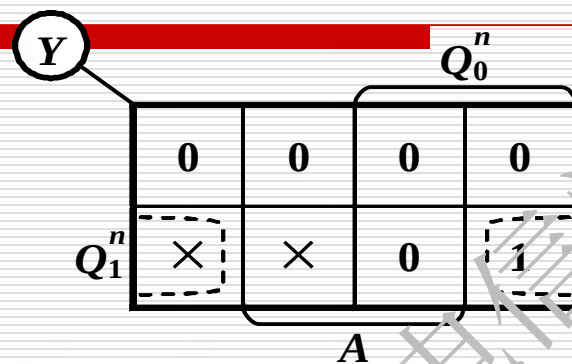
状态转换真值表及激励信号

Q_1^n	Q_0^n	A	Q_1^{n+1}	Q_0^{n+1}	Y	激励信号	
						D_1	D_0
0	0	0	0	0	0	0	0
0	0	1	0	1	0	0	1
0	1	0	0	0	0	0	0
0	1	1	1	1	0	1	1
1	1	0	0	0	1	0	0
1	1	1	1	1	0	1	1

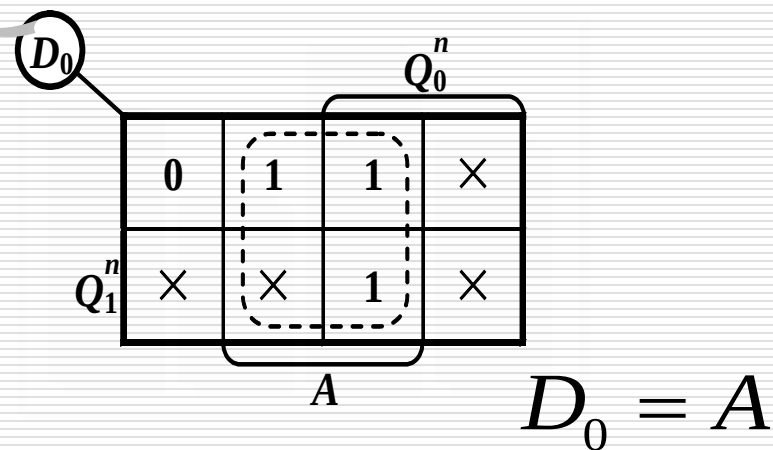
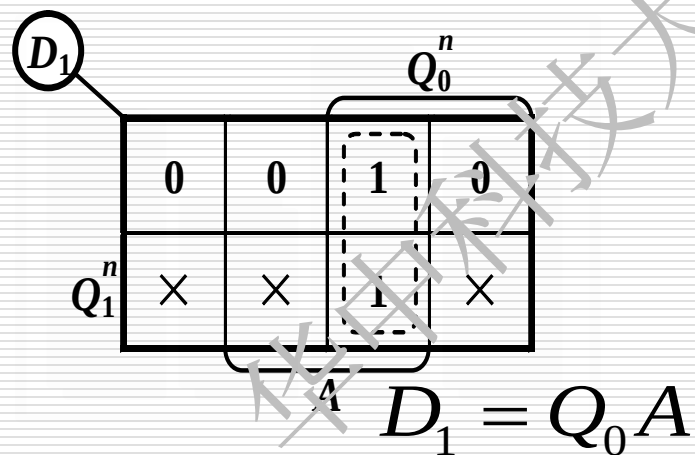
卡诺图化简得

输出方程

$$Y = Q_1 \overline{A}$$



激励方程



6. 根据激励方程和输出方程画出逻辑图,并检查自启动能力

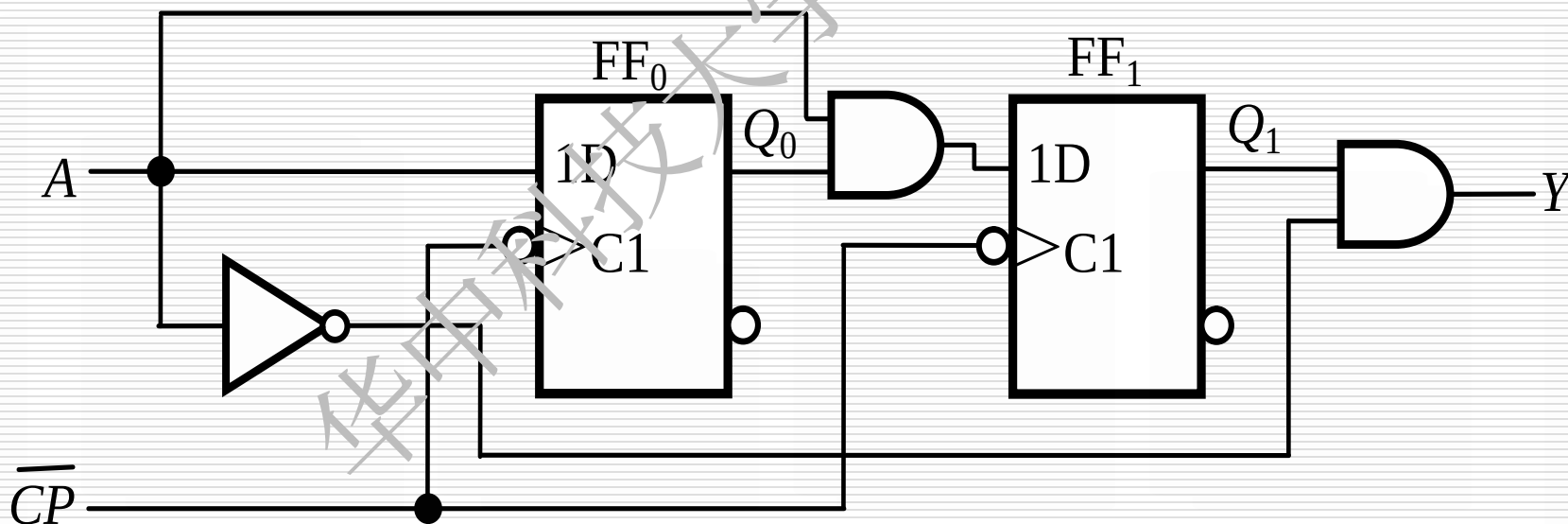
激励方程

$$D_1 = Q_0 A$$

$$D_0 = A$$

输出方程

$$Y = Q_1 \overline{A}$$

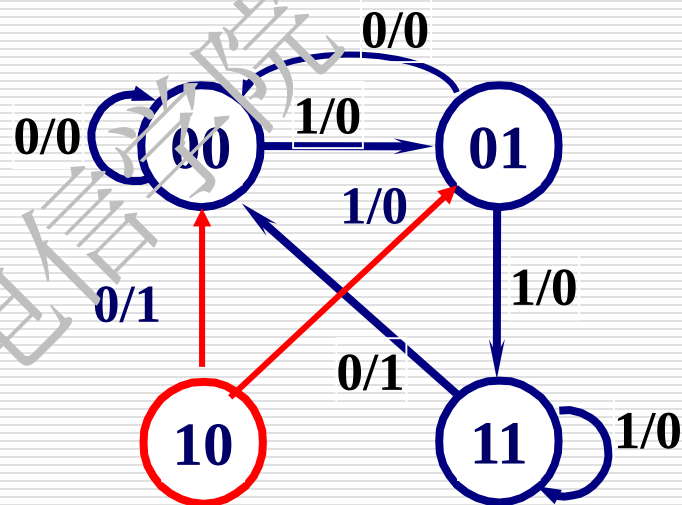


检查自启动能力和输出

当 $Q_1^n Q_0^n = 10$ 时

$$A=0 \quad Q_1^{n+1} Q_0^{n+1} = 00 \quad Y = 1$$

$$A=1 \quad Q_1^{n+1} Q_0^{n+1} = 01 \quad Y = 0$$



能自启动

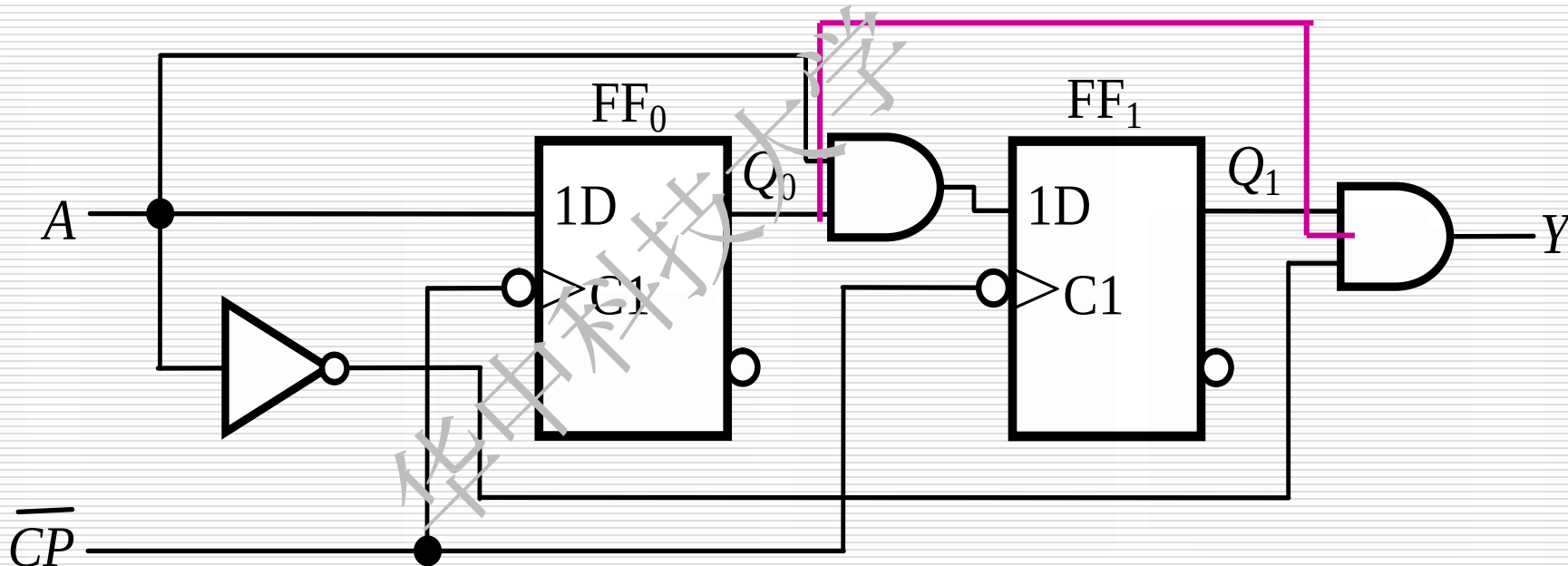
输出方程 $Y = Q_1 \bar{A}$ $\Rightarrow$ $Y = Q_1 Q_0 \bar{A}$

修改电路

输出方程 $Y = Q_1 \bar{A}$ 

卡诺图化简去掉无关项

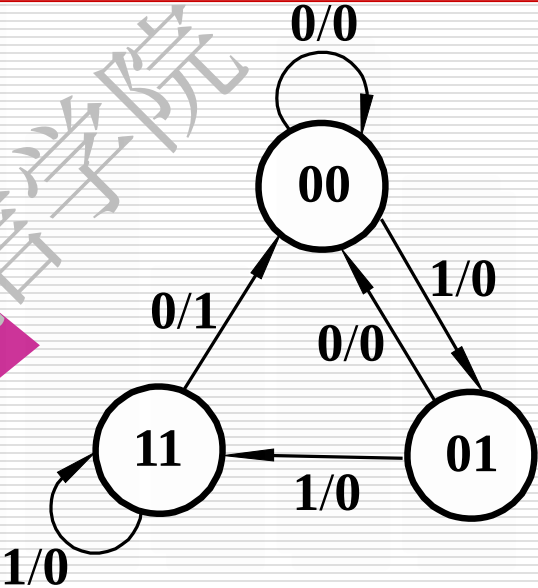
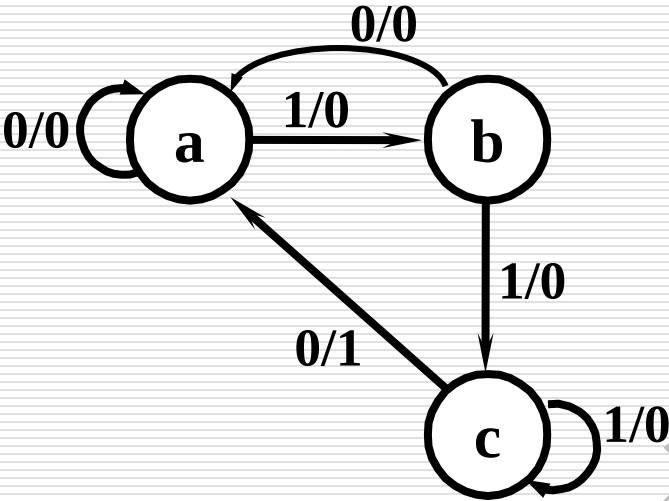
$$Y = Q_1 Q_0 \bar{A}$$



方法2：采用对 CP 下降沿敏感的JK 触发器。

3、状态分配

令 $a = 00$, $b = 01$, $c = 11$,



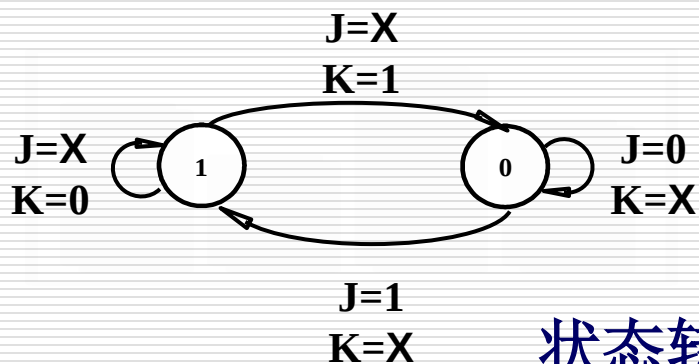
4、选择触发器的类型

触发器个数：两个。

类型：采用对 CP 下降沿敏感的 JK 触发器。

现态 $Q_1 Q_0$	$Q_1^{n+1} Q_0^{n+1} / Y$	
	A=0	A=1
00	00 / 0	01 / 0
01	00 / 0	11 / 0
11	00 / 1	11 / 0

5. 求激励方程和输出方程



现态 Q_1Q_0	$Q_1^{n+1}Q_0^{n+1} / Y$	
	A=0	A=1
00	00 / 0	01 / 0
01	00 / 0	11 / 0
11	00 / 1	11 / 0

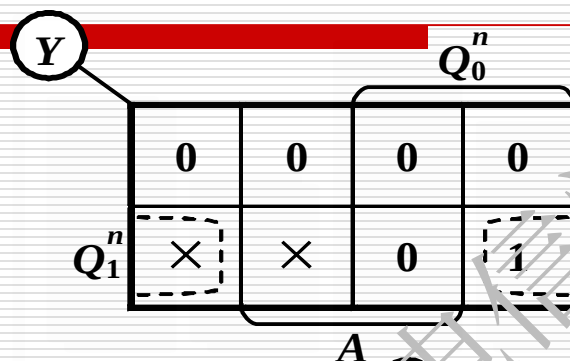
状态转换真值表及激励信号

Q_1^n	Q_0^n	A	Q_1^{n+1}	Q_0^{n+1}	Y	激励信号			
						J_1	K_1	J_0	K_0
0	0	0	0	0	0	0	×	0	×
0	0	1	0	1	0	0	×	1	×
0	1	0	0	0	0	0	×	×	1
0	1	1	1	1	0	1	×	×	0
1	1	0	0	0	1	×	1	×	1
1	1	1	1	1	0	×	0	×	0

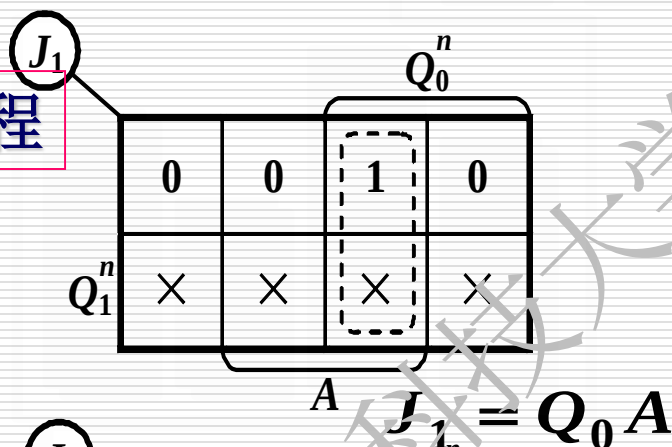
卡诺图化简得

输出方程

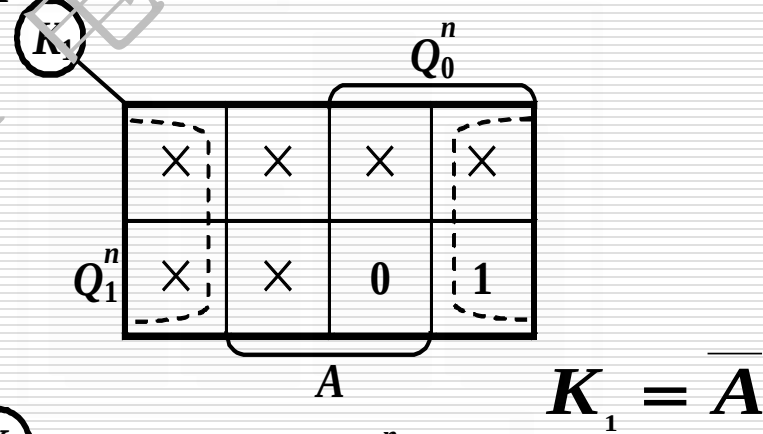
$$Y = Q_1 \overline{A}$$



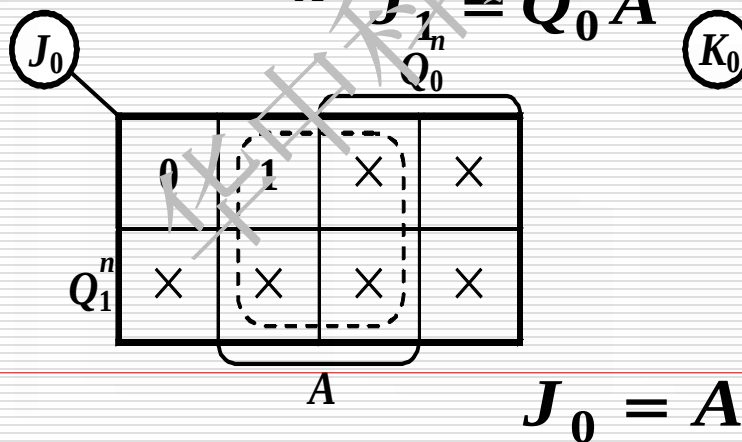
激励方程



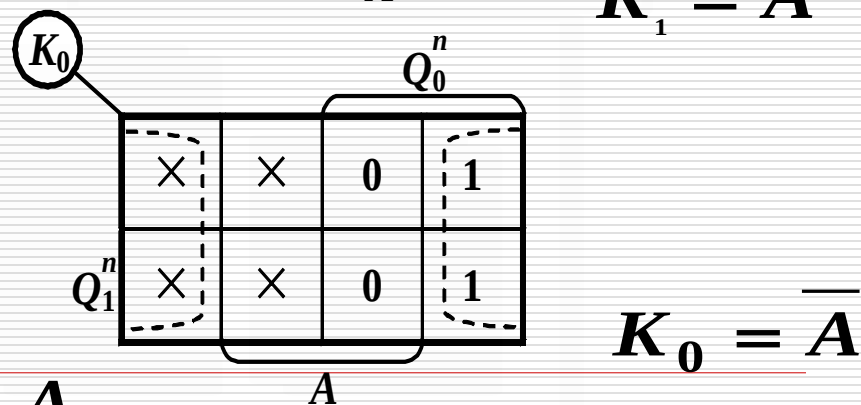
$$J_1 = Q_0 A$$



$$K_1 = \overline{A}$$



$$J_0 = A$$



$$K_0 = \overline{A}$$

6. 根据激励方程和输出方程画出逻辑图,并检查自启动能力

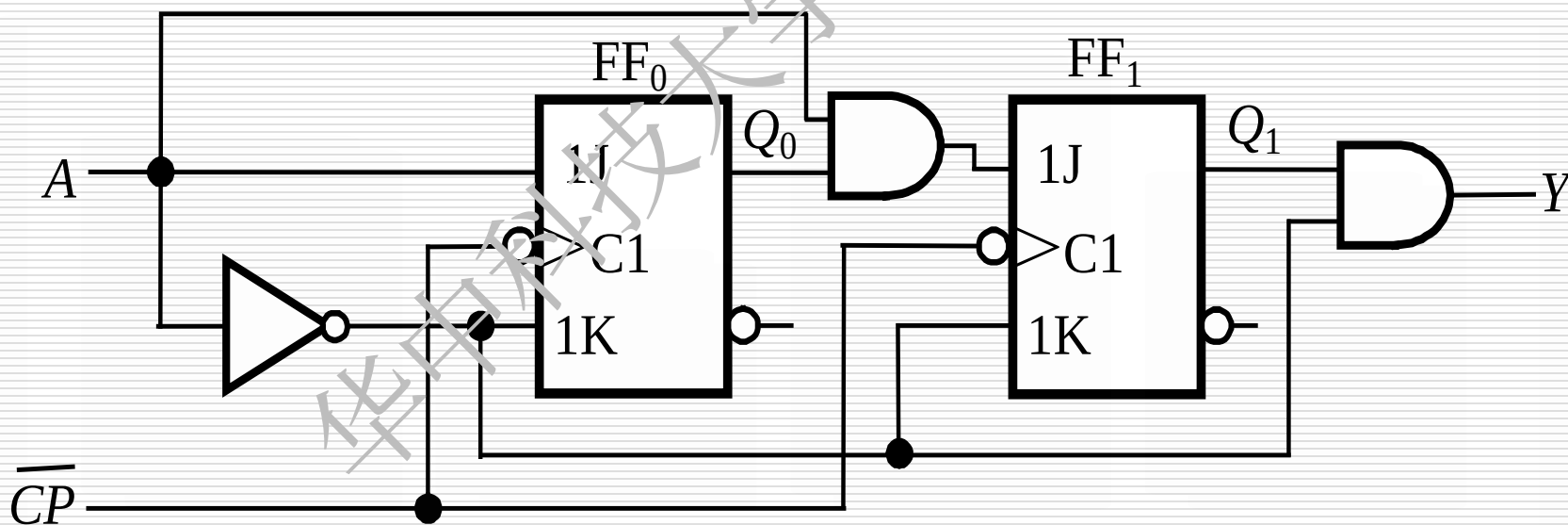
激励方程

$$J_1 = Q_0 A \quad K_1 = \overline{A}$$

$$J_0 = A \qquad K_0 = \overline{A}$$

输出方程

$$\mathbf{Y} = \mathbf{Q}_1 \overline{\mathbf{A}}$$



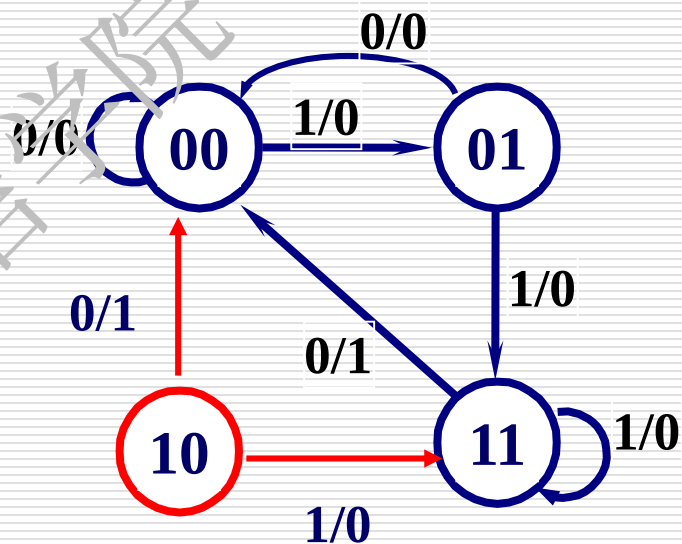
检查自启动能力和输出

当 $Q_1^n Q_0^n = 10$ 时

$$A=0 \quad Q_1^{n+1} Q_0^{n+1} = 00 \quad Y = 1$$

$$A=1 \quad Q_1^{n+1} Q_0^{n+1} = 11 \quad Y = 0$$

输出方程 $Y = Q_1 \bar{A}$ $\Rightarrow$ $Y = Q_1 Q_0 \bar{A}$



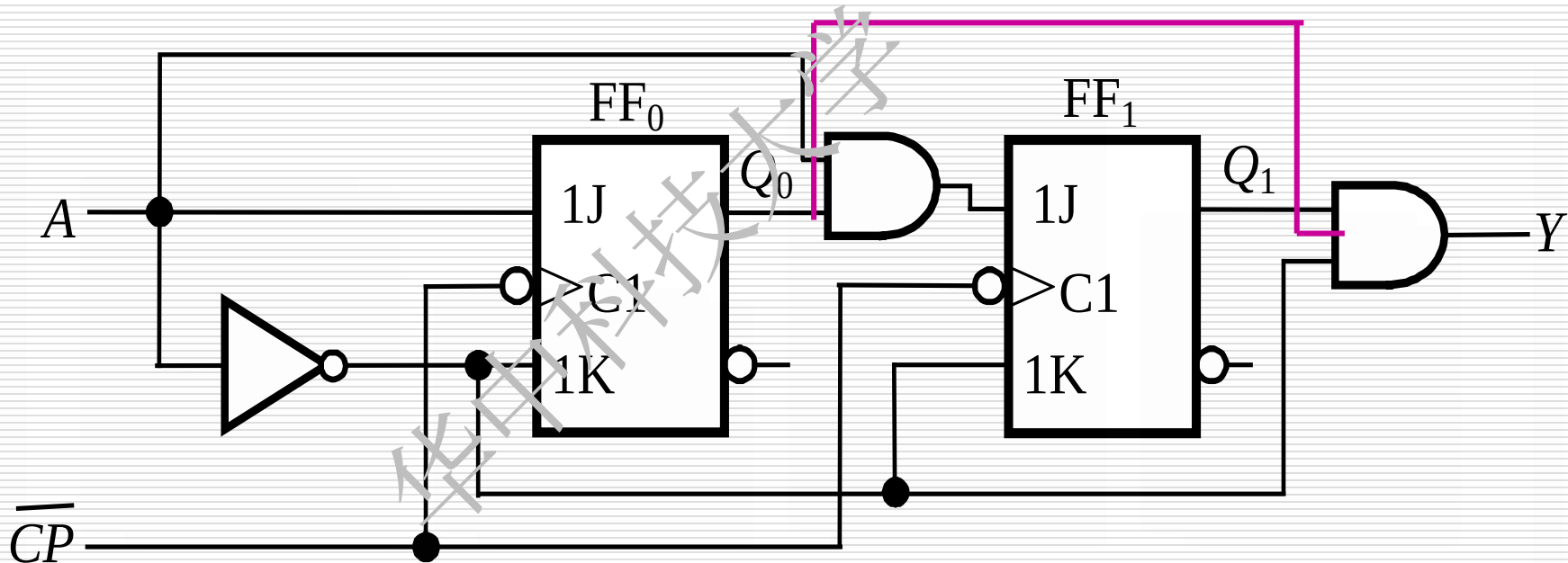
能自启动

修改电路

输出方程 $Y = Q_1 \bar{A}$ 

卡诺图化简去掉无关项

$$Y = Q_1 Q_0 \bar{A}$$

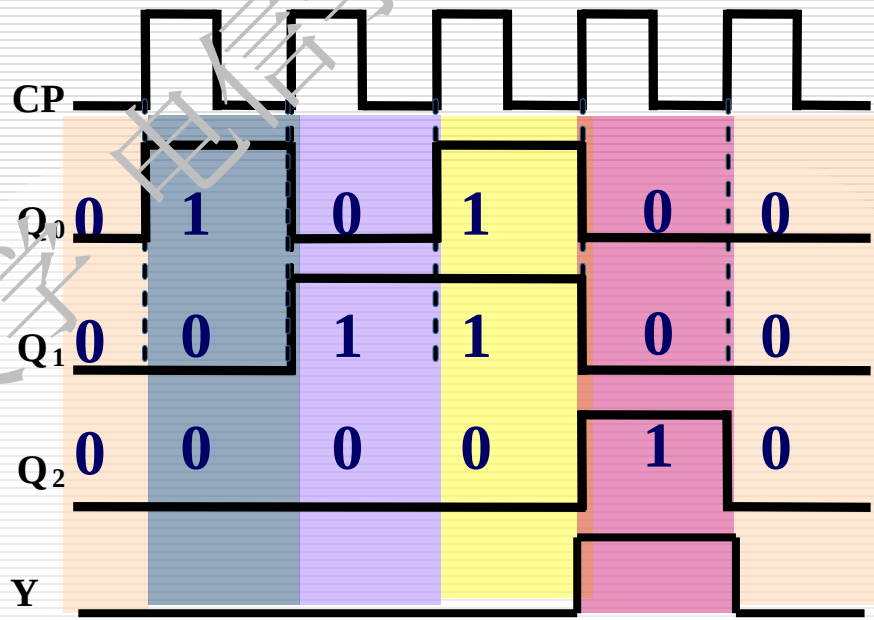
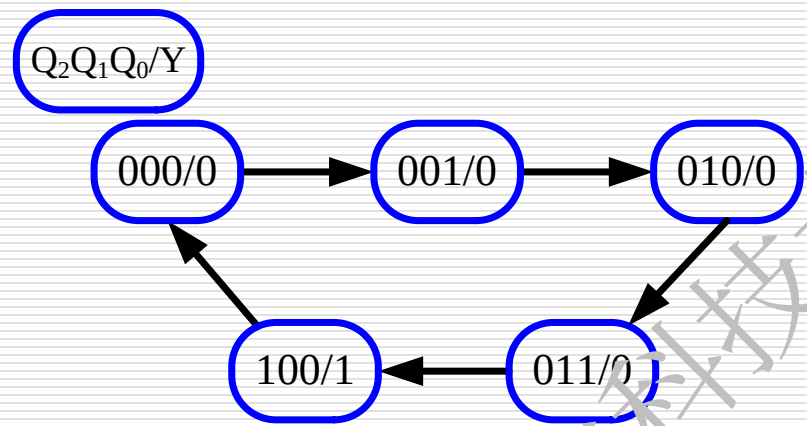


例3:试设计一个同步时序电路，要求电路中触发器 Q_0 、 Q_1 、 Q_2 及输出Y端的信号与CP时钟信号波形满足下图所示的时序关系。

解法1: T触发器

解：据题意可直接由波形图

1、画出电路状态图。



2、确定触发器的类型和个数

触发器个数：3个

触发器类型：上升沿触发的T触发器。

3、求出电路的激励方程和输出方程；

Q_2^n	Q_1^n	Q_0^n	Q_2^{n+1}	Q_1^{n+1}	Q_0^{n+1}	Y	T_2	T_1	T_0
0	0	0	0	0	1	0	0	0	1
0	0	1	0	1	0	0	0	1	1
0	1	0	0	1	1	0	0	0	1
0	1	1	1	0	0	0	1	1	1
1	0	0	0	0	0	1	1	0	0

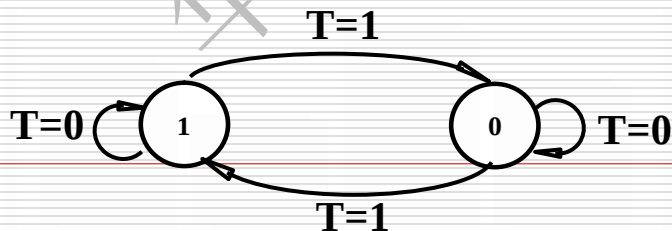
T_2		$Q_1^n Q_0^n$			
		00	01	11	10
Q_2^n	0	0	0	1	0
	1	1	X	X	X

$$T_2 = Q_0^n Q_1^n + Q_2^n$$

$$T_1 = Q_0^n$$

$$T_0 = \overline{Q_2^n}$$

$$Y = Q_2^n$$



(3) 画出逻辑图(略)



(4) 检查自启动能力

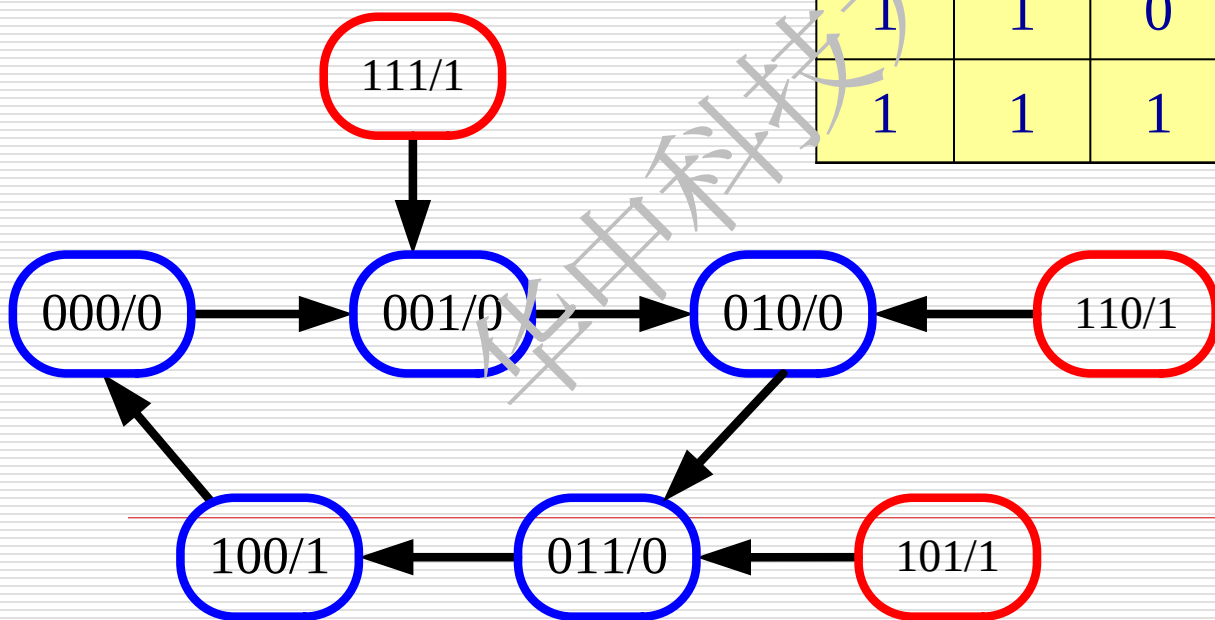
$$Q_0^{n+1} = \overline{Q_2^n} \cdot \overline{Q_0^n} + Q_2^n \cdot Q_0^n$$

$$Q_1^{n+1} = Q_0^n \overline{Q_1^n} + \overline{Q_0^n} Q_1^n$$

$$Q_2^{n+1} = Q_0^n Q_1^n \overline{Q_2^n}$$

Q_2^n	Q_1^n	Q_0^n	Q_2^{n+1}	Q_1^{n+1}	Q_0^{n+1}	Y
0	0	0	0	0	1	0
0	0	1	0	1	0	0
0	1	0	0	1	1	0
0	1	1	1	0	0	0
1	0	0	0	0	0	1

1	0	1	0	1	1	1
1	1	0	0	1	0	1
1	1	1	0	0	1	1



电路具备自启动能力

Q_2^n	Q_1^n	Q_0^n	Q_2^{n+1}	Q_1^{n+1}	Q_0^{n+1}	Y
0	0	0	0	0	1	0
0	0	1	0	1	0	0
0	1	0	0	1	1	0
0	1	1	1	0	0	0
1	0	0	0	0	0	1

Y	$Q_1^n Q_0^n$				
	Q_2^n	00	01	11	10
0	0	0	0	0	0
1	0	1	X	X	X

1	0	1	0	1	1	0
1	1	0	0	1	0	0
1	1	1	0	0	1	0

(5) 修改逻辑图(略)

修改输出方程:

$$Y = Q_2^n$$

$$Y = Q_2^n \cdot \overline{Q_1^n} \cdot \overline{Q_0^n}$$

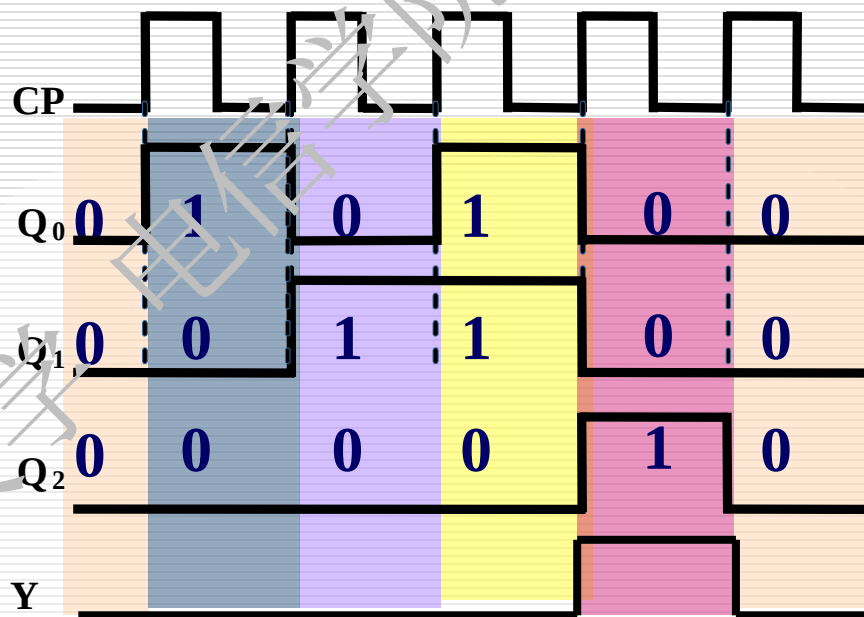
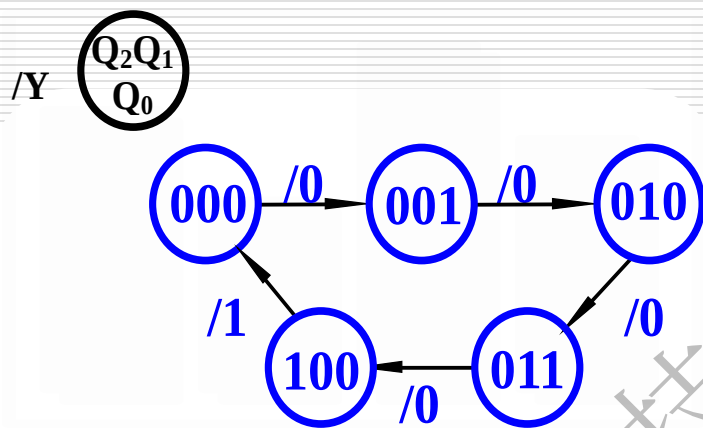
电路的输出有错!



解法2: JK触发器

解: 据题意可直接由波形图

1、画出电路状态图。



2、确定触发器的类型和个数

触发器个数: 3个

触发器类型: 上升沿触发的JK边沿触发器。

3、求出电路的激励方程和输出方程;



Q_2^n	Q_1^n	Q_0^n	Q_2^{n+1}	Q_1^{n+1}	Q_0^{n+1}	Y	J_2	K_2	J_1	K_1	J_0	K_0
0	0	0	0	0	1	0	0	X	0	X	1	X
0	0	1	0	1	0	0	0	X	1	X	X	1
0	1	0	0	1	1	0	0	X	X	0	1	X
0	1	1	1	0	0	0	1	X	X	1	X	1
1	0	0	0	0	0	1	X	1	0	X	0	X

J_2

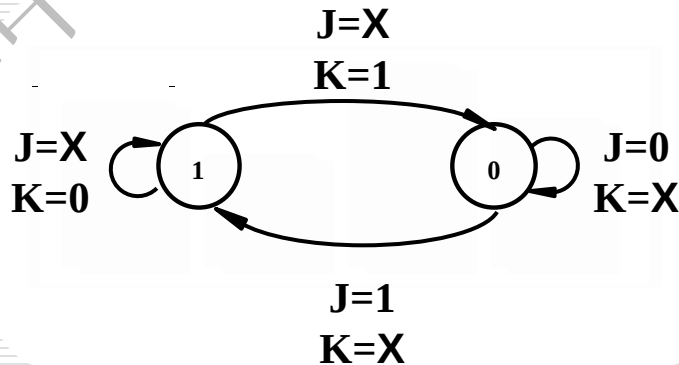
$Q_2^n \backslash Q_1^n Q_0^n$	00	01	11	10
0	0	0	1	0
1	X	X	X	X

$$J_2 = Q_0^n Q_1^n$$

K_2

$Q_2^n \backslash Q_1^n Q_0^n$	00	01	11	10
0	X	X	X	X
1	1	X	X	X

$$K_2 = 1$$



$$J_1 = Q_0^n \quad K_1 = \overline{Q_0^n}$$

$$J_0 = \overline{Q_2^n} \quad K_0 = 1$$

求激励方程的第二种方法

Y	$Q_1^n Q_0^n$			
	00	01	11	10
Q_2^n				
0	0	0	0	0
1	1	X	X	X

$$Y = Q_2^n$$

Q_2^n	Q_1^n	Q_0^n	Q_2^{n+1}	Q_1^{n+1}	Q_0^{n+1}	Y
0	0	0	0	0	1	0
0	0	1	0	1	0	0
0	1	0	0	1	1	0
0	1	1	1	0	0	0
1	0	0	0	0	0	1

Q_2^n	$Q_1^n Q_0^n$			
	00	01	11	10
0	0	0	1	0
1	0	×	×	×

$$Q_2^{n+1}$$

Q_2^n	$Q_1^n Q_0^n$			
	00	01	11	10
0	0	1	0	1
1	0	×	×	×

$$Q_1^{n+1}$$

Q_2^n	$Q_1^n Q_0^n$			
	00	01	11	10
0	1	0	0	1
1	0	×	×	×

$$Q_0^{n+1}$$

$$Q_2^{n+1} = Q_1^n Q_0^n Q_2^n$$

$$Q_1^{n+1} = \overline{Q_1^n} Q_0^n + Q_1^n \overline{Q_0^n}$$

$$Q_0^{n+1} = \overline{Q_2^n} \cdot \overline{Q_0^n}$$



$$Q_2^{n+1} = Q_0^n Q_1^n \overline{Q_2^n} + 0 \cdot Q_2^n$$

$$Q^{n+1} = J\overline{Q}^n + \overline{K}Q^n$$

$$Q_1^{n+1} = Q_0^n \overline{Q_1^n} + \overline{Q_0^n} Q_1^n$$

$$J_2 = Q_0^n Q_1^n \quad K_2 = 1$$

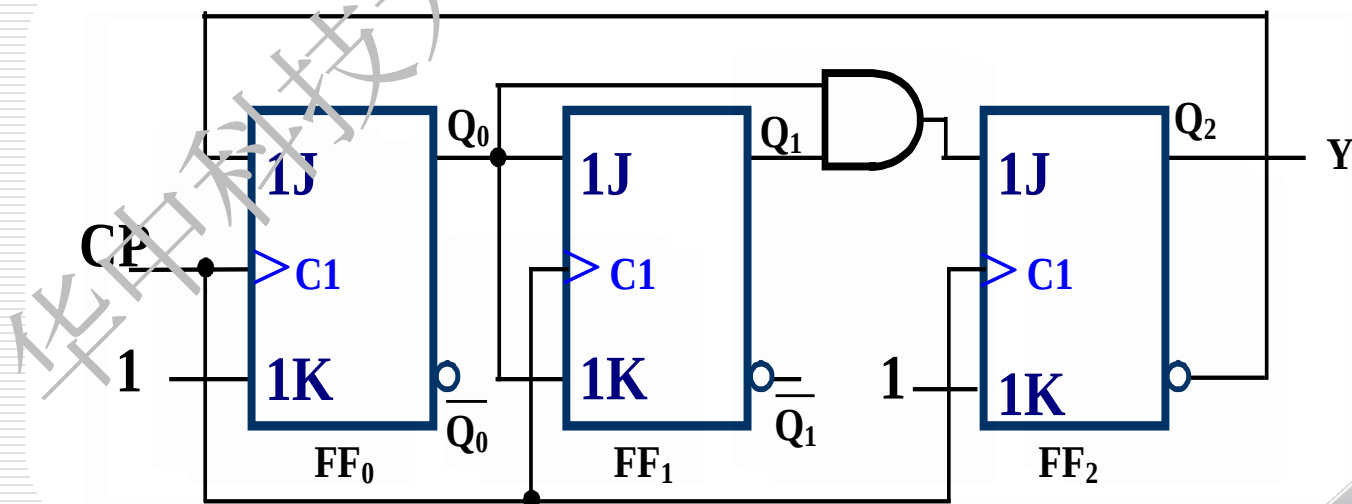
$$J_1 = Q_0^n \quad K_1 = \overline{Q_0^n}$$

$$Q_0^{n+1} = \overline{Q_2^n} \cdot \overline{Q_0^n}$$

$$J_0 = \overline{Q_2^n} \quad K_0 = 1$$

$$Y = Q_2^n$$

(3) 画出逻辑图



(4) 检查自启动能力

$$Q_0^{n+1} = \overline{Q_2^n} \cdot \overline{Q_0^n}$$

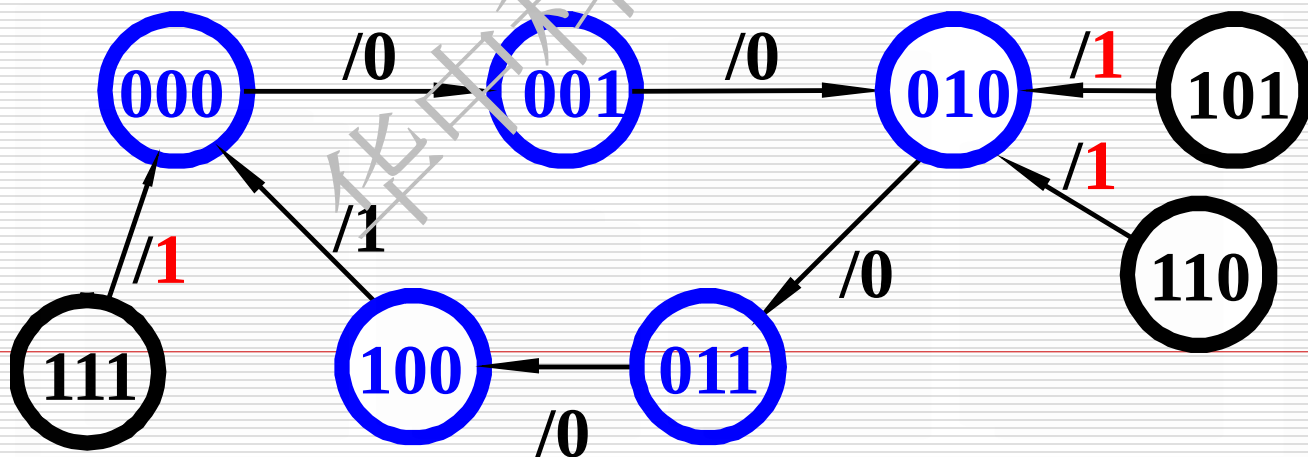
$$Q_1^{n+1} = Q_0^n \overline{Q_1^n} + \overline{Q_0^n} Q_1^n$$

$$Q_2^{n+1} = Q_0^n Q_1^n \overline{Q_2^n}$$

电路具备自启动能力

Q_2^n	Q_1^n	Q_0^n	Q_2^{n+1}	Q_1^{n+1}	Q_0^{n+1}	Y
0	0	0	0	0	1	0
0	0	1	0	1	0	0
0	1	0	0	1	1	0
0	1	1	1	0	0	0
1	0	0	0	0	0	1

1	0	1	0	1	0	1
1	1	0	0	1	0	1
1	1	1	0	0	0	1



Q_2^n	Q_1^n	Q_0^n	Q_2^{n+1}	Q_1^{n+1}	Q_0^{n+1}	Y
0	0	0	0	0	1	0
0	0	1	0	1	0	0
0	1	0	0	1	1	0
0	1	1	1	0	0	0
1	0	0	0	0	0	1

Y Q_2^n	$Q_1^n Q_0^n$			
	00	01	11	10
0	0	0	0	0
1	1	X	X	X

1	0	1	0	1	0	0
1	1	0	0	1	0	0
1	1	1	0	0	0	0

修改输出方程:

$$Y = Q_2^n$$

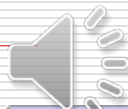
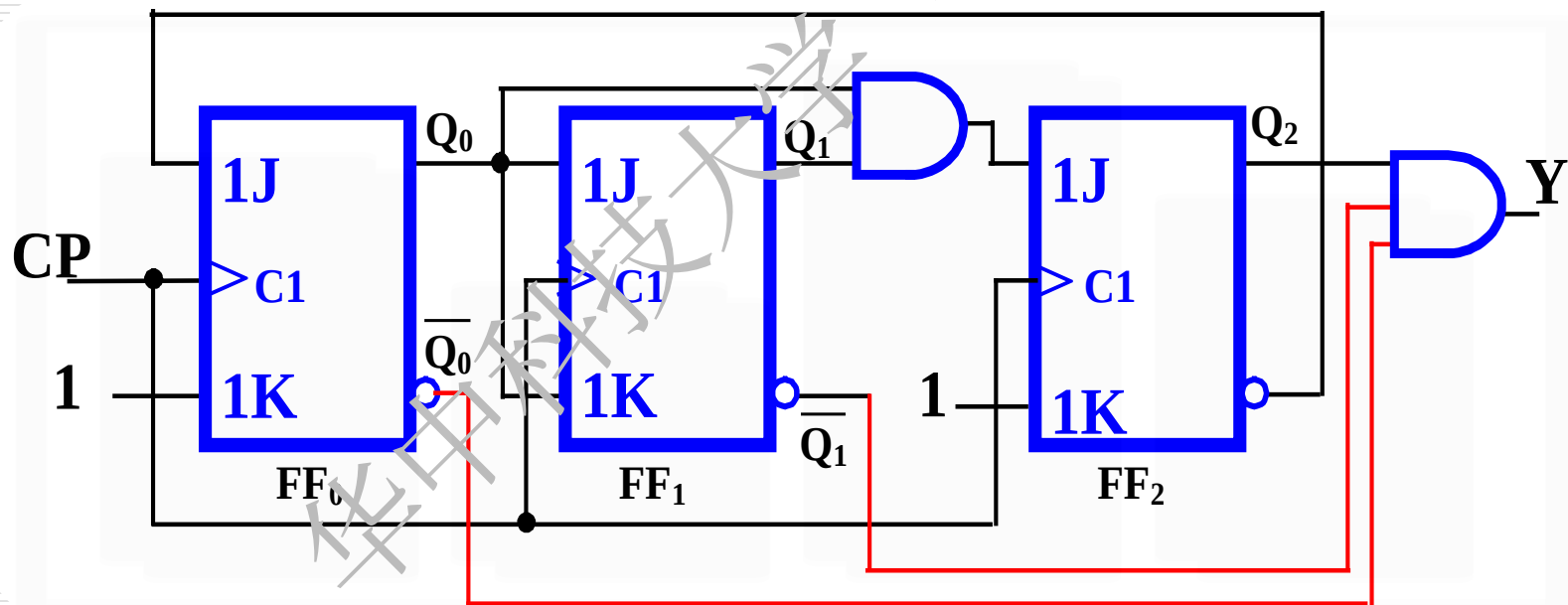
$$Y = Q_2^n \cdot \overline{Q_1^n} \cdot \overline{Q_0^n}$$

电路的输出有错!

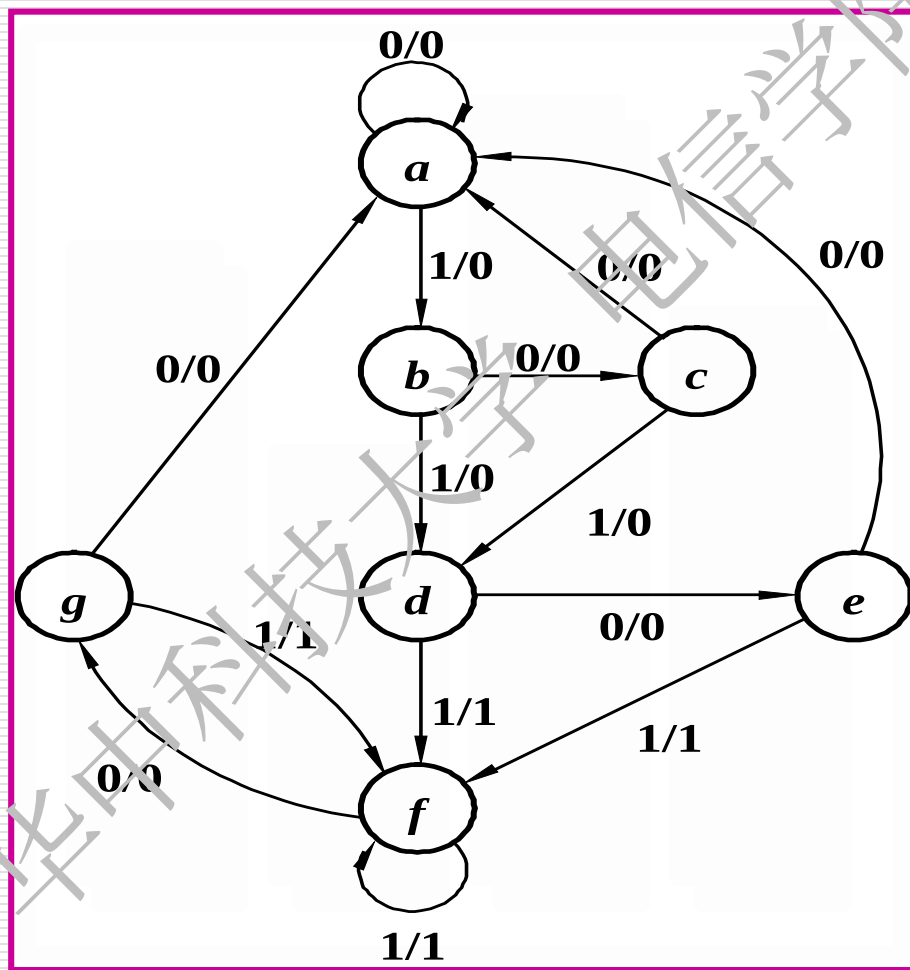


•修改后的逻辑图

$$Y = Q_2^n \quad \longrightarrow \quad Y = Q_2^n \cdot Q_1^n \cdot \overline{Q_0^n}$$

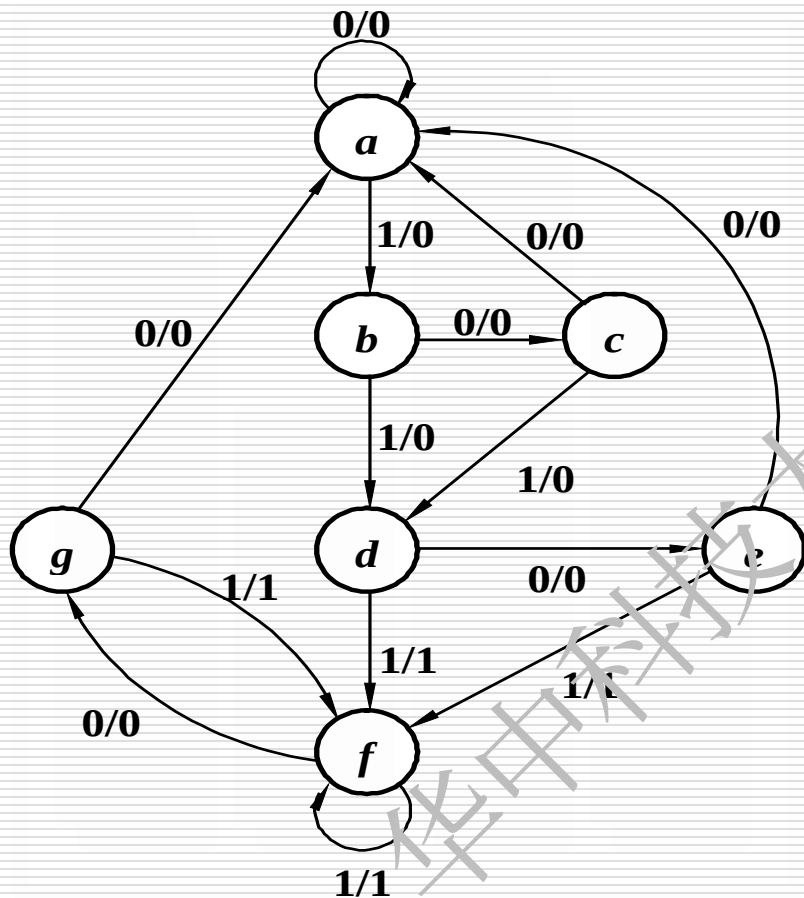


例4：用D 触发器设计状态变化满足下状态图的时序逻辑电路



1、列出原始状态表

原始状态表



现态 (S^n)	次态/输出 (S^{n+1}/Y)	
	$A=0$	$A=1$
a	$a / 0$	$b / 0$
b	$c / 0$	$d / 0$
c	$a / 0$	$d / 0$
d	$e / 0$	$f / 1$
e	$a / 0$	$f / 1$
f	$g / 0$	$f / 1$
g	$a / 0$	$f / 1$



2、状态表化简

现态 (S^n)	次态/输出 (S^{n+1}/Y)	
	A=0	A=1
<i>a</i>	<i>a</i> / 0	<i>b</i> / 0
<i>b</i>	<i>c</i> / 0	<i>d</i> / 0
<i>c</i>	<i>a</i> / 0	<i>d</i> / 0
<i>d</i>	<i>e</i> / 0	<i>f</i> / 1
<i>e</i>	<i>a</i> / 0	<i>f</i> / 1
<i>f</i>	<i>g</i> / 0	<i>f</i> / 1
<i>g</i>	<i>a</i> / 0	<i>f</i> / 1

第一次化简状态表

现态 (S^n)	次态/输出 (S^{n+1}/Y)	
	A=0	A=1
<i>a</i>	<i>a</i> / 0	<i>b</i> / 0
<i>b</i>	<i>c</i> / 0	<i>d</i> / 0
<i>c</i>	<i>a</i> / 0	<i>d</i> / 0
<i>d</i>	<i>e</i> / 0	<i>f</i> / 1
<i>e</i>	<i>a</i> / 0	<i>f</i> / 1
<i>f</i>	<i>e</i> / 0	<i>f</i> / 1

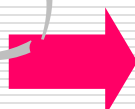


2、状态编码

a=000;b=001;c=010 ;d=011;e=100

最后简化的状态表

现态 (S^n)	次态/输出 (S^{n+1}/Y)	
	A=0	A=1
a	a / 0	b / 0
b	c / 0	d / 0
c	a / 0	a / 0
d	e / 0	d / 1
e	a / 0	d / 1



已分配状态的状态表

现态 (S^n)	次态/输出 (S^{n+1}/Y)	
	A=0	A=1
000	000 / 0	001 / 0
001	010 / 0	011 / 0
010	000 / 0	011 / 0
011	100 / 0	011 / 1
100	000 / 0	011 / 1



三种状态分配方案

状态	方案1 自然二进制 码	方案2 格雷码	方案3 “一对一”
<i>a</i>	0 0 0	0 0 0	0 0 0 0 1
<i>b</i>	0 0 1	0 0 1	0 0 0 1 0
<i>c</i>	0 1 0	0 1 1	0 0 1 0 0
<i>d</i>	0 1 1	0 1 0	0 1 0 0 0
<i>e</i>	1 0 0	1 1 0	1 0 0 0 0



3、求激励方程、输出方程

状态转换真值表

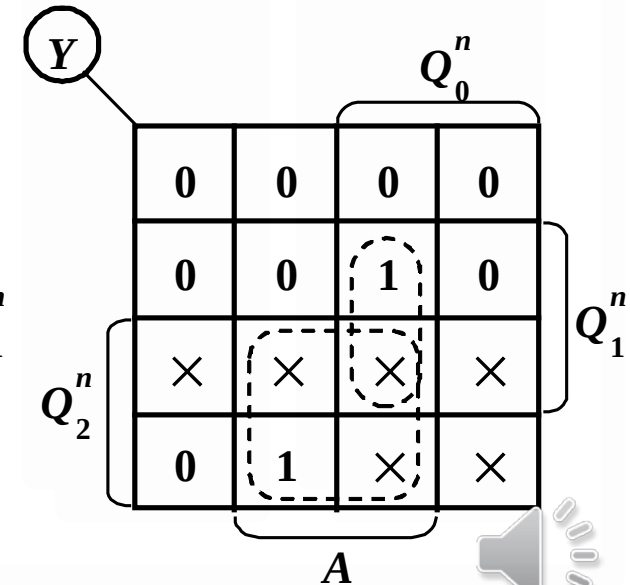
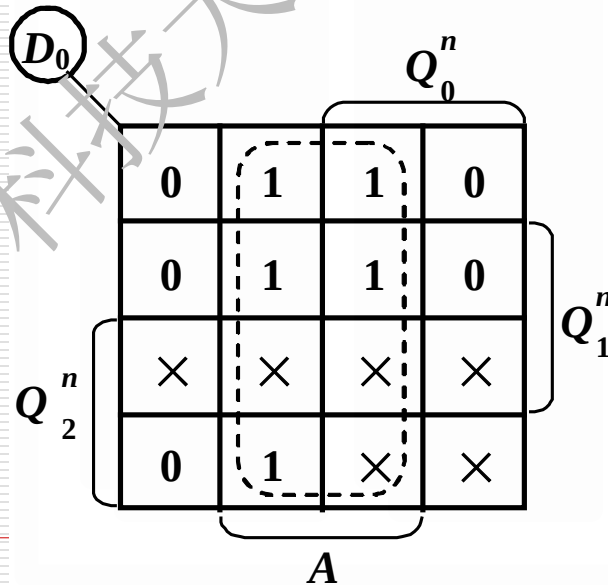
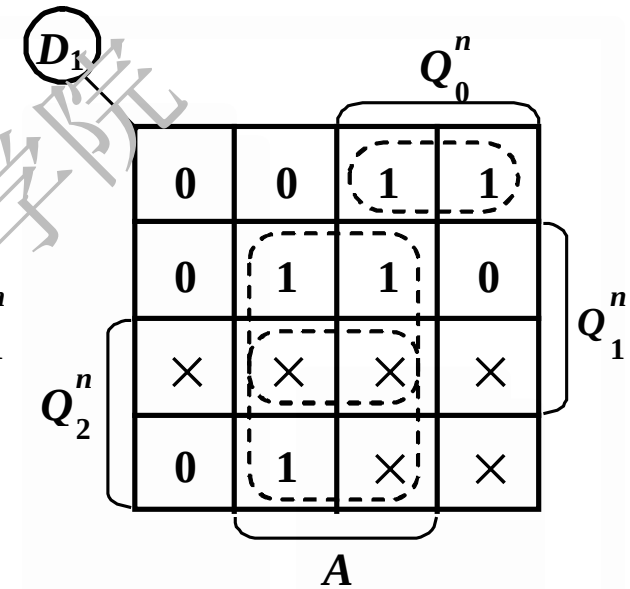
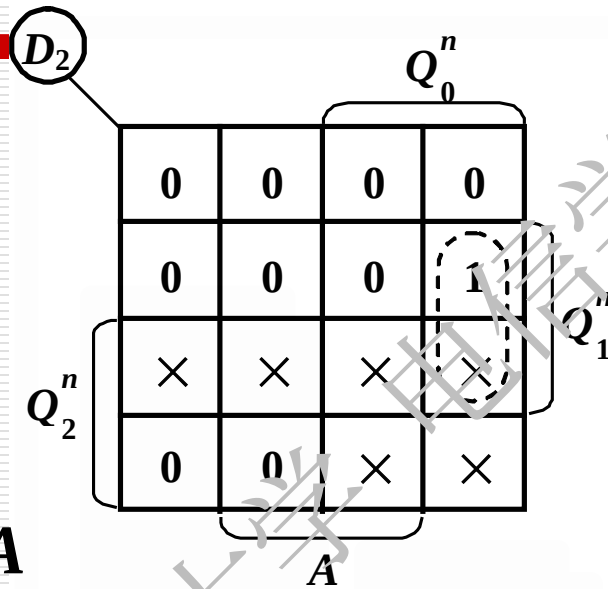
Q_2^n	Q_1^n	Q_0^n	A	$Q_2^{n+1}(D_2)$	$Q_1^{n+1}(D_1)$	$Q_0^{n+1}(D_0)$	Y
0	0	0	0	0	0	0	0
0	0	0	1	0	0	1	0
0	0	1	0	0	1	0	0
0	0	1	1	0	1	1	0
0	1	0	0	0	0	0	0
0	1	0	1	0	1	1	0
0	1	1	0	1	0	0	0
0	1	1	1	0	1	1	1
1	0	0	0	0	0	0	0
1	0	0	1	0	1	1	1

$$D_2 = Q_2^{n+1} = Q_1^n Q_0^n \overline{A}$$

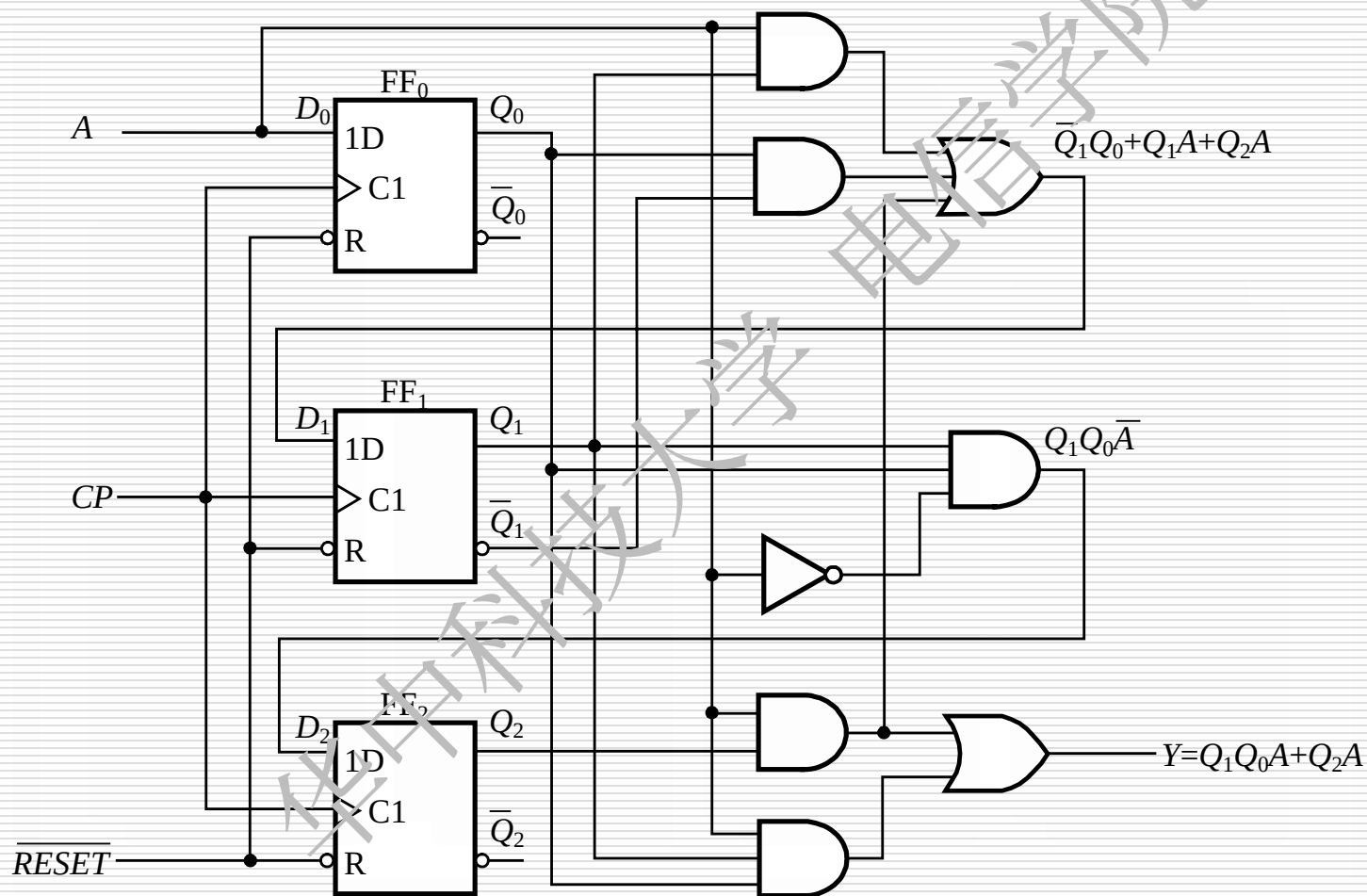
$$D_1 = Q_0^n \overline{Q_1^n} + Q_1^n A + Q_2^n A$$

$$D_0 = Q_0^{n+1} = A$$

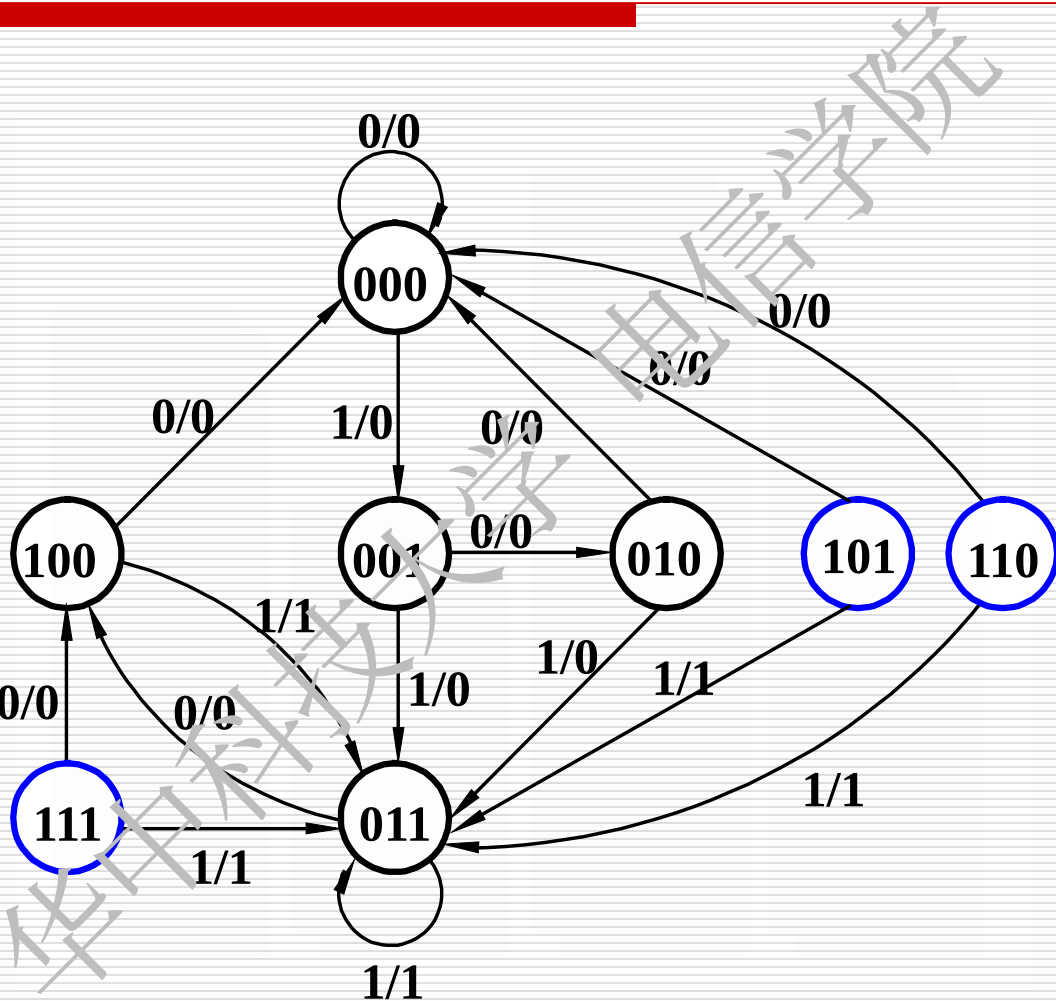
$$Y = Q_1^n Q_0^n A + Q_2^n A$$



画出逻辑电路



Downloaded from <http://ajphaphysocpharm.sagepub.com/> at 11:04 11 September 2014



1. 等价状态和最小化状态表

等价状态——在相同输入条件下，它们的输出相同而且次态等价。

次态等价3的种情况：

- (a) 次态相同。(0, 7)
- (b) 次态交错。例5 (B, C)
- (c) 次态循环。

Y	Y_{n+1}/Z	
	X	
	0	1
0	0/0	1/0
1	0/0	2/0
2	0/0	3/0
3	4/0	3/0
4	5/0	1/0
5	0/0	6/0
6	7/1	2/0
7	0/0	1/0

不考

例5 简化表5所示的原始状态表。

表5

Y	Y _{n+1} /Z	
	X	
	0	1
A	C/1	B/0
B	C/1	E/0
C	B/1	E/0
D	D/1	B/1
E	D/1	B/1

合并等价状态
(B, C) (D, E)

Y	Y _{n+1} /Z	
	X	
	0	1
A	B/1	B/0
B	B/1	D/0
D	D/1	B/1

A → A'
(B, C) → B'
(D, E) → C'

Y	Y _{n+1} /Z	
	X	
	0	1
A'	B'/1	B'/0
B'	B'/1	C'/0
C'	C'/1	B'/1



等价关系的传递性，等价类和最大等价类

等价关系的传递性 假若A与B等价，又有A与C等价，则B与C也等价。记为
 $(A, B), (A, C) \rightarrow (B, C)$

等价类-----若干等价状态的集合

最大等价类-----包含了原始状态表中全部等价状态的等价类



2 隐含表法

例6 简化表6所示的原始状态表。

解 隐含表法步骤

- (1) 画隐含表表格
- (2) 顺序比较, 将比较结果采用3种方法标明在相应方格上。
 - (a) 输出不完全相同的状态对, 标注“×”以示不等价, 例如, A与C, B与E等。
 - (b) 输出完全相同, 次态相同或交错者, 标注“√”以示等价, 称为等价对, 例如, B与D。
 - (c) 输出完全相同, 次态不相同又不交错者, 标注次态, 以便进一步比较。例如, A与B对应方格中标注BD。

B				
C				
D				
E				
	A	B	C	D



2. 隐含表法

(3) 关联比较：检查隐含表中所填状态对，观察这些状态对是否等价。

(4) 列最大等价类：

(A、B)，(B、D) (A、D) ————— (A、B、D)
(A、B、D)，(C)，(E) 3个。

(5) 求最小化状态表



不考	Y_{n+1}/Z	
Y	X	
	0	1
A	B/0	B/1
B	D/0	A/1
C	D/1	B/0
D	B/0	A/1
E	C/1	B/0

B				
C				
D				
E				
	A	B	C	D

B	BD √			
C	×	×		
D	AB √	√	×	
E	×	×	CD ×	×
	A	B	C	D

B	BD			
C	×	×		
D	AB	√	×	
E	×	×	CD	×
	A	B	C	D

不考

Y	Y _{n+1} /Z	
	X	
	0	1
A	B/0	B/1
B	D/0	A/1
C	D/1	B/0
D	B/0	A/1
E	C/1	B/0

(A、B、D), (C)
, (E)

(A、B、D) → A'
(C) → B'
(E) → C'

Y	Y _{n+1} /Z	
	X	
	0	1
A	A/0	A/1
C	A/1	A/0
E	C/1	A/0

Y	Y _{n+1} /Z	
	X	
	0	1
A'	A'/0	A'/1
B'	A'/1	A'/0
C'	B'/1	A'/0



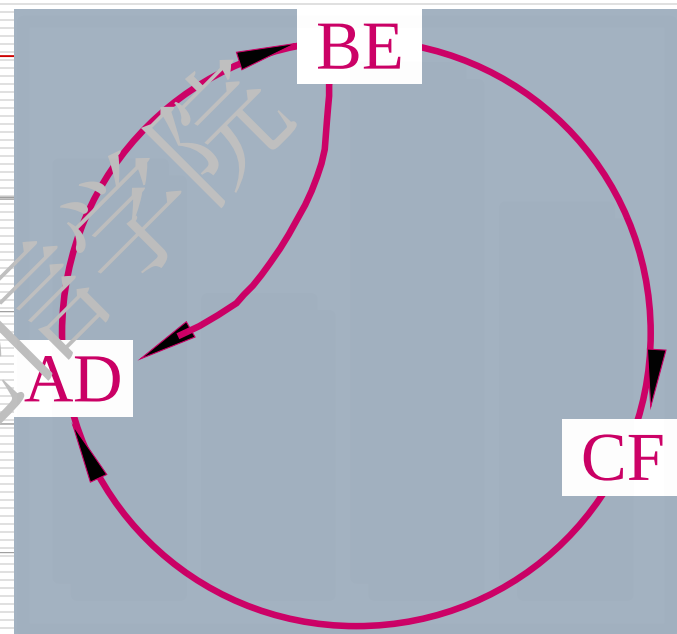
•(c) 次态循环

Y	Y_{n+1}/Z	
	X	
	0	1
A	E/0	D/0
B	A/1	F/0
C	C/0	A/1
D	B/0	A/0
E	D/1	C/0
F	C/0	D/1
G	H/1	G/1
H	C/1	B/1



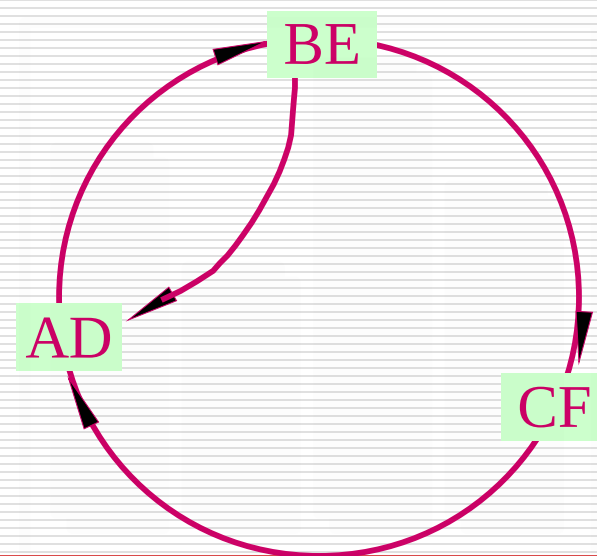
•(c) 次态循环

B	×						
C	×	×					
D	BE ✓	×	×				
E	×	AD CF ✓	×	×			
F	×	×	AD ✓	×	×		
G	×	×	×	×	×	×	
H	×	×	×	×	×	×	CH BG ×
	A	B	C	D	E	F	G



次态循环

- AD是否等价取决于BE，而BE又取决于AD和CF，CF又取决于AD，这些状态构成循环。这些状态对在输入相同的条件下，输出完全相同；另一组次态是相同或交错。
- (A、D)，(B、E)，(C、F)



6.4 异步时序逻辑电路的分析

一. 异步时序逻辑电路的分析方法：
分析步骤：

1. 写出下列各逻辑方程式：

a) 时钟方程

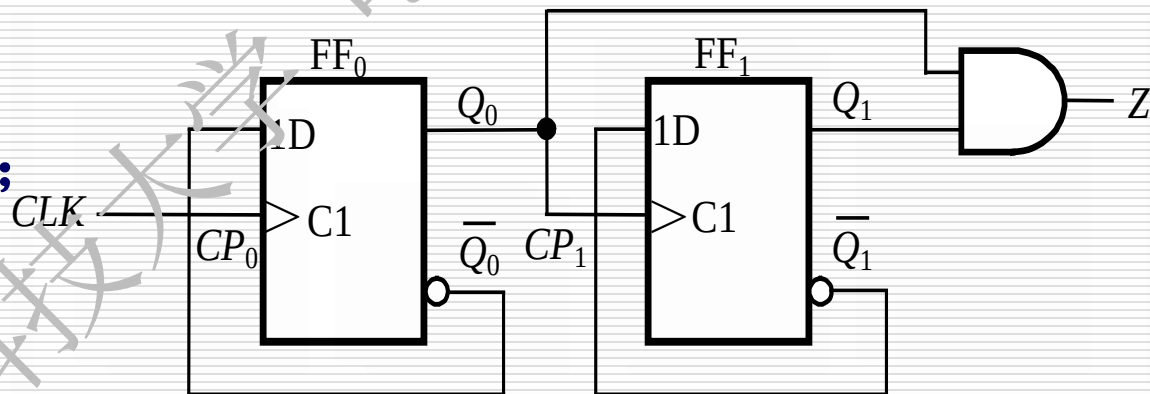
b) 触发器的激励方程；

c) 输出方程

d) 状态方程

2. 列出状态转换表或画出状态图和波形图；

3. 确定电路的逻辑功能。



注意:

(1) 分析状态转换时必须考虑各触发器的时钟信号作用情况

有作用, 则令 $cp_n=1$; 否则 $cp_n=0$

根据激励信号确定那些 $cp_n=1$ 的触发器的次态, $cp_n=0$ 的触发器则保持原有状态不变。

(2) 每一次状态转换必须从输入信号所能触发的第一个触发器开始逐级确定

(3) 每一次状态转换都有一定的时间延迟

同步时序电路的所有触发器是同时转换状态的, 与之不同, 异步时序电路各个触发器之间的状态转换存在一定的延迟, 也就是说, 从现态 S^n 到次态 S^{n+1} 的转换过程中有一段“不稳定”的时间。在此期间, 电路的状态是不确定的。只有当全部触发器状态转换完毕, 电路才进入新的“稳定”状态, 即次态 S^{n+1} 。

二. 异步时序逻辑电路的分析举例

例1 分析如图所示异步电路

1. 写出电路方程式

① 时钟方程

$$CP_0 = CLK \quad CP_1 = Q_0$$

② 输出方程

$$Z = Q_1^n Q_0^n$$

③ 激励方程

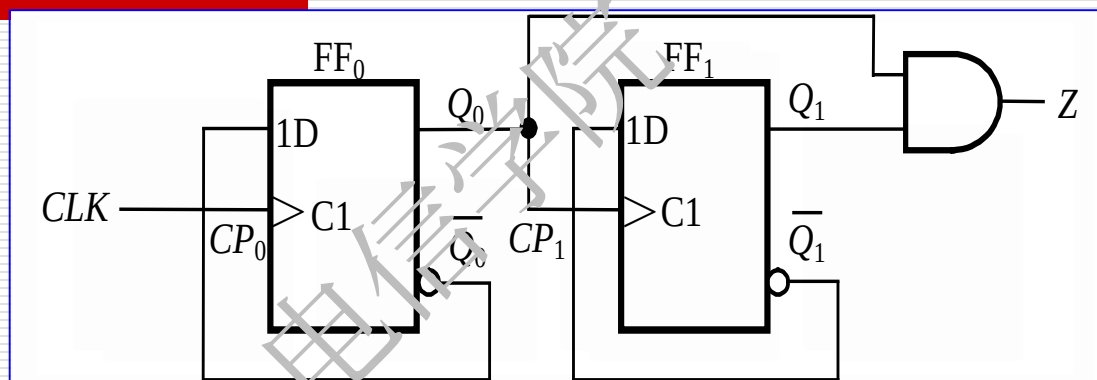
$$D_0 = \bar{Q}_0 \quad D_1 = \bar{Q}_1$$

④ 求电路状态方程

触发器如有时钟脉冲的上升沿作用时，其状态变化；
如无时钟脉冲上升沿作用时，其状态不变。

$$Q_0^{n+1} = D_0 cp_0 + Q_0^n \overline{cp_0} = \bar{Q}_0^n cp_0 + Q_0^n \overline{cp_0}$$

$$Q_1^{n+1} = D_1 cp_1 + Q_1^n \overline{cp_1} = \bar{Q}_1^n cp_1 + Q_1^n \overline{cp_1}$$



3. 列状态表、画状态图、波形图

$$CP_0 = CLK$$

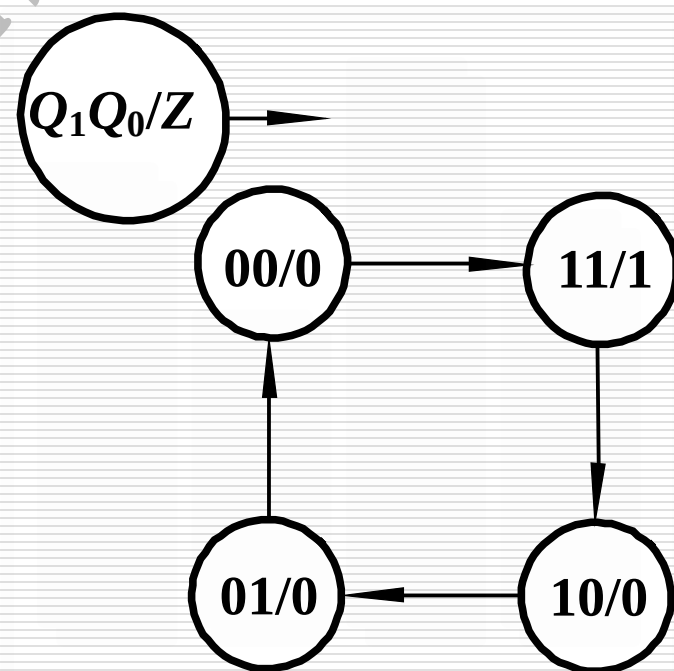
$$CP_1 = Q_0$$

$$Q_0^{n+1} = D_0 cp_0 + Q_0^n \overline{cp_0} = \overline{Q_0^n} cp_0 + Q_0^n \overline{cp_0}$$

$$Q_1^{n+1} = D_1 cp_1 + Q_1^n \overline{cp_1} = \overline{Q_1^n} cp_1 + Q_1^n \overline{cp_1}$$

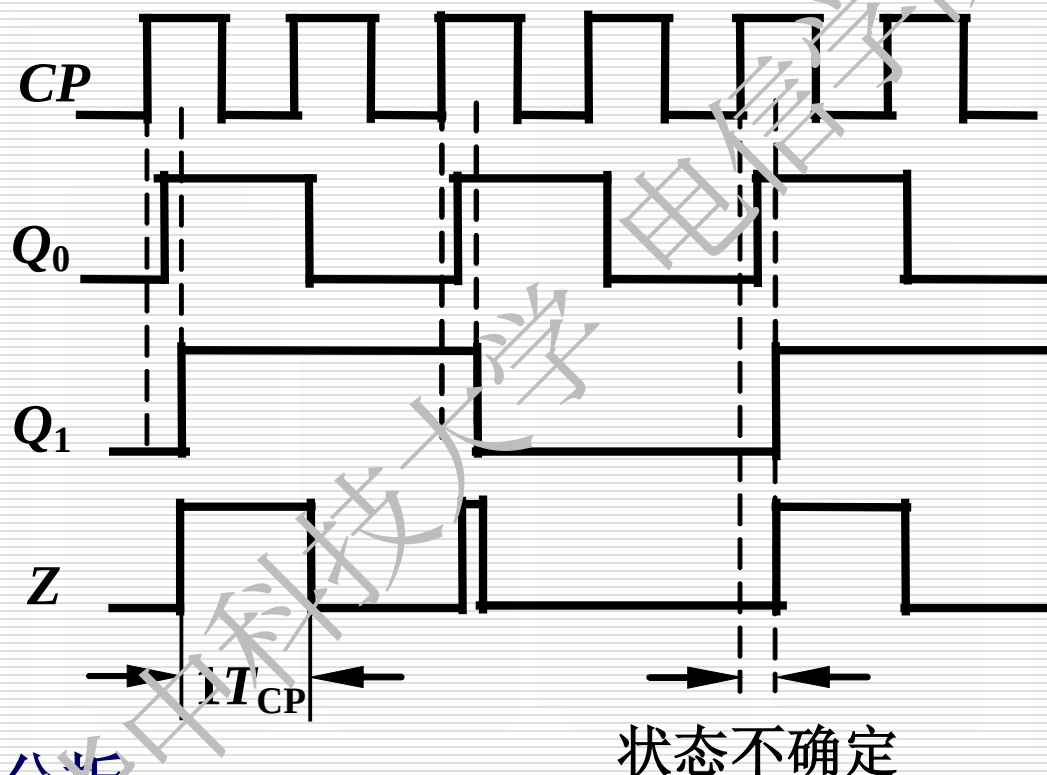
(X----无触发沿, ↑----有触发沿)

CLK	Q_1	Q_0	CP_1	CP_0	Q_1^{n+1}	Q_0^{n+1}
↑	0	0	↑	↑	1	1
↑	1	1	x	↑	1	0
↑	1	0	↑	↑	0	1
↑	0	1	x	↑	0	0
↑	0	0	↑	↑	1	1



根据状态图和具体触发器的传输延迟时间 t_{pLH} 和 t_{pHL} ,

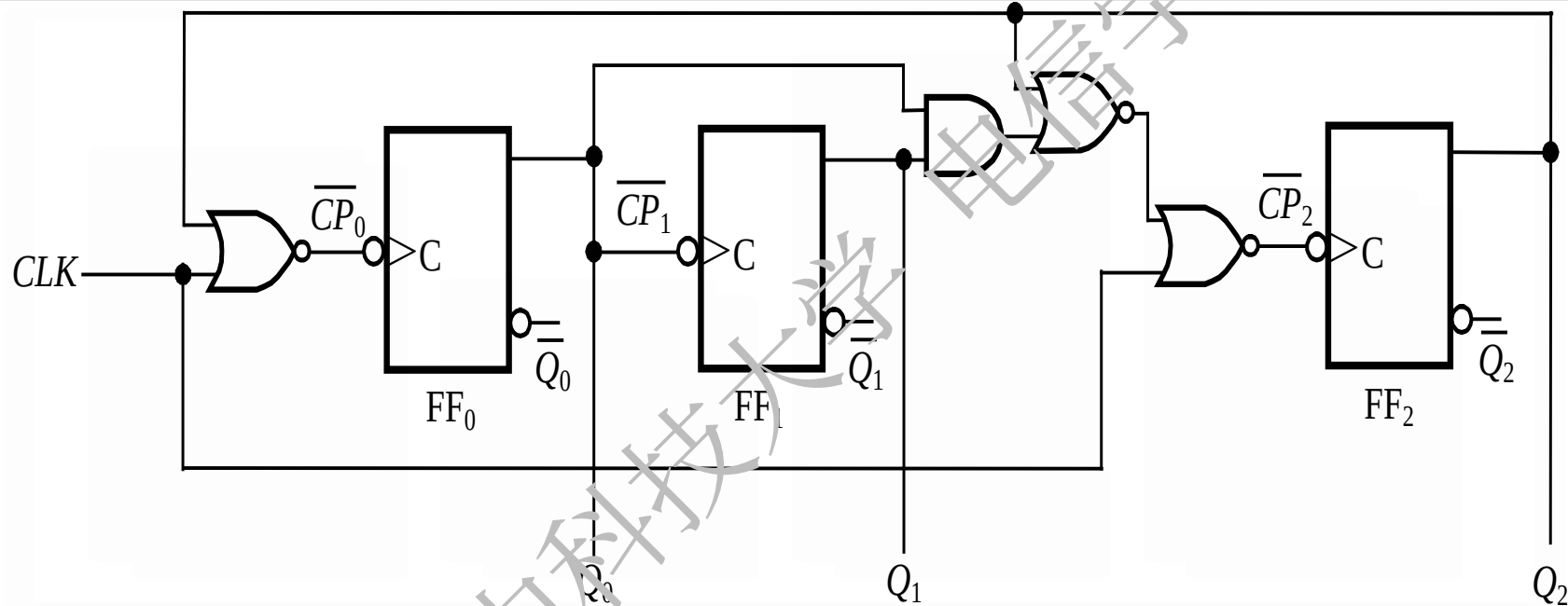
可以画出时序图



4. 逻辑功能分析

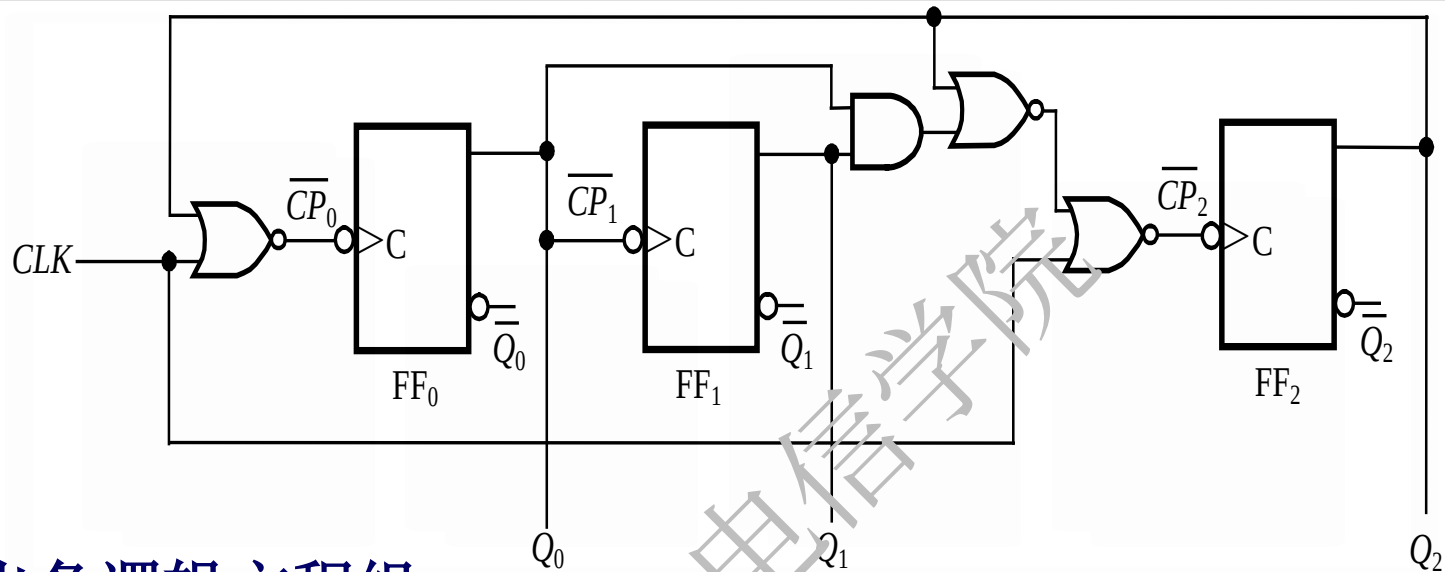
该电路是一个异步二进制减计数器， Z 信号的上升沿可触发借位操作。也可把它看作为一个序列信号发生器。

例2 分析如图所示异步时序逻辑电路。



解 1、了解电路组成

是由3个下降沿敏感的T' 触发器构成的异步时序电路



解 (1) 列出各逻辑方程组

时钟方程

$$\overline{CP_0} = \overline{Q_2 + CLK} = \overline{Q_2} \overline{CLK} \quad \overline{CP_1} = Q_0$$

$$\overline{CP_2} = \overline{Q_0 Q_1 + Q_2 + CLK} = (\overline{Q_0 Q_1 + Q_2}) \overline{CLK}$$

状态方程

$$Q_0^{n+1} = \overline{Q_0^n} cp_0 + Q_0^n \overline{cp_0}$$

$$Q_1^{n+1} = \overline{Q_1^n} cp_1 + Q_1^n \overline{cp_1}$$

$$Q_2^{n+1} = \overline{Q_2^n} cp_2 + Q_2^n \overline{cp_2}$$

(2) 列出 状态表

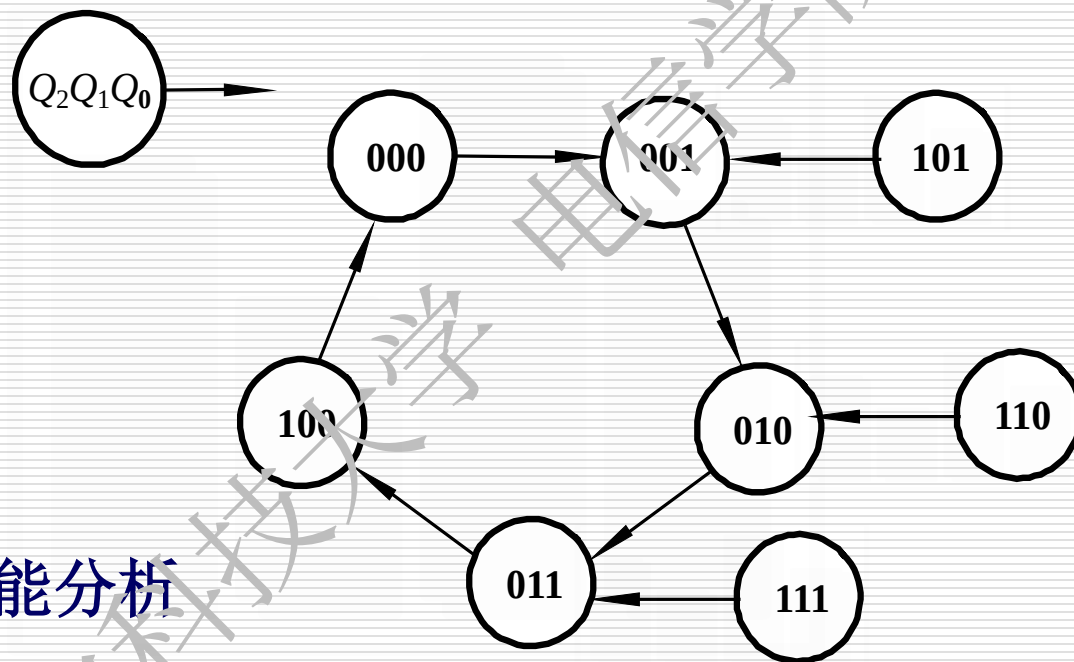
$$\overline{CP}_0 = \overline{Q}_2 \overline{CLK}$$

$$\overline{CP}_1 = Q_0$$

$$\overline{CP}_2 = (Q_0 Q_1 + Q_2) \overline{CLK}$$

CLK	Q_2^n	Q_1^n	Q_0^n	$\overline{CP}_2$	$\overline{CP}_1$	$\overline{CP}_0$	Q_2^{n+1}	Q_1^{n+1}	Q_0^{n+1}
0	0	0	0	0	0	1	0	0	0
1	0	0	0	0	1	0	0	0	1
0	0	0	1	0	1	1	0	0	1
1	0	0	1	0	0	0	0	1	0
0	0	1	0	0	0	1	0	1	0
1	0	1	0	0	1	0	0	1	1
0	0	1	1	1	1	1	0	1	1
1	0	1	1	0	0	0	1	0	0
0	1	0	0	1	0	0	1	0	0
1	1	0	0	0	0	0	0	0	0
0	1	0	1	1	1	0	1	0	1
1	1	0	1	0	1	0	0	0	1
0	1	1	0	1	0	0	1	1	0
1	1	1	0	0	0	0	0	1	0
0	1	1	1	1	1	0	1	1	1
1	1	1	1	0	1	0	0	1	1

(3) 画出状态图



(4) 逻辑功能分析

电路是一个异步五进制加计数电路。

6.5 若干典型的时序逻辑集成电路

6.5.1 寄存器和移位寄存器

6.5.2 计数器

6.5 若干典型的时序逻辑集成电路

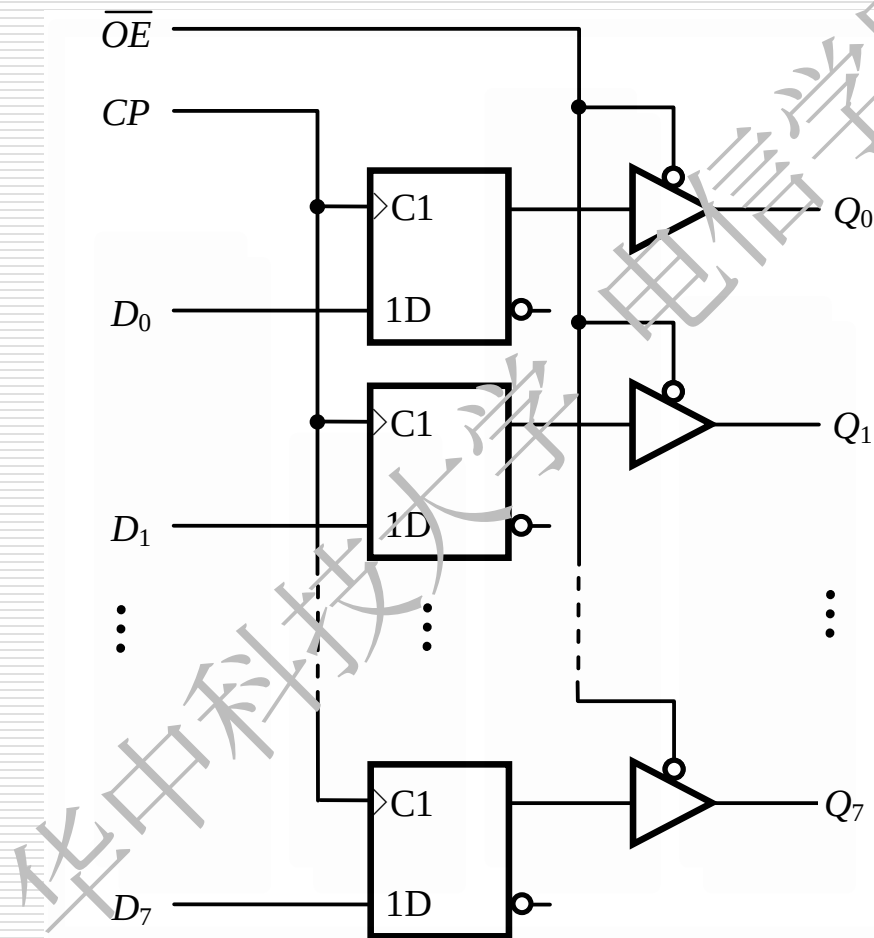
6.5.1 寄存器和移位寄存器

1、 寄存器

寄存器:是数字系统中用来存储代码或数据的逻辑部件。它的主要组成部分是触发器。

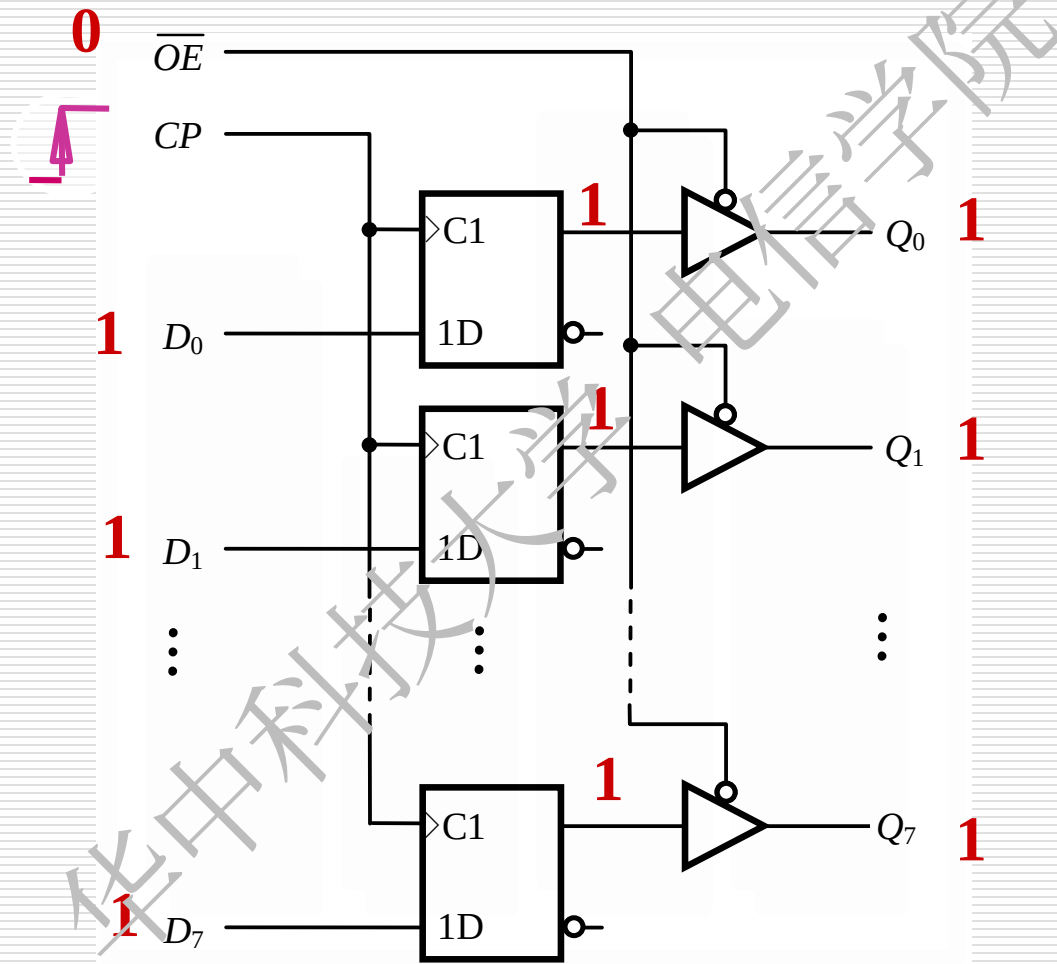
一个触发器能存储1位二进制代码, 存储 n 位二进制代码的寄存器需要用 n 个触发器组成。寄存器实际上是若干触发器的集合。

8位CMOS寄存器74HC374



脉冲边沿敏感的寄存器

8位CMOS寄存器74HC/HCT374



8位CMOS寄存器74LV374

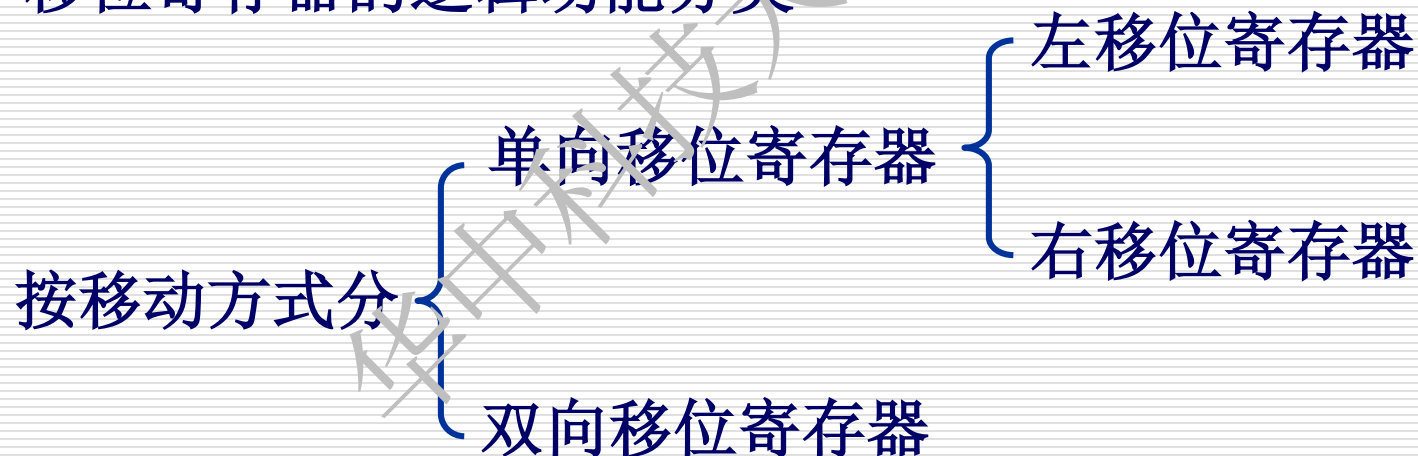
工作模式	输 入			内部触发器 Q_N^{n+1}	输出
	$\overline{OE}$	CP	D_N		$Q_0 \sim Q_7$
存入和读出数据	L	↑	L	L	对应内部触发器的状态
	L	↑	H	H	
存入数据，禁止输出	H	↑	L	L	高阻
	H	↑	H	H	高阻

2、 移位寄存器

- 移位寄存器的逻辑功能

移位寄存器是既能寄存数码，又能在时钟脉冲的作用下使数码向高位或向低位移动的逻辑功能部件。

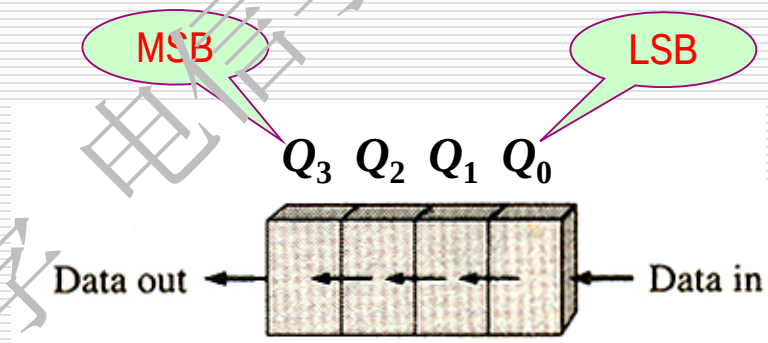
- 移位寄存器的逻辑功能分类



常见的几种移位寄存器数据传送转换方式

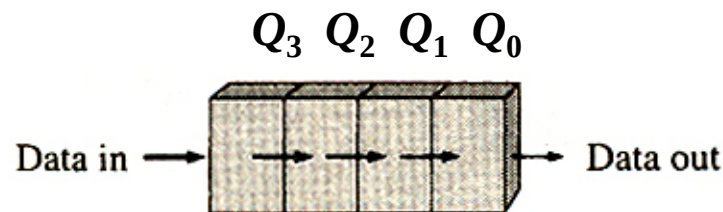
□ 右移(串行输入/串行输出)

数据从低位移向高位

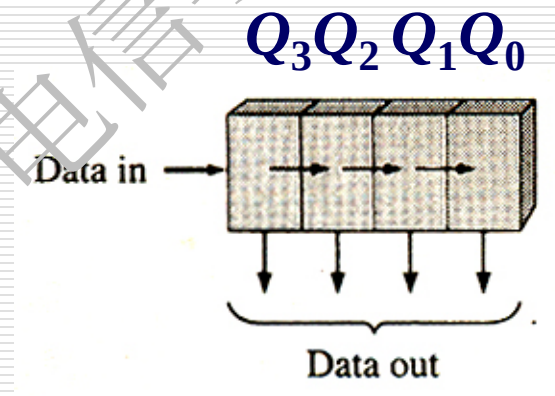
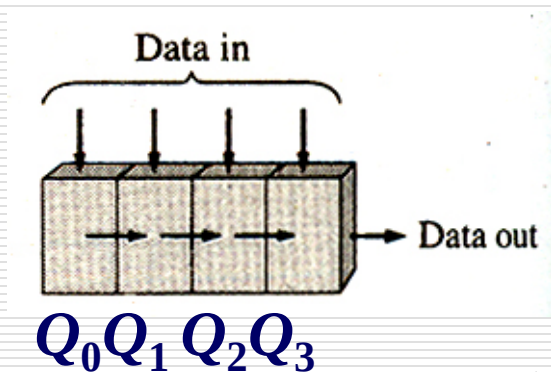


□ 左移(串行输入/串行输出)

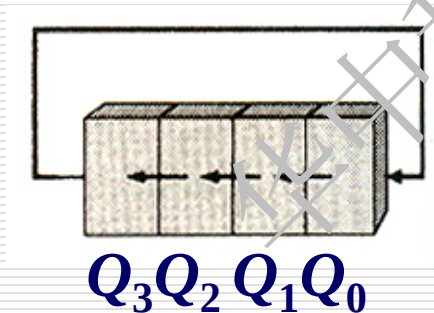
数据从高位移向低位



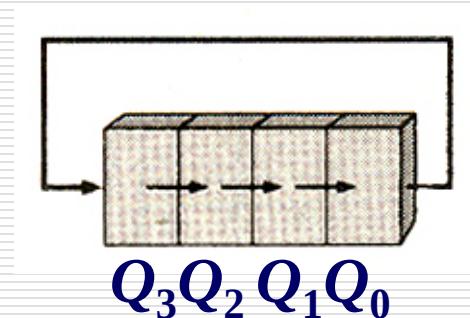
- 右移 (并行输入/串行输出)
- 左移 (串行输入/并行输出)



- 环行右移

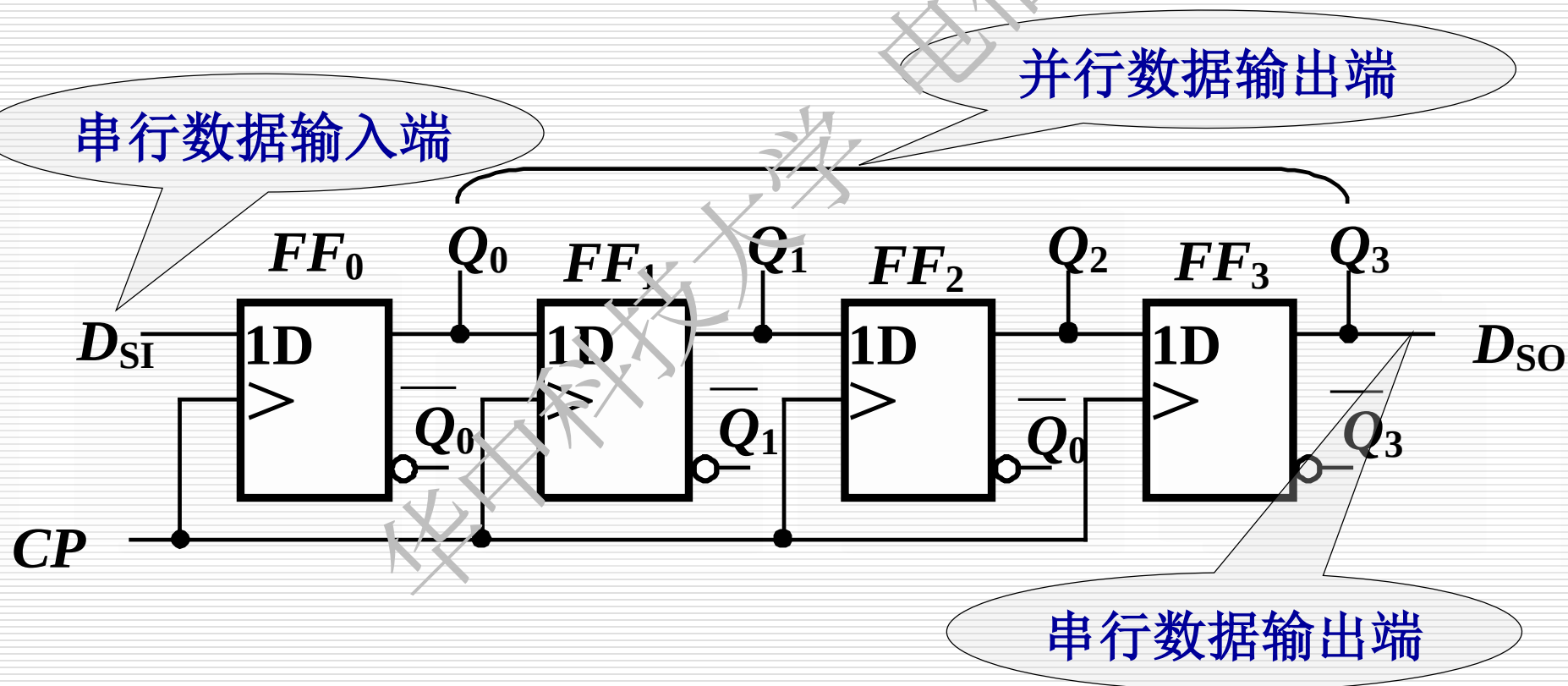


- 环行左移



(1) 基本移位寄存器

(a) 电路



(b). 工作原理

写出激励方程:

$$D_0 = D_{SI} \quad D_1 = Q_0^n$$

$$D_2 = Q_1^n$$

$$D_3 = Q_2^n$$

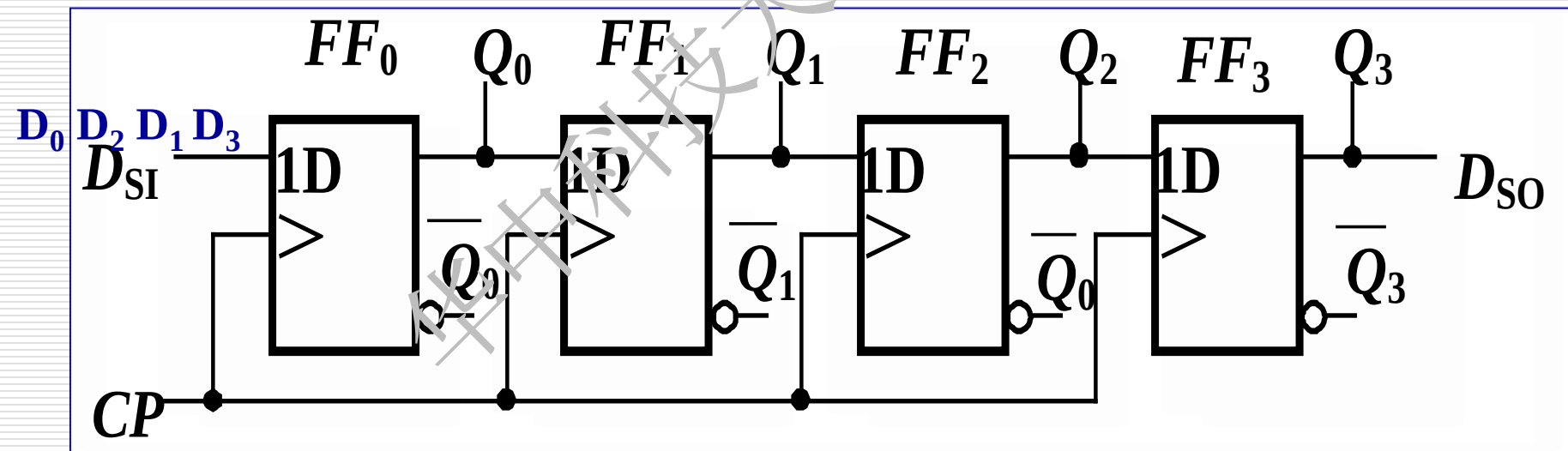
写出状态方程:

$$Q_0^{n+1} = D_{SI}$$

$$Q_1^{n+1} = D_1 = Q_0^n$$

$$Q_2^{n+1} = D_2 = Q_1^n$$

$$Q_3^{n+1} = D_3 = Q_2^n$$



FF_0	FF_1	FF_2	FF_3
0	0	0	0

$$Q_0^{n+1} = D_{SI}$$

1CP 后 1→

1	0	0	0
---	---	---	---

$$Q_1^{n+1} = Q_0^n$$

2CP 后 1→

1	1	0	0
---	---	---	---

$$Q_2^{n+1} = Q_1^n$$

3CP 后 0→

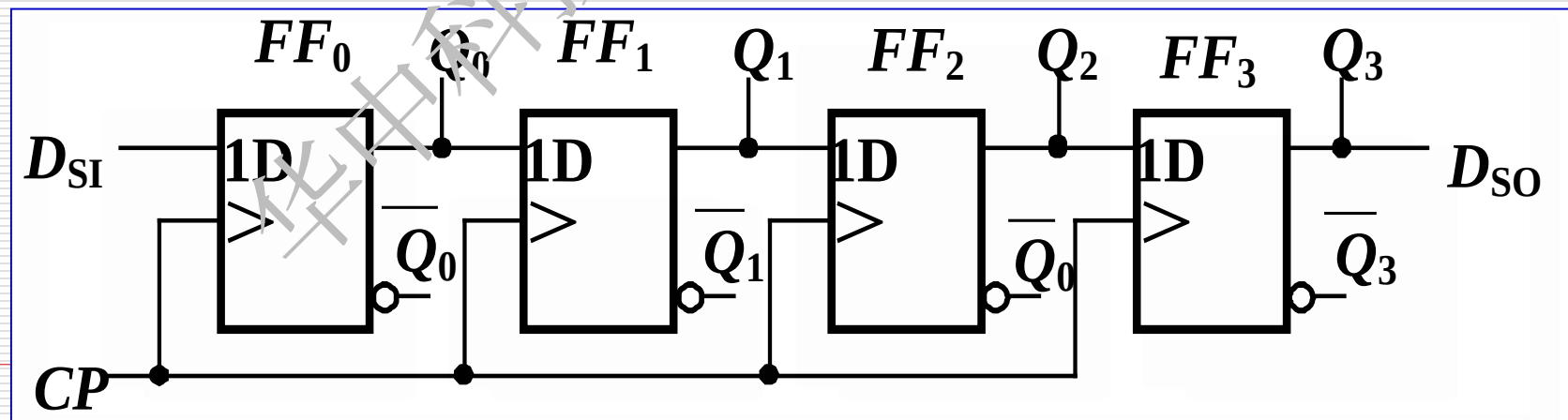
0	1	1	0
---	---	---	---

$$Q_3^{n+1} = Q_2^n$$

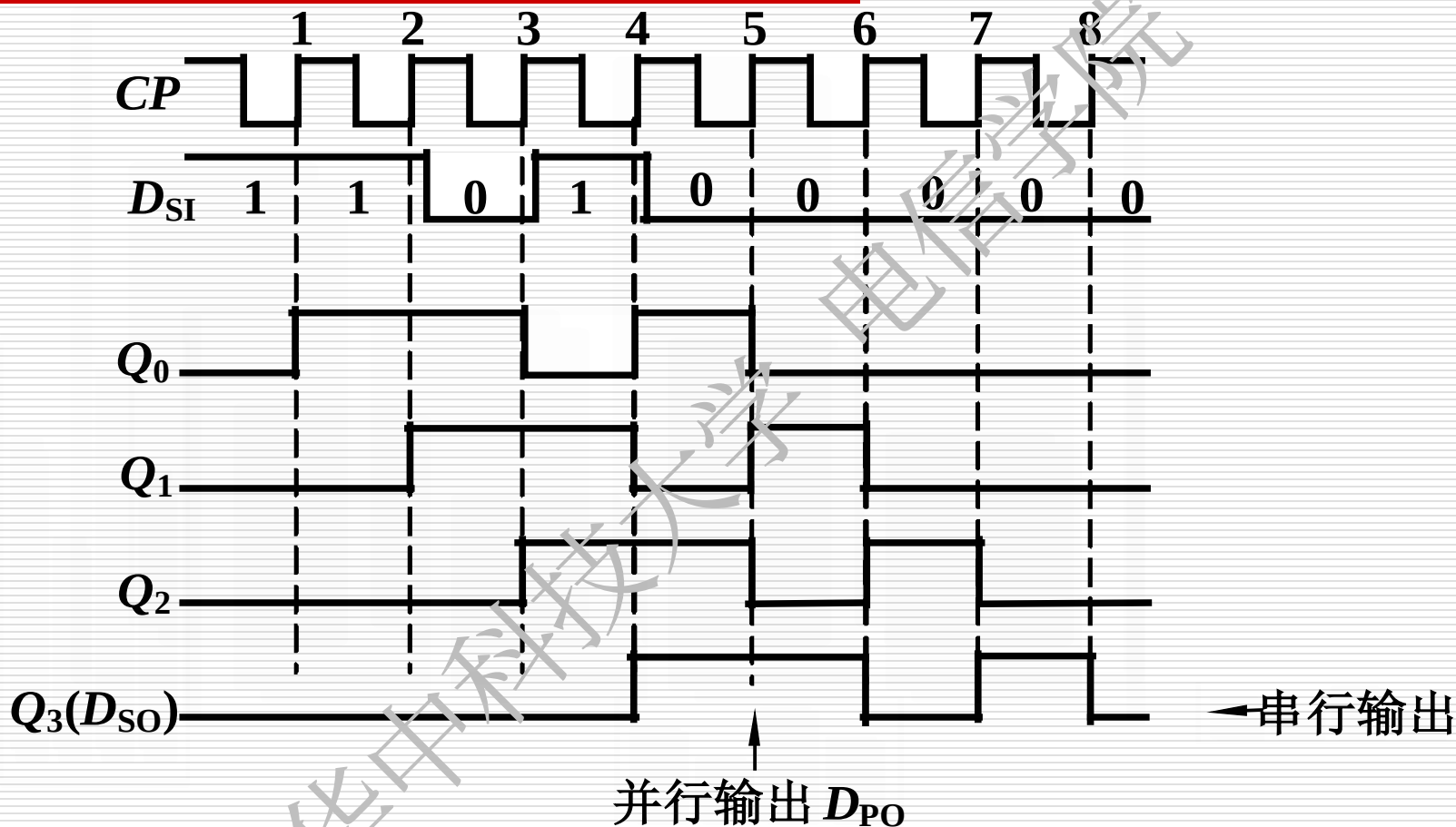
4CP 后 1→

1	0	1	1
---	---	---	---

1011



$D_{SI} = 11010000$, 从高位开始输入



经过7个CP脉冲作用后，从 D_{SI} 端串行输入的数码就可以从 D_O 端串行输出。
串入→串出

(2) 多功能双向移位寄存器

(a) 工作原理

高位移向低位----左移

低位移向高位----右移

多功能移位寄存器工作模式简图



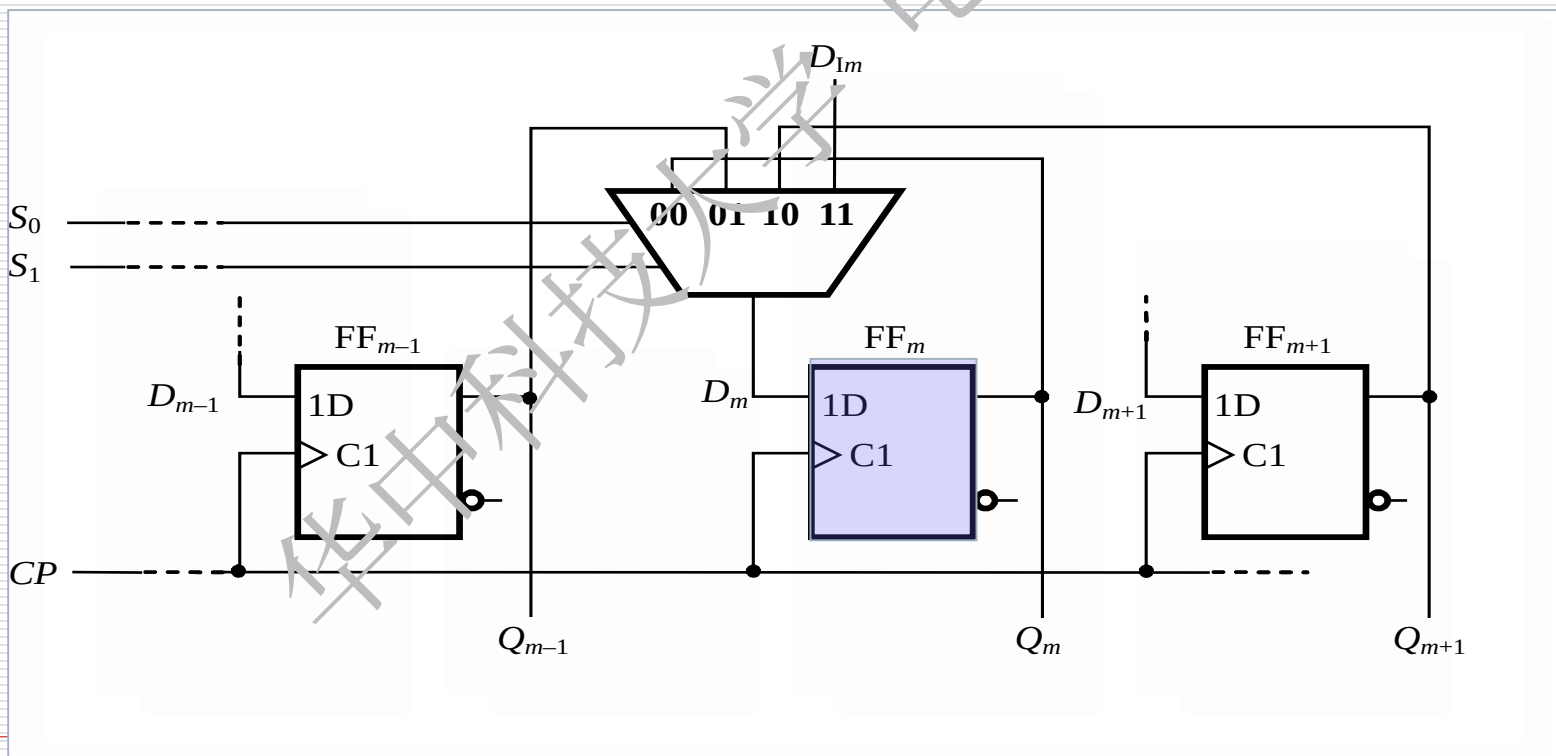
实现多种功能双向移位寄存器的一种方案(仅以 FF_m 为例)

$S_1S_0=00$ $Q_m^{n+1} = Q_m^n$ 不变

$S_1S_0=10$ $Q_m^{n+1} = Q_{m+1}^n$ 高位移向低位

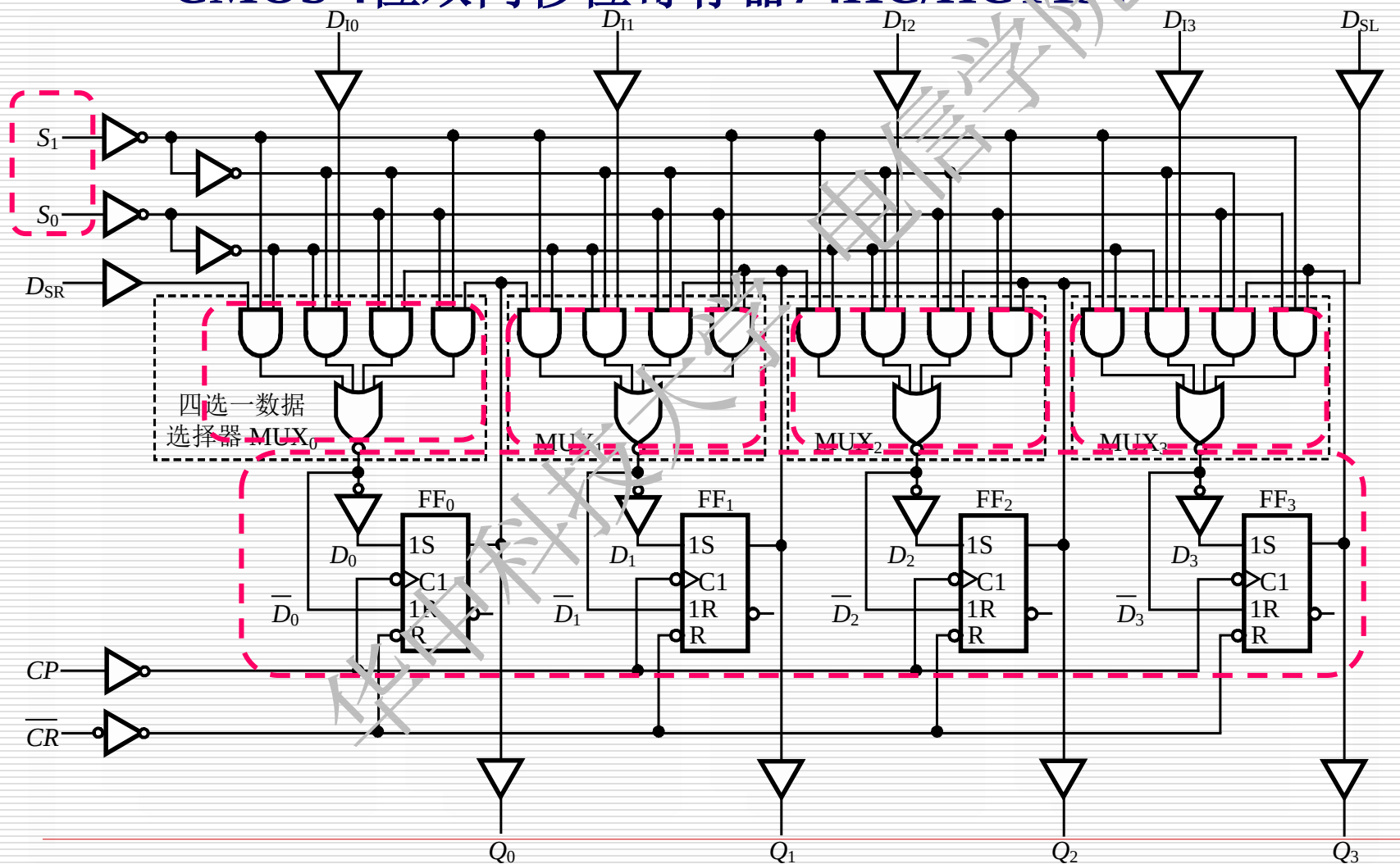
$S_1S_0=01$ $Q_m^{n+1} = Q_{m-1}^n$ 低位移向高位

$S_1S_0=11$ $Q_m^{n+1} = D_m$ 并入



(2) 典型集成电路

CMOS 4位双向移位寄存器74HC/HCT194



(2) 典型集成电路

CMOS 4位双向移位寄存器74HC/HCT194

□ 并行置数

当 $S_1S_0 = 11$

$Clear = 1$

数据选择器输入3

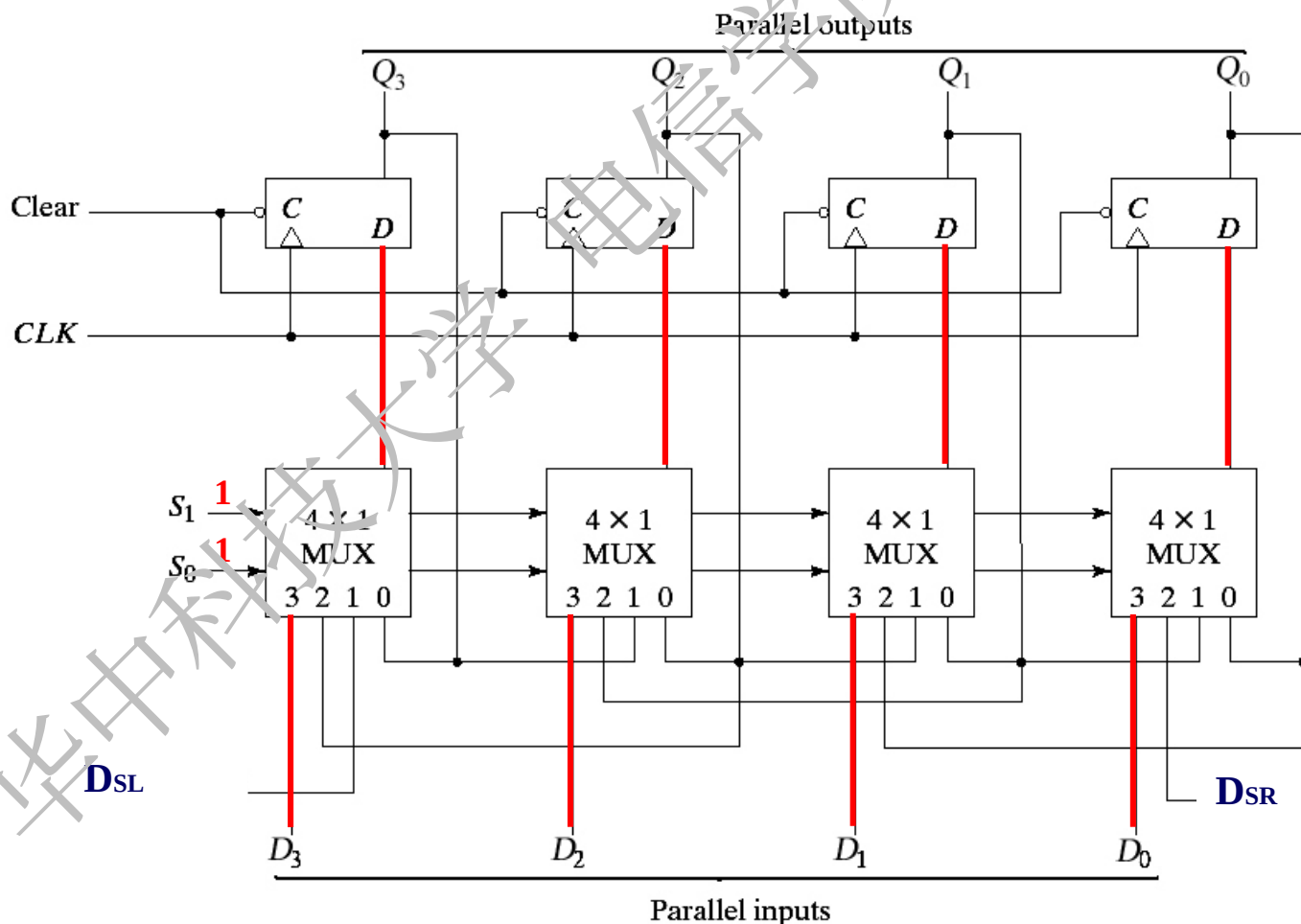
触发器的D端等
于并行输入数据

$$Q_0^{n+1} = D_0$$

$$Q_1^{n+1} = D_1$$

$$Q_2^{n+1} = D_2$$

$$Q_3^{n+1} = D_3$$



(2) 典型集成电路

CMOS 4位双向移位寄存器74HC/HCT194

□ 右移

当 $S_1S_0 = 01$

$Clear = 1$

数据选择起输入2

每个触发器的D
输入端等于低位的
输出状态。

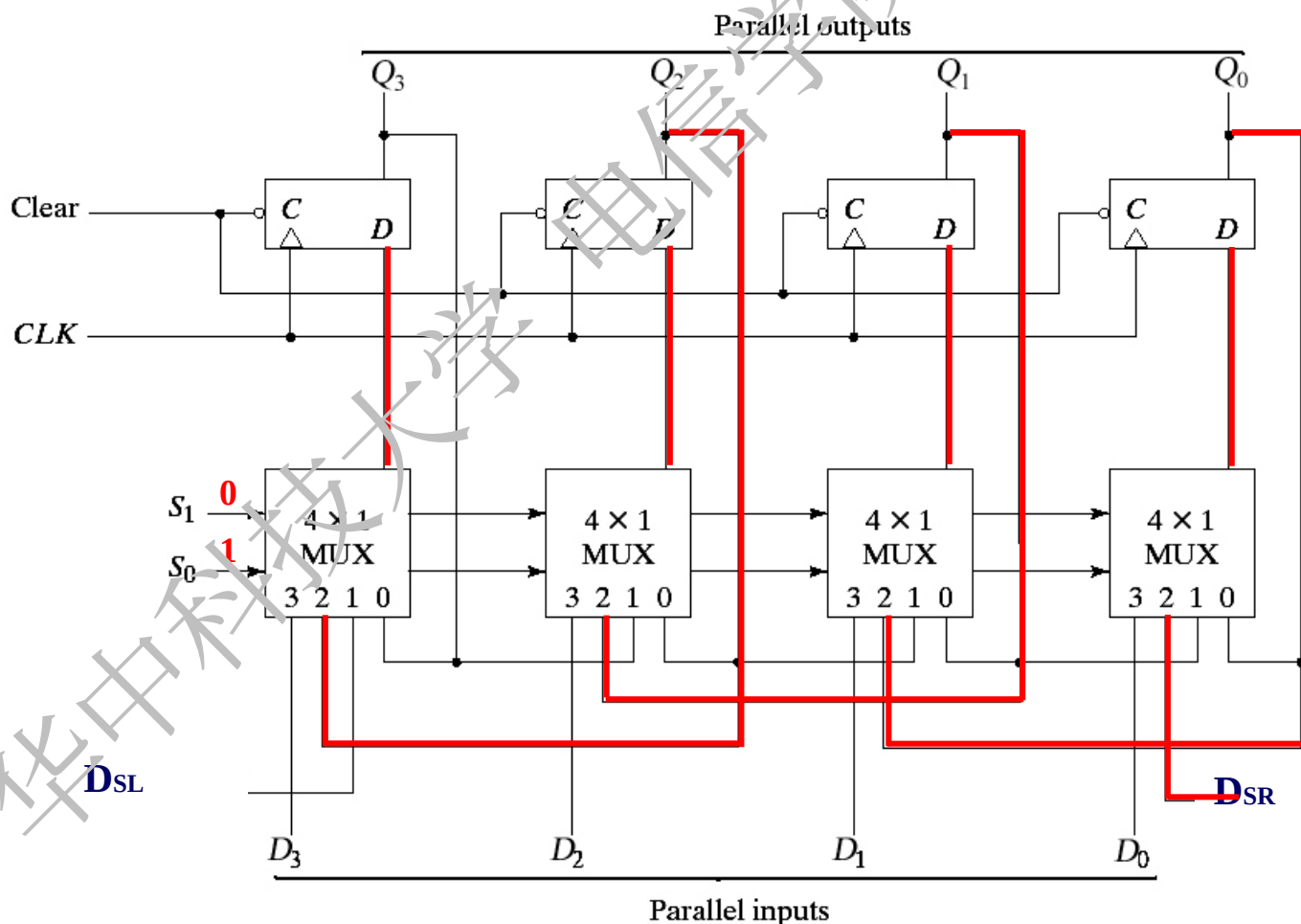
在CLK的作用下，
数据右移

$$Q_0^{n+1} = D_{SR}$$

$$Q_1^{n+1} = Q_0^n$$

$$Q_2^{n+1} = Q_1^n$$

$$Q_3^{n+1} = Q_2^n$$



(2) 典型集成电路

CMOS 4位双向移位寄存器74HC/HCT194

□ 左移

当 $S_1 S_0 = 10$

$Clear = 1$

数据选择器输入1

每个触发器 D
端的输入等于高
位的输出状态

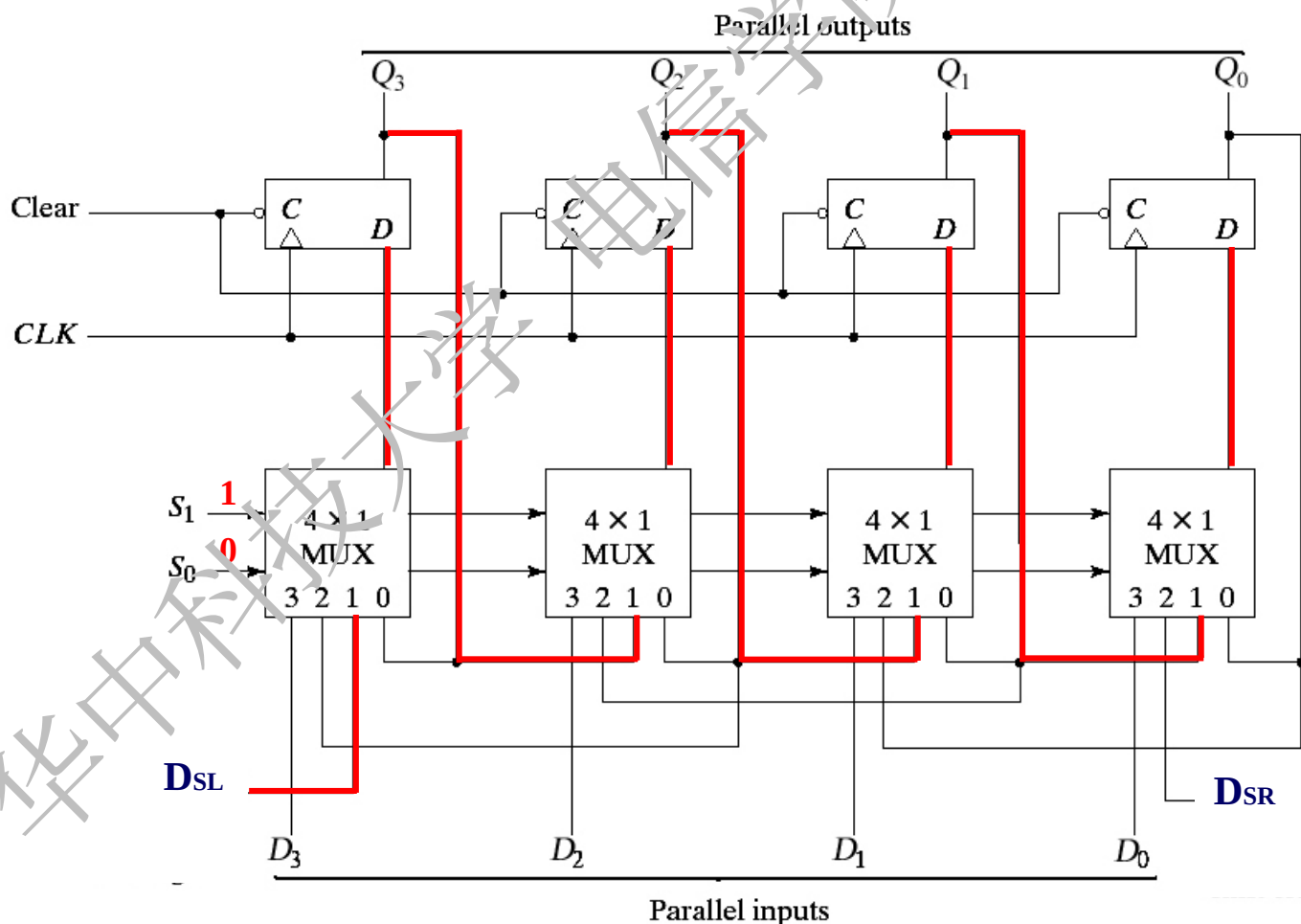
在 CLK 的作用
下,数据串行左移

$$Q_0^{n+1} = Q_1^n$$

$$Q_1^{n+1} = Q_2^n$$

$$Q_2^{n+1} = Q_3^n$$

$$Q_3^{n+1} = D_{SL}$$



(2) 典型集成电路

CMOS 4位双向移位寄存器74HC/HCT194

□ 状态不变

当 $S_1S_0 = 00$

$Clear = 1$

数据选择器输入0

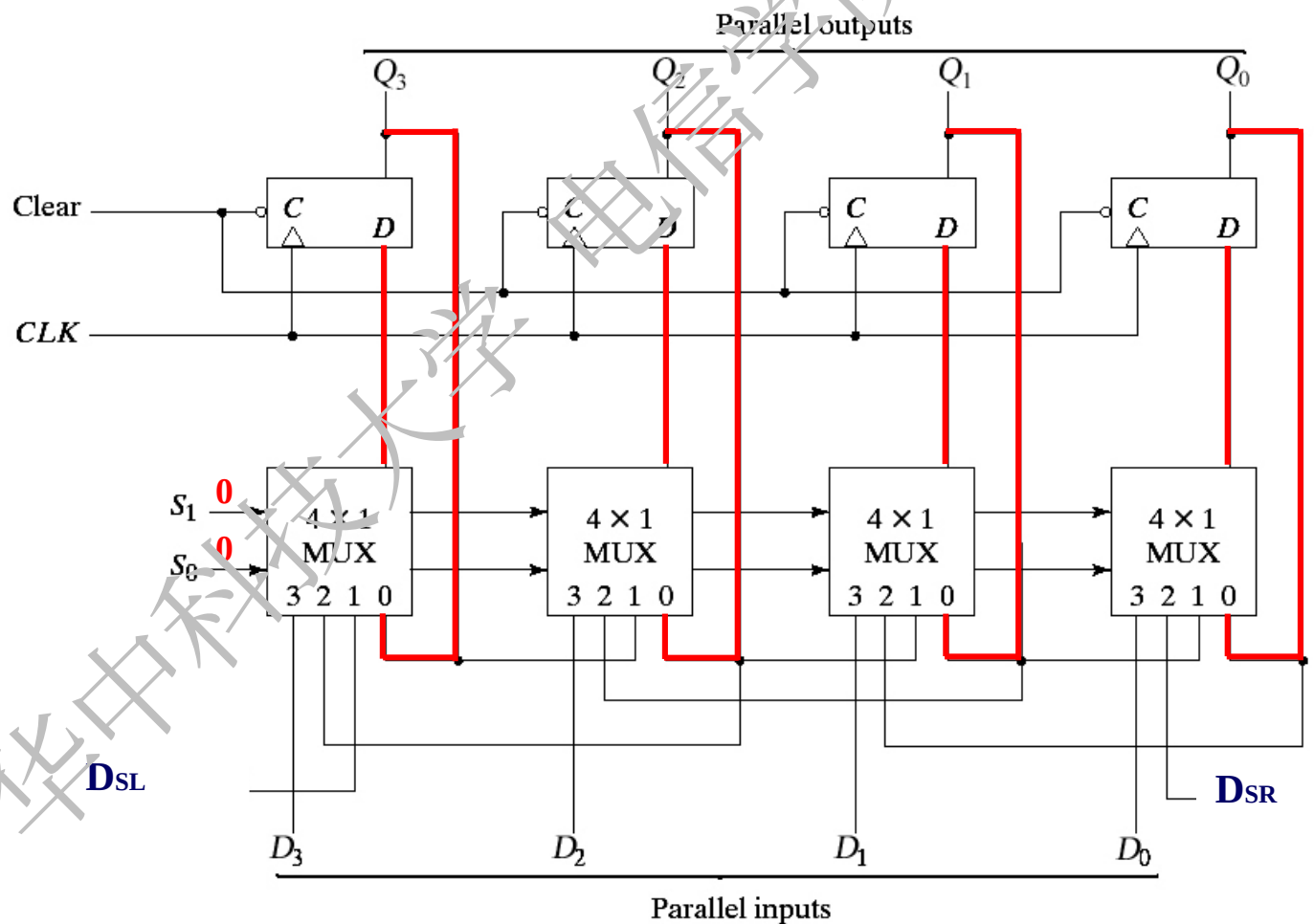
每个触发器的 **D**
端等于其输出状态

$$Q_0^{n+1} = Q_0^n$$

$$Q_1^{n+1} = Q_1^n$$

$$Q_2^{n+1} = Q_2^n$$

$$Q_3^{n+1} = Q_3^n$$



(2) 典型集成电路

CMOS 4位双向移位寄存器74HC/HCT194

□ 清零

当 $Clear = 0$

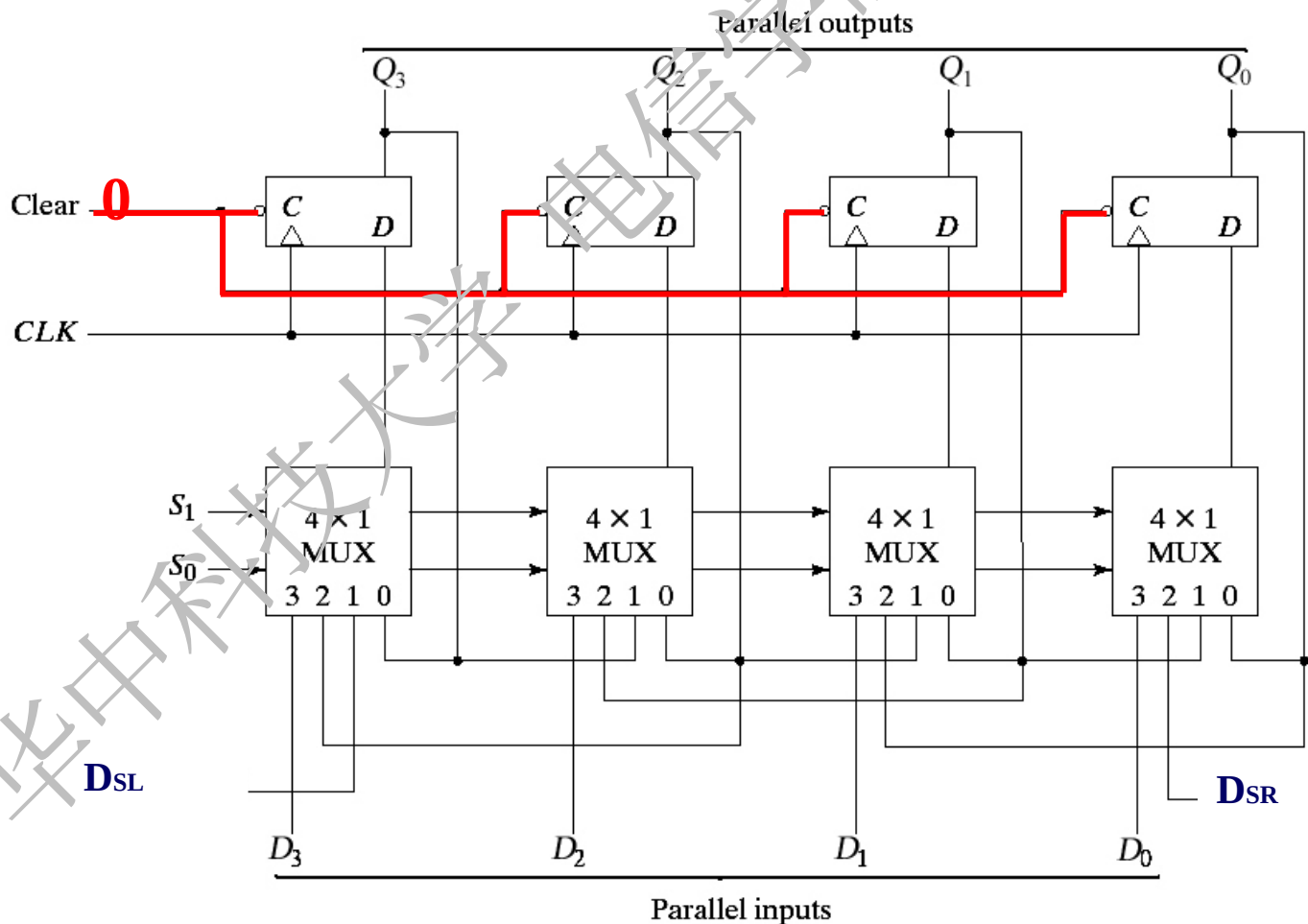
所有触发器
同时被清零

$$Q_0 = 0$$

$$Q_1 = 0$$

$$Q_2 = 0$$

$$Q_3 = 0$$



74HCT194 的功能表

输 入							输 出				行			
清 零	控制信 号		串行输 入		时 钟 CP	并行输入				Q_0^{n+1} Q_1^{n+1} Q_2^{n+1} Q_3^{n+1}				
$\overline{CR}$	S_1	S_0	右 移 D_{SR}	左 移 D_{SL}		DI_0	DI_1	DI_2	DI_3					
L	×	×	×	×	×	×	×	×	×	L	L	L	L	1
H	L	L	×	×	×	×	×	×	×	Q_0^n	Q_1^n	Q_2^n	Q_3^n	2
H	L	H	L	×	↑	×	×	×	×	L	Q_0^n	Q_1^n	Q_2^n	3
H	L	H	H	×	↑	×	×	×	×	H	Q_0^n	Q_1^n	Q_2^n	4
H	H	L	×	L	↑	×	×	×	×	Q_1^n	Q_2^n	Q_3^n	L	5
H	H	L	×	H	↑	×	×	×	×	Q_1^n	Q_2^n	Q_3^n	H	6
H	H	H	×	×	↑	DI_0^*	DI_1^*	DI_2^*	DI_3^*	D_0	D_1	D_2	D_3	7

异步清零

右移---低位向高位移动

同步置数

保持

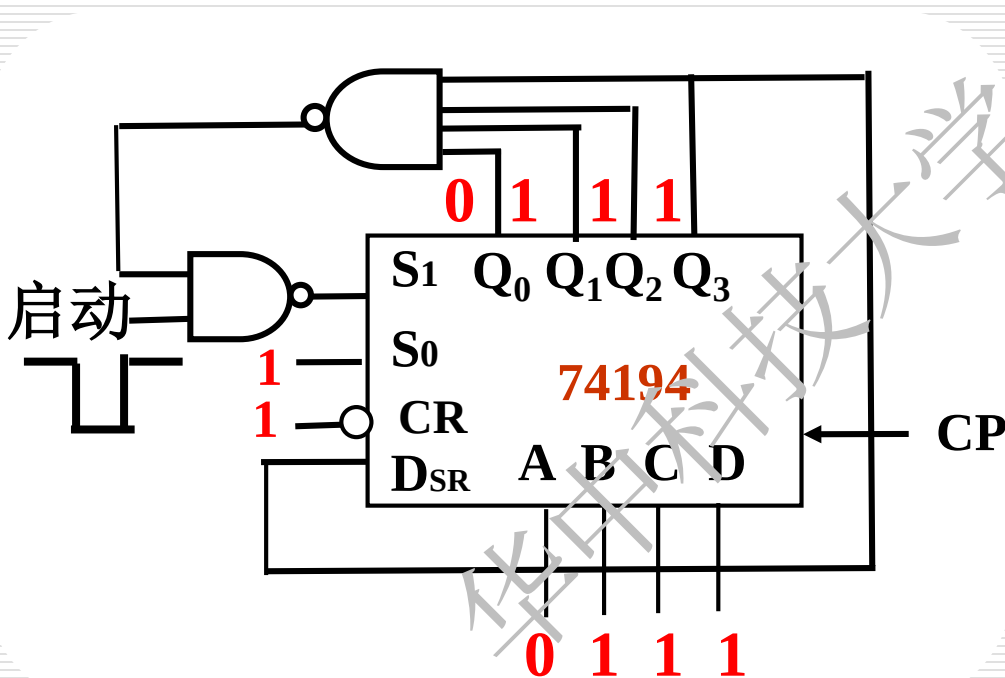
左移---高位向低位移动



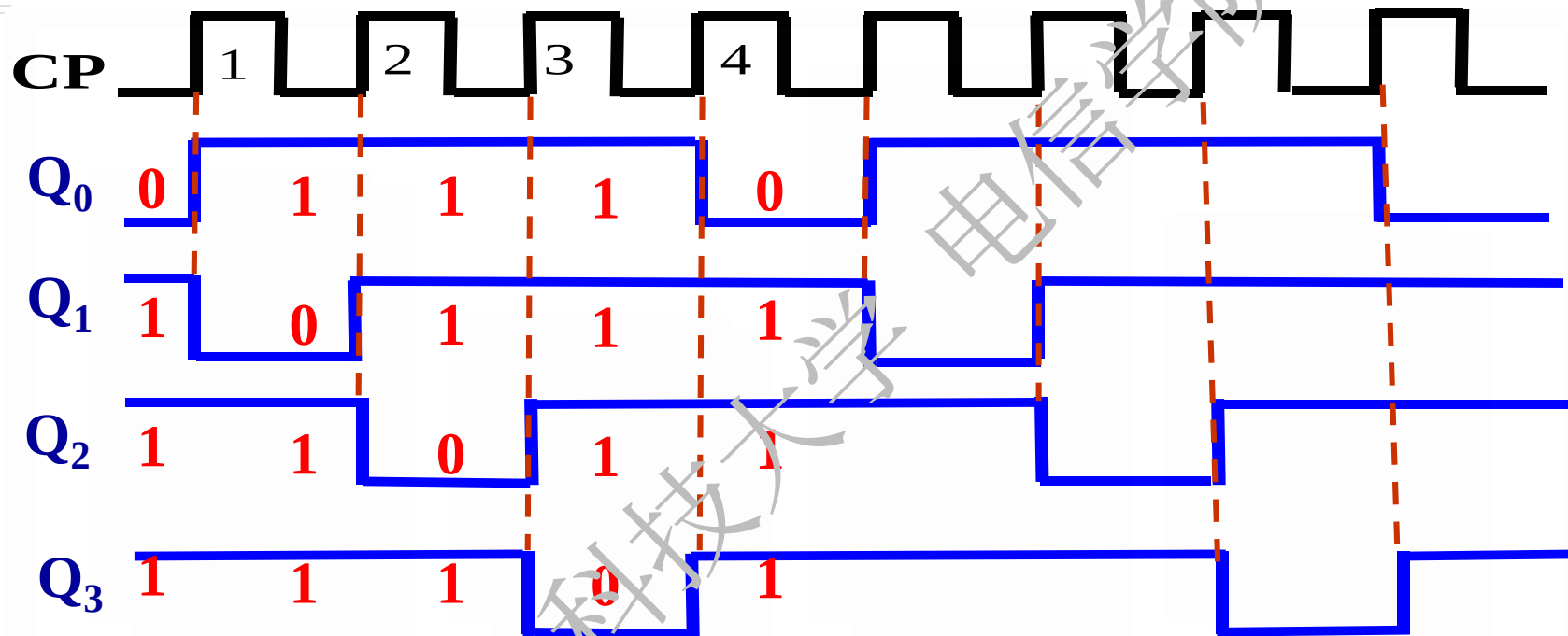
例：时序脉冲产生器。电路如图所示。画出 $Q_0—Q_3$ 波形，分析逻辑功能。

解：启动信号为0： $S_1=1$ $S_0=1$, 同步置数 $Q_A \sim Q_D = 0111$

启动信号为1后： $S_1=0$ $S_0=1$, 低位移向高位状态, $Q_3 = D_{SR}$

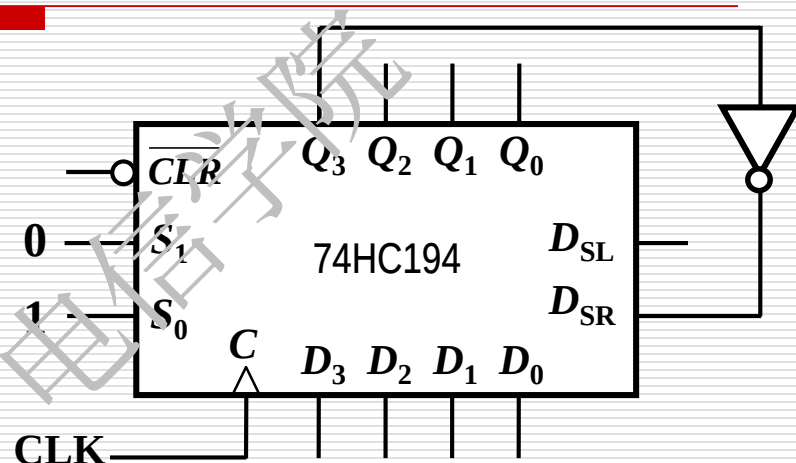
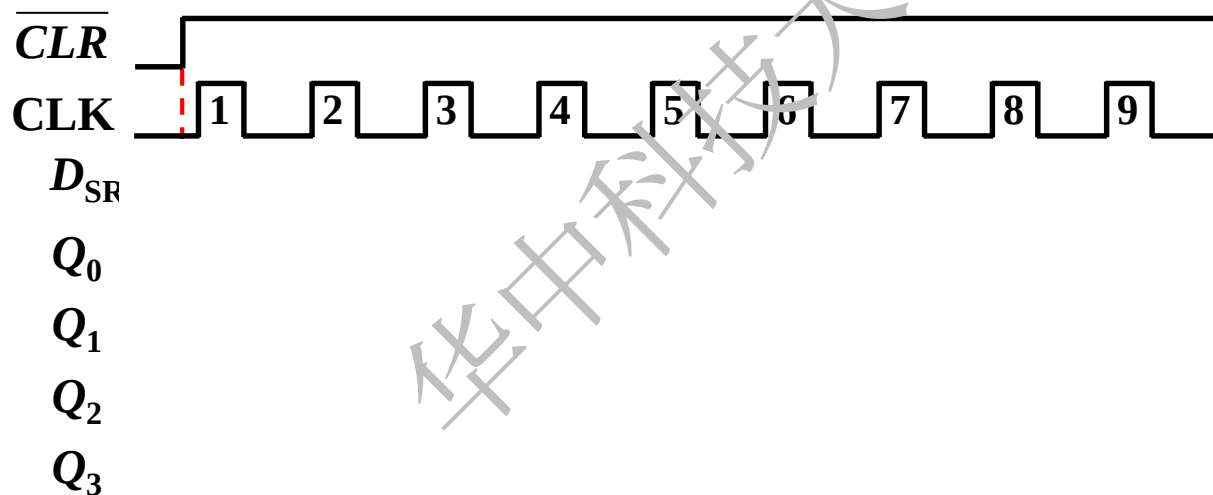


因为 $Q_0 \sim Q_3$ 总有一个为0， $S_1 S_0 = 01$ ，则74194始终工作在低位向高位移动循环移位的状态。



□ 例2

分析如图所示电路画出图中
 D_{SL} , Q_0 , Q_1 , Q_2 , Q_3 的工作波形



Clock pulse	$Q_3Q_2Q_1Q_0$
0	0 0 0 0
1	0 0 0 1
2	0 0 1 1
3	0 1 1 1
4	1 1 1 1
5	1 1 1 0
6	1 1 0 0
7	1 0 0 0
8	0 0 0 0

这是一个 **Johson** 计数器.

共有8个计数状态

6.5.2 计 数 器

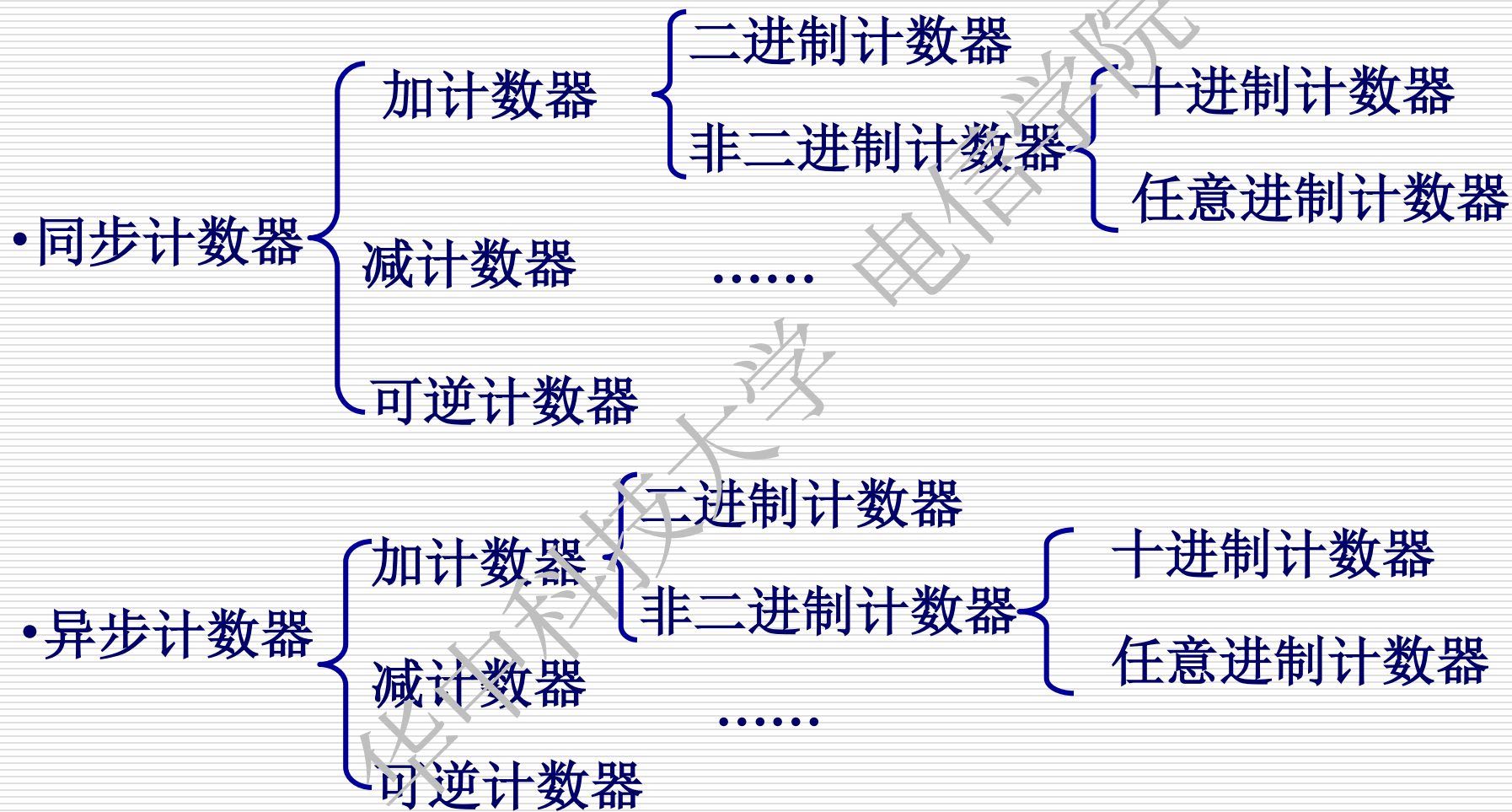
概 述

(1) 计数器的逻辑功能

计数器的基本功能是对输入时钟脉冲进行计数。它也可用于分频、定时、产生节拍脉冲和脉冲序列及进行数字运算等等。

(2) 计数器的分类

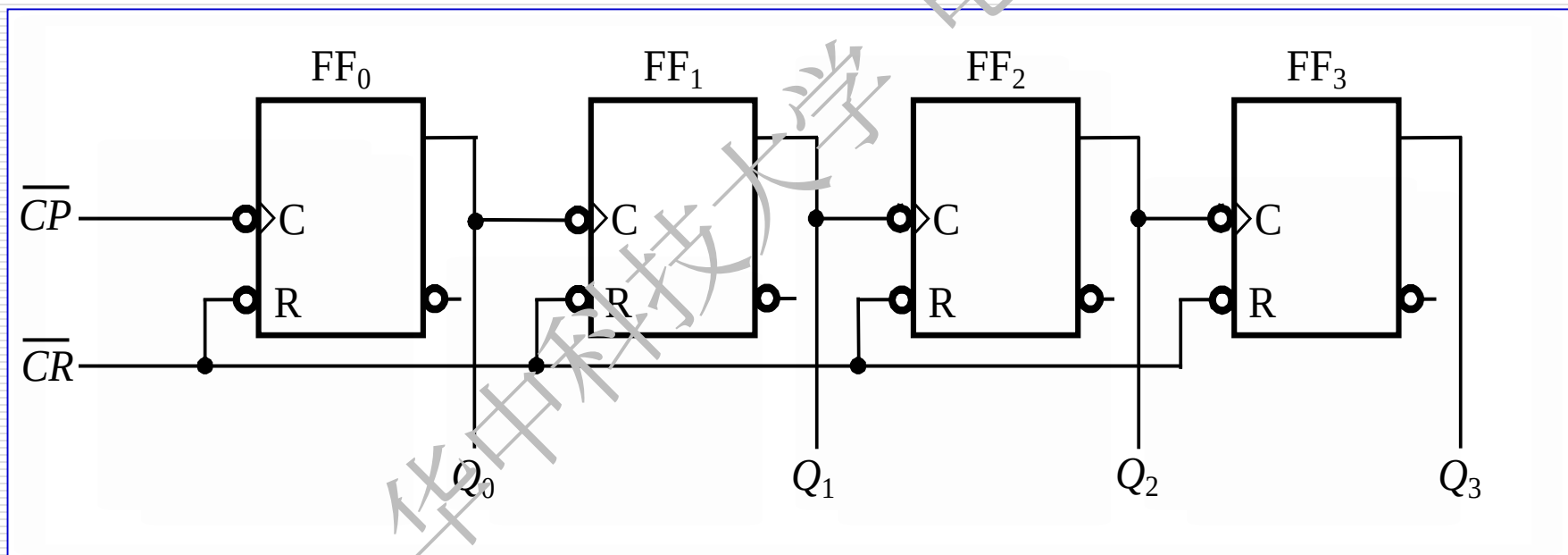
- 按脉冲输入方式，分为同步和异步计数器
- 按进位体制，分为二进制、十进制和任意进制计数器
- 按逻辑功能，分为加法、减法和可逆计数器

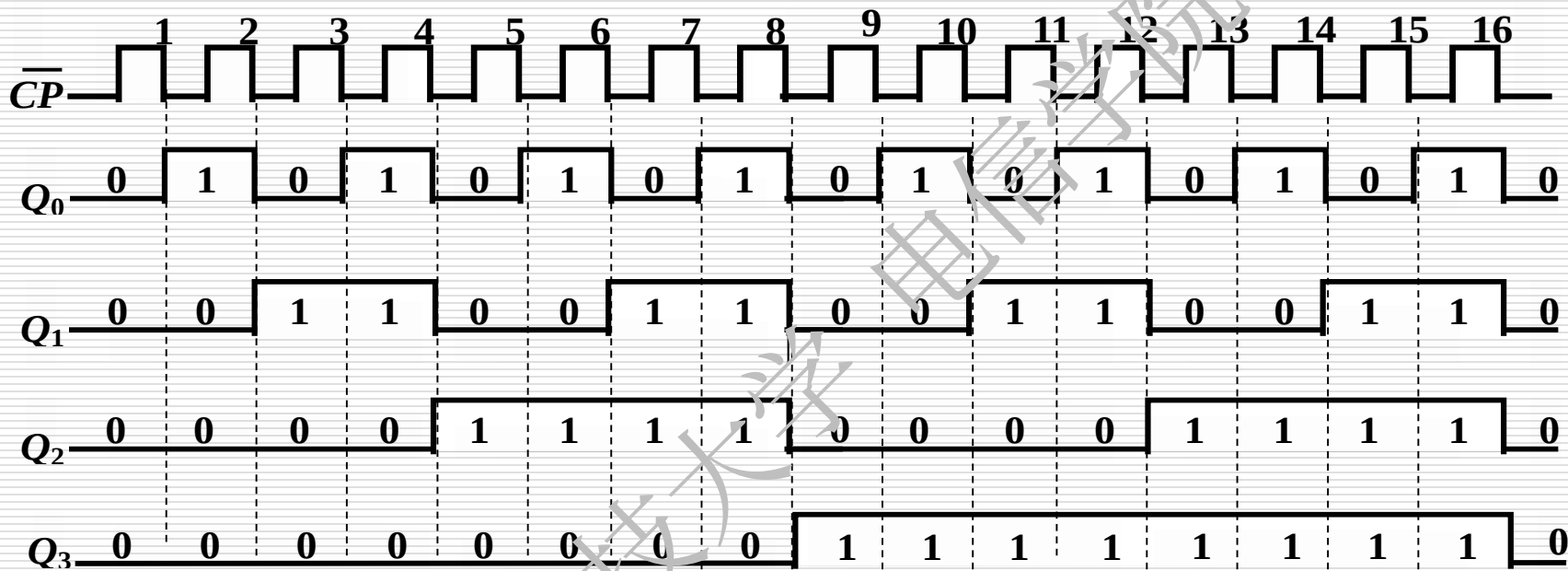


1、 二进制计数器

(1) 异步二进制计数器---4位异步二进制加法计数器

工作原理

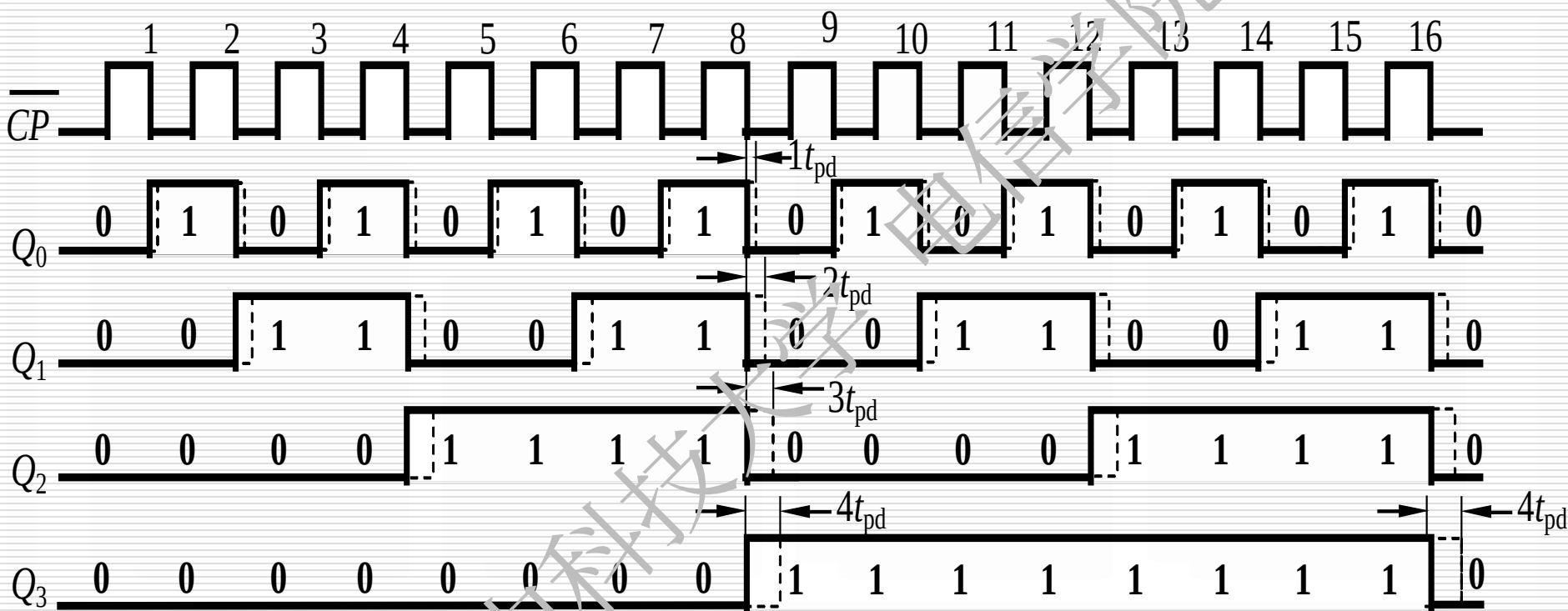




$$f_{Q_0} = \frac{1}{2} f_{CP} \quad f_{Q_1} = \frac{1}{4} f_{CP} \quad f_{Q_2} = \frac{1}{8} f_{CP} \quad f_{Q_3} = \frac{1}{16} f_{CP}$$

结论: ➤ 计数器的功能: 不仅可以计数也可作为分频器。

如考虑每个触发器都有 $1t_{pd}$ 的延时，电路会出现什么问题？



► 异步计数脉冲的最小周期 $T_{min} = n t_{pd}$ 。（ n 为位数）

(2) 二进制同步加计数器
(设计)

Q_0 在每个CP都翻转一次

FF_0 可采用 $T=1$ 的T触发器

Q_1 仅在 $Q_0=1$ 后的下一个CP到来时翻转

FF_1 可采用 $T=Q_0$ 的T触发器

Q_2 仅在 $Q_0=Q_1=1$ 后的下一个CP到来时翻转

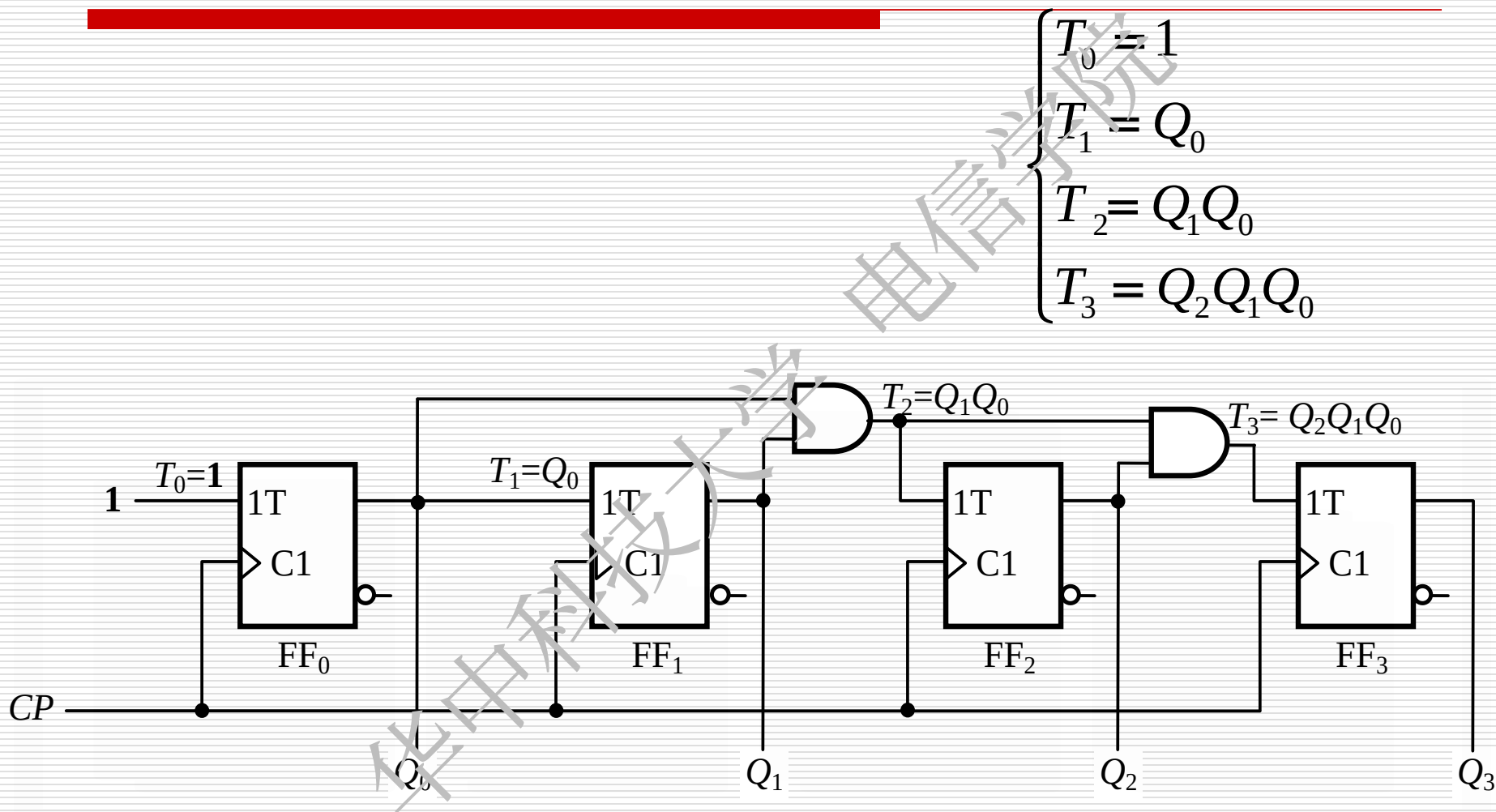
FF_2 可采用 $T=Q_0Q_1$ 的T触发器

Q_3 仅在 $Q_0=Q_1=Q_2=1$ 后的下一个CP到来时翻转

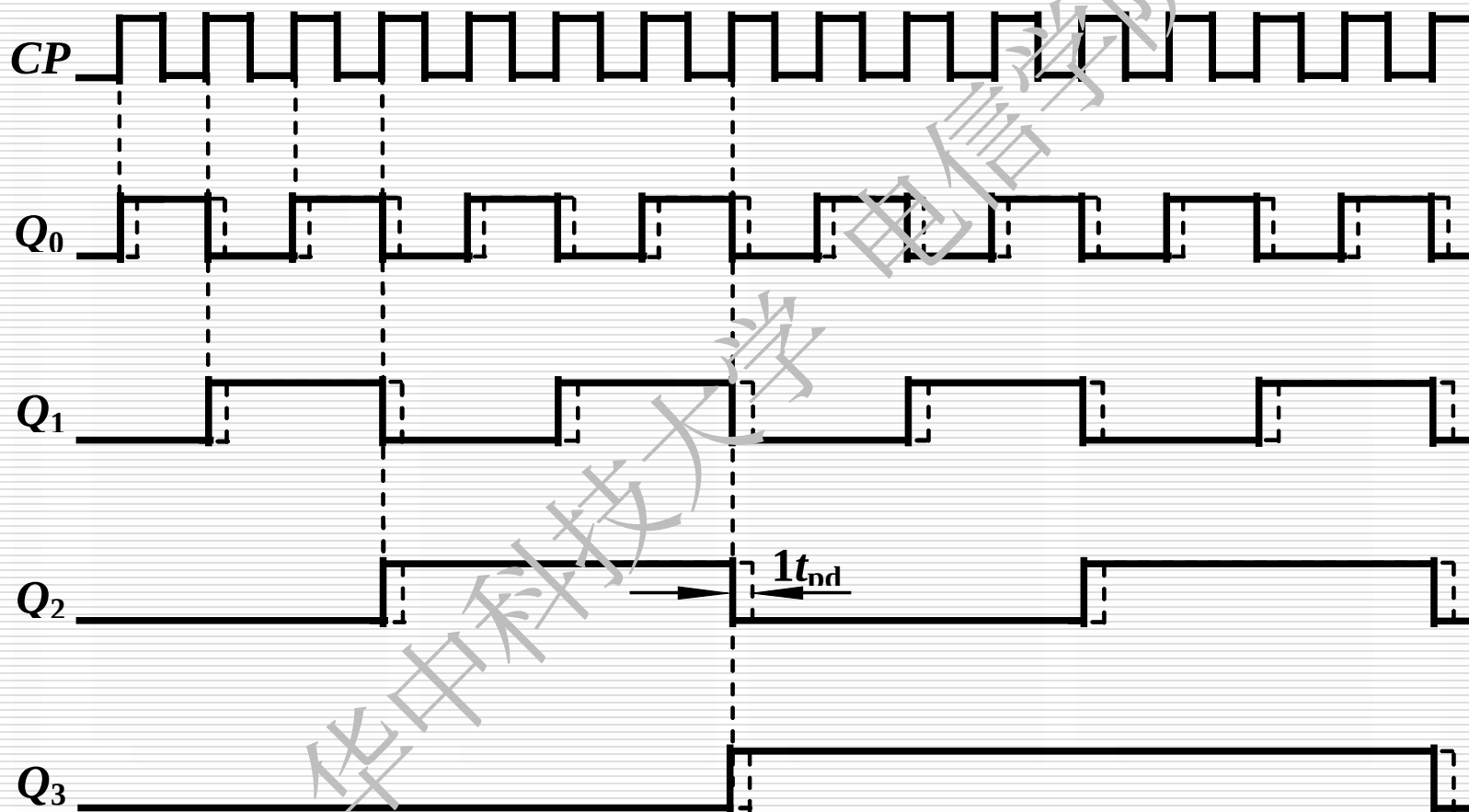
FF_3 可采用 $T=Q_0Q_1Q_2$ 的T触发器

计数顺序	电路状态				进位输出
	Q_3	Q_2	Q_1	Q_0	
0	0	0	0	0	0
1	0	0	0	1	0
2	0	0	1	0	0
3	0	0	1	1	0
4	0	1	0	0	0
5	0	1	0	1	0
6	0	1	1	0	0
7	0	1	1	1	0
8	1	0	0	0	0
9	1	0	0	1	0
10	1	0	1	0	0
11	1	0	1	1	0
12	1	1	0	0	0
13	1	1	0	1	0
14	1	1	1	0	0
15	1	1	1	1	1
16	0	0	0	0	0

(a) 4位二进制同步加计数器逻辑图---由T触发器构成

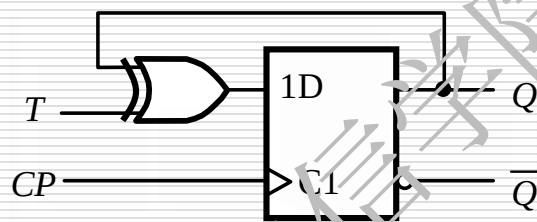


4位二进制同步加计数器时序图

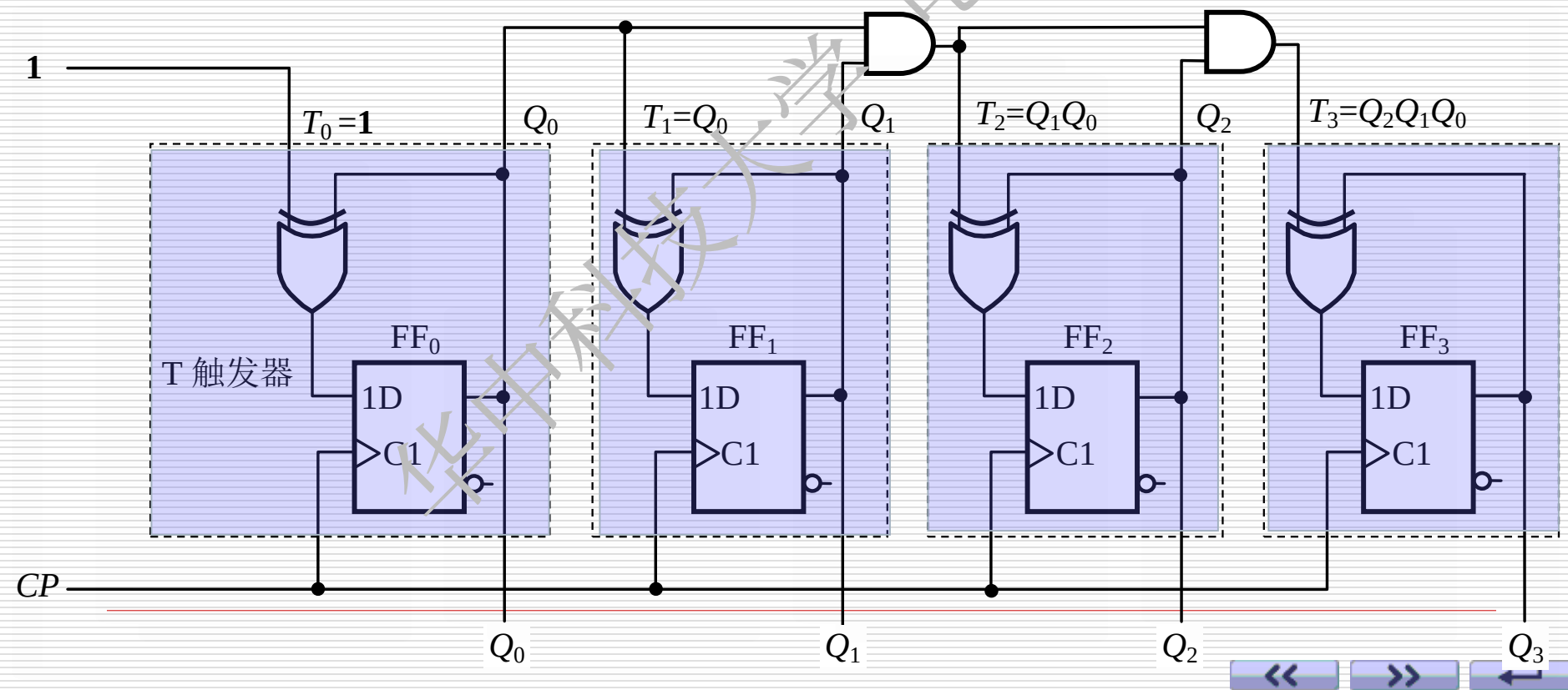


(b) 4位二进制同步加计数器逻辑图---由D触发器构成

$$Q^{n+1} = D = T\overline{Q}^n + \overline{T}Q^n$$

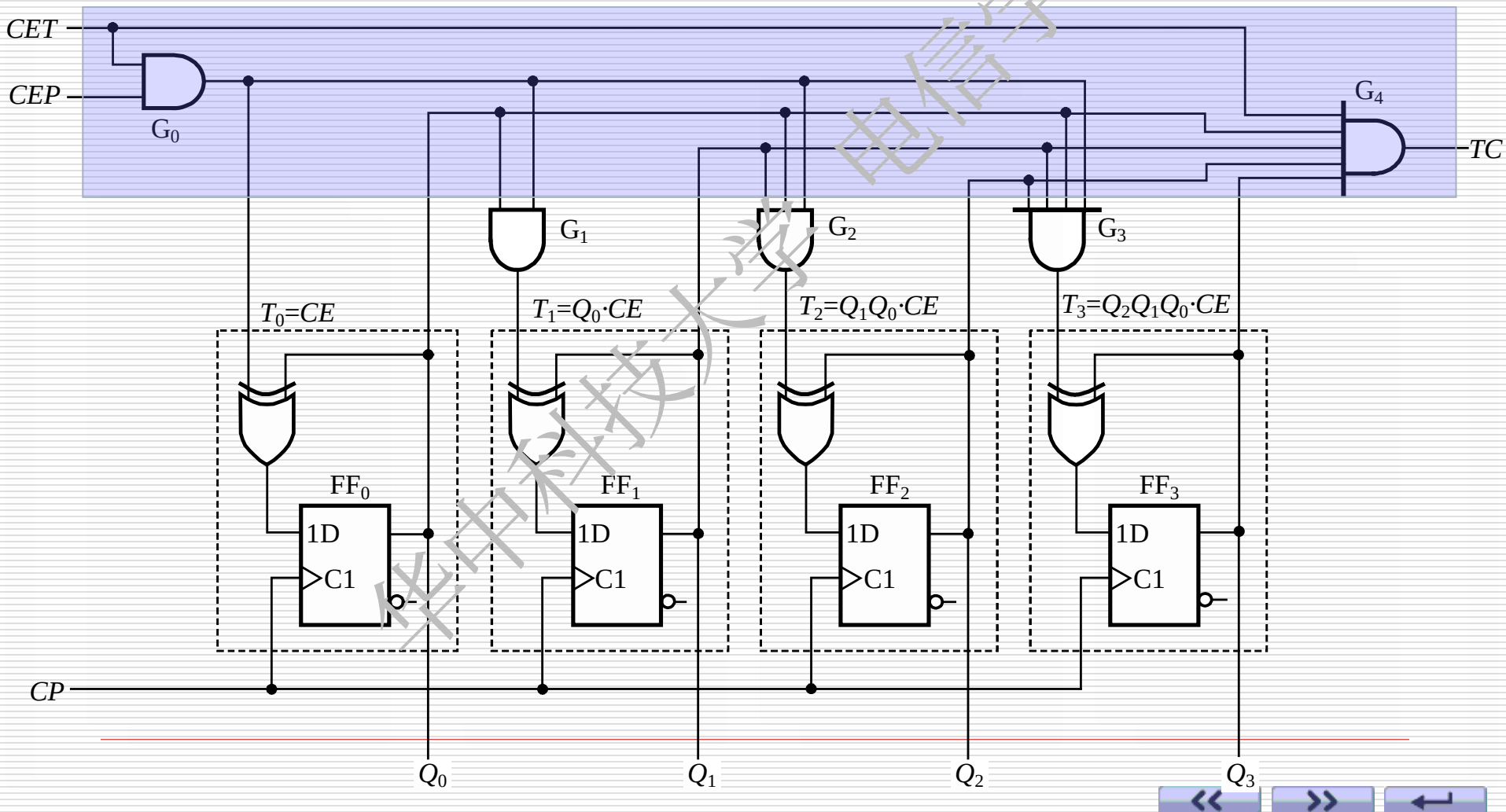


$$\begin{cases} T_0 = 1 \\ T_1 = Q_0 \\ T_2 = Q_1 Q_0 \\ T_3 = Q_2 Q_1 Q_0 \end{cases}$$

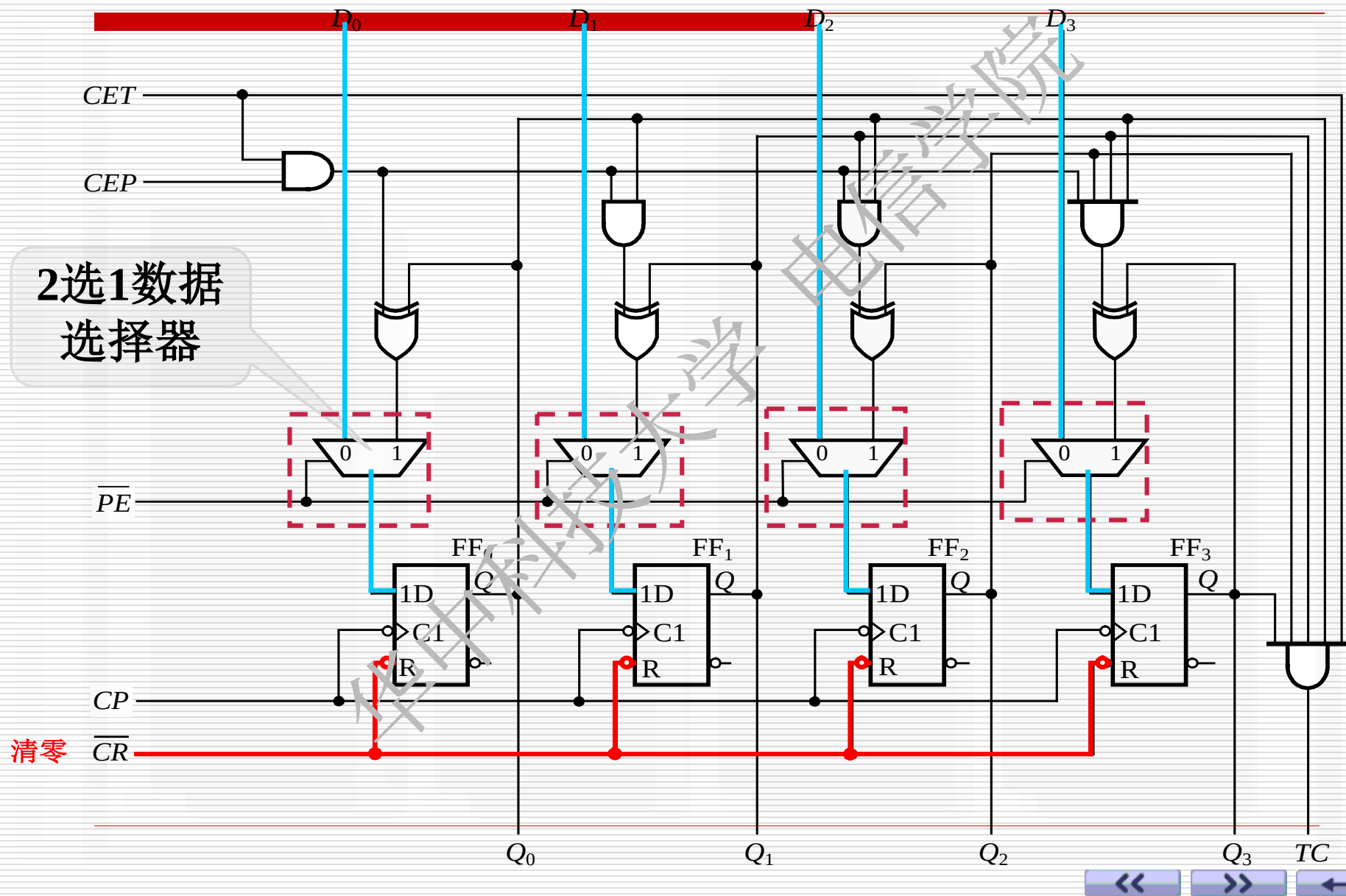


(c) 计数使能和并行进位

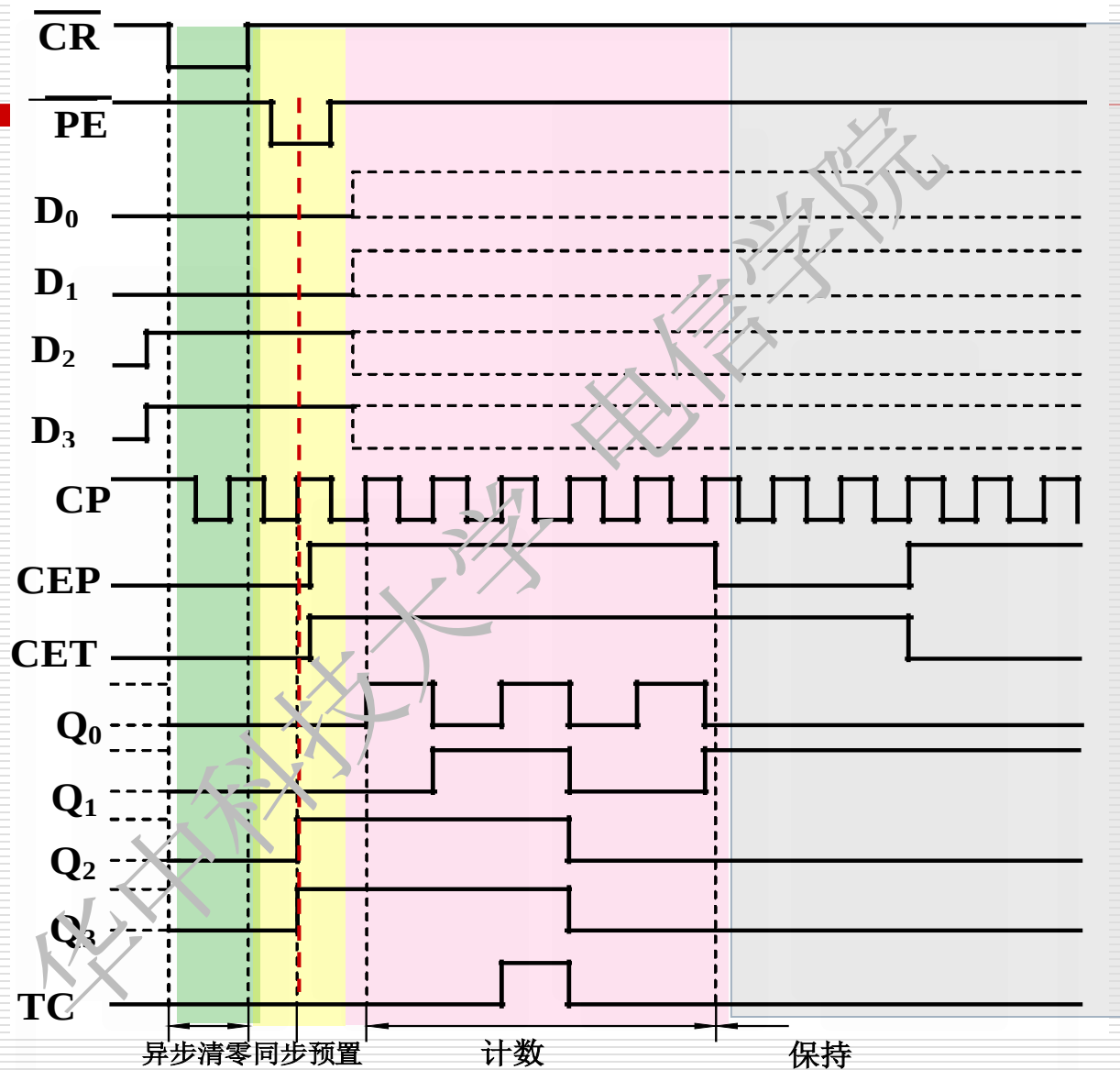
CET 、 CEP 为计数使能，并行进位 $TC=Q_3Q_2Q_1Q_0 \cdot CET$



(d) 异步清零和同步并行置数



(2) 时序图



$$TC = CET \cdot Q_3 Q_2 Q_1 Q_0$$

74LVC161逻辑功能表

输 入								输 出					
清零	预置	使能		时钟	预置数据输入				计 数				进位
$\overline{CR}$	$\overline{PE}$	CEP	CE_T	CP	D_3	D_2	D_1	D_0	Q_3	Q_2	Q_1	Q_0	TC
L	×	×	×	×	×	×	×	×	L	L	L	L	L
H	L	×	×	↑	D_3	D_2	D_1	D_0	D_3	D_2	D_1	D_0	*
H	H	L	×	×	×	×	×	×	保 持				*
H	H	×	L	×	×	×	×	×	保 持				*
H	H	H	H	↑	×	×	×	×	计 数				*

$\overline{CR}$ 的作用?

$\overline{PE}$ 的作用?

1、设置初始状态为0000

1、设置初始状态为 $D_3D_2D_1D_0$

2、在计数过程中置0，去除若干状态,构成任意进制计数器

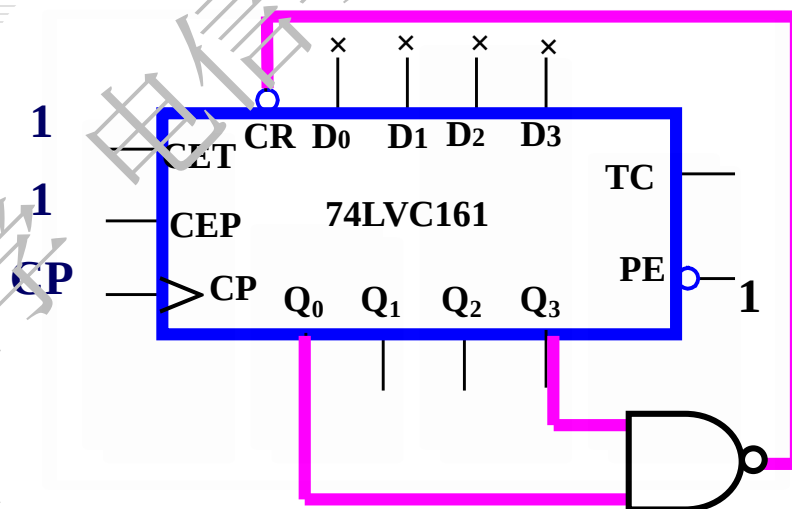
2、在计数过程中置数，去除若干状态，构成任意进制计数器

3) 应用

例 用74LVC161构成九进制加计数器。

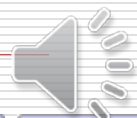
(a) 反馈清零法：利用异步置零输入端，在M进制计数器的计数过程中，跳过M-N个状态，得到N进制计数器的方法。

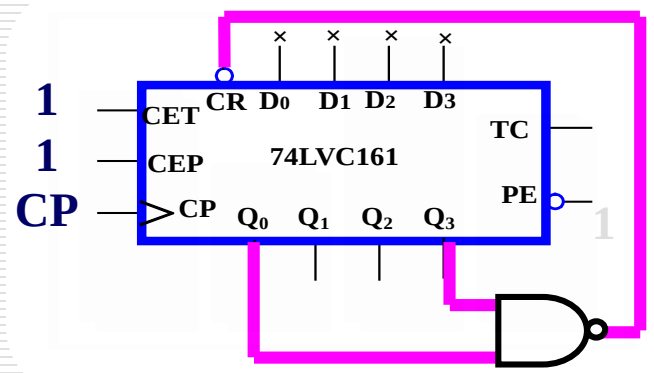
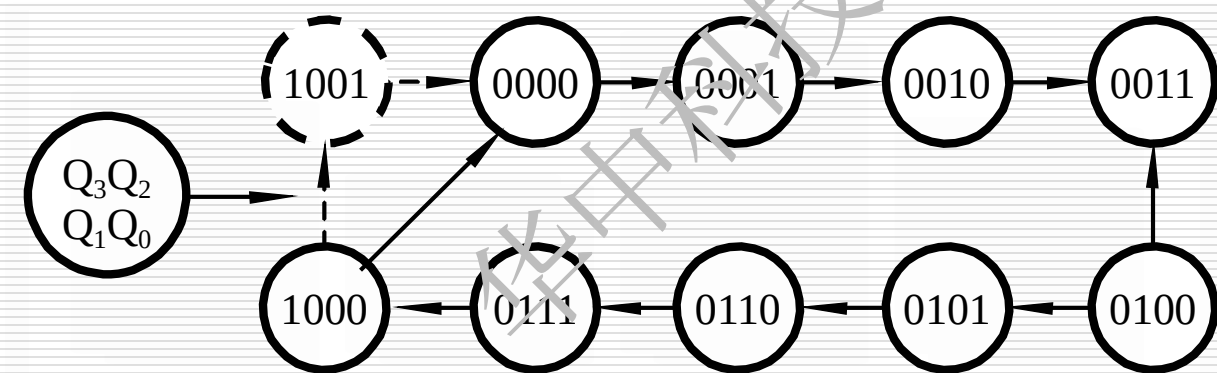
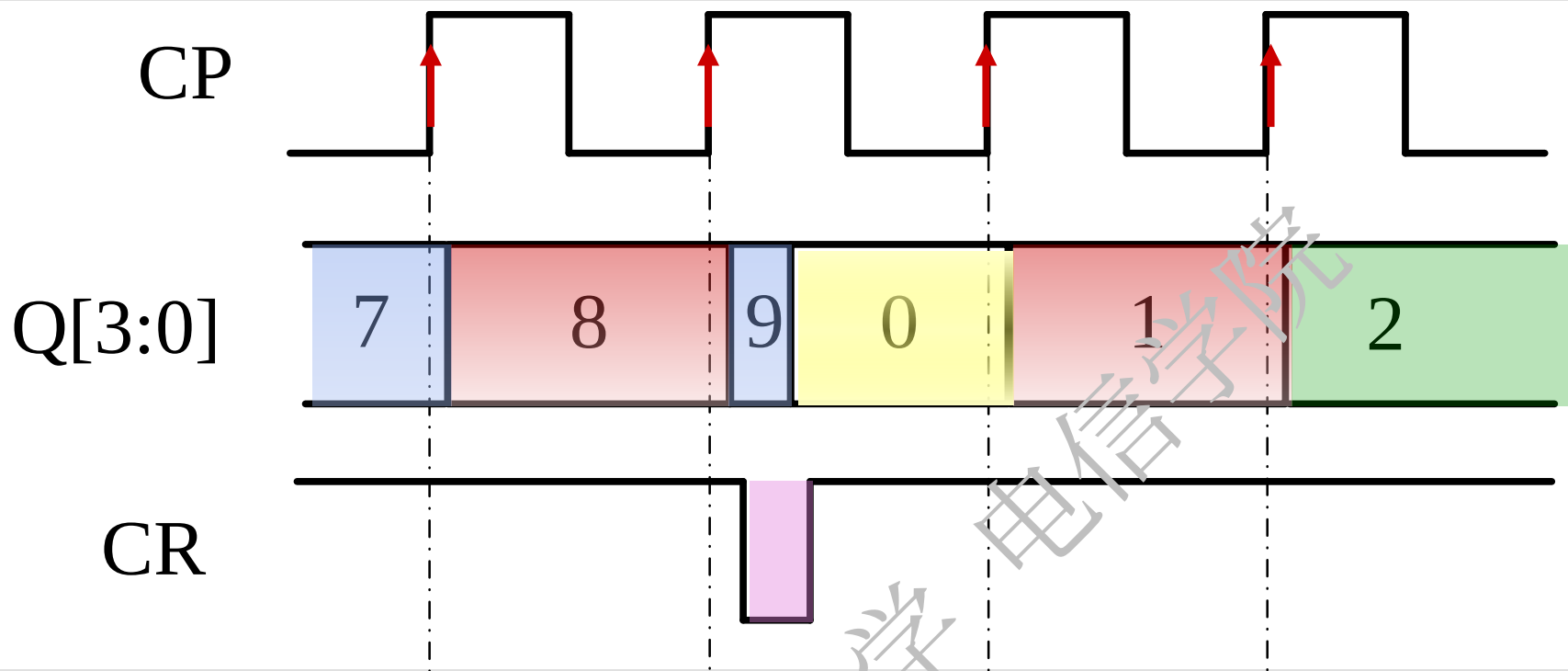
CP	Q ₃	Q ₂	Q ₁	Q ₀
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
...	...	...	...	...
8	1	0	0	0
9	1	0	0	1
...	...	...	...	...
15	1	1	1	1



$$CR = \overline{Q_0} \cdot Q_3 = 0$$

设法跳过16-9=7个状态

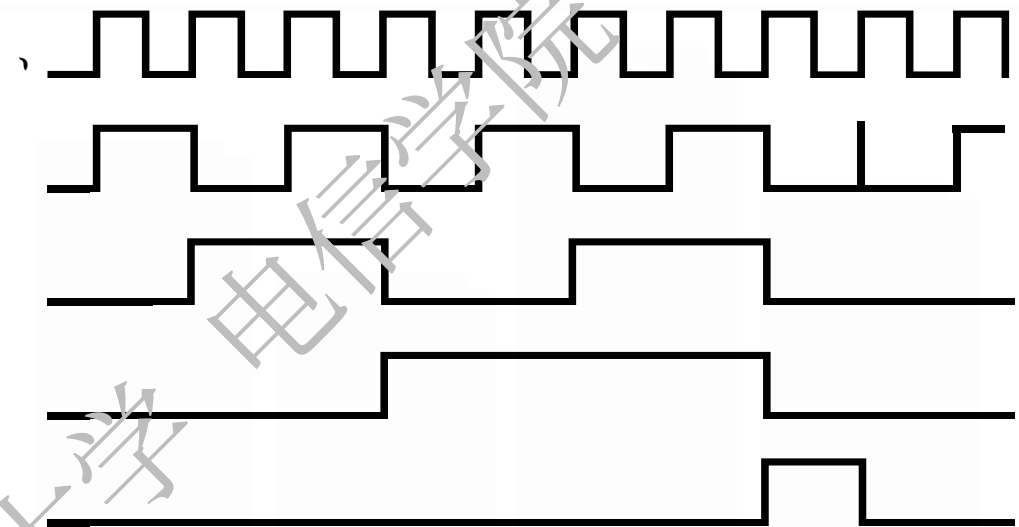
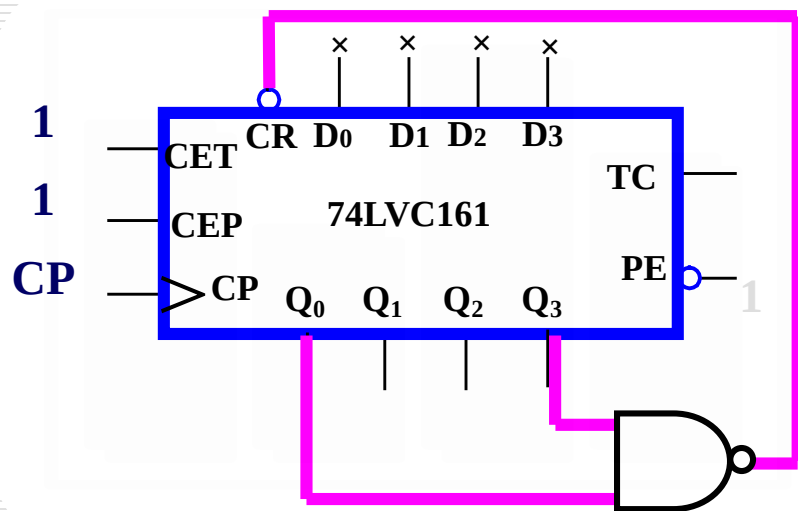




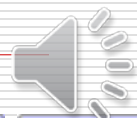
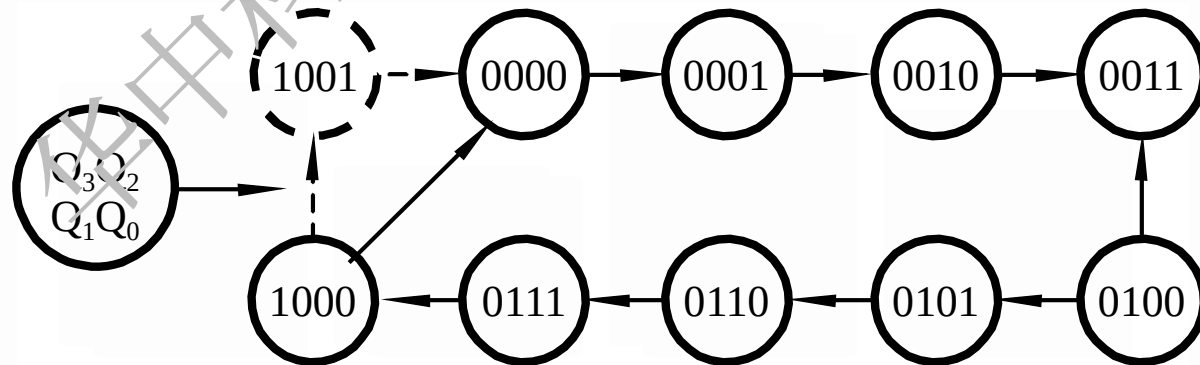
状态图



工作波形



状态图

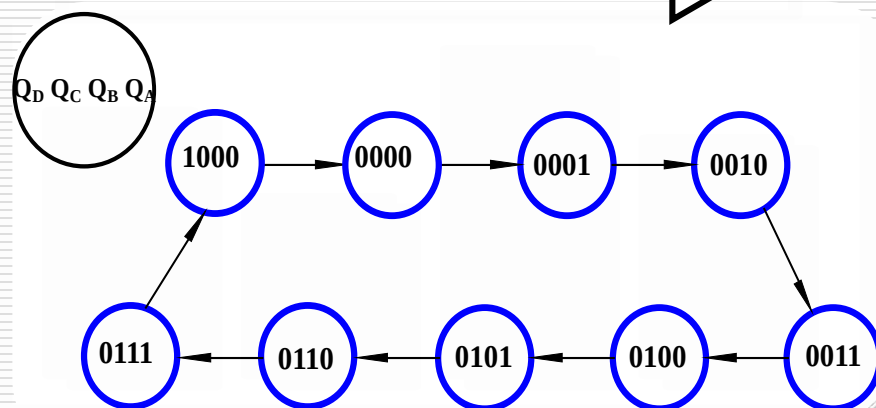
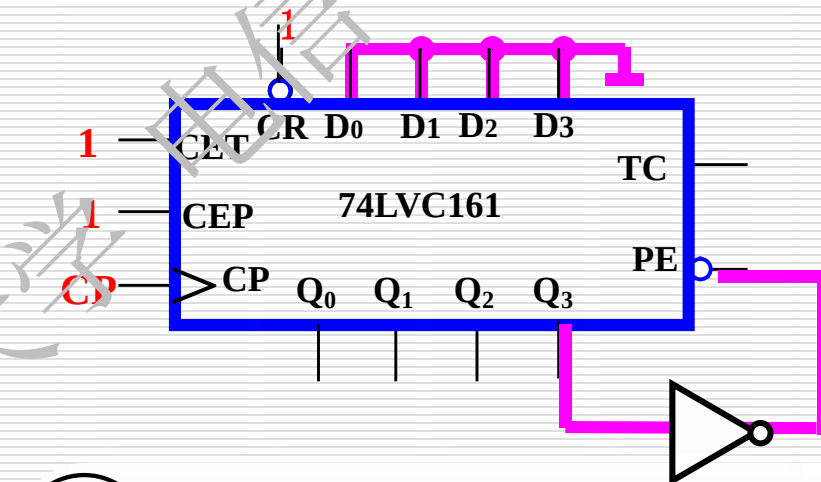


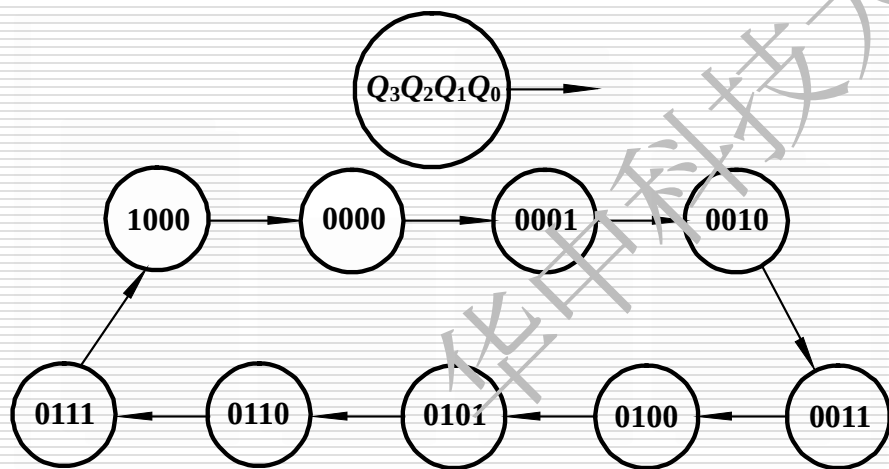
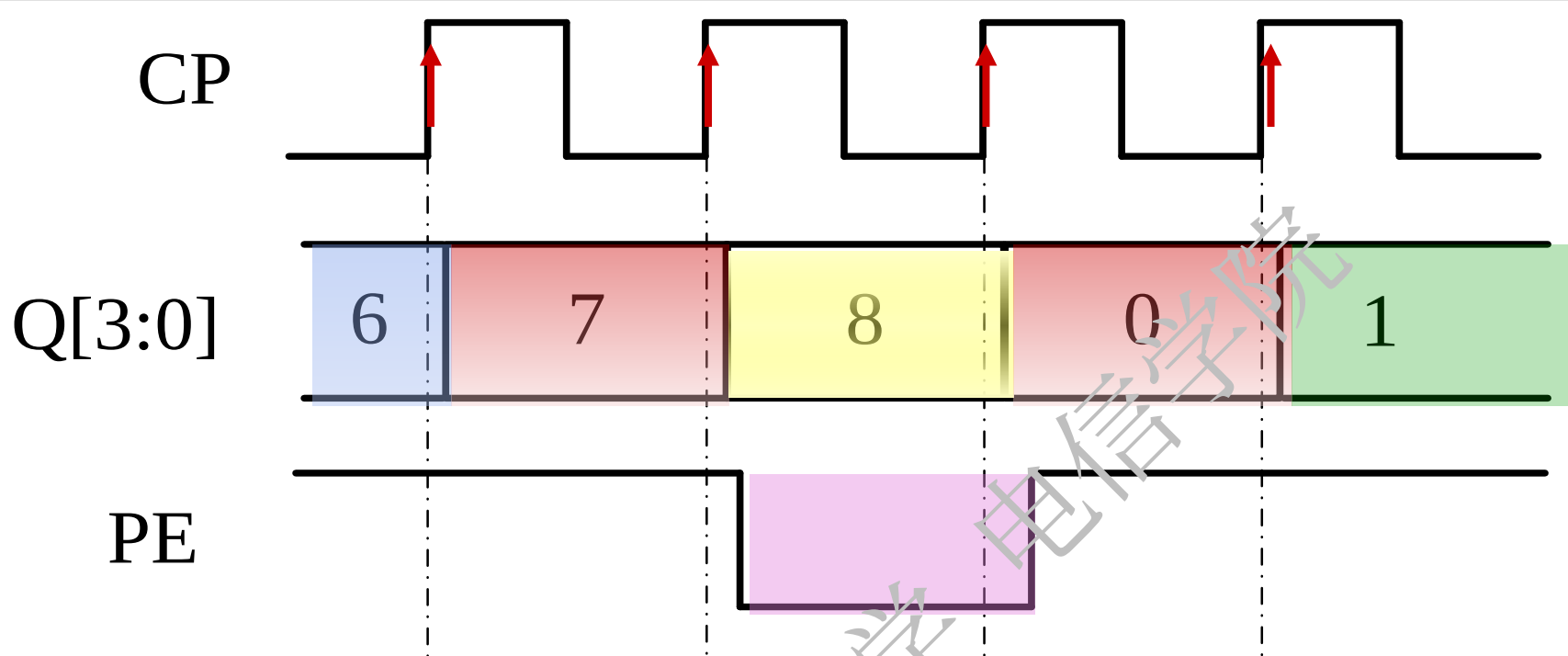
利用同步置数端构成九进制计数器

(b) 反馈置数法:利用同步置数端, 在M进制计数器的计数过程中, 跳过M-N个状态, 得到N进制计数器的方法。

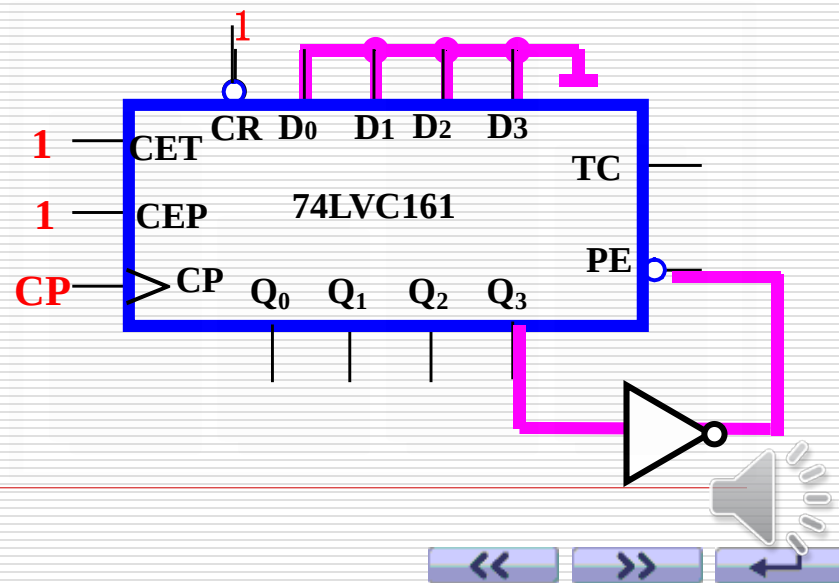
CP	Q ₃	Q ₂	Q ₁	Q ₀
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
...	...	...	...	...
8	1	0	0	0

$$PE = \overline{Q_3} = 0$$



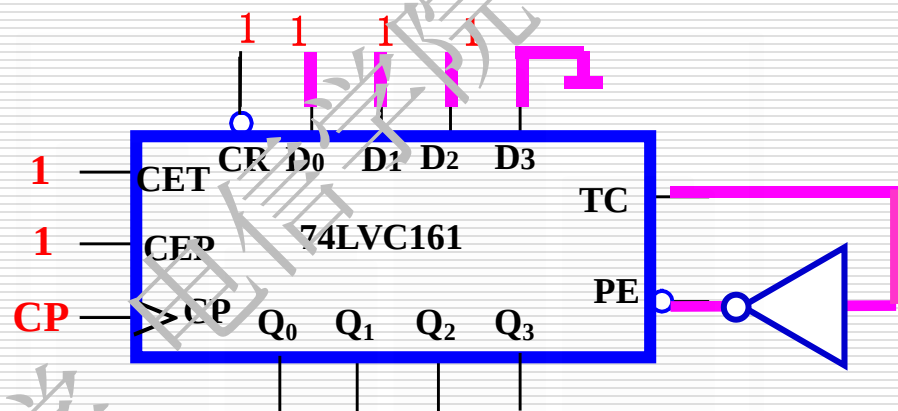


状态图

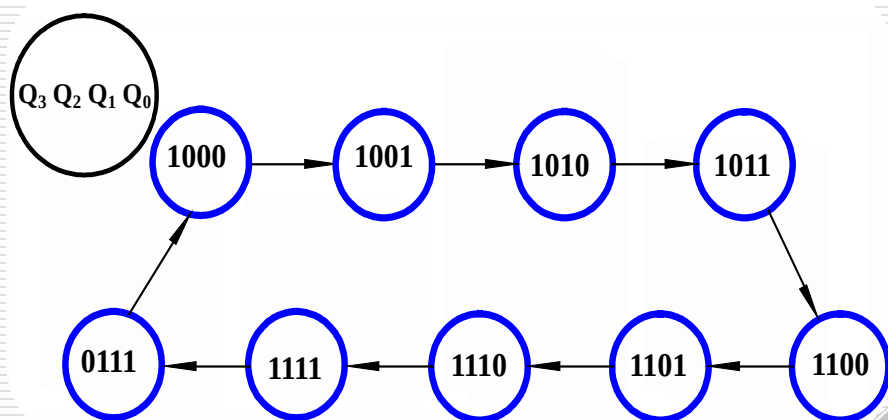


采用后九种状态作为有效状态，用反馈置数法 构成九进制加计数器。

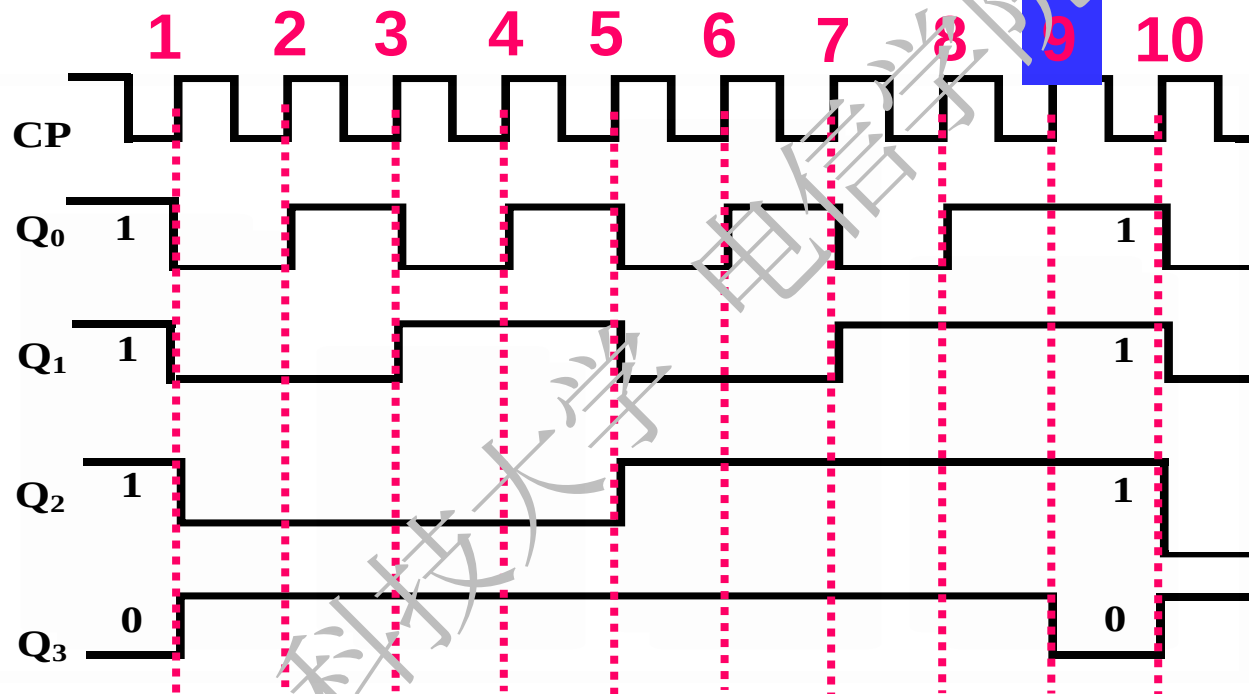
Q_3	Q_2	Q_1	Q_0
0	1	1	1
1	0	0	0
1	0	0	1
1	0	1	0
1	0	1	1
1	1	0	0
1	1	0	1
1	1	1	0
1	1	1	1



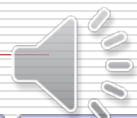
$$TC = CET \cdot Q_3 \cdot Q_2 \cdot Q_1 \cdot Q_0 = 1$$



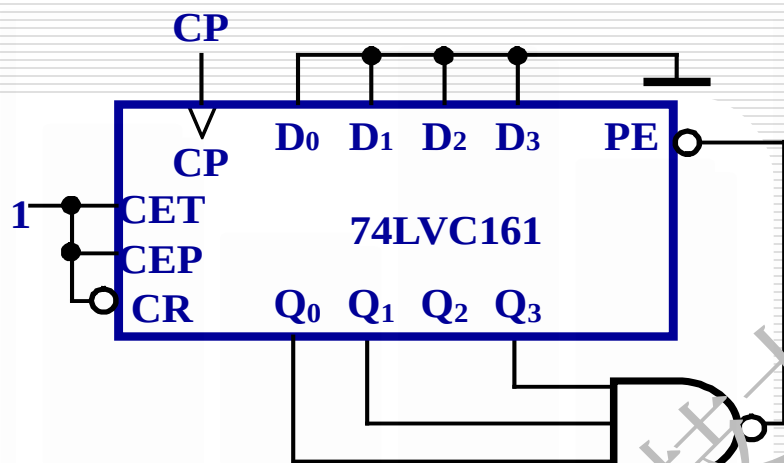
波形图:



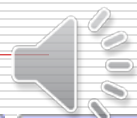
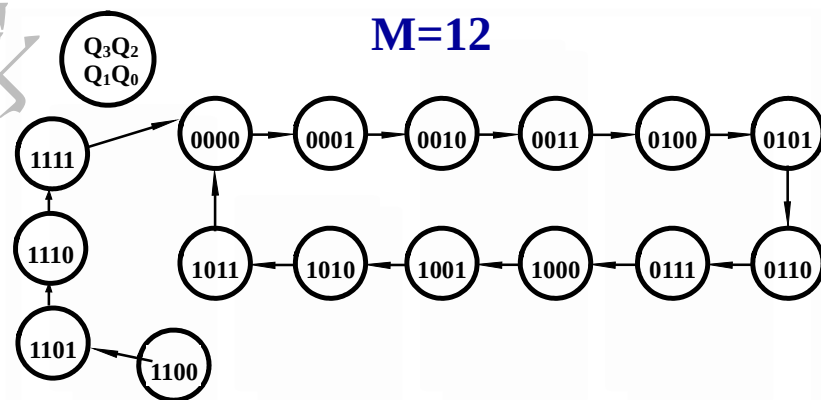
该计数器的模为9。



分析下图所示的时序逻辑电路，试画出其状态图和在CP脉冲作用下 Q_3 、 Q_2 、 Q_1 、 Q_0 的波形，并指出计数器的模是多少？



$$PE = \overline{Q_3 \cdot Q_1 \cdot Q_0} = 0$$



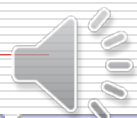
例 用74VC161组成256进制计数器。

解：设计思想

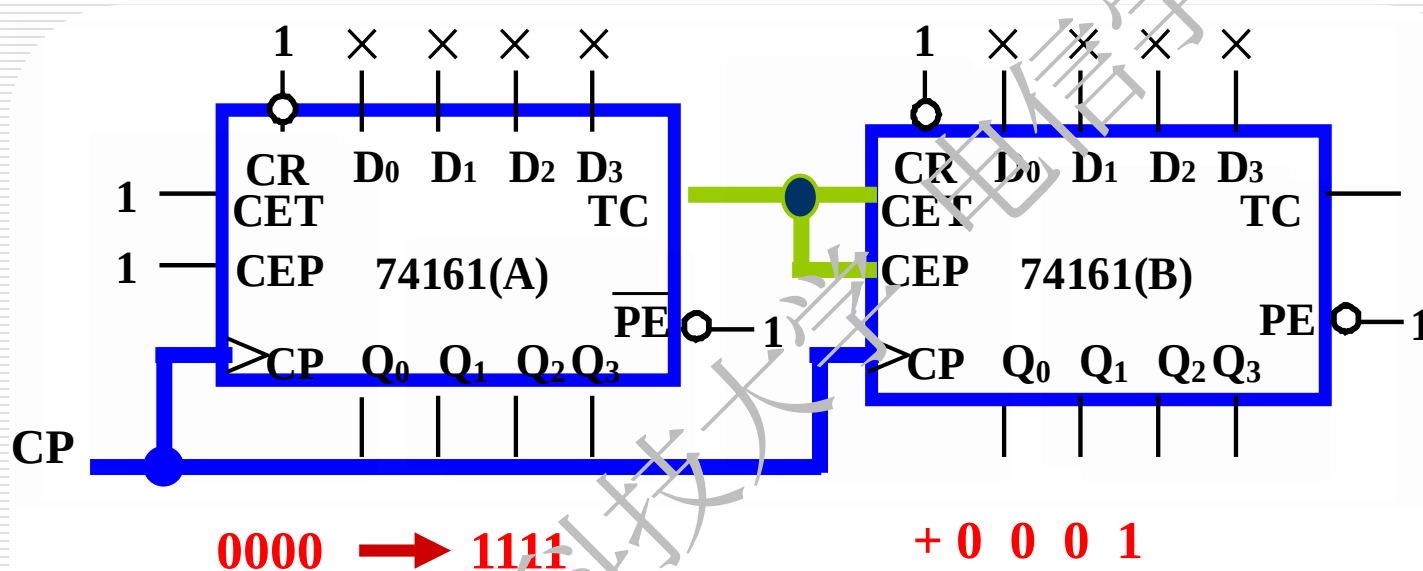
- 1片74161是16进制计数器
- $256 = 16 \times 16$
- 所以256进制计数器需用两片74161构成
- 片与片之间的连接通常有两种方式：

{ 并行进位 (低位片的进位信号作为高位片的使能信号)

{ 串行进位 (低位片的进位信号作为高位片的时钟脉冲，
即异步计数方式)



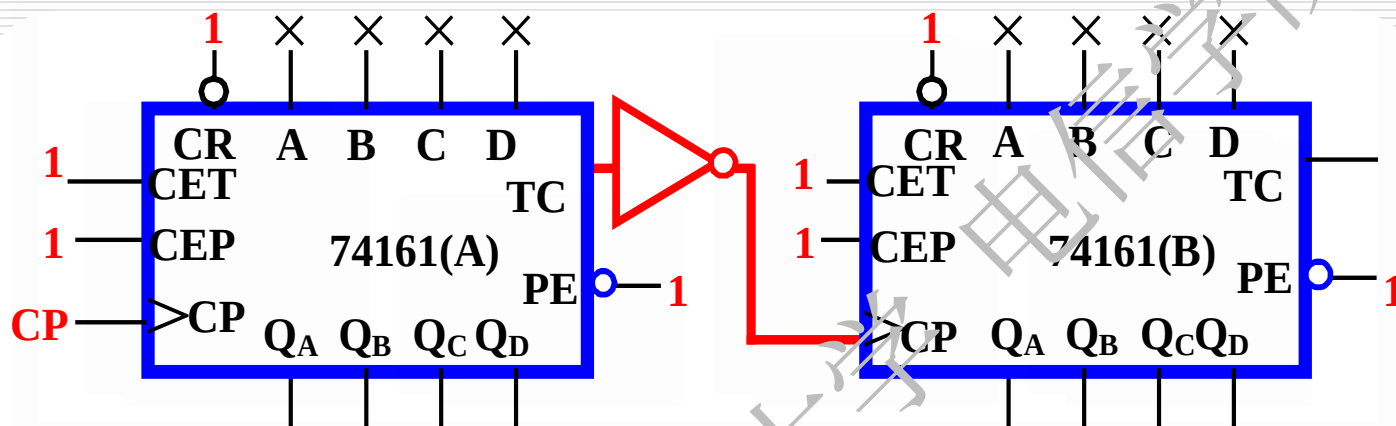
并行进位：低位片的进位作为高位片的使能



计数状态：0000 0000 ~ 1111 1111

$$N = 16 \times 16 = 256$$

串行进位：低位片的进位作为高位片的时钟



0000 → 1111

+ 0 0 0 1

计数状态：0000 0000 ~ 1111 1111

采用串行进位时，为什么低TC要经反相器后作为高位的CP？

小结

用集成计数器构成任意进制计数器的一般方法

1) $N < M$ 的情况：

(已有的集成计数器是 M 进制，需组成的是 N 进制计数器)

实现的方法：

反馈清零法

利用清零输入端，使电路计数到某状态时产生清零操作，清除 $M-N$ 个状态实现 N 进制计数器。

反馈置数法

利用计数器的置数功能，通过给计数器重复置入某个数码的方法减少 $(M-N)$ 个独立状态，实现 N 进制计数器。

2) $N > M$ 的情况

实现的方法：---采用多片M进制计数器构成。

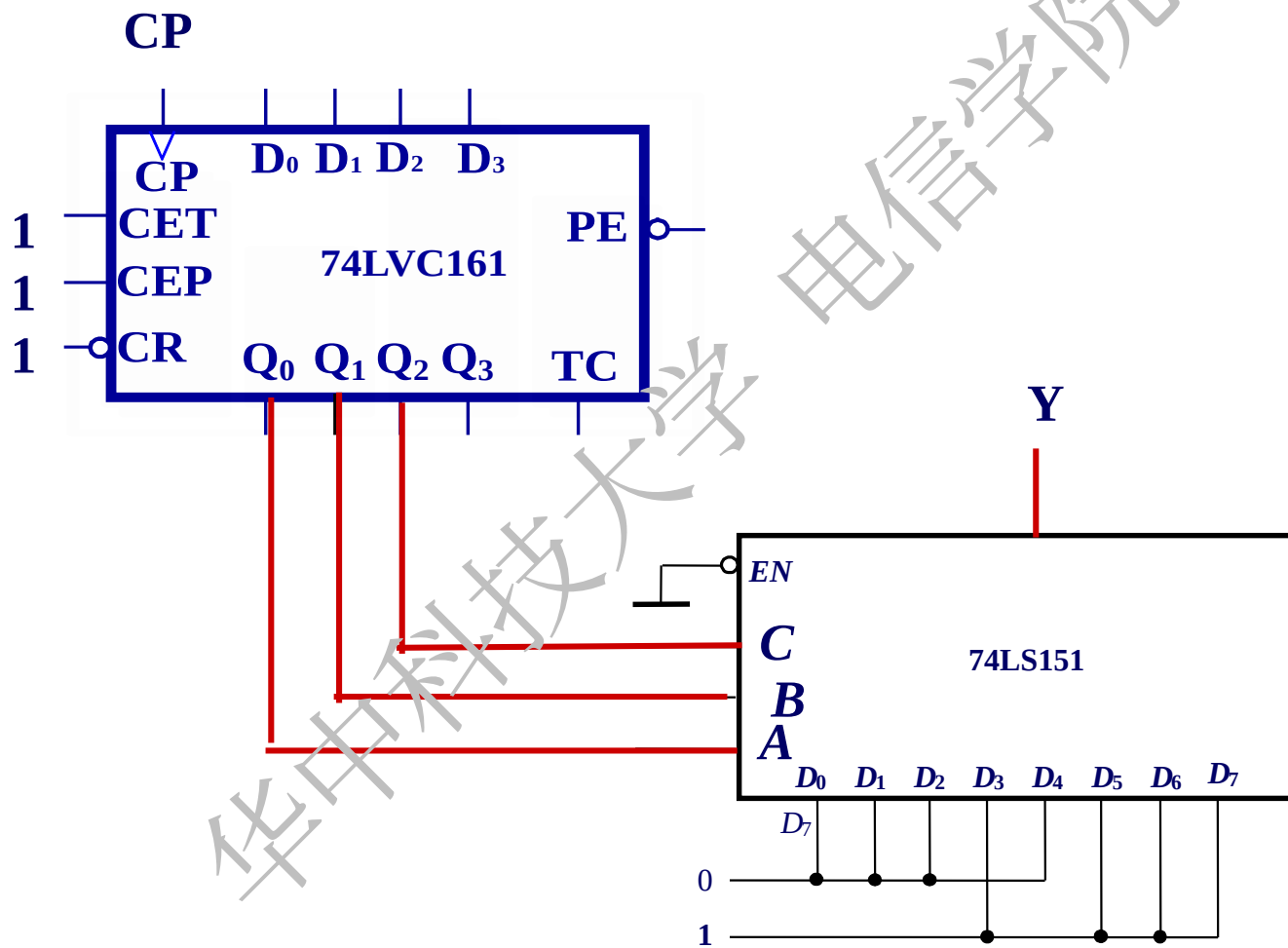
按芯片连接方式可分为：

(1) 串行进位方式：构成异步计数器

(2) 并行进位方式：构成同步计数器

应用举例 序列信号发生器

在CP的作用下，Y端产生00010111循环序列信号



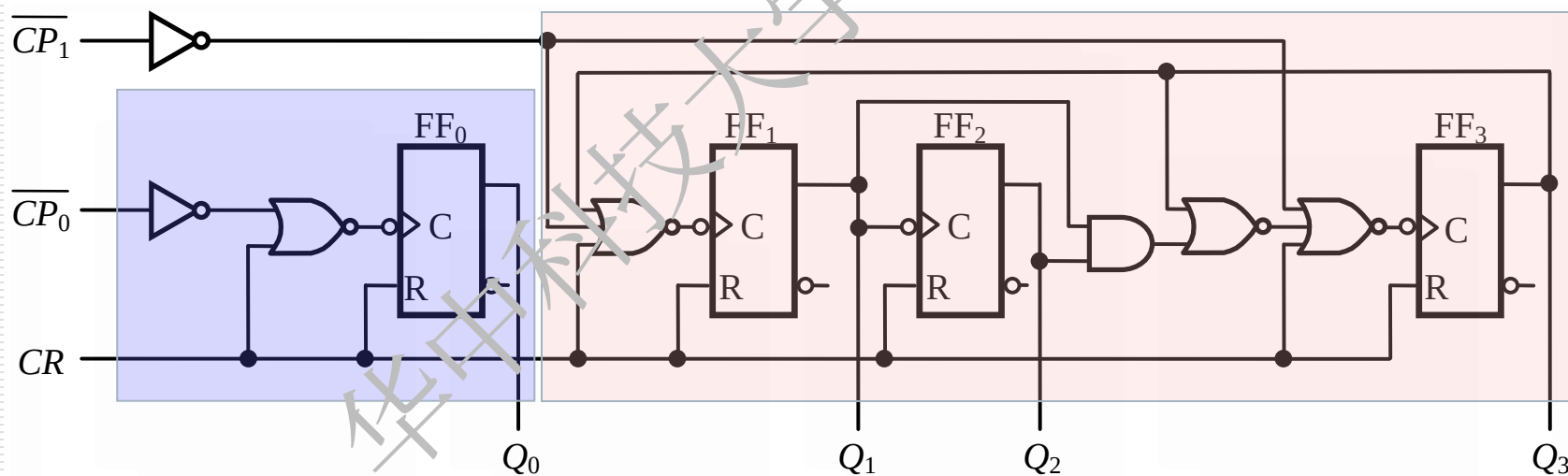
如要求Y端产生10110010循环序列信号，如何改变电路的连接？

1. 异步二-十进制计数器

将图中电路按以下两种方式连接：

- (1) $\overline{CP}_0$ 接计数脉冲信号，将 Q_0 与 $\overline{CP}_1$ 相连；
- (2) $\overline{CP}_1$ 接计数脉冲信号，将 Q_3 与 $\overline{CP}_0$ 相连

试分析它们的逻辑输出状态。



二进制

五进制

两种连接方式的状态表

计数顺序	连接方式1（8421码）				连接方式2（5421码）			
	Q_3	Q_2	Q_1	Q_0	Q_0	Q_3	Q_2	Q_1
0	0	0	0	0	0	0	0	0
1	0	0	0	1	0	0	0	1
2	0	0	1	0	0	0	1	0
3	0	0	1	1	0	0	1	1
4	0	1	0	0	0	1	0	0
5	0	1	0	1	1	0	0	0
6	0	1	1	0	1	0	0	1
7	0	1	1	1	1	0	1	0
8	1	0	0	0	1	0	1	1
9	1	0	0	1	1	1	0	0

第一个 $CP: Q_3Q_2Q_1Q_0=0010$,

第二个 $CP: Q_3Q_2Q_1Q_0=0100$,

第三个 $CP: Q_3Q_2Q_1Q_0=1000$,

第四个 $CP: Q_3Q_2Q_1Q_0=0001$,

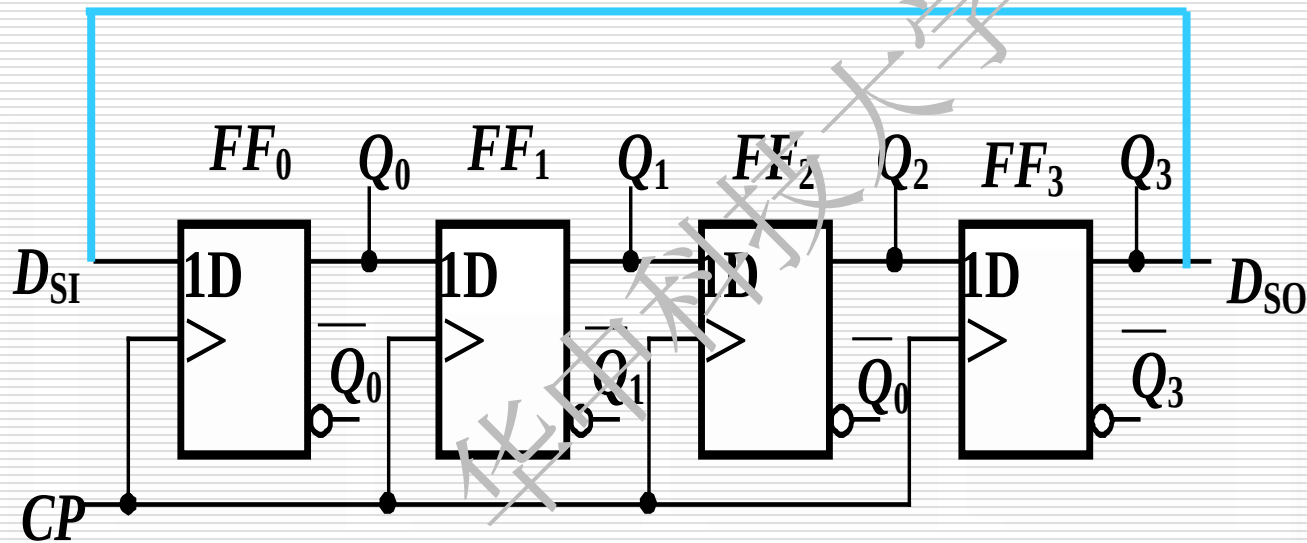
第五个 $CP: Q_3Q_2Q_1Q_0=0010$,

3. 环形计数器

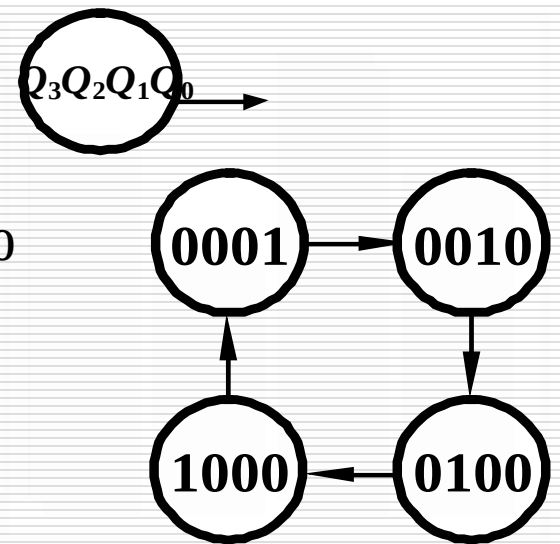
(1) 工作原理

① 基本环形计数器

置初态 $Q_3Q_2Q_1Q_0=0001$,



状态图



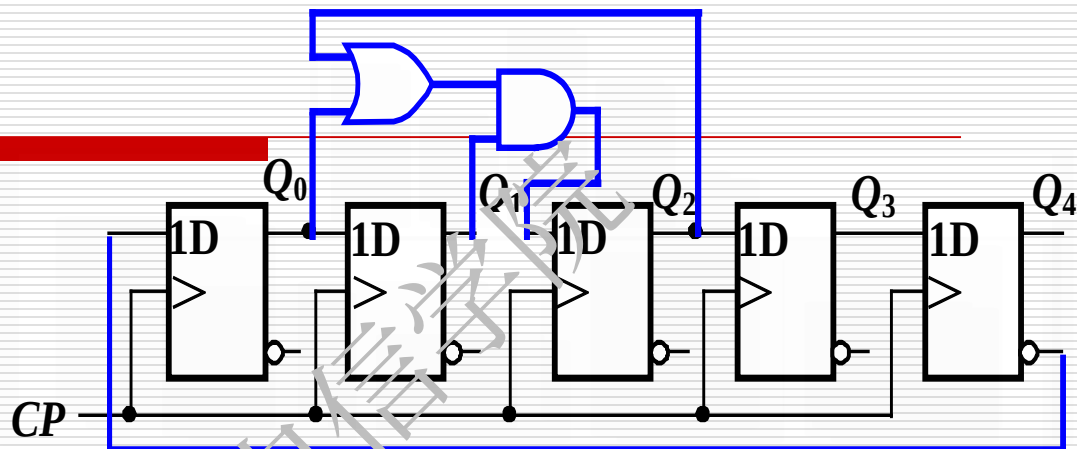
② 扭环形计数器

a、电路

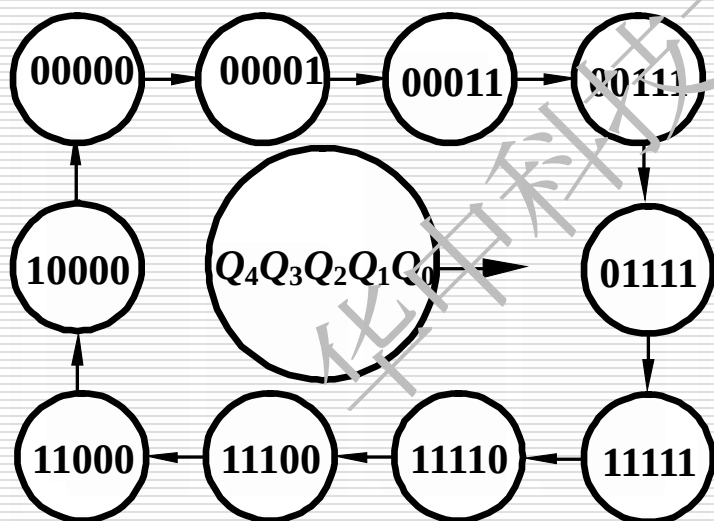
b、状态表

置初态 $Q_3Q_2Q_1Q_0=0001$,

c、状态图



状态编号	Q_4	Q_3	Q_2	Q_1	Q_0
0	0	0	0	0	0
1	0	0	0	0	1
2	0	0	0	1	1
3	0	0	1	1	1
4	0	1	1	1	1
5	1	1	1	1	1
6	1	1	1	1	0
7	1	1	1	0	0
8	1	1	0	0	0
9	1	0	0	0	0



状态编号	Q_4	Q_3	Q_2	Q_1	Q_0
0	0	0	0	0	0
1	0	0	0	0	1
2	0	0	0	1	1
3	0	0	1	1	1
4	0	1	1	1	1
5	1	1	1	1	1
6	1	1	1	1	0
7	1	1	1	0	0
8	1	1	0	0	0
9	1	0	0	0	0

$$Y_0 = \overline{Q_4} \overline{Q_0}$$

$$Y_1 = \overline{Q_1} Q_0$$

$$Y_2 = \overline{Q_2} Q_1$$

$$Y_3 = \overline{Q_3} Q_2$$

$$Y_4 = \overline{Q_4} Q_3$$

$$Y_5 = Q_4 Q_0$$

$$Y_6 = Q_1 \overline{Q_0}$$

$$Y_7 = Q_2 \overline{Q_1}$$

$$Y_8 = Q_3 \overline{Q_2}$$

$$Y_9 = Q_4 \overline{Q_3}$$

译码电路简单, 且不会出现竞争冒险

6.6 简单的时序可编程逻辑器件(GAL)

6.6.1 GAL的结构

6.6.2 GAL的输出逻辑宏单元

6.6.3 GAL的控制字

1. 时序可编程逻辑器件的主要类型

(1) 通用阵列逻辑 (GAL)

在PLA和PAL基础上发展起来的增强型器件.电路设计者可根据需要编程,对宏单元的内部电路进行不同模式的组合,从而使输出功能具有一定的灵活性和通用性。

(2) 复杂可编程逻辑器件 (CPLD)

集成了多个逻辑单元块,每个逻辑块就相当于一个GAL器件。这些逻辑块可以通过共享可编程开关阵列组成的互连资源,实现它们之间的信息交换,也可以与周围的I/O模块相连,实现与芯片外部交换信息。

(3) 现场可编程门阵列 (FPGA)

芯片内部主要由许多不同功能的可编程逻辑模块组成，靠纵横交错的分布式可编程互联线连接起来，可构成极其复杂的逻辑电路。它更适合于实现多级逻辑功能，并且具有更高的集成密度和应用灵活性在软件上，亦有相应的操作系统配套。这样，可使整个数字系统（包括软、硬件系统）都在单个芯片上运行，即所谓的SOC技术。

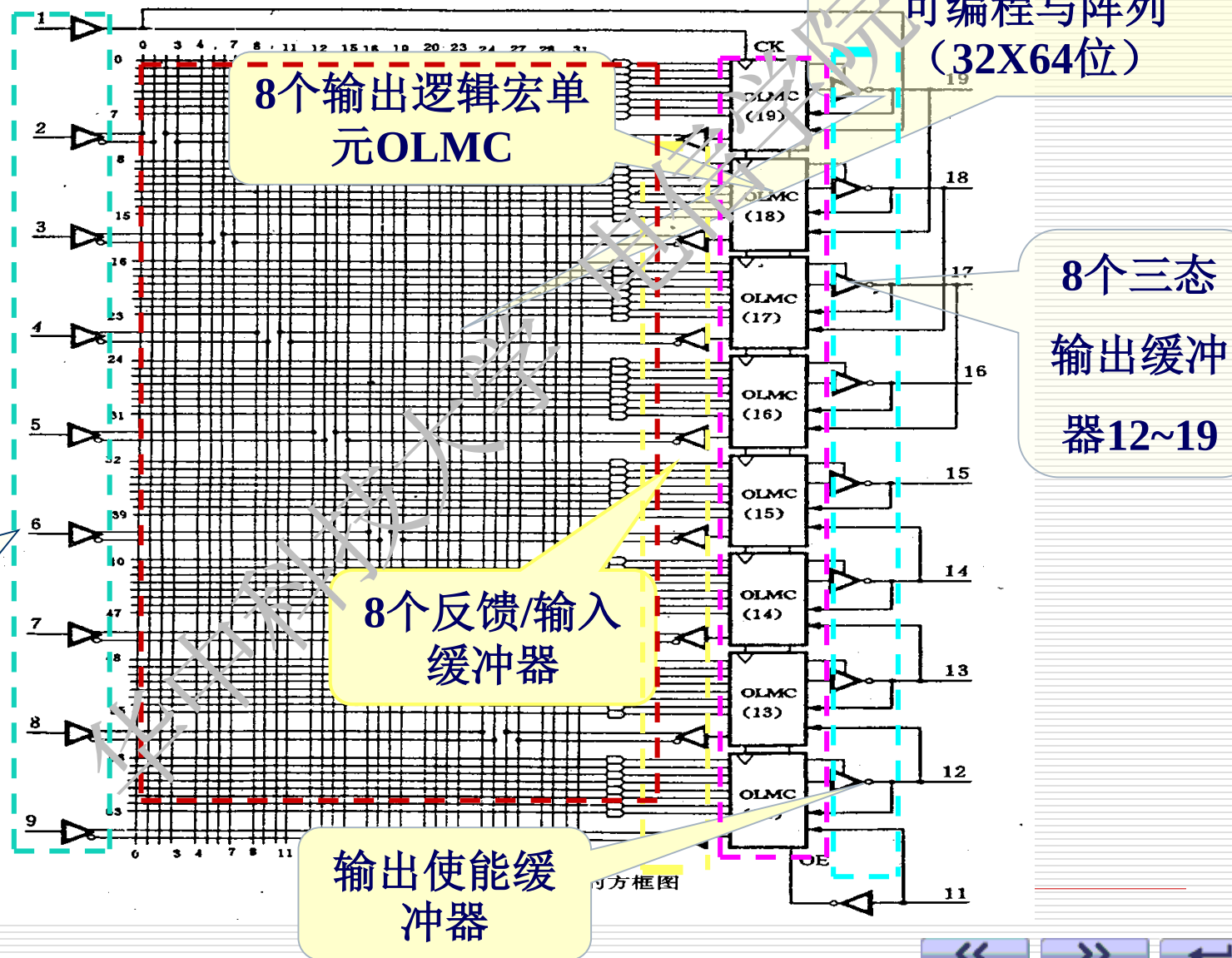
2. PAL的不足:

- (1) 由于采用的是双极型熔丝工艺，一旦编程后不能修改；
- (2) 输出结构类型太多，给设计和使用带来不便。

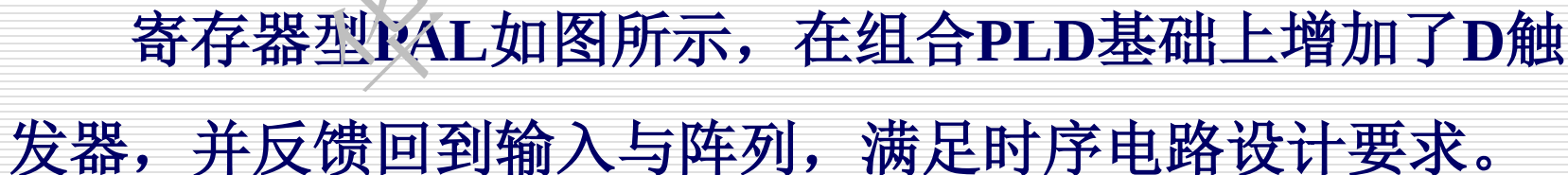
3. GAL的优点:

- (1) 采用电可擦除的E²CMOS工艺可以多次编程；
- (2) 输出端设置了可编程的输出逻辑宏单元（OLMC）通过编程可将OLMC设置成不同的工作状态，即一片GAL便可实现PAL的5种输出工作模式。器件的通用性强；
- (3) GAL工作速度快，功耗小

6.6.1 GAL的结构—GAL16V8的结构为例



1. 寄存器型PAL

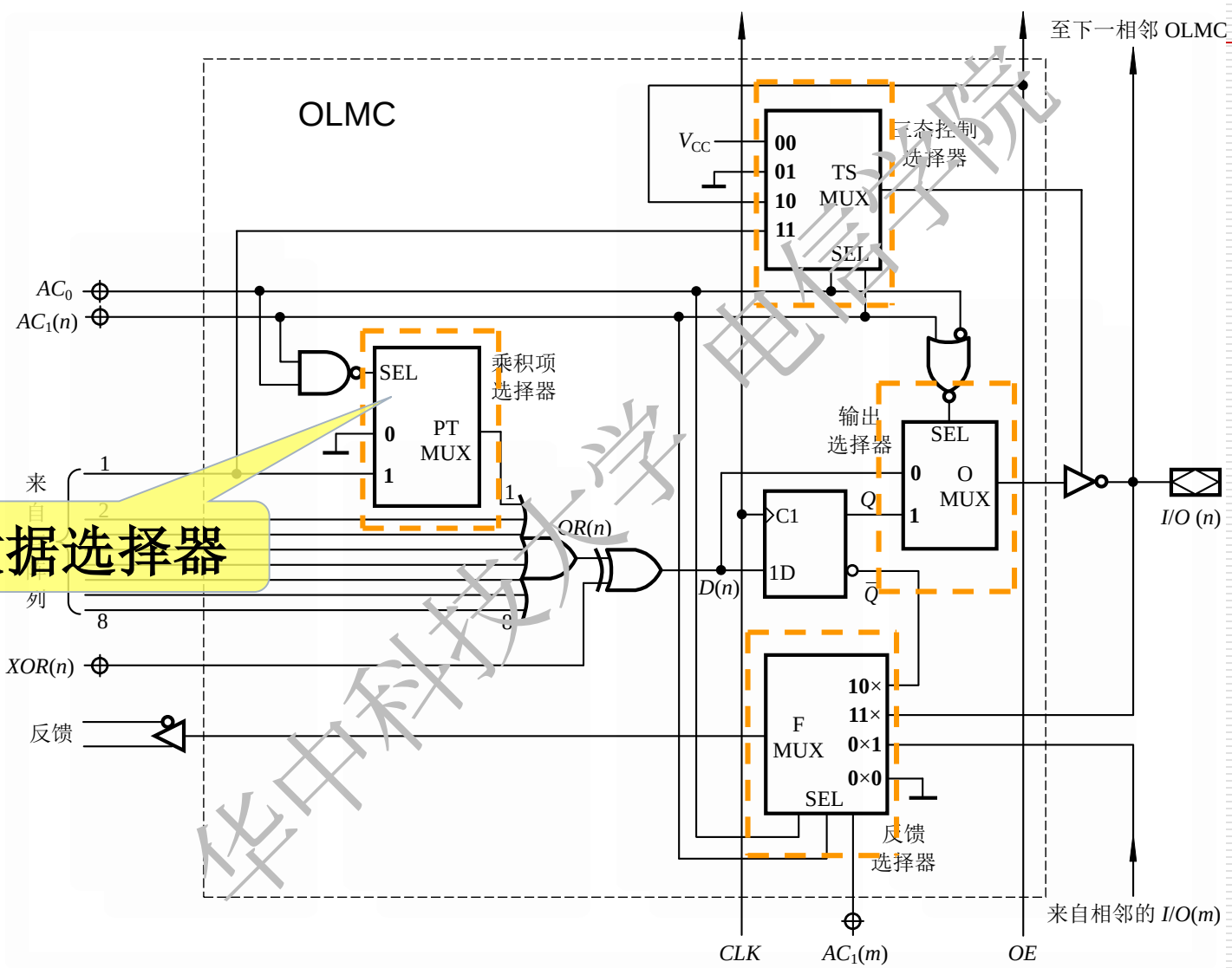


2. GAL中的输出宏单元

GAL的电路结构与PAL类似，由可编程的与逻辑阵列、固定的或逻辑阵列和输出电路组成，但GAL的输出端增设了可编程的输出逻辑宏单元（OLMC）。通过编程可将OLMC设置为不同的工作状态，可实现PAL的所有输出结构，产生组合、时序逻辑电路输出。

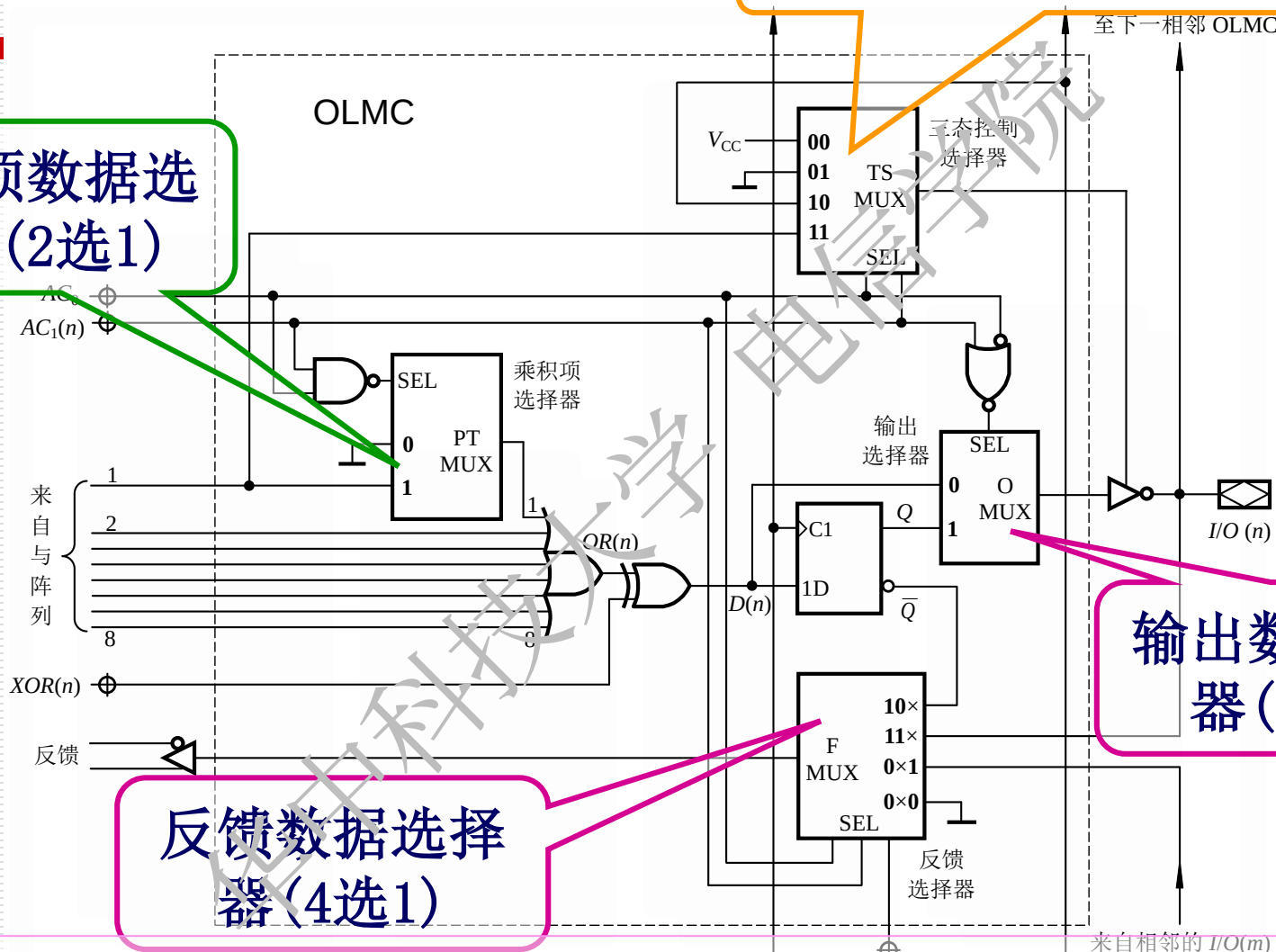


数据选择器



三态数据选择器(4选1)

乘积项数据选择器(2选1)

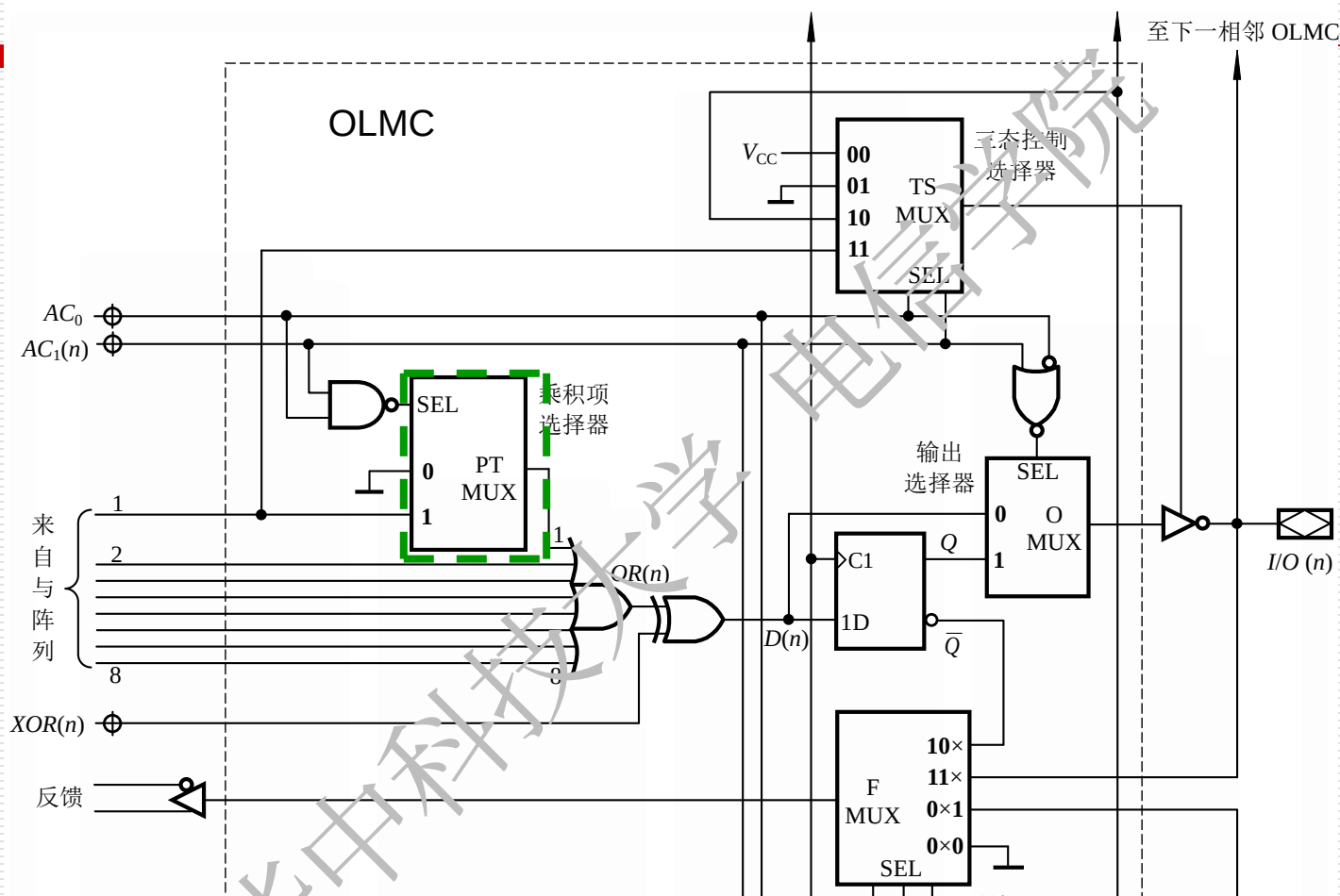


输出数据选择器(2选1)

反馈数据选择器(4选1)

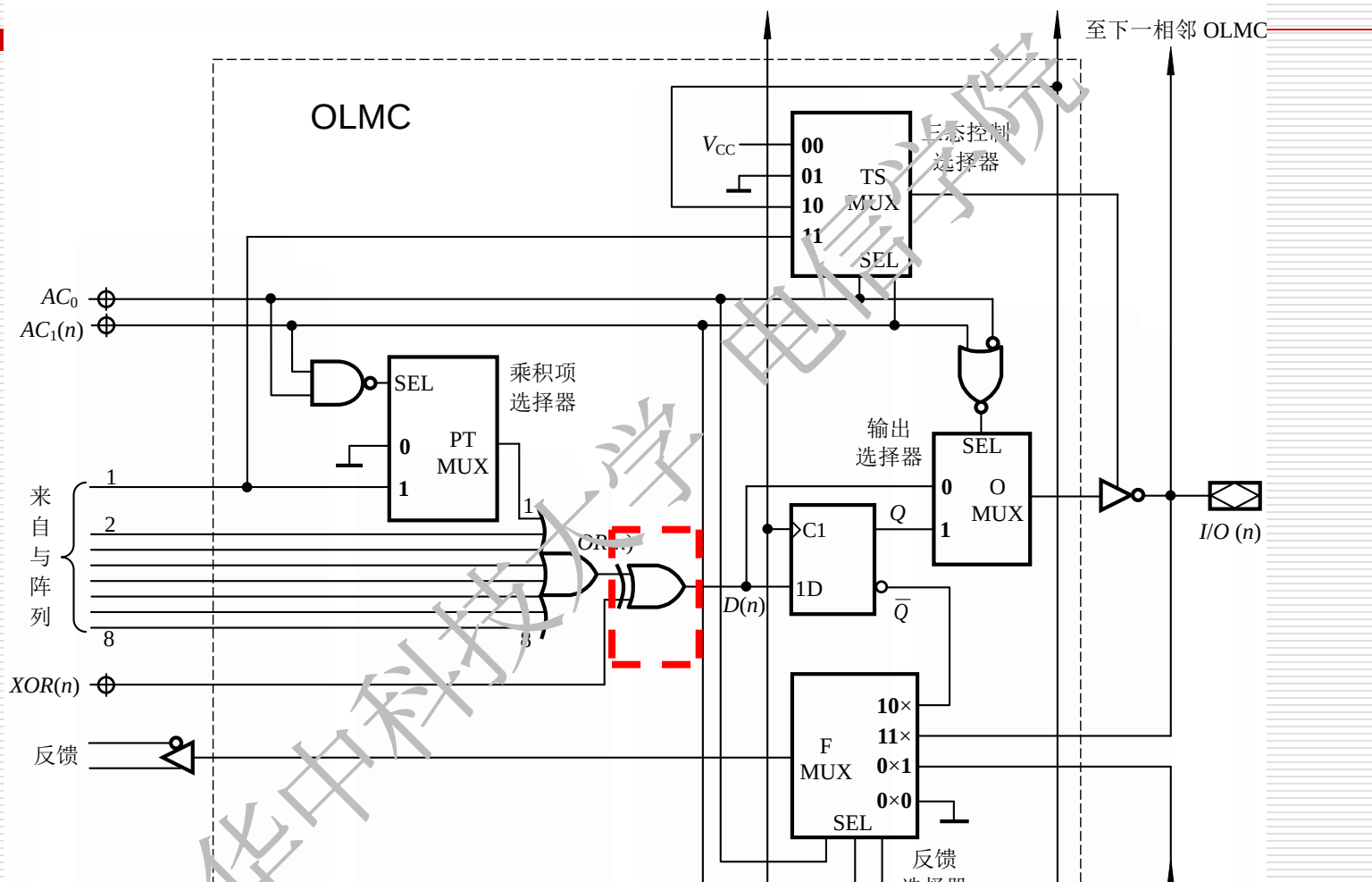
4个数据选择器：用不同的控制字实现不同的输出电路结构形式

(1) 输入电路—由乘积项数据选择器 (2选1)PTMUX控制



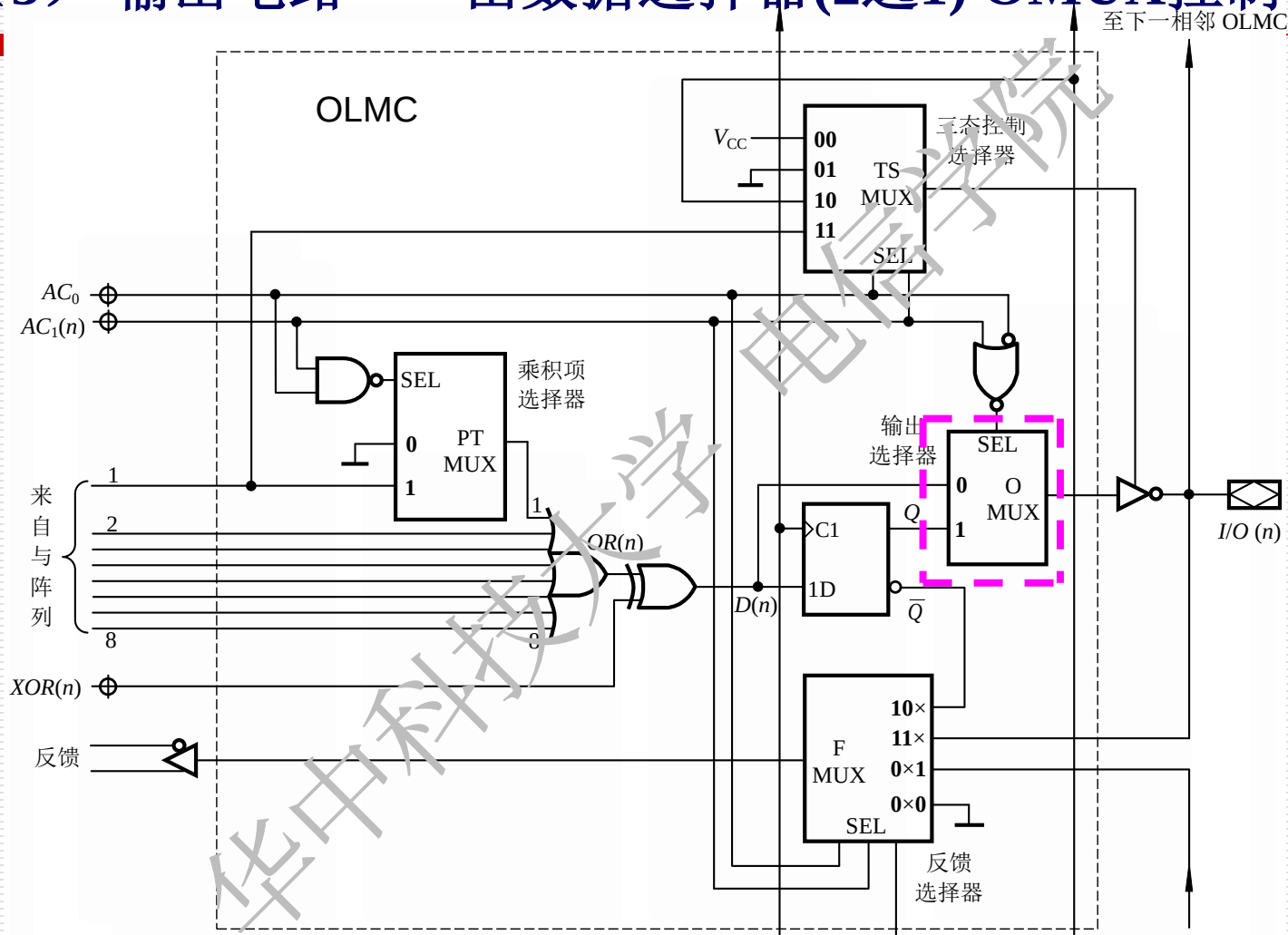
乘积项数据选择器： 根据 AC_0 和 $AC_1(n)$ 决定与逻辑阵列的第一乘积项是否作为或门的一个输入端。只有在 G_2 的输出为1时，第一乘积项是或门的一个输入端。

(2) 原变量/非变量输出电路—由异或门控制



异或门输出为或门输出 $OR(n)$ 与 $XOR(n)$ 进行异或运算。
 $XOR(n)=0$, 则 $D(n)=OR(n)$, 若 $XOR(n)=1$, 则 $D(n)=\overline{OR(n)}$ 。

(3) 输出电路——由数据选择器(2选1) OMUX控制

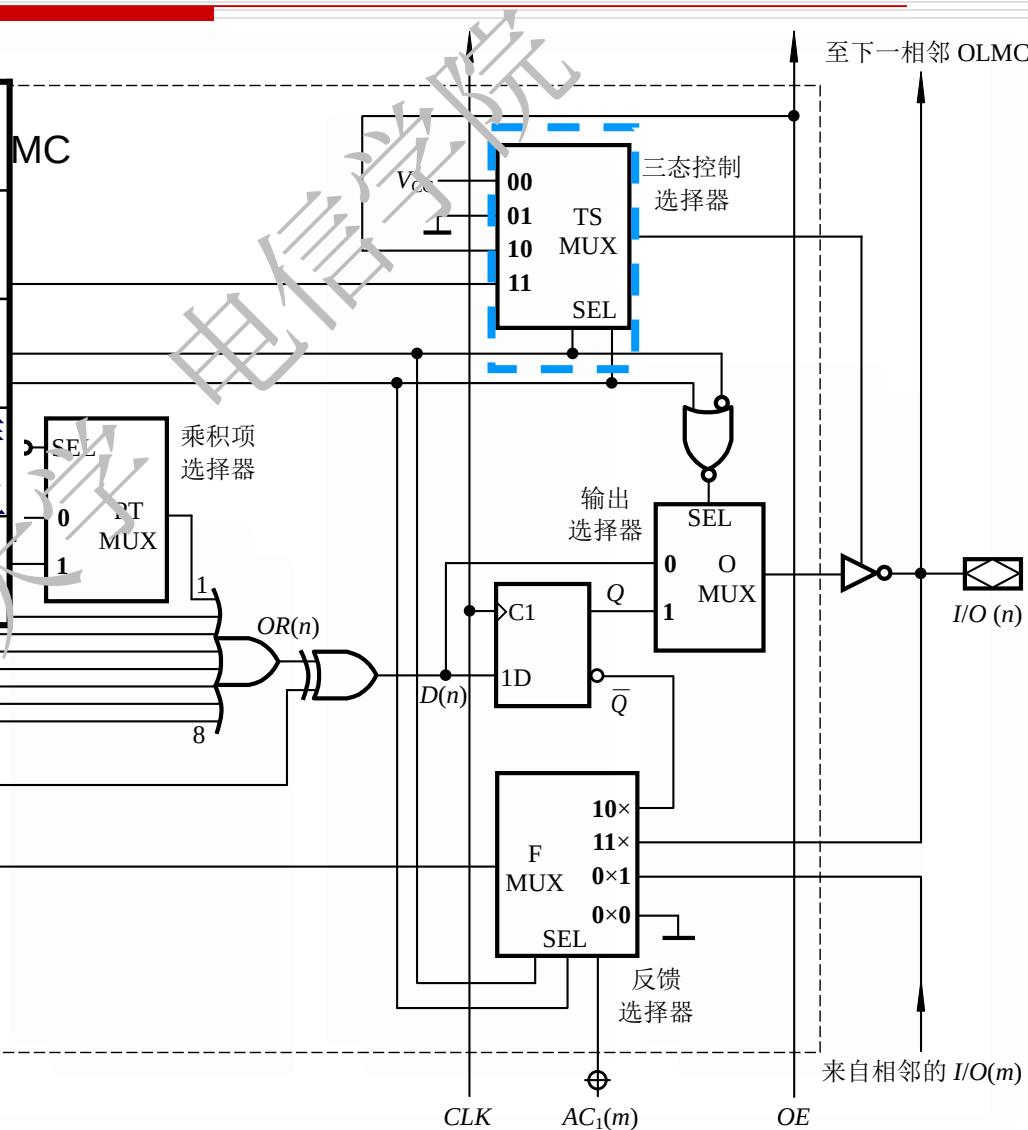


OMUX: 根据 AC_0 和 $AC_1(n)$ 决定OLMC是组合输出还是寄存器输出模式

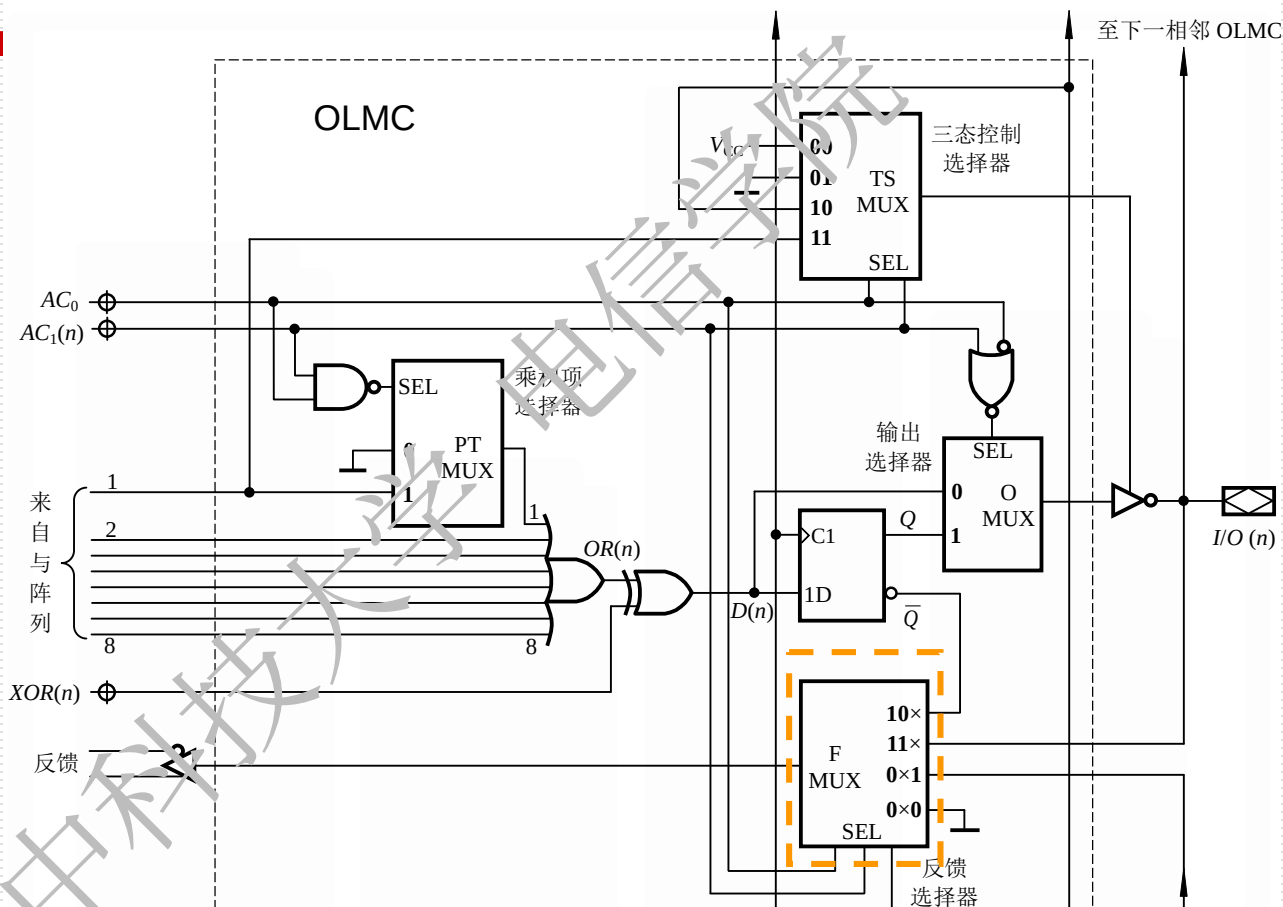
由三态数据选择器(4选1)控制输出选择器的选通端SEL

AC0 AC1(n)	TX (输出)	三态缓冲器的工作状态
0 0	V_{CC}	工作
0 1	地电平	高阻
1 0	OE	OE=1, 工作 OE=0, 高阻
1 1	第一乘积项	1, 工作 0, 高阻

三态数据选择器受AC0和AC1(n)的控制，用于选择输出三态缓冲器的选通信号。可分别选择 V_{CC} 、地、OE和第一乘积项。



反馈数据选择器(4选1)——FMUX

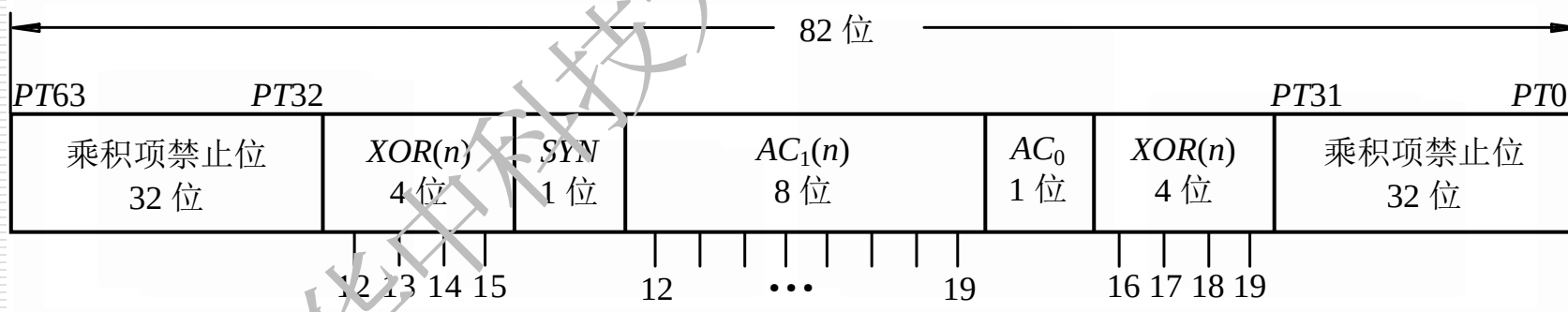


FMUX:

根据 AC_0 和 $AC_1(n)$ 的不同编码，使反向传输的电信号也对应不同。

6.6.3 GAL的结构控制字

GAL16V8的结构控制字共有82位，它们的定义如图。每个OLMC有2个编程单元 $AC1(n)$ 和 $XOR(n)$ ，一个全局编程单元 $AC0$ ，同步控制单元 SYN 。



小 结

- 时序电路的分析，首先按照给定电路列出各逻辑方程组、进而列出状态表、画出状态图和时序图，最后分析得到电路的逻辑功能。时序电路的设计，首先根据逻辑功能的需求，导出原始状态图或原始状态表，有必要时需进行状态化简，继而对状态进行编码，然后根据状态表导出激励方程组和输出方程组，最后画出逻辑图完成设计任务。。

- 时序逻辑电路一般由组合电路和存储电路两部分构成。它们在任一时刻的输出不仅是当前输入信号的函数，而且还与电路原来的状态有关。时序电路可分为同步和异步两大类。逻辑方程组、状态表、状态图和时序图从不同方面表达了时序电路的逻辑功能，是分析和设计时序电路的主要依据和手段。

6.7 用Verilog HDL描述时序逻辑电路

6.7.1 移位寄存器的Verilog建模

6.7.2 计数器的Verilog建模

6.7.3 状态图的Verilog建模

6.7.4 数字钟的Verilog建模

6.7.1 移位寄存器的Verilog建模

用行为级描述always描述一个4位双向移位寄存器，有异步清零、同步置数、左移、右移和保持。功能同74xx194。

```
module shift74x194 (S1, S0, D, Dsl, Dsr, Q, CP, CR);  
    input S1, S0;                //控制输入  
    input Dsl, Dsr;              //串行输入  
    input CP, CR;                //时钟及清零  
    input [3:0] D;               //并行输入  
    output [3:0] Q;              //寄存器输出  
    reg [3:0] Q;
```

6.7.1 移位寄存器的Verilog建模

```
always @ (posedge CP or negedge CR)
```

```
    if (~CR) Q <= 4'b0000;
```

```
    else
```

```
        case ({S1,S0})
```

```
            2'b00: Q <= Q;                //保持
```

```
            2'b01: Q <= {Q[2:0],Dsr};    //右移
```

```
            2'b10: Q <= {Dsl,Q[3:1]};    //左移
```

```
            2'b11: Q <= D;                //并行输入
```

```
        endcase
```

```
endmodule
```

6.7.2 计数器的Verilog建模实例

用Verilog描述具有使能端、异步置零、同步置数、计数、保持的10进制计数器

```
module counter74x161_beh ( //Verilog 2001, 2005 syntax
    input CEP, CET, PE, CP, CR, //输入端口声明
    input [3:0] D,                //并行数据输入
    output TC,                    //进位输出
    output reg [3:0] Q            //数据输出端口及变量的数据类型声明
);
    wire CE;                      //中间变量声明
```

6.7.2 计数器的Verilog建模实例

```
assign CE=CEP&CET; //CE=1时，计数器计数
assign TC=CET&PE&(Q == 4'b1111); //产生进位输出信号
always @(posedge CP, negedge CR) //Verilog 2001, 2005 syntax
    if (~CR) Q<=4'b0000; //实现异步清零功能
    else if (~PE) Q<=D; //PE=0, 同步装入输入数据
    else if (CE) Q<=Q+1'b1; //加1计数
    else Q<=Q; //输出保持不变
endmodule
```


6.7.3 状态图的Verilog建模实例

用Verilog描述状态图非常方便，常用always或case语句

```
module Mealy_sequence_detector (A, CP, CR, Y);
```

```
input A, CP, CR;
```

```
output Y;
```

```
reg Y;
```

```
reg [1:0] current_state, next_state;
```

```
parameter S0=2'b00, S1=2'b01, S2=2'b11;
```

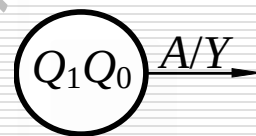
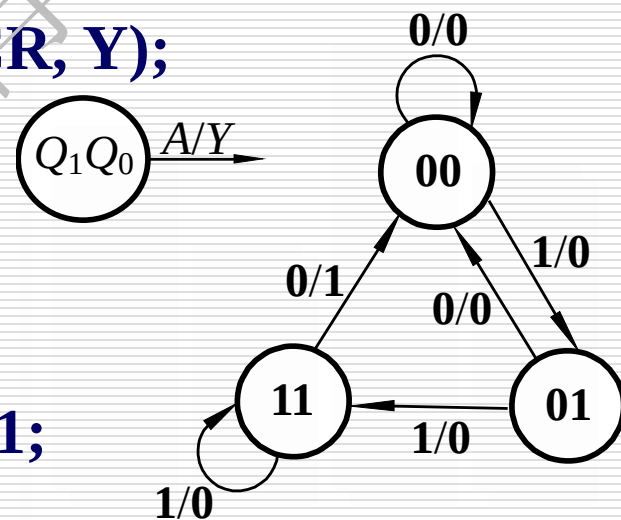
```
always @( negedge CP or negedge CR)
```

```
begin
```

```
    if (~CR) current_state <= S0;    //在CR下降沿设s0为初态
```

```
    else      current_state <= next_state;
```

```
end
```



6.7.3 状态图的Verilog建模实例

第二个always是将current_state和输入A作为敏感变量

```
always @(current_state or A)
```

```
begin
```

```
  case(current_state)
```

```
    S0: begin Y<=0; next_state=(A==1)? S1: S0; end
```

```
    S1: begin Y<=0; next_state=(A==1)? S2: S0; end
```

```
    S2: if (A==1)
```

```
      begin Y<=0; next_state<=S2; end
```

```
    else
```

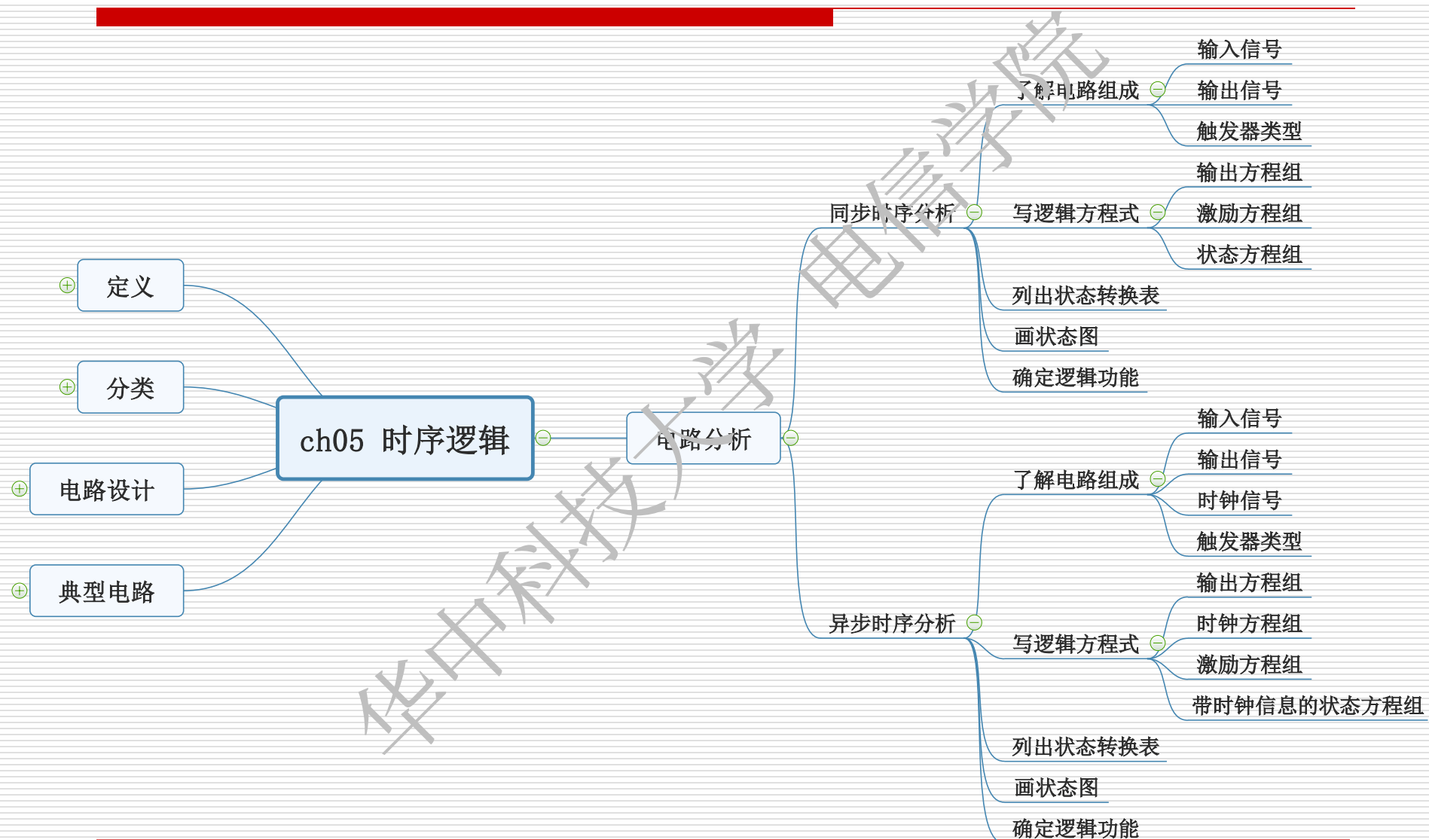
```
      begin Y=1; next_state<=S0; end
```

```
    default: begin Y<=0; next_state<=S0; end
```

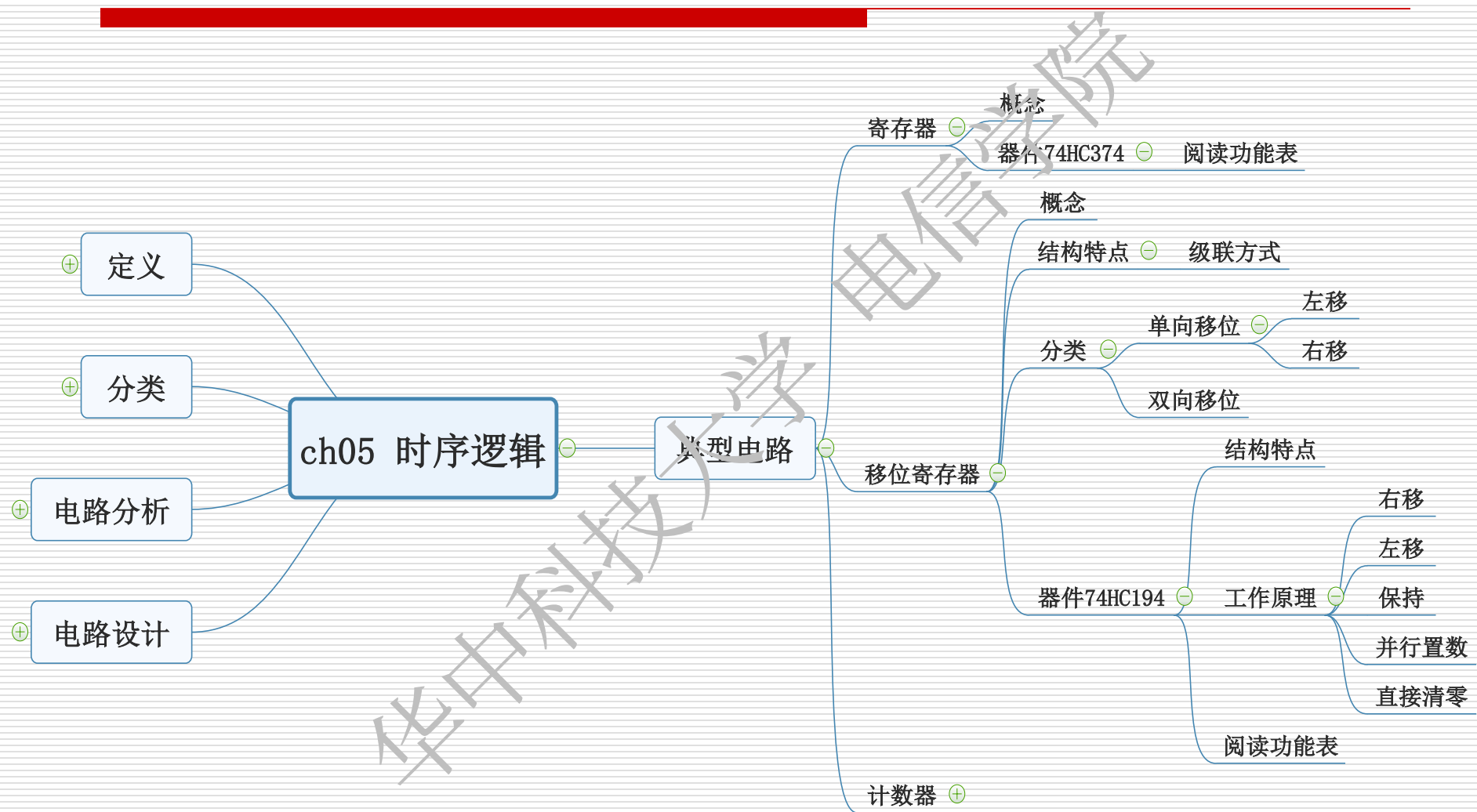
```
  endcase
```

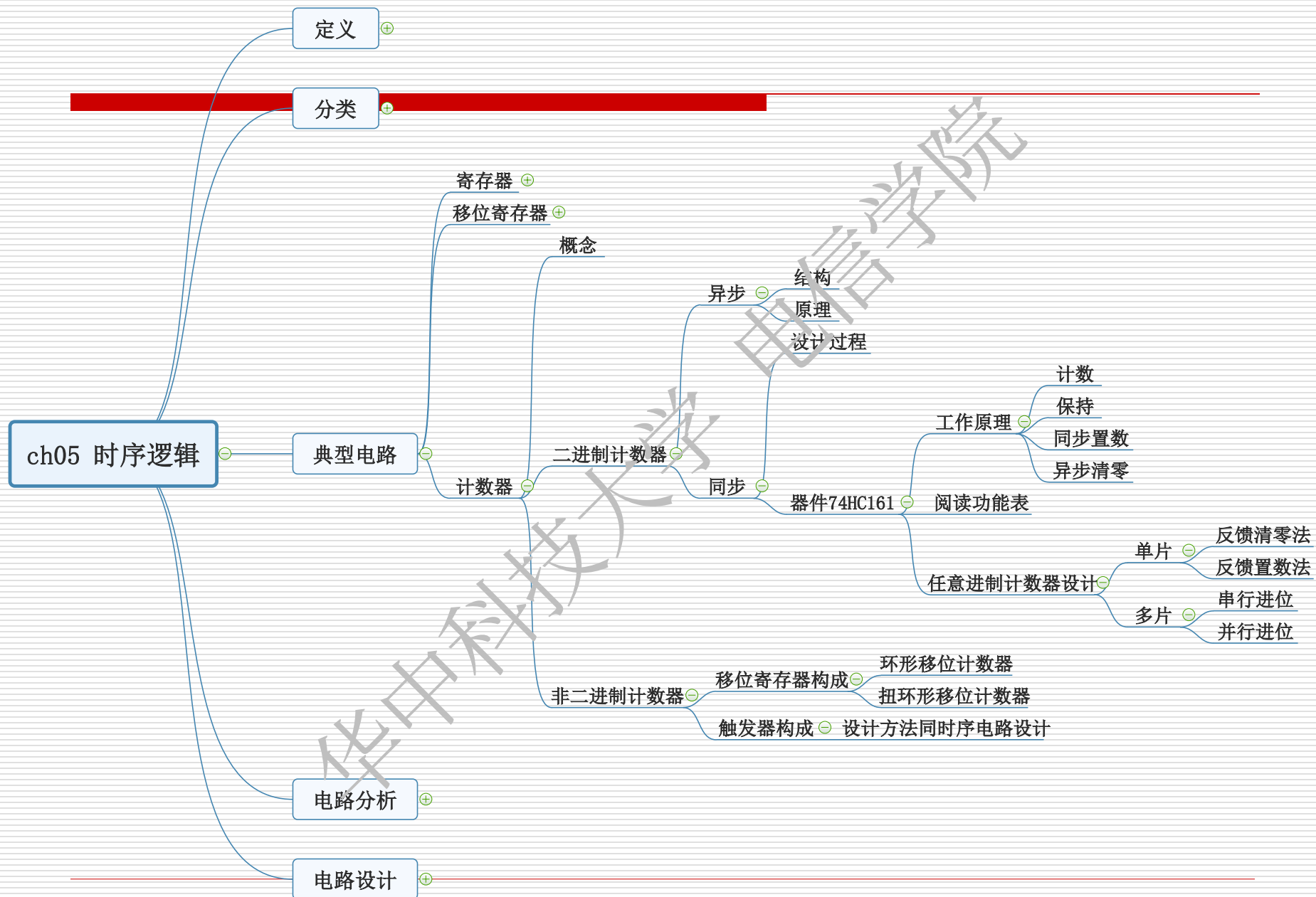
```
end
```

```
endmodule
```







ch05 时序逻辑

定义

- 结构特征
- 工作特征
- 逻辑方程组
- 输出方程组
- 激励方程组
- 状态方程组
- 表示方法
 - 状态转换真值表
 - 状态转换表
 - 状态图
 - 时序图

分类

- 时钟源
 - 同步时序
 - 异步时序
- 输出
 - 摩尔型
 - 米利型

电路分析

- 同步时序分析
 - 了解电路组成
 - 输入信号
 - 输出信号
 - 触发器类型
 - 写逻辑方程式
 - 输出方程组
 - 激励方程组
 - 状态方程组
 - 列出状态转换表
 - 画状态图
 - 确定逻辑功能
- 异步时序分析
 - 了解电路组成
 - 输入信号
 - 输出信号
 - 时钟信号
 - 触发器类型
 - 写逻辑方程式
 - 输出方程组
 - 时钟方程组
 - 激励方程组
 - 带时钟信息的状态方程组
 - 列出状态转换表
 - 画状态图
 - 确定逻辑功能

典型电路

- 寄存器
 - 概念
 - 器件74HC374
 - 级联方式
 - 结构特点
 - 分类
 - 单向移位
 - 左移
 - 右移
 - 双向移位
 - 移位寄存器
 - 结构特点
 - 工作原理
 - 右移
 - 左移
 - 保持
 - 器件74HC194
 - 并行置数
 - 直接清零
 - 阅读功能表
- 计数器
 - 概念
 - 异步
 - 设计过程
 - 同步
 - 工作原理
 - 计数
 - 保持
 - 同步置数
 - 异步清零
 - 器件74HC161
 - 任意进制计数器设计
 - 单片
 - 反馈清零法
 - 反馈置数法
 - 多片
 - 串行进位
 - 并行进位
 - 非二进制计数器
 - 移位寄存器构成
 - 环形移位计数器
 - 扭环形移位计数器
 - 设计方法同时序电路设计
 - 触发器构成

电路设计

- 设计步骤
 - 确定输入输出变量
 - 建立原始状态图/表
 - 确定状态跳转关系
 - 画出原始状态图/表
 - 状态化简
 - 合并等价状态
 - 定义
 - 判断方法
 - 状态编码
 - 二进制码
 - 格雷码
 - 独热码
 - 状态数与触发器个数的关系
 - 选择触发器
 - 触发器类型
 - 特性方程
 - 敏感边沿类型
 - 求激励方程和输出方程
 - 真值表法
 - 状态方程法
 - 画出逻辑图
 - 检查自启动
 - 定义
 - 判断方法
 - 检查输出信号
 - 修改方法





A Guide to Verilog HDL for Sequential Circuit



Introductory information

- ✓ `3'b101` `width'base formatnumber`
- ✓ parameter BIT=1, BYTE=8, PI=3.1415926;
- ✓ `'define (global)`
- ✓ `reg [3:0] Counter;`
- ✓ reductions operator(缩位)
e.g., `A=4'b1010,`
`^A=A[0]^A[1]^A[2]^A[3]=1'b1;&A=0`
- ✓ Concatenation operators(位拼接)
e.g., `A=1'b1,C=2'b10, Y={A,C}=3'b110`

Introductory information (cont.)

```
module module_name(ports)
    ports declaration;
    parameter;
    data type definitions;
    behavior description
endmodule
```

Introductory information (cont.)

✓ Blocking Assignment Statement

阻塞型赋值 =

✓ Non-Blocking Assignment Statement

非阻塞型赋值 <=

e.g., A=2,B=4

begin

B=A;

C is 3

C=B+1;

end

begin

B<=A;

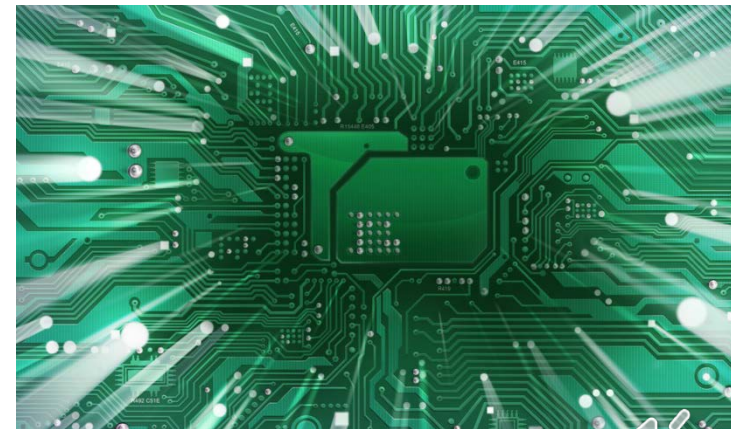
C is 5

C<=B+1;

end

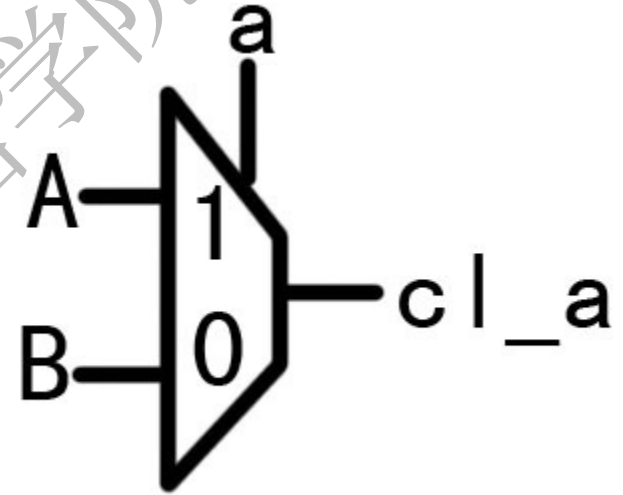
Outline

- Behavior of flip-flops
- Behavior of shift registers
- Behavior of counters
- State machine
- Home work



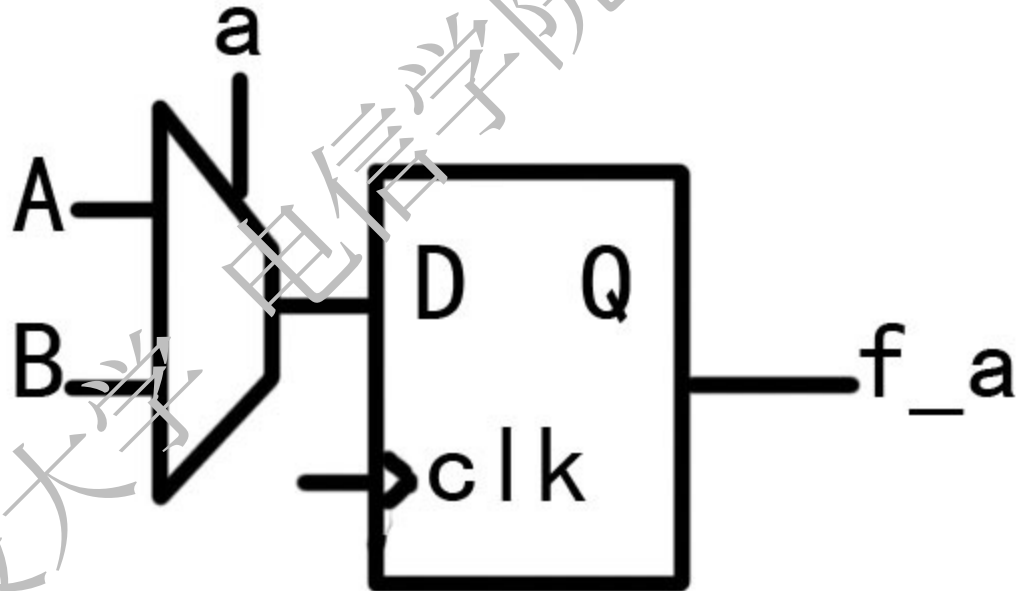
Combinational-logic-circuit-based reg

```
wire cl_a;  
assign cl_a = a ? A : B;
```



```
reg cl_a;  
always @(a or A or B)  
begin cl_a = a ? A : B; end
```

Flip-flop-based reg



- reg f_a;
- always @(posedge clk)
- begin f_a **<=** a ? A : B; end

Behavior of D Flip-flop

```
module A_DFF (Q,D,CP,Rd);  
output Q;  
input D,CP,Rd;  
reg Q;  
always @(posedge CP or  
negedge Rd)  
if (~Rd) Q<=1'b0;  
else Q<=D;  
endmodule;
```

//with asynchronous reset

```
module S_DFF (Q,D,CP,Rd);  
output Q;  
input D,CP,Rd;  
reg Q;  
always @(posedge CP )  
if (~Rd) Q<=1'b0;  
else Q<=D;  
endmodule;
```

//with synchronous reset



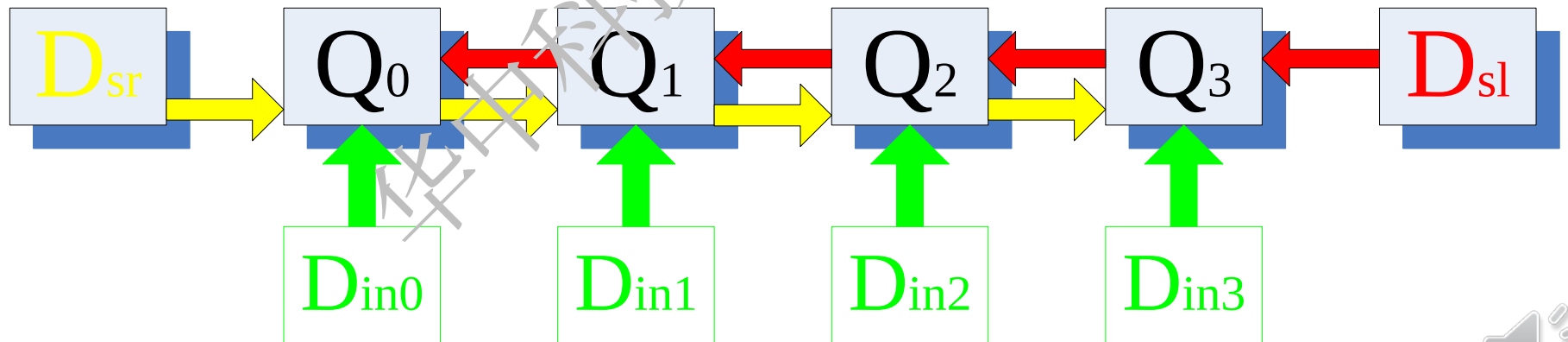
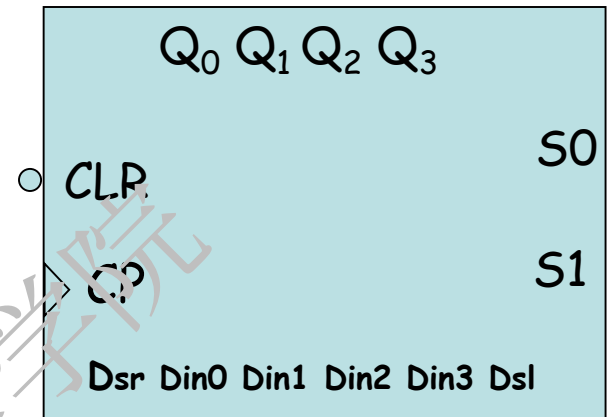
Behavior of JK Flip-flop

```
module JK_FF (J,K,CP,Q,Qn);  
  output Q,Qn; input J,K,CP; reg Q;  
  assign Qn=~Q;  
  always @(negedge CP)  
    case ({J,K})  
      2'b00: Q<=Q;  
      2'b01: Q<=1'b0;  
      2'b10: Q<=1'b1;  
      2'b11: Q<=~Q;  
    endcase;  
endmodule;
```

Behavior of universal shift registers

- Serial input / Serial output Shift register
- Universal Shift Register
- SN74LS194
 - Parallel input / parallel output
 - Four Parallel data inputs permit parallel loading of external data.
 - Two serial input, one for left-shift, and the other for right-shift.
 - Serial output data are taken from either Q_0 (LSB) or Q_3 (MSB) depending on shift direction.

- Positive edge triggered
- serial mode control : **S₁, S₀**
- CLR can clear the data freely
- Combined with clock pulse
 - S₁S₀=11, load the external data
 - S₁S₀=01, shift right,
 - S₁S₀=10, shift left,
 - S₁S₀=00, inhibit (remains same)



Behavior of universal shift registers

```
module shift74x194 (S1,S0,Din,Dsl,Dsr,Q,CP,CLR);
```

```
input S1,S0;
```

```
input Dsl,Dsr;
```

```
input CP,CLR;
```

```
input [3:0] Din;
```

```
output [3:0] Q;
```

```
reg [3:0] Q;
```

```
always @(posedge CP or  
negedge CLR)
```

```
if (~CLR) Q<=4'b0000;
```

```
else
```

```
case ({S1,S0})
```

```
2'b00: Q <= Q;
```

```
2'b01: Q <= {Q[2:0], Dsr};
```

```
2'b10: Q <= {Dsl, Q[3:1]};
```

```
2'b11: Q <= Din;
```

```
endcase
```

```
endmodule
```

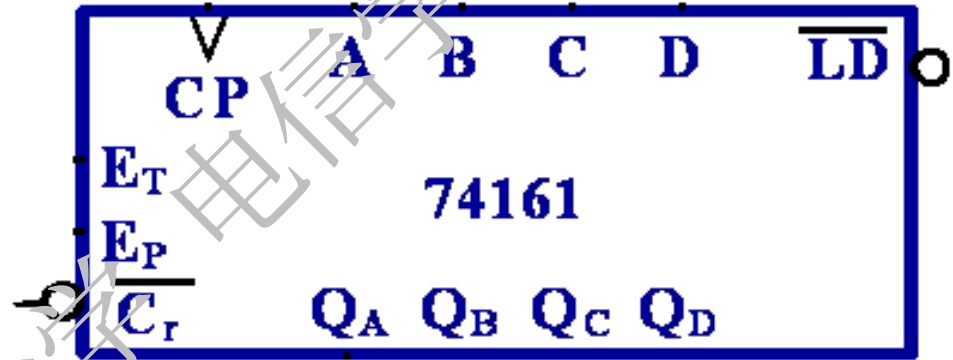
Behavior of counters

```
module _4bit_counter(CP,CLR,Q,Mod);  
input CP,CLR,Mod;  
output [3:0] Q;  
reg [3:0] Q;  
always @(posedge CP or negedge CLR)  
if (~CLR) Q<=4'b0000;  
else if (Mod==1) Q<=Q+1'b1;  
else Q<=Q-1'b1;  
endmodule
```

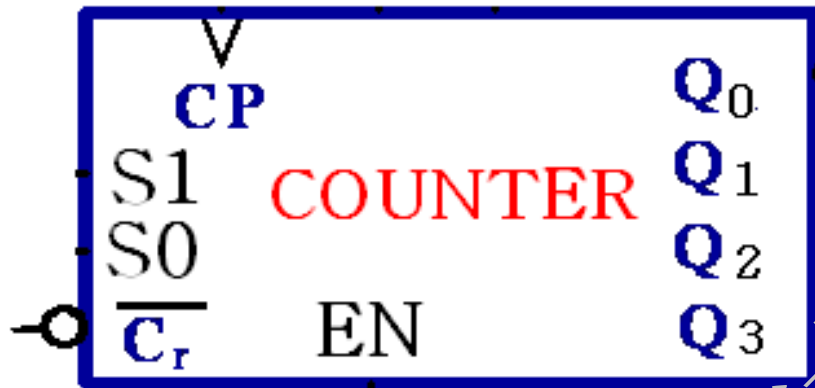
- 4-bit binary counter with asynchronous reset
- Mod=1 counting up
- Mod=0 counting down

Behavior of counters

```
module _74x161(EP,ET,Load,Din,CP,CLR,Q,RCO);
input EP,ET,Load,CP,CLR,Mod;
input [3:0] Din
output RCO;
output [3:0] Q;
reg [3:0] Q;
wire EN;
assign EN=EP & ET;
assign RCO= ET& (Q==4'b1111);
always @(posedge CP or negedge CLR)
if (~CLR) Q<=4'b0000;
else if (~Load) Q<=Din;
else if (~EN) Q<=Q;
else Q<=Q+1'b1;
endmodule
```



Modulo-Variable counter

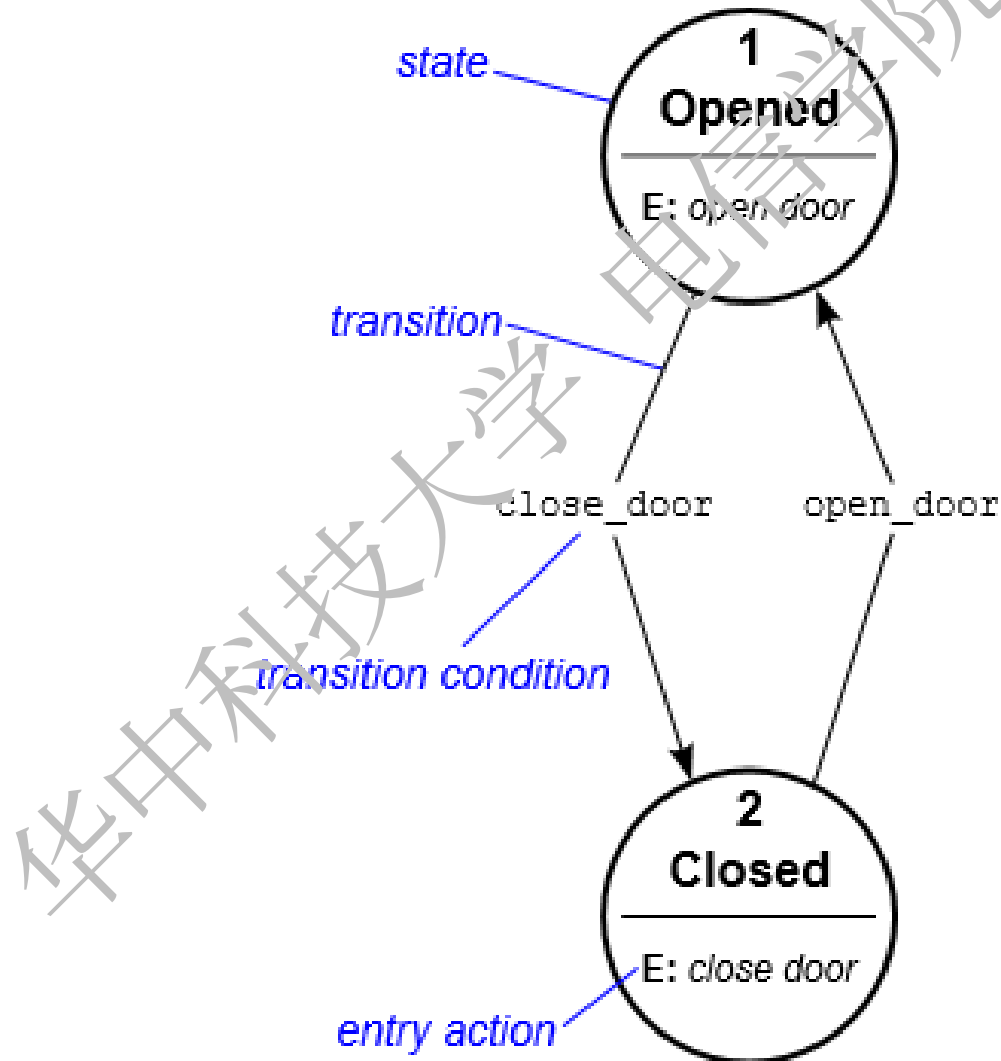


Mode control		Modulo-
S1	S0	
0	0	six
0	1	eight
1	0	ten
1	1	fifteen

Modulo-Variable counter

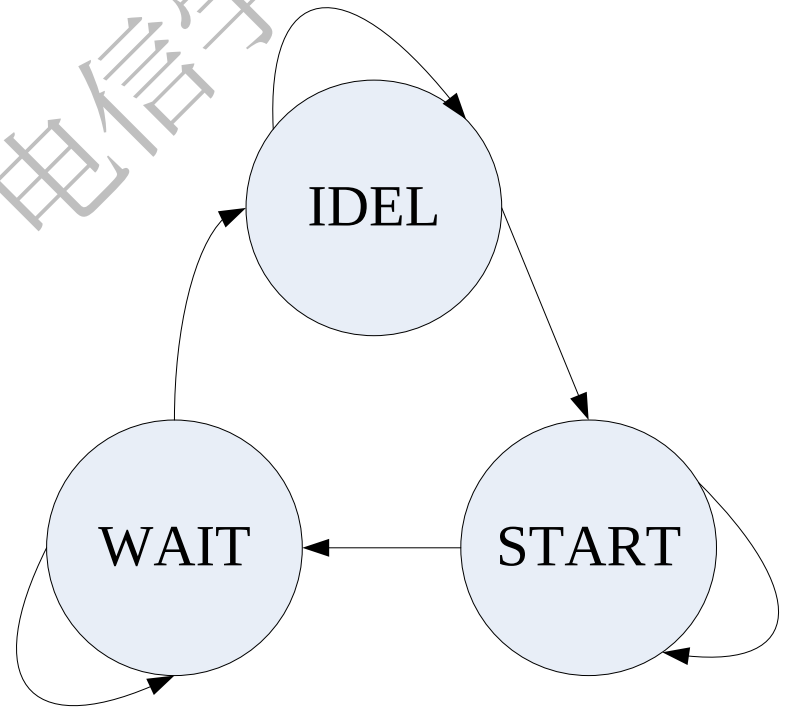
```
module var_cnt(CP,CLR,EN,S1,S0,Q);
input CP,CLR,EN,S1,S0;
output [3:0] Q;
reg [3:0] Q;
always @(posedge CP or negedge CLR)
if (~CLR)    Q<=4'b0000;
else if (EN)
begin
    case ({S1,S0})
        2'b00: if (Q>=4'd5) Q<=4'd0;    else Q<=Q+1'd1;
        2'b01: if (Q>=4'd7) Q<=4'd0;    else Q<=Q+1'd1;
        2'b10: if (Q>=4'd9) Q<=4'd0;    else Q<=Q+1'd1;
        2'b11: if (Q>=4'd14) Q<=4'd0; else Q<=Q+1'd1;
    endcase
end
else Q <= Q;
endmodule
```


Finite State Machine/Automata



State machine

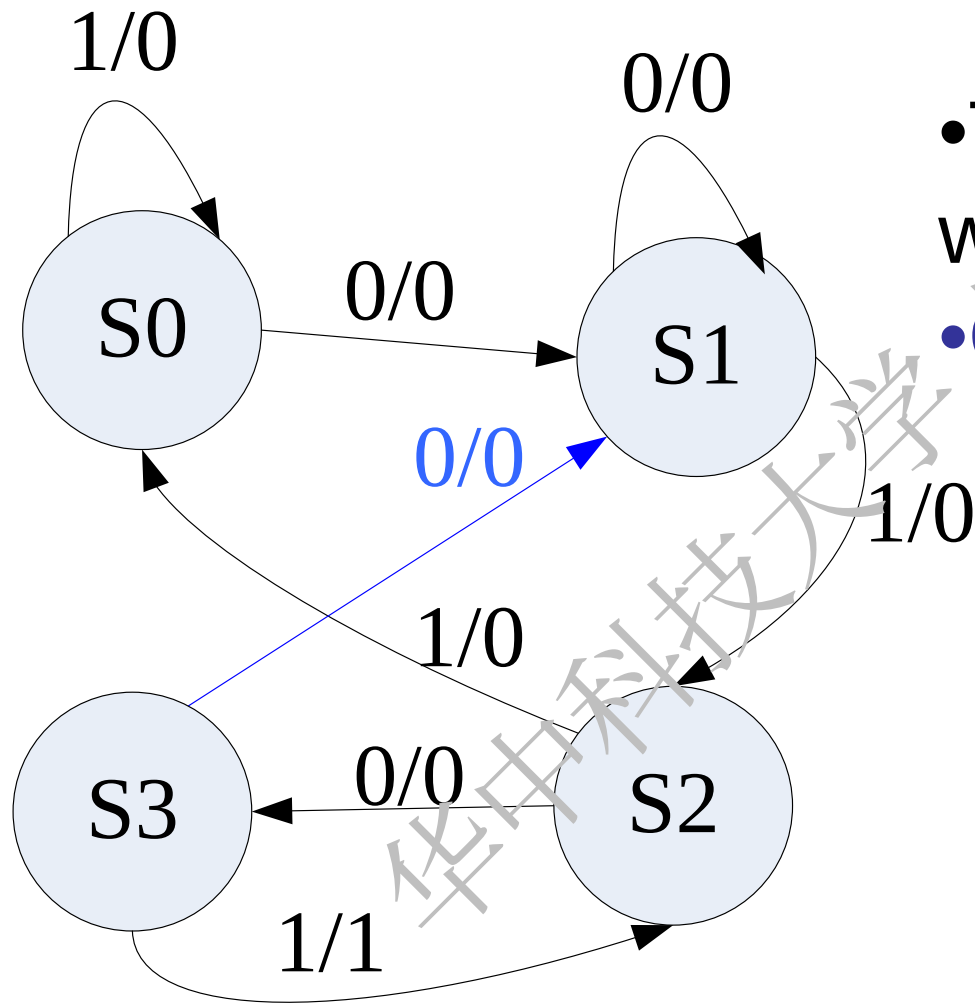
- parameter
- always
- case / if-else
- output logic



State encoding/assignment

Plain binary coding	Gray coding	One-hot coding
S0=2'b00	S0=2'b00	S0=4'b000 1
S1=2'b01	S1=2'b01	S1=4'b00 1 0
S2=2'b10	S2=2'b11	S2=4'b0 1 00
S3=2'b11	S3=2'b10	S3=4'b 1 000
		IDLE=4'b0000

e.g., Sequence detector



- Tag=1 while **0101** detected
- **Overlap** is available

Description

```
module Det1(Din,CP,nCLR,Tag);  
Input Din, CP, nCLR;  
output Tag;  
reg Tag;  
reg [1:0] state;  
parameter S0=2'b00, S1=2'b01, S2=2'b10,  
          S3=2'b11;
```

Solution 1#: Single always

always @(posedge CP or negedge nCLR)

begin

if (~nCLR) state <= S0;

else

case (state)

S0: begin Tag <= 1'b0; state <= (Din == 1)? S0: S1; end

S1: begin Tag <= 1'b0; state <= (Din == 1)? S2: S1; end

S2: begin Tag <= 1'b0; state <= (Din == 1)? S0: S3; end

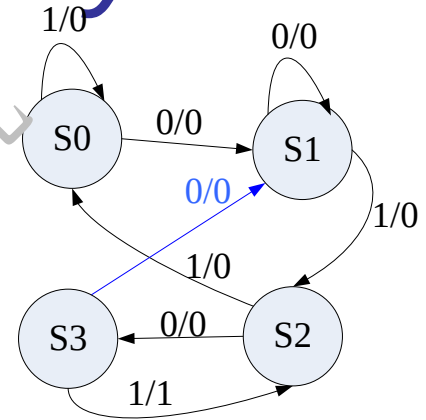
S3: if (Din == 1) begin Tag <= 1'b1; state <= S2; end

else begin Tag <= 1'b0; state <= S1; end

endcase

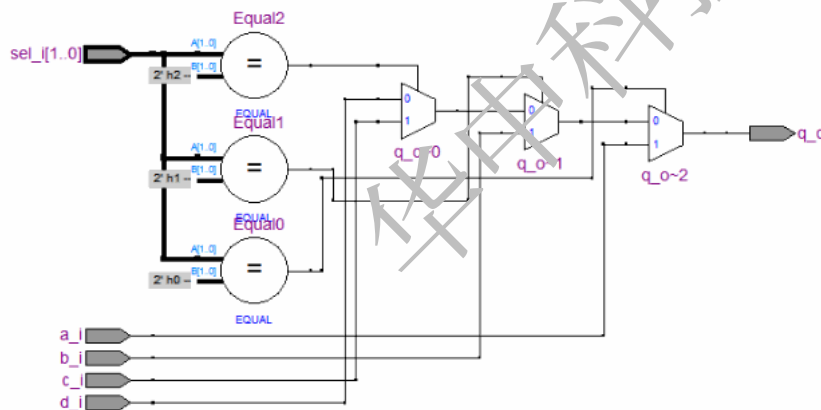
end

endmodule



Coding style

- Describing sequential logic and combinational logic, respectively
- Current state and current outputs
- Timing constraints and synthesis



Solution 2#: Two always

```
module Det2(Din,CP,nCLR,Tag);  
  Input Din, CP, nCLR;  
  output Tag;  
  reg Tag;  
  reg [1:0] Current_state, Next_state;  
  parameter S0=2'b00, S1=2'b01, S2=2'b10,  
            S3=2'b11;
```


State transition

always @(posedge CP or negedge nCLR)

begin

if (~nCLR)

Current_state <= S0;

else

Current_state <= Next_state;

end

//一个always模块采用同步时序描述状态转移

Next state pointers and output logic

always @(Current_state or Din)

//另一个模块采用组合逻辑判断状态转移条件，描述状态转移规律以及输出

begin

Next_state= 2'bxx; Tag=1'b0;

case (Current_state)

S0: begin Tag=1'b0; Next_state=(Din==1)?S0:S1; end

S1: begin Tag=1'b0; Next_state =(Din==1)?S2:S1; end

S2: begin Tag=1'b0; Next_state =(Din==1)?S0:S3; end

S3: if (Din==1) begin Tag=1'b1; Next_state = S2; end

else begin Tag=1'b0; Next_state =S1;end

endcase

end

endmodule

2段式设计的隐患是：描述当前状态的输出用组合逻辑实现，组合逻辑很容易产生毛刺，而且不利于约束，不利于综合器和布局布线器实现高性能的设计。

Solution 3#: triple always

```
module Det3(Din,CP,nCLR,Tag);  
Input Din, CP, nCLR;  
output Tag;  
reg Tag;  
reg [1:0] Current_state,Next_state;  
parameter S0=2'b0001, S1=2'b0010,  
          S2=2'b0100, S3=2'b1000;
```

State transition

```
always @(posedge CP or negedge nCLR)
```

```
begin
```

```
  if (~nCLR)
```

```
    Current_state <= S0;
```

```
  else
```

```
    Current_state <= Next_state;
```

```
end
```

//第一个进程，同步时序always模块

//格式化描述次态寄存器迁移到现态寄存器

Next state pointers

always @(Current_state or Din) //电平触发

begin

Next_state= 2'bxx; //初始化，系统复位后能进入正确的状态

case (Current_state)

S0: begin Next_state=(Din==1)? S0:S1; end

S1: begin Next_state =(Din==1)? S2:S1; end

S2: begin Next_state =(Din==1)? S0:S3; end

S3: begin Next_state =(Din==1)? S2:S1; end

endcase //阻塞赋值

end

//第二个进程，组合逻辑always模块，描述状态转移条件判断

Registered output to eliminate glitch

always @(posedge CP or negedge nCLR)

begin //同步时序always模块，格式化描述外部输出

if (~nCLR) Tag<=1'b0;

else begin

Tag <= 1'b0;

case (Current_state)

S0,S1,S2: Tag <= 1'b0; //注意：非阻塞赋值

S3: if (Din==1)

Tag <= 1'b1;

else Tag <= 1'b0;

endcase

end

endmodule

3段式的好处：关键在于根据状态转移规律，在上一状态根据输入条件判断出当前状态的输出，从而在不插入额外时钟节拍的前提下，实现了寄存器输出。消除了组合逻辑输出的不稳定与毛刺的隐患，更利于时序路径分组，在FPGA/CPLD等可编程逻辑器件上的综合与布局布线效果更佳。



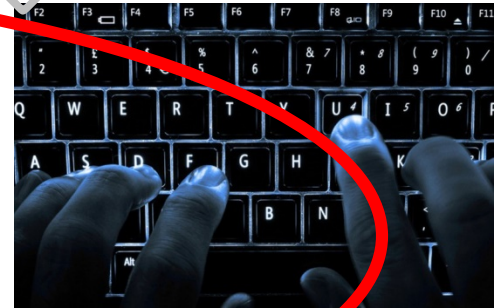
Homework

Design “110” Sequence detector

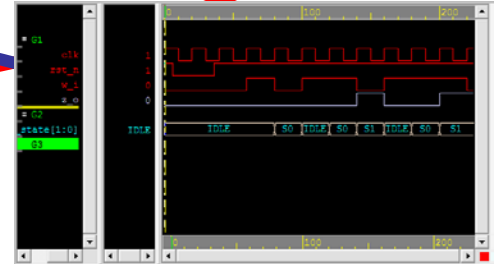
1. Draw State diagram and state assignment
2. Design via Verilog HDL (triple-always based)
3. Design via D-flip-flop and basic gates

Current and Future works

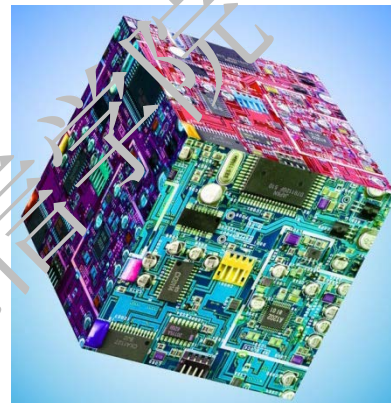
- ✓ Fundamental of Verilog HDL
- ✓ Behavioral modeling
- ✓ Combinational and sequential logic



- Project building
- Drive excitation and debug in GUI
- Measure and Debug in testbed/prototype
- Optimization



自主可控 积于**硅**步

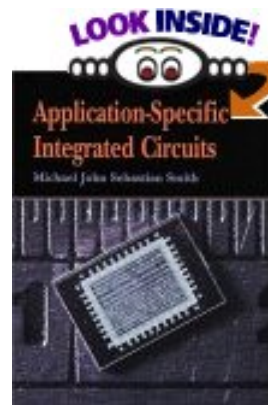


Verilog on one page

From 《ASICs... THE COURSE》

Author: Michael John Sebastian Smith

Publisher: Addison-Wesley



Summary(1)

Verilog feature	Example
Comments 注释	<pre>a = 0; // comment ends with new line /* This is a multiline or block comment */</pre>
Constants: string and numeric 常量：字符串和数字	<pre>parameter BW = 32 // local, BW `define G_BUS 32 // global, `G_BUS</pre>

Summary(2)

Verilog feature	Example
Names (case-sensitive, start with letter or '_') 变量的命名规则(大小写敏感)	<code>_12name</code> <code>A_name</code> <code>\$BAD</code> <code>NotSame</code> <code>notsame</code>
Two basic types of logic signals: wire and reg 基本逻辑信号类型：线、寄存器	<code>wire myWire;</code> <code>reg myReg;</code>

Summary(3)

Verilog feature	Example
Use continuous assignment statement with wire 持续给线型变量赋值	assign myWire = 1;
Use procedural assignment statement with reg 使用过程给寄存器型变量赋值	always myReg = myWire;

Summary(4)

Verilog feature	Example
Buses and vectors use square brackets 总线 and 向量定义中的[]	<pre>reg [31:0] DBus; DBus[12] = 1'bx;</pre>
We can perform arithmetic on bit vectors 位向量的计算	<pre>reg [31:0] DBus; DBus = DBus + 2;</pre>

Summary(5)

Verilog feature	Example
Arithmetic is performed modulo 2^n 以 2^n 为模	<pre>reg [2:0] R; R = 7 + 1; // now R = 0</pre>
Fixed logic-value system 固定逻辑值	1, 0, x (unknown), z (high-impedance)
Operators: as in C (but not ++ or --) 运算符与c一样	Not listed

Summary(6)

Verilog feature	Example
Basic unit of code is the module 模块的基本代码	<pre>module bake (chips, dough, cookies); input chips, dough; output cookies; assign cookies = chips & dough; endmodule</pre>

Summary(7)

Verilog feature	Example
Ports 端口	input or input/output ports are wire output ports are wire or reg
Functions and tasks 函数和任务	function ... endfunction task ... endtask

Summary(8)

Verilog feature	Example
Procedures happen at the same time and may be sensitive to an edge, posedge , negedge , or to a level. 进程同时并发 上升沿、下降沿、电平敏感	always @rain sing; always @rain dance; always @(posedge clock) Q = D; //flip-flop always @(a or b) c = a & b; // AND Gate

Summary(9)

Verilog feature	Example
Sequential blocks model repeating things: always: executes forever一直执行 initial: executes once only at start of simulation仅执行一次	initial born; always @alarm_clock begin : a_day metro = commute; bulot = work; dodo = sleep; end

Summary(10)

Verilog feature	Example
Output 输出命令	<code>\$display("a=%f", a);</code> <code>\$dumpvars;</code> <code>\$monitor(a)</code>
Control simulation 仿真控制命令	<code>\$stop; \$finish</code> <code>// sudden/gentle</code> <code>halt</code>

Summary(11)

Verilog feature	Example
Compiler directives 编译器的时基设置	<code>`timescale 1ns/1ps</code> <code>//units/resolution</code>
Delay 延时命令	<code>#1 a = b;</code> <code>// delay then</code> <code>sample b</code> <code>a = #1 b;</code> <code>// sample b then</code> <code>delay</code>

作业

➤ 6.2.3

➤ 6.5.13

➤ 6.2.7

➤ 6.5.14

➤ 6.3.2

➤ 6.5.19

➤ 6.3.4

➤ 6.5.20

➤ 6.5.3

➤ 6.7.5(不仿真)

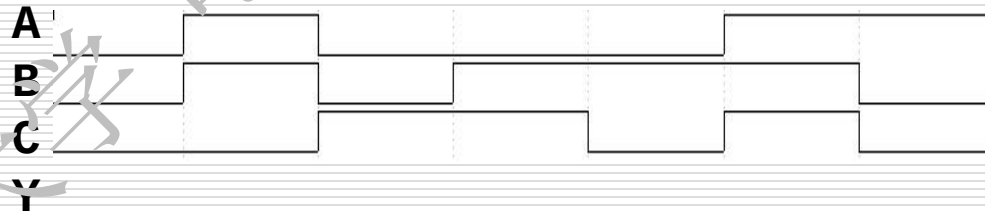
➤ 6.5.10

补充作业1

```
module ch4ex1(A,B,C,Y);  
input A,B,C;  
output Y;  
reg Y;  
always @(A or B or C)  
    if(A == 0)  
        if (B||C == 1)  
            Y = 1;  
        else  
            Y = 0;  
    else if(B^C == 1)  
        Y = 1;  
    else  
        Y = 0;  
endmodule
```

分析Verilog HDL程序,并完成

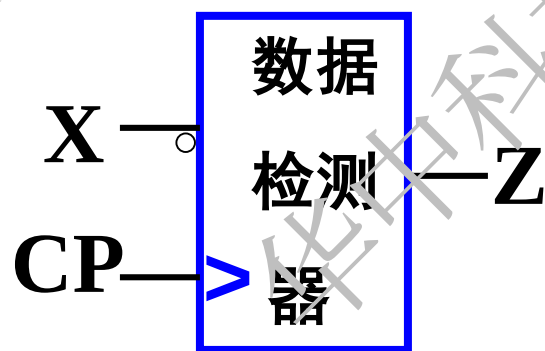
- 1) 画出Y的波形图
- 2) 写出Y的最小项表达式
- 3) 并化简为最简与或式
- 4) 画出用一片138和门电路实现Y的逻辑电路图



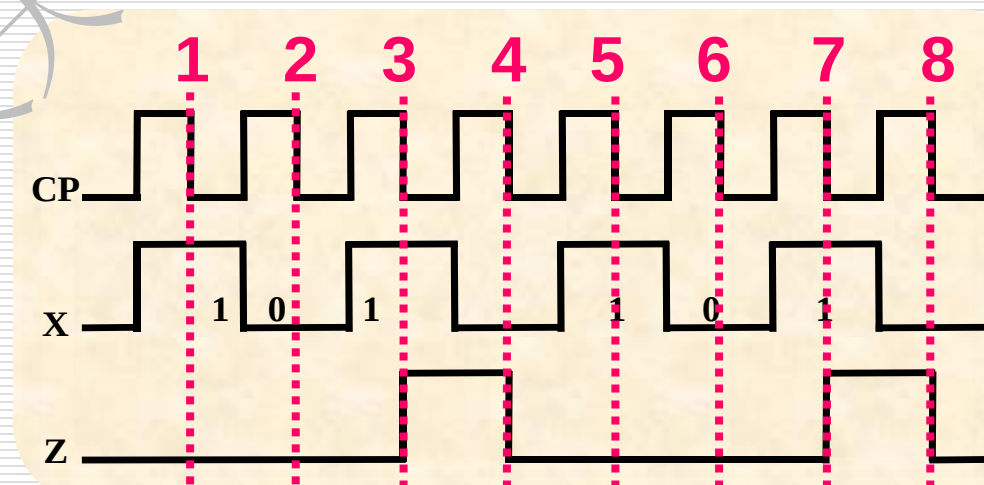
补充作业2

设计一个串行数据检测器。电路的输入信号是与时钟脉冲同步的串行数据X，输出信号为Z；要求电路在X信号输入出现101序列时，输出信号Z为1，否则为0。输入信号X序列及相应输出信号Z的波形示意图如图所示。（输入序列不允许重迭）

- (1) 画出原始状态转换图；
- (2) 列出原始状态表；
- (3) 画出最简状态表和最简状态转换图；
- (4) 若上题中的条件改为输入序列允许重迭，完成题目要求的 (1) (2) (3)



电路框图



不允许重迭：序列尾部的比特不能作为下一个序列的首比特

1 0 1 0 1 0 1

允许重迭：序列尾部的比特可以作为下一个序列的首比特

1 0 1 0 1 0 1

例：

1 0 1 0 1 0 1 0 1 0 1 0 1 0 1

不允许重迭：

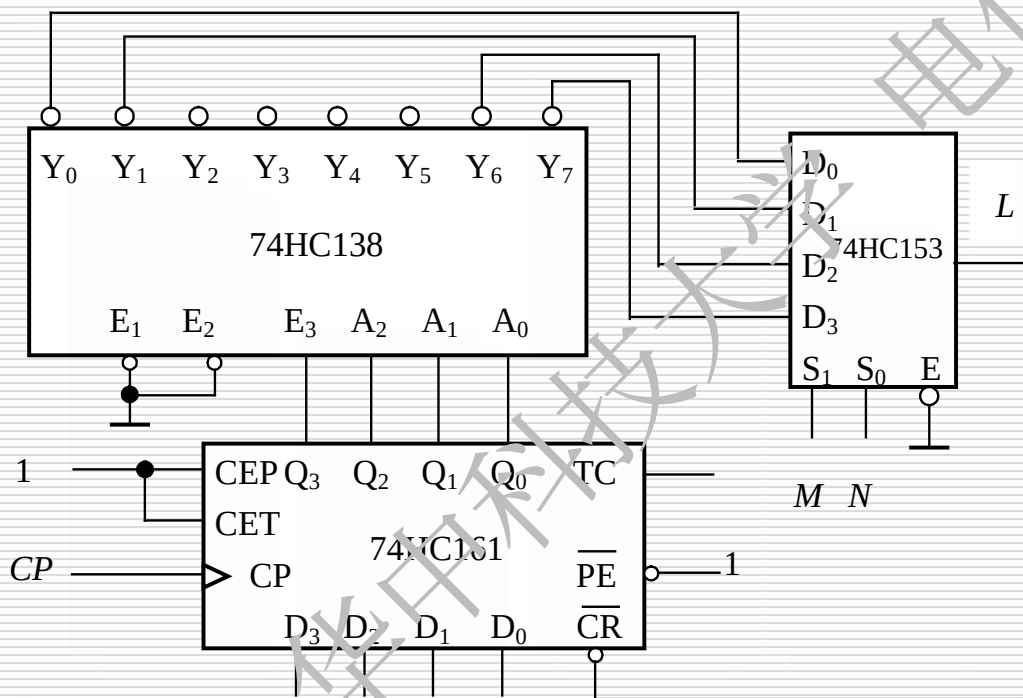
允许重迭：

检测到4次

检测到8次

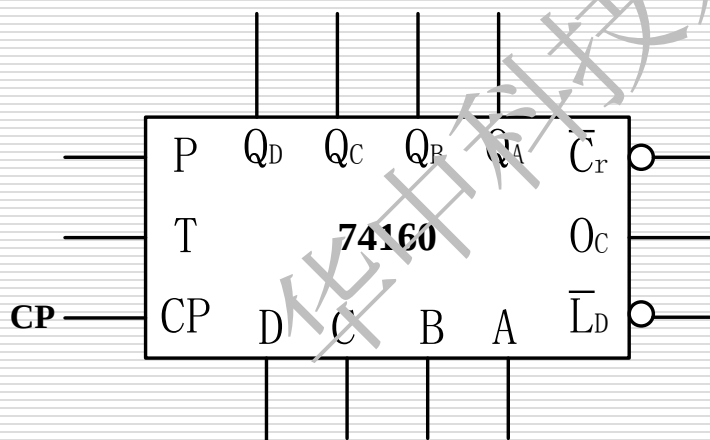
补充作业3

电路如图所示，图中74HC153为四选一数据选择器。试问当 MN 为各种不同输入时，电路分别是那几种不同进制的计数器，写出分析过程。



补充作业4

利用8421BCD码十进制同步加法计数器74160设计一个模8的加法计数器，计数规律为
 $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 6 \rightarrow 7 \rightarrow 8 \rightarrow 9 \rightarrow 0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 6 \rightarrow 7 \rightarrow 8 \rightarrow 9 \dots$ 。要求写出简要设计过程，并画出电路图



补充作业5

分析如图所示电路，画出波形图(不必写过程)

